

COMBINATORIAL OPTIMIZATION

# **The Traveling Salesman Problem and Its Variations**

Gregory Gutin and Abraham P. Punnen (Eds.)

Kluwer Academic Publishers

## The Traveling Salesman Problem and Its Variations

# COMBINATORIAL OPTIMIZATION

---

## VOLUME 12

---

Through monographs and contributed works the objective of the series is to publish state of the art expository research covering all topics in the field of combinatorial optimization. In addition, the series will include books which are suitable for graduate level courses in computer science, engineering, business, applied mathematics, and operations research.

Combinatorial (or discrete) optimization problems arise in various applications, including communications network design, VLSI design, machine vision, airline crew scheduling, corporate planning, computer-aided design and manufacturing, database query design, cellular telephone frequency assignment, constraint directed reasoning, and computational biology. The topics of the books will cover complexity analysis and algorithm design (parallel and serial), computational experiments and applications in science and engineering.

### *Series Editors:*

Ding-Zhu Du, *University of Minnesota*  
Panos M. Pardalos, *University of Florida*

### *Advisory Editorial Board:*

Afonso Ferreira, *CNRS-LIP ENS Lyon*  
Jun Gu, *University of Calgary*  
David S. Johnson, *AT&T Research*  
James B. Orlin, *M.I.T.*  
Christos H. Papadimitriou, *University of California at Berkeley*  
Fred S. Roberts, *Rutgers University*  
Paul Spirakis, *Computer Tech Institute (CTI)*

# The Traveling Salesman Problem and Its Variations

edited by

Gregory Gutin

*Royal Holloway,  
University of London, U.K.*

and

Abraham P. Punnen

*University of New Brunswick,  
Saint John, Canada*

**KLUWER ACADEMIC PUBLISHERS**  
NEW YORK, BOSTON, DORDRECHT, LONDON, MOSCOW

eBook ISBN: 0-306-48213-4  
Print ISBN: 1-4020-0664-0

©2004 Kluwer Academic Publishers  
New York, Boston, Dordrecht, London, Moscow

Print ©2002 Kluwer Academic Publishers  
Dordrecht

All rights reserved

No part of this eBook may be reproduced or transmitted in any form or by any means, electronic, mechanical, recording, or otherwise, without written consent from the Publisher

Created in the United States of America

Visit Kluwer Online at: <http://kluweronline.com>  
and Kluwer's eBookstore at: <http://ebooks.kluweronline.com>

# Contents

|                                                                           |     |
|---------------------------------------------------------------------------|-----|
| Preface                                                                   | xi  |
| Contributing Authors                                                      | xv  |
| 1                                                                         |     |
| The Traveling Salesman Problem: Applications, Formulations and Variations | 1   |
| <i>Abraham P. Punnen</i>                                                  |     |
| 1.1 Introduction                                                          | 1   |
| 1.2 Simple Variations of the TSP                                          | 7   |
| 1.3 Applications of TSP                                                   | 9   |
| 1.4 Alternative representations of the TSP                                | 15  |
| 1.5 Matrix Transformations                                                | 23  |
| 1.6 More Variations of the TSP                                            | 24  |
| 2                                                                         |     |
| Polyhedral Theory and Branch-and-Cut Algorithms for the Symmetric TSP     | 29  |
| <i>Denis Naddef</i>                                                       |     |
| 2.1 Introduction                                                          | 29  |
| 2.2 Integer linear programming models                                     | 30  |
| 2.3 STSP polytope and relaxations                                         | 35  |
| 2.4 The graphical relaxation Framework                                    | 44  |
| 2.5 The Comb inequalities                                                 | 58  |
| 2.6 The Star and Path inequalities                                        | 62  |
| 2.7 The Clique Tree and Bipartition inequalities                          | 67  |
| 2.8 The Ladder inequalities                                               | 71  |
| 2.9 A general approach to some TSP valid inequalities                     | 73  |
| 2.10 A unifying family of inequalities                                    | 77  |
| 2.11 Domino inequalities                                                  | 78  |
| 2.12 Other inequalities                                                   | 81  |
| 2.13 The separation problem                                               | 82  |
| 2.14 Greedy heuristic for minimum cut                                     | 84  |
| 2.15 Graph associated to a vector $x^* \in \mathbb{R}^E$                  | 85  |
| 2.16 Heuristics for Comb Separation                                       | 86  |
| 2.17 Separation of multi-handle inequalities                              | 94  |
| 2.18 Separation outside the template paradigm                             | 100 |
| 2.19 Branch-and-Cut implementation of the STSP                            | 105 |
| 2.20 Computational results                                                | 113 |
| 2.21 Conclusion                                                           | 114 |

|   |                                                                 |     |
|---|-----------------------------------------------------------------|-----|
| 3 |                                                                 |     |
|   | Polyhedral Theory for the Asymmetric Traveling Salesman Problem | 117 |
|   | <i>Egon Balas and Matteo Fischetti</i>                          |     |
|   | 3.1 Introduction                                                | 117 |
|   | 3.2 Basic ATS inequalities                                      | 120 |
|   | 3.3 The monotone ATS polytope                                   | 128 |
|   | 3.4 Facet-lifting procedures                                    | 133 |
|   | 3.5 Equivalence of inequalities and canonical forms             | 142 |
|   | 3.6 Odd closed alternating trail inequalities                   | 145 |
|   | 3.7 Source-destination inequalities                             | 150 |
|   | 3.8 Lifted cycle inequalities                                   | 155 |
| 4 |                                                                 |     |
|   | Exact Methods for the Asymmetric Traveling Salesman Problem     | 169 |
|   | <i>Matteo Fischetti, Andrea Lodi and Paolo Toth</i>             |     |
|   | 4.1 Introduction                                                | 169 |
|   | 4.2 AP-Based Branch-and-Bound Methods                           | 172 |
|   | 4.3 An Additive Branch-and-Bound Method                         | 176 |
|   | 4.4 A Branch-and-Cut Approach                                   | 181 |
|   | 4.5 Computational Experiments                                   | 194 |
| 5 |                                                                 |     |
|   | Approximation Algorithms for Geometric TSP                      | 207 |
|   | <i>Sanjeev Arora</i>                                            |     |
|   | 5.1 Background on Approximation                                 | 208 |
|   | 5.2 Introduction to the Algorithm                               | 209 |
|   | 5.3 Simpler Algorithm                                           | 214 |
|   | 5.4 Better Algorithm                                            | 215 |
|   | 5.5 Faster Algorithm                                            | 219 |
|   | 5.6 Generalizations to other Problems                           | 220 |
| 6 |                                                                 |     |
|   | Exponential Neighborhoods and Domination Analysis for the TSP   | 223 |
|   | <i>Gregory Gutin, Anders Yeo and Alexei Zverovitch</i>          |     |
|   | 6.1 Introduction, Terminology and Notation                      | 223 |
|   | 6.2 Exponential Neighborhoods                                   | 228 |
|   | 6.3 Upper Bounds for Neighborhood Size                          | 237 |
|   | 6.4 Diameters of Neighborhood Structure Digraphs                | 240 |
|   | 6.5 Domination Analysis                                         | 244 |
|   | 6.6 Further Research                                            | 254 |
| 7 |                                                                 |     |
|   | Probabilistic Analysis of the TSP                               | 257 |
|   | <i>A. M. Frieze and J. E. Yukich</i>                            |     |
|   | 7.1 Introduction                                                | 257 |
|   | 7.2 Hamiltonian Cycles in Random Graphs                         | 259 |
|   | 7.3 Traveling Salesman Problem: Independent Model               | 274 |
|   | 7.4 Euclidean Traveling Salesman Problem                        | 282 |
| 8 |                                                                 |     |
|   | Local Search and Metaheuristics                                 | 309 |
|   | <i>César Rego and Fred Glover</i>                               |     |
|   | 8.1 Background on Heuristic Methods                             | 309 |

|                                                                                                           |     |
|-----------------------------------------------------------------------------------------------------------|-----|
| <i>Contents</i>                                                                                           | vii |
| 8.2 Improvement Methods                                                                                   | 313 |
| 8.3 Tabu Search                                                                                           | 345 |
| 8.4 Recent Unconventional Evolutionary Methods                                                            | 355 |
| 8.5 Conclusions and Research Opportunities                                                                | 367 |
| 9                                                                                                         |     |
| Experimental Analysis of Heuristics for the STSP                                                          | 369 |
| <i>David S. Johnson and Lyle A. McGeoch</i>                                                               |     |
| 9.1 Introduction                                                                                          | 369 |
| 9.2 DIMACS STSP Implementation Challenge                                                                  | 371 |
| 9.3 Heuristics and Results                                                                                | 381 |
| 9.4 Conclusions and Further Research                                                                      | 438 |
| 10                                                                                                        |     |
| Experimental Analysis of Heuristics for the ATSP                                                          | 445 |
| <i>David S. Johnson, Gregory Gutin, Lyle A. McGeoch, Anders Yeo, Weixiong Zhang and Alexei Zverovitch</i> |     |
| 10.1 Introduction                                                                                         | 446 |
| 10.2 Methodology                                                                                          | 447 |
| 10.3 Heuristics                                                                                           | 457 |
| 10.4 Results                                                                                              | 463 |
| 10.5 Conclusions and Further Research                                                                     | 486 |
| 11                                                                                                        |     |
| Polynomially Solvable Cases of the TSP                                                                    | 489 |
| <i>Santosh N. Kabadi</i>                                                                                  |     |
| 11.1 Introduction                                                                                         | 489 |
| 11.2 Constant TSP and its generalizations                                                                 | 489 |
| 11.3 The Gilmore-Gomory TSP (GG-TSP)                                                                      | 494 |
| 11.4 GG Scheme: a generalization of Gilmore-Gomory scheme for GG-TSP                                      | 506 |
| 11.5 Minimum cost connected directed pseudograph problem with node deficiency requirements (MCNDP)        | 539 |
| 11.6 Solvable cases of geometric TSP                                                                      | 547 |
| 11.7 Generalized graphical TSP                                                                            | 560 |
| 11.8 Solvable classes of TSP on specially structured graphs                                               | 564 |
| 11.9 Classes of TSP with known compact polyhedral representation                                          | 566 |
| 11.10 Other solvable cases and related results                                                            | 576 |
| 12                                                                                                        |     |
| The Maximum TSP                                                                                           | 585 |
| <i>Alexander Barvinok, Edward Kh. Gimadi and Anatoliy I. Serdyukov</i>                                    |     |
| 12.1 Introduction                                                                                         | 585 |
| 12.2 Hardness Results                                                                                     | 588 |
| 12.3 Preliminaries: Factors and Matchings                                                                 | 589 |
| 12.4 MAX TSP with General Non-Negative Weights                                                            | 590 |
| 12.5 The Symmetric MAX TSP                                                                                | 591 |
| 12.6 The Semimetric MAX TSP                                                                               | 595 |
| 12.7 The Metric MAX TSP                                                                                   | 597 |
| 12.8 TSP with Warehouses                                                                                  | 598 |
| 12.9 MAX TSP in a Space with a Polyhedral Norm                                                            | 600 |
| 12.10 MAX TSP in a Normed Space                                                                           | 602 |



|       |                                                                         |     |
|-------|-------------------------------------------------------------------------|-----|
| 12.11 | Probabilistic Analysis of Heuristics                                    | 605 |
| 13    |                                                                         |     |
|       | The Generalized Traveling Salesman and Orienteering Problems            | 609 |
|       | <i>Matteo Fischetti, Juan-José Salazar-González and Paolo Toth</i>      |     |
| 13.1  | Introduction                                                            | 609 |
| 13.2  | The Generalized Traveling Salesman Problem                              | 617 |
| 13.3  | The Orienteering Problem                                                | 642 |
| 14    |                                                                         |     |
|       | The Prize Collecting Traveling Salesman Problem<br>and Its Applications | 663 |
|       | <i>Egon Balas</i>                                                       |     |
| 14.1  | Introduction                                                            | 663 |
| 14.2  | An Application                                                          | 664 |
| 14.3  | Polyhedral Considerations                                               | 667 |
| 14.4  | Lifting the Facets of the ATS Polytope                                  | 668 |
| 14.5  | Primitive Inequalities from the ATSP                                    | 671 |
| 14.6  | Cloning and Clique Lifting for the PCTSP.                               | 680 |
| 14.7  | A Projection: The Cycle Polytope                                        | 686 |
| 15    |                                                                         |     |
|       | The Bottleneck TSP                                                      | 697 |
|       | <i>Santosh N. Kabadi and Abraham P. Punnen</i>                          |     |
| 15.1  | Introduction                                                            | 697 |
| 15.2  | Exact Algorithms                                                        | 699 |
| 15.3  | Approximation Algorithms                                                | 705 |
| 15.4  | Polynomially Solvable Cases of BTSP                                     | 714 |
| 15.5  | Variations of the Bottleneck TSP                                        | 734 |
| 16    |                                                                         |     |
|       | TSP Software                                                            | 737 |
|       | <i>Andrea Lodi and Abraham P. Punnen</i>                                |     |
| 16.1  | Introduction                                                            | 737 |
| 16.2  | Exact algorithms for TSP                                                | 739 |
| 16.3  | Approximation Algorithms for TSP                                        | 741 |
| 16.4  | Java Applets                                                            | 745 |
| 16.5  | Variations of the TSP                                                   | 745 |
| 16.6  | Other Related Problems and General-Purpose Codes                        | 747 |
|       | Appendix: A. Sets, Graphs and Permutations                              | 750 |
|       | <i>Gregory Gutin</i>                                                    |     |
| A-.1  | Sets                                                                    | 750 |
| A-.2  | Graphs                                                                  | 750 |
| A-.3  | Permutations                                                            | 753 |
|       | Appendix: B. Computational Complexity                                   | 754 |
|       | <i>Abraham P. Punnen</i>                                                |     |
| B-.1  | Introduction                                                            | 754 |
| B-.2  | Basic Complexity Results                                                | 756 |
| B-.3  | Complexity and Approximation                                            | 758 |

|                 |     |
|-----------------|-----|
| <i>Contents</i> | ix  |
| References      | 761 |
| List of Figures | 807 |
| List of Tables  | 813 |
| Index           | 817 |

*This page intentionally left blank*

## Preface

The traveling salesman problem (TSP) is perhaps the most well known combinatorial optimization problem. The book “The Traveling Salesman Problem: A guided tour of combinatorial optimization” edited by Lawler, Lenstra, Rinooy Kan and Shmoys provides the state of the art description of the topic up to 1985. Since then, several significant developments have taken place in the area of combinatorial optimization in general and the traveling salesman problem in particular. This warrants the need for an updated book. We discussed this matter with many distinguished colleagues. These consultations culminated in the project of compiling a new book on the TSP. Enthusiastic support from the research community provided us with the courage and motivation to take up this challenging task.

The pattern and style of this new book closely resemble those of its predecessor [548]. In addition to standard and more traditional topics, we also cover domination analysis of approximation algorithms and some important variations of the TSP. The purpose of the book is to serve as a self-contained reference source which updates the book by Lawler et. al [548]. We believe that the book can also be used for specialized graduate and senior undergraduate courses and research projects.

Roughly speaking, the traveling salesman problem is to find a shortest route of a traveling salesperson that starts at a home city, visits a prescribed set of other cities and returns to the starting city. The distance travelled in such a tour obviously depends on the order in which the cities are visited and, thus, the problem is to find an ‘optimal’ ordering of the cities. There are several applications of the TSP that extend beyond the route planning of a traveling salesman. Chapter 1 describes several formulations, applications, and variations of the problem.

As one can see, it does not take much mathematical sophistication to understand many of the formulations of the TSP. However, TSP is a typical ‘hard’ optimization problem and solving very large instances of it is very difficult if not impossible. Nevertheless, recent developments in polyhedral theory and branch-and-cut algorithms have significantly in-

creased the size of instances which can be solved to optimality. Chapters 2–4 provides a thorough description of polyhedral theory, and implementation and testing of exact algorithms. Chapter 2 discusses polyhedral theory and experimental results of branch-and-cut algorithms for the symmetric TSP. Chapter 3 deals with polyhedral results for the asymmetric TSP developed mostly in last 15 years. Chapter 4 deals with implementation and testing of branch-and-bound and branch-and-cut algorithms for the asymmetric version of the TSP.

Despite the fact that significant progress have been made in our ability to solve TSP by exact algorithms, still there are several instances of the problem to be solved to optimality, that are hard for exact algorithms. Moreover, even when an instance is solvable by an exact algorithm, the running time may become prohibitively large for certain applications. In some cases, the problem data of the instance in hand may not be exact, for various reasons, and thus solving such an instance to optimality may not be viable, especially when it takes a significant amount of computational time. Therefore, researchers have investigated a large number of heuristic algorithms, some of which normally produce near optimal solutions. Chapters 5–10 are devoted to various aspects of heuristic algorithms for the TSP.

Chapter 5 presents a compact account on recent developments on approximation algorithms for the geometric TSP in general and Euclidean TSP in particular. Chapter 6 discusses very large, exponential, size neighborhoods for the TSP and domination analysis, a new tool to compare the quality of heuristics. Probabilistic approaches to the TSP are studied in Chapter 7. Probabilistic approaches try to elucidate the properties of typical rather than worst-case instances. Chapter 8 describes well-established and new heuristic approaches to the symmetric TSP. Chapter 9 provides results of extensive computational experiments with numerous heuristic algorithms for the Symmetric TSP. Computational experience with heuristics for the less well-studied Asymmetric TSP is described in Chapter 10.

Chapter 11 is an extensive survey on polynomial time solvable cases of the TSP. Chapters 12–15 are devoted to variations and generalizations of the TSP. Interesting differences between the TSP and its maximization version are described in Chapter 12, where several approximation algorithms for the maximum TSP are presented. Chapter 13 deals with the Generalized TSP and Orienteering Problem. Polyhedral results as well as implementation of exact algorithms, are discussed in detail. The Prize Collecting TSP, where the traveling salesperson needs to visit only part of the prescribed set of cities as long as he/she collected enough “prize”

items, is the topic of Chapter 14. Chapter 15 considers the Bottleneck TSP, where the largest inter-city distance along the route is minimized.

A summary of available TSP software is discussed in Chapter 16. This chapter will be of particular interest to the readers looking for a “quick” way to solve their TSP instances without getting deeply involved with algorithmic ideas and coding techniques. The book has two appendices. Appendix A is a short overview on graphs, sets and permutations for the benefit of readers who want to refresh their knowledge on these topics. Appendix B discusses complexity issues.

Our foremost thanks go to all authors for their enthusiasm and hard work without which this book would not have been completed. Every chapter of this book was read by several researchers who provided comments and suggestions. We would like to thank, among others, Norbert Ascheuer, Rafi Hassin, Gilbert Laporte, Adam Letchford, Francois Margot, Pablo Moscato, K.G. Murty, K.P.K. Nair, Prabha Sharma, Mike Steele, Stefan Voss, and Klaus Wenger for their valuable comments and suggestions. Some of the authors, in particular, Matteo Fischetti, Alan Frieze, Fred Glover, David Johnson, Santosh Kabadi, Andrea Lodi, Denis Naddef, Cesar Rego, Paolo Toth, Anders Yeo and Alexei Zverovitch also participated actively in refereeing chapters and our gratitude goes to each one of them. We highly appreciate the help and support from our publisher especially Gary Folven and Ramesh Sharda. Special thanks are due to John Martindale, for persuading us to publish the book with Kluwer and retaining his patience and optimism as one deadline after another did not materialize.

GREGORY GUTIN AND ABRAHAM PUNNEN

*This page intentionally left blank*

## Contributing Authors

**Sanjeev Arora** is an Associate Professor of Computer Science at Princeton University. His doctoral dissertation, submitted at UC Berkeley in 1994, was cowinner of ACM's doctoral dissertation award in 1995. He has also received the NSF Career award, and Packard and Sloan fellowships. He was a cowinner of the ACM-Sigact Gödel prize in 2001.

**Egon Balas** is University Professor and the Lord Professor of Operations Research at Carnegie Mellon University. He received the John von Neumann Theory Prize of INFORMS in 1995, and the EURO Gold Medal in 2001.

**Alexander Barvinok** is Professor of Mathematics at the University of Michigan. His research interests include computational complexity and algorithms in algebra, geometry and combinatorics.

**Matteo Fischetti** is Full Professor of Operations Research at the Faculty of Engineering of the University of Padua. In 1987 his Ph.D. dissertation was awarded the first prize *Best PhD Dissertation on Transportation* by the Operations Research Society of America. He is the author of more than 50 papers published in international scientific journals.

**Alan Frieze** is Professor of Mathematics at Carnegie Mellon University. He was the joint recipient of the 1991 Fulkerson Prize for Discrete Mathematics, awarded jointly by the American Mathematical Society and the Mathematical Programming Society. He is the author of more than 200 scientific papers.

**Edward Gimadi** is Professor at Novosibirsk State University, PhD (1971), ScD (1988), and Head Researcher at Sobolev Institute of mathematics of Russian Academy of Sciences.



**Fred Glover** is the MediaOne Chaired Professor in Systems Science at the University of Colorado, Boulder. He has authored or co-authored more than three hundred published articles and four books in the fields of mathematical optimization, computer science and artificial intelligence. Professor Glover is the recipient of the distinguished von Neumann Theory Prize, as well as of numerous other awards and honorary fellowships, including those from the American Association for the Advancement of Science, the NATO Division of Scientific Affairs, the Institute of Management Science, the Operation Research Society, the Decision Sciences Institute, the U.S. Defense Communications Agency, the Energy Research Institute, the American Assembly of Collegiate Schools of Business, Alpha Iota Delta, and the Miller Institute for Basic Research in Science. He serves on the advisory boards of several organizations and is co-founder of OptTek Systems, Inc. and Heuristic, Inc.

**Gregory Gutin** is Professor of Computer Science at Royal Holloway, University of London. He earned his PhD from Tel Aviv University in 1993. He wrote, together with J. Bang-Jensen, *Digraphs: Theory, Algorithms and Applications*, Springer, London, 2000.

**David S. Johnson** is the Head of the Algorithms and Optimization Department at AT&T Labs – Research in Florham Park, New Jersey, and has been with the Labs in its various guises since 1973. He and Michael Garey wrote the Lancaster Prize-winning book *Computers and Intractability: A Guide to the Theory of NP-Completeness*. He has been active in the field of experimental analysis of algorithms since 1983, and has overseen the series of DIMACS Implementation Challenges since its inception in 1990.

**Santosh Narayan Kabadi** is Professor of Management Science at the Faculty of Administration, University of New Brunswick, Fredericton, NB, Canada. He earned his Ph.D from University of Texas at Dallas.

**Andrea Lodi** is Assistant Professor of Operations Research at the Faculty of Engineering of the University of Bologna. He earned the PhD in System Engineering from the same university in March 2000.

**Lyle A. McGeoch** is Professor of Computer Science at Amherst College, where he taught since 1987. He earned his Ph.D at Carnegie Mellon University.

**Denis Naddef** is Professor of Operations Research and Combinatorial Optimization at the Institut National Polytechnique de Grenoble. He received his Ph.D from the University of Grenoble in 1978.

**Abraham P. Punnen** is Professor of Operations Research at University of New Brunswick, Saint John. He earned his Ph.D at Indian Institute of Technology, Kanpur in 1990. He published extensively in the area of combinatorial optimization and serves on the editorial boards of various journals. He is a member of the board of directors of the Canadian Mathematical Society.

**Cesar Rego** is Associate Professor at School of Business Administration and a Senior Researcher at the Hearin Center for Enterprise Science, University of Mississippi, USA. He received his Ph.D. in Computer Science from the University of Versailles, France, after earning a MSc in Operations Research and Systems Engineering from the Technical School (IST) of the University of Lisbon. His undergraduate degree in Computer Science and Applied Mathematics is from the Portucalense University in Portugal. His publications have appeared in books on metaheuristics and in leading journals on optimization such as EJOR, JORS, Parallel Computing, and Management Science.

**Juan José Salazar González** is Associate Professor of the “Departamento de Estadística, Investigación Operativa y Computación”, University of La Laguna (Tenerife, Spain), since 1997. He earned his Ph.D in Mathematics from University of Bologna (Italy) in 1992, and Ph.D in Computer Science from University of La Laguna (Spain) in 1995.

**Anatoly Serdyukov** was Senior Researcher at Sobolev Institute of Mathematics of Russian Acadimy of Sciences, PhD (1980). He passed away on 7 February 2001.

**Paolo Toth** is Full Professor of Combinatorial Optimization at the Faculty of Engineering of the University of Bologna. He is the author of more than 90 papers published in international scientific journals, and co-author of the book *Knapsack Problems: Algorithms and Computer Implementations*, Wiley, 1990. He was President of EURO (Association of the European Operational Research Societies) from 1995 to 1996, and is President of IFORS (International Federation of the Operational Research Societies) for the period 2001-2003.

**Anders Yeo** is a Lecturer in Computer Science, Royal Holloway, University of London. He earned his PhD from Odense University, Denmark. His main research interests are in graph theory and combinatorial optimization.

**Joseph Yukich** is Professor of Mathematics at Lehigh University. He is the recipient of two Fulbright awards and has authored the monograph *Probability Theory of Classical Euclidean Optimization Problems*, Lecture Notes in Mathematics, volume 1675, 1998.

**Weixiong Zhang** is an Associate Professor of Computer Science at Washington University in St. Louis. He earned his PhD from University of California at Los Angeles. He is the author of *State-space search: Algorithms, Complexity, Extensions, and Applications* published by Springer-Verlag in 1999.

**Alexei Zverovitch** is a PhD student at Royal Holloway, University of London, UK. His supervisor is Gregory Gutin.

## Chapter 1

# THE TRAVELING SALESMAN PROBLEM: APPLICATIONS, FORMULATIONS AND VARIATIONS

Abraham P. Punnen

*Department of Mathematical Sciences*

*University of New Brunswick-Saint John*

*New Brunswick, Canada*

punnen@unbsj.ca

This chapter is dedicated to the memory of my grandparents  
Mr. Korah Abraham and Mrs. Sosamma Abraham

### 1. Introduction

The traveling salesman problem (TSP) is to find a routing of a salesman who starts from a home location, visits a prescribed set of cities and returns to the original location in such a way that the total distance travelled is minimum and each city is visited exactly once. Although a business tour of a modern day traveling salesman may not seem to be too complex in terms of route planning, the TSP in its generality represents a typical ‘hard’ combinatorial optimization problem.

TSP was known among mathematicians and statisticians under various names. Karl Menger considered a variation of TSP called the *messenger problem* (Botenproblem) [116, 451, 593]. It may be noted that the messenger problem and TSP are equivalent in the sense that one problem can be reduced to the other using very simple transformations. Bock [116] gives an English translation of the description of messenger problem from [593] as follows:

“We designate as the Messenger Problem (since this problem is encountered by every postal messenger, as well as by many travellers) the task

of finding, for a finite number of points whose pairwise distances are known, the shortest path connecting the points. This problem is naturally always solvable by making a finite number of trials below the number of permutations of the given points. The rule, that one should first go from the starting point to the nearest point, then to the point nearest to this etc., does not in general result in the shortest path.”

It is believed that the report by Menger [593] is the first published work on TSP. He pointed out that complete enumeration is a possible strategy to compute an optimal solution but a ‘nearest neighbor’ algorithm does not guarantee an optimal solution.

Mahalanobis [575] presented a talk at the Indian Science Congress in 1939 on the problem of estimating the area under jute (*Corchorus Capsularis*) farm in Bengal (currently spread over West Bengal in India and Bangladesh). The objective is to keep the cost involved in the land survey operations below a threshold level while maximizing the accuracy of the result. As part of the cost calculations, he was interested in a least cost routing through a finite number of points in the Euclidean plane. Although no attempt was reported in calculating the least possible cost for such a traversal, it is clear from the discussions that Mahalanobis was aware of the difficulties involved in finding such a minimum cost routing. His cost calculations however are based on expected values considering random points in the plane which forms a small part of a more general cost function.

In a 1955 paper, Morton and Land [608] indicated that “the TSP may be familiar to statisticians as *the mean minimum distances problem*” by citing supporting references [358, 458, 583]. In their brief historic account on TSP, Morton and Land wrote:

“In the United States this problem is known as the Traveling-Salesman problem; the salesman wishes to visit one city in each of the 48 States and Washington, D.C, in such a sequence as to minimize the total road distance traveled. Thinking of the problem independently and on a smaller geographical scale, we used to call it the laundry van problem, where the conditions were a daily service by a one-van laundry. Since the American term was used by Robinson (1949) we propose to adopt it here.”

The name “traveling salesman problem” for the optimization problem of our discussion is believed to have originated in the United States. Perhaps the first report using this name was published in 1949 [725]. However, it would be reasonable to say that a systematic study of the TSP as a combinatorial optimization problem began with the work of Dantzig, Fulkerson, and Johnson [239]. The survey papers [474, 590, 592, 591] and the books [548, 711] summarize various developments on the subject.

Let us now give a formal mathematical definition of the TSP. For details of basic terminology and notations in graph theory we refer to Appendix A. Let  $G = (V, E)$  be a graph (directed or undirected) and  $\mathbb{F}$  be the family of all Hamiltonian cycles (tours) in  $G$ . For each edge  $e \in E$  a cost (weight)  $c_e$  is prescribed. Then the traveling salesman problem is to find a tour (Hamiltonian cycle) in  $G$  such that the sum of the costs of edges of the tour is as small as possible.

Without loss of generality, we assume that  $G$  is a complete graph (digraph), for otherwise we could replace the missing edges with edges of very large cost. Let the node set  $V = \{1, 2, \dots, n\}$ . The matrix  $C = (c_{ij})_{n \times n}$ , is called the cost matrix (also referred to as the distance matrix or weight matrix), where the  $(i, j)^{th}$  entry  $c_{ij}$  corresponds to the cost of the edge joining node  $i$  to node  $j$  in  $G$ .

The TSP can also be viewed as a permutation problem. Generally, two variations of the permutation representation of TSP are in use; a *quadratic representation* and a *linear representation*. Let  $\mathbb{P}_n$  be the collection of all permutations of the set  $\{1, 2, \dots, n\}$ . Then the TSP is to find a  $\pi = (\pi(1), \pi(2), \dots, \pi(n))$  in  $\mathbb{P}_n$  such that  $c_{\pi(n)\pi(1)} + \sum_{i=1}^{n-1} c_{\pi(i)\pi(i+1)}$  is minimized. In this case  $(\pi(1), \pi(2), \dots, \pi(n))$  gives the order in which cities are visited, starting with city  $\pi(1)$ . For example, if  $n = 5$  and  $\pi = (2, 1, 5, 3, 4)$  then the corresponding tour will be  $(2, 1, 5, 3, 4, 2)$ . Every cyclic shift of  $\pi$  also give the same tour. Thus there are  $n$  different permutations which represent the same tour. This representation of TSP is generally used in the context of some sequencing problems and gives rise to a binary quadratic programming formulation of the TSP (see Section 4). Thus, we refer to this the *quadratic permutation representation* of TSP. Chapter 12 of this book uses this quadratic permutation representation of the TSP.

In another permutation representation of TSP, referred to as *linear permutation representation*, only special types of permutations, called cyclic permutations, are considered feasible. Let  $\mathbb{C}_n$  be the collection of all cyclic permutations of  $\{1, 2, \dots, n\}$ . Then the TSP is to find a  $\sigma = (\sigma(1), \sigma(2), \dots, \sigma(n)) \in \mathbb{C}_n$  such that,  $\sum_{i=1}^n c_{i\sigma(i)}$  is minimized. Under this representation,  $\sigma(i)$  is the successor of city  $i$  in the resulting tour, for  $i = 1, 2, \dots, n$ . For example if  $n = 5$  and  $(\sigma(1), \sigma(2), \dots, \sigma(5)) = (2, 4, 5, 3, 1)$ , the resulting tour is given by  $(1, 2, 4, 3, 5, 1)$ . This representation of TSP leads to a binary liner programming formulation of TSP and hence we call it the *linear permutation representation*. Chapter 11 and part of Chapter 15 use the linear permutation representation of the TSP.

Depending on the nature of the cost matrix (equivalently the nature of  $G$ ), TSP is divided in to two classes. If  $C$  is symmetric (i.e.  $G$  is

undirected) then the TSP is called the symmetric traveling salesman problem (STSP). If  $C$  is not necessarily symmetric (equivalently the graph  $G$  is directed) then it is called the asymmetric traveling salesman problem (ATSP). Since every undirected graph can be viewed as a directed graph, by duplicating edges one in the forward direction and the other in the backward direction, STSP can be viewed as a special case of ATSP. Interestingly, it is also possible to formulate ATSP as a STSP [471] by doubling the number of nodes. Consider the ATSP on a digraph  $D = (N, A)$  with  $c_{ij}$  as the cost of arc  $(i, j)$ . Construct the undirected graph  $G^* = (V^*, E^*)$  with node set  $V^* = N \cup \{1^*, 2^*, \dots, n^*\}$  where  $N = \{1, 2, \dots, n\}$ . For each arc  $(i, j)$  in  $D$ , create an edge  $(i, j^*)$  in  $G^*$  with cost  $c_{ij}$ . Also introduce  $n$  dummy edges  $(i, i^*)$  of cost  $-M$  where  $M$  is a very large number. All other edges have cost  $M$ . It can be verified that solving the ATSP on  $D$  is equivalent to solving the STSP on  $G^*$ .

### 1.1. The Shoelace Problem - A simple TSP

According to Bata shoe museum (Toronto, Canada) evidence of shoe making exists as early as 10,000 BC; the average person walks the equivalent of three-and-a-half times around the earth in a life time; and North Americans spend almost \$18 billion a year on footwear! Having a comfortable pair of shoes doesn't ensure comfortable walking. Proper lacing of the shoe is also important in increasing foot comfort and perhaps increasing the life span of the shoe. Needless to say, proper lacing of shoe is important for a traveling salesman. Our salesman however is interested in lacing his shoes with a lace of minimum length! Optimal shoe-lacing as a mathematical entertainment problem was introduced by Halton [431]. It can be described as follows:

Let  $A_0, A_1, A_2, \dots, A_n$  and  $B_0, B_1, B_2, \dots, B_n$  be the eyelets of a shoe arranged on two parallel lines. (See Fig: 1.1).

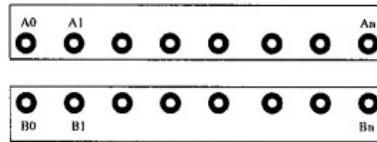


Figure 1.1. Eyelets on a shoe

A feasible lacing is a 'path' from  $A_0$  to  $B_0$  visiting each eyelet exactly once, choosing  $A_i$ 's and  $B_j$ 's alternately, and closing the path by moving from  $B_0$  to  $A_0$ . It can be represented as an ordered pair  $(\pi, \sigma)$  of two

permutations  $\pi$  and  $\sigma$  of  $\{1, 2, \dots, n\}$ . Note that  $(\pi, \sigma)$  represents the unique lacing

$$A_0 \rightarrow B_{\sigma(1)} \rightarrow A_{\pi(1)} \dots \rightarrow A_{\pi(n-1)} \rightarrow B_{\sigma(n)} \rightarrow A_{\pi(n)} \rightarrow B_0 \rightarrow A_0.$$

The length of a feasible lacing  $(\pi, \sigma)$ , denoted by  $L(\pi, \sigma)$ , is given by

$$\begin{aligned} & \sum_{i=1}^{n-1} [d(\sigma(i), \pi(i)) + d(\pi(i), \sigma(i+1)) + d(0, \sigma(1)) + d(\sigma(n), \pi(n)) \\ & + d(\pi(n), 0) + d(0, 0), \end{aligned}$$

where  $d(i, j)$  is the Euclidean distance between  $A_i$  and  $B_j$ . Let the distance between two consecutive eyelets on the same side be  $\alpha$  and the distance between  $A_i$  and  $B_i$  be  $\beta$ . Then, it can be verified that

$$\begin{aligned} L^{\alpha\beta}(\pi, \sigma) &= \sqrt{\sigma(1)^2\alpha^2 + \beta^2} + \sqrt{\pi(n)^2\alpha^2 + \beta^2} + \sqrt{(\pi(n) - \sigma(n))^2\alpha^2 + \beta^2} \\ &+ \sum_{i=1}^{n-1} (\sqrt{(\pi(i) - \sigma(i))^2\alpha^2 + \beta^2} + \sqrt{(\sigma(i+1) - \pi(i))^2\alpha^2 + \beta^2}) + \beta \end{aligned}$$

The *shoelace problem*  $SP(\alpha, \beta)$  is to find a feasible lacing  $(\bar{\pi}, \bar{\sigma})$  of minimum length  $L^{\alpha\beta}(\bar{\pi}, \bar{\sigma})$ . Note that there are  $(n!)^2$  feasible lacing patterns. Interestingly we will see that one of the most commonly used lacing patterns is indeed optimal. Let us consider four special lacing patterns  $(\pi, \sigma)$ . When  $\pi = (2, 4, \dots, 5, 3, 1)$  and  $\sigma = (5, 3, 1, \dots, 2, 4)$ ,  $(\pi, \sigma)$  is called a *zig-zag lacing*. When  $\pi = (2, 4, \dots, 3, 1)$  and  $\sigma = (1, 2, 4, 6, \dots, 5, 3)$  we have a *straight lacing*. *Quick lacing* corresponds to  $\pi = (n, n-1, \dots, 3, 2, 1)$  and  $\sigma = (n, n-1, \dots, 2, 1)$  and *diagonal lacing* corresponds to  $\pi = (1, 2, 3, \dots, n-1, n)$  and  $\sigma = (n, n-1, \dots, 2, 1)$ .

It can be verified that the zig-zag lacing is of length  $2(\beta + n\sqrt{\alpha^2 + \beta^2})$ , straight lacing is of length  $(n+1)\beta + 2\sqrt{\beta^2 + \alpha^2} + (n-1)\sqrt{4\alpha^2 + \beta^2}$ , quick lacing is of length  $(n+1)\beta + n\sqrt{\alpha^2 + \beta^2} + \sqrt{n^2\alpha^2 + \beta^2}$ , and diagonal lacing is of length  $2(\beta + \sum_{i=1}^{n-1} \sqrt{(n-i)^2\alpha^2 + \beta^2})$ . By choosing  $n = 10, \alpha = 1$  and  $\beta = 2$ , the zig-zag, straight, quick, and diagonal lacings have approximate lengths 48.72, 51.92, 54.55, and 103.19, respectively. Thus the zig-zag lacing comes out to be the winner. Indeed, it can be shown that,

**Theorem 1** [431] *For  $n \geq 3$ , zig-zag lacing is the unique optimal solution to the shoelace problem  $SP(\alpha, \beta)$ .*

For  $n = 2$ , all four lacing patterns discussed earlier have the same length. So far we have assumed that  $A_0, A_1, A_2, \dots, A_n$  are arranged



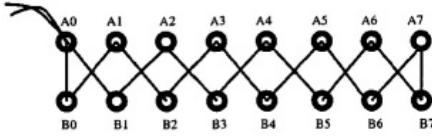


Figure 1.2. Zig-zag Lacing

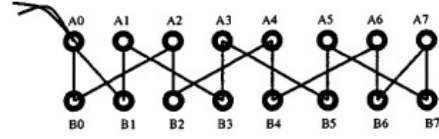


Figure 1.3. Straight Lacing

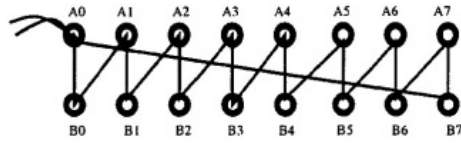


Figure 1.4. quick Lacing

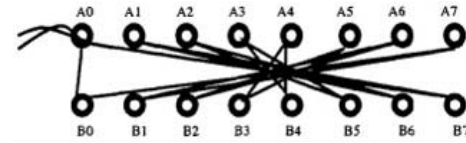


Figure 1.5. diagonal Lacing

on a straight line and  $B_0, B_1, B_2, \dots, B_n$  are arranged on a parallel line. However, this assumption can be relaxed. If  $A_i A_j B_k B_l$  for all  $i < j$  and  $k > l$  forms a convex quadrilateral, then also the zig-zag lacing can be shown to be optimal [600, 789]. What can we say about the diagonal lacing? Is it the longest? We leave this question for the reader.

Choosing the eyelets  $A_0, A_1, \dots, A_n$  and  $B_0, B_1, \dots, B_n$  as nodes of a complete bipartite graph where the edge costs correspond to the Euclidean distances between these points except for the edge  $(A_0, B_0)$  the cost of which is  $-M$  where  $M$  is a very large number, the shoelace problem can be considered as a TSP restricted to a complete bipartite graph which in turn can be viewed as traveling salesman problem on a complete graph by adding edges of large cost.

Although we have a closed form solution for this special case, TSP in general is NP-hard. (For a summary of computational complexity results on TSP, we refer to Appendix B.) This inherent difficulty of the problem and its various practical applications inspired researchers and practitioners to investigate various aspects of the problem. In particular, significant developments have taken place in the recent past in the study of structural properties of the problem, development of efficient exact and approximation algorithms, and identification of efficiently solvable special cases. In the following chapters we study each of these aspects of TSP and its variations. In the rest of this chapter, we will concentrate on applications, variations and mathematical formulations of the problem.

## 2. Simple Variations of the TSP

Several variations of the TSP that are studied in the literature have been originated from various real life or potential applications. Let us first consider some of these variations that can be reformulated as a TSP using relatively simple transformations.

**The MAX TSP:** Unlike the TSP, the objective in this case is to find a tour in  $G$  where the total cost of edges of the tour is maximum. MAX TSP can be solved as a TSP by replacing each edge cost by its additive inverse. If we require that the edge costs are to be non-negative, a large constant could be added to each of the edge-costs without changing the optimal solutions of the problem. For details on MAX TSP, we refer to Chapter 12.

**The bottleneck TSP:** In the case of a bottleneck TSP, the objective is to find a tour in  $G$  such that the largest cost of edges in the tour is as small as possible. A bottleneck TSP can be formulated as a TSP with exponentially large edge costs. This problem is studied in detail in Chapter 15.

**TSP with multiple visits(TSPM):** Here we want to find a routing of a traveling salesman, who starts at a given node of  $G$ , visits each node *at least* once and comes back to the starting node in such a way that the total distance traveled is minimized. This problem can be transformed into a TSP by replacing the edge costs with the shortest path distances in  $G$ . In the absence of negative cycles, shortest path distances between all pairs of nodes of a graph can be computed using efficient algorithms [7]. If  $G$  contains a negative cycle, then TSPM is unbounded. Several shortest path algorithms are capable of detecting negative cycles in a graph [7].

**Messenger problem:** This problem[593] is also known as the wandering salesman problem [474]. Given two specified nodes  $u$  and  $v$  in  $G$ , we want to find a least cost Hamiltonian path in  $G$  from  $u$  to  $v$ . This can be solved as a TSP by choosing a cost of  $-M$  for the edge  $(v, u)$  where  $M$  is a large number. If no nodes are specified, and one wishes to find a least cost Hamiltonian path in  $G$ , it can also be achieved by solving a TSP. Modify the graph  $G$  by creating a new node and connecting it with all the nodes of  $G$  by edges (for directed graphs, introduce two arcs, one in forward and other in backward directions) of cost  $-M$ . From an optimal solution to the TSP on this modified graph, an optimal solution

to the original problem can be recovered.

**Clustered TSP:** In this case, the node set of  $G$  is partitioned into clusters  $V_1, V_2, \dots, V_k$ . Then the clustered TSP [426, 469] is to find a least cost tour in  $G$  subject to the constraint that cities within the same cluster must be visited consecutively. This problem can be reduced to a TSP by adding a large cost  $M$  to the cost of each inter-cluster edge.

**Generalized TSP(GTSP):** As in the previous case, let  $V_1, V_2, \dots, V_k$  be a partition of the node set of  $G$ . In a GTSP, we want to find a shortest cycle in  $G$  which passes through exactly one node from each cluster  $V_i$ ,  $1 \leq i \leq k$ . If  $|V_i| = 1$  for all  $i$ , GTSP is the same as TSP. Using the reduction described in [629] we now show that GTSP can be reduced to a TSP for arbitrary  $|V_i|$ 's. Without loss of generality, assume that  $G$  is a digraph and the partitions are numbered in such a way that  $|V_i| \geq 2$  for  $1 \leq i \leq r$  and  $|V_i| = 1$  for  $r+1 \leq i \leq k$ . For any  $i \leq r$  let  $V_i = \{i_1, i_2, \dots, i_{n_i}\}$ . For each arc  $e \in E$ , consider a new cost  $d_e$  defined as follows:

$$d_{i_j i_{j+1}} = -M, \quad j = 1, \dots, n_i \text{ with } n_i + 1 \equiv 1, i = 1, \dots, r$$

This ensures that if an optimal TSP tour in  $G$  enters a cluster  $V_i$  through node  $i_j$ , it visits vertices of  $V_i$  in the order  $i_j, i_{j+1}, \dots, i_{n_i}, i_1, \dots, i_{j-1}$  and leaves the cluster  $V_i$  from the node  $i_{j-1}$ . We want to translate this outcome equivalent to a GTSP tour entering and leaving the cluster  $V_i$  by visiting just node  $i_j$ . To replicate this fact, we make the new cost of arcs going out of  $i_{j-1}$  corresponds to the original cost of arcs leaving  $i_j$ . More precisely.

$$d_{i_{j-1}p} = c_{i_j p}, \quad p \notin V_i, j = 1, 2, \dots, n_i \text{ with index } 0 \equiv n_i, i = 1, \dots, r$$

and  $d_{uv} = c_{uv}$  for all other edges. From an optimal solution to the TSP on  $G$  with the modified costs  $d_e$  for  $e \in E$ , an optimal solution to GTSP can be recovered. GTSP is discussed in detail in Chapter 13.

**The  $m$ -salesmen TSP:** Assume that  $m$  salesmen are located at node 1 of  $G$ . Starting from node 1, each salesman visits a subset  $X_i$  of nodes of  $G$  exactly once and returns to node 1. We are interested in finding a partition  $X_1, X_2, \dots, X_m$  of  $V - \{1\}$  and a routing of each of the  $m$  salesmen such that (i)  $|X_i| \geq 1$  for each  $i$ , (ii)  $\cup_{i=1}^m X_i = V - \{1\}$ , (iii)  $X_i \cap X_j = \emptyset$  for  $i \neq j$ , and (iv) total distance travelled by all the salesmen is minimized [474]. If  $G$  is undirected, the  $m$ -salesmen TSP can be formulated as a TSP on a graph  $G' = (V', E')$  with  $m-1$  additional nodes, where  $V' = V \cup \{n+2, n+3, \dots, n+m\}$ ,  $E' = E \cup \{(i, n+k) : 2 \leq$

$k \leq m, 2 \leq i \leq n\}$ . Let  $c'_{ij}$  be the cost of edges in  $G'$ , where  $c'_{ij} = c_{ij}$  if  $(i, j) \in E$ , and  $c'_{i, n+k} = c_{i1}$ , for  $2 \leq k \leq m, 2 \leq i \leq n$ . From an optimal TSP solution on  $G'$  an optimal solution to the  $m$ -salesmen TSP can be recovered. The case when  $G$  is a directed graph can be handled similarly with simple modifications in the above transformation. For variations of the  $m$ -salesmen TSP that can be formulated as a TSP we refer to [96, 695].

### 3. Applications of TSP

Applications of the TSP and its variations go way beyond the route planning problem of a traveling salesman and spans over several areas of knowledge including mathematics, computer science, operations research, genetics, engineering, and electronics. In this section, we discuss some selected applications of TSP.

#### 3.1. Machine Scheduling Problems

Perhaps the most well studied application area of the TSP is machine sequencing and scheduling. A simple scheduling application can be described as follows. There are  $n$  jobs  $\{1, 2, \dots, n\}$  to be processed sequentially on a machine. Let  $c_{ij}$  be the set up cost required for processing job  $j$  immediately after job  $i$ . When all the jobs are processed, the machine is reset to its initial state at a cost of  $c_{j1}$ , where  $j$  is the last job processed. The sequencing problem is to find an order in which the jobs are to be processed such that the total setup cost is minimized. Clearly, finding a permutation  $\pi$  of  $\{1, 2, \dots, n\}$  that minimizes  $c_{\pi(n)\pi(1)} + \sum_{i=1}^{n-1} c_{\pi(i)\pi(i+1)}$  solves the problem.

In many practical applications, the jobs are not arbitrary but often can be clustered together so that the setup time, if any, between jobs within a cluster is relatively small compared to setup time between jobs in two different clusters. This is a typical scenario in assembly line. Assembling similar products need minimal setup time in between; but if a different product needs to be assembled, the setup time may be larger as one may require new parts, tools etc. The sequencing problem with such a specialized cost matrix reduces to the clustered TSP. This cluster property can be effectively exploited in developing efficient algorithms. Ozgur and Brown [639] developed a two stage heuristic for such a problem originated from the mechanical push-pull cable control systems.

Let us now consider another scheduling problem - a no wait flow shop problem. There are  $n$  jobs each requiring processing on  $m$  machines in the order  $1, 2, 3, \dots, m$ . No job is allowed to have a waiting time between

processing on two consecutive machines. The objective is to find an optimum sequencing of jobs to be processed so that the makespan (total completion time) is minimum. Applications of this type of sequencing problems arise in a variety of situations [272]. The no-wait flow shop problem is strongly NP-hard, but solvable in polynomial time when  $m = 2$  in which case it reduces to the well known Gilmore-Gomory problem. (See Chapter 11 for more details on this.)

From an instance of the no-wait flow shop problem, we construct an equivalent TSP instance using the reduction proposed in [272]. Create a complete directed graph  $G$  on  $n + 1$  nodes, where node  $j$  corresponds to job  $j$ ,  $1 \leq j \leq n$  and node  $n + 1$  represents both the start and end of processing. The cost  $c_{ij}$  of arc  $(i, j)$  in  $G$  represents the additional schedule length if job  $j$  is the immediate successor of job  $i$  in the schedule. Thus a minimum cost tour in  $G$  corresponds to a schedule with minimum makespan. To complete the reduction, we must identify the values of  $c_{ij}$ . Let  $p_{jk}$  be the processing time of job  $j$  on processor  $k$ ,  $1 \leq j \leq n$ ,  $1 \leq k \leq m$ . Then  $c_{ij}$  can be obtained [272] using the equations,

$$\begin{aligned} c_{n+1,i} &= \sum_{r=1}^m p_{ir}, \quad i = 1, 2, \dots, n \\ c_{ij} &= \max_{1 \leq k \leq m} \left\{ \sum_{r=1}^k p_{ir} + \sum_{r=k}^m p_{jr} \right\} - \sum_{r=1}^m p_{ir}, \quad 1 \leq i, j \leq n, i \neq j \\ c_{i,n+1} &= c_{ii} = 0, \quad i = 1, 2, \dots, n. \end{aligned}$$

The applications we considered so far reduce a given problem to the TSP. Let us now discuss another application of TSP where it is used as a subroutine to develop efficient heuristics for a more complex problem involved in the control of a pick and placement machine. This application is taken from Ahmadi and Mamer [4].

A pick-and-place machine includes a work platform, a placement arm, pick and place heads, component delivery carrier (feeder magazines), and tool magazines. Generally these types of machines are used in printed circuit board assembly [4, 83] and available in different architectures such as the placement arm is moving while work platform remains stationary or the placement arm remains stationary while the work platform is moving or a combination of the two. In our model the placement arm moves between two fixed locations - the pick up position and the delivery position. The work platform moves in the  $x$  and  $y$  directions and feeder-tool carrier moves in the  $x$  direction only. The head attached to the placement arm needs to be changed if a different component type is to be placed on the board. Normally tool change operations are more time

consuming in comparison to pick and placement operations. The cycle time - i.e. the time required to move the placement arm between the pickup and delivery positions, perform a pick up operation and perform a delivery operation- is assumed to be constant. Let  $p_i$  be the position of the  $i^{th}$  placement site,  $c_{ij}$  be the time required to move the placement board from site  $p_i$  to site  $p_j$ . When the placement head moves to the pickup position, simultaneously the work platform moves to adjust the next placement site. Let  $t_{ij}$  denote the time lost in switching between part  $i$  and part  $j$  and  $N_k$  denote the index set corresponding to placement sites requiring part type  $k$ ,  $k = 1, 2, \dots, m$ . It is assumed that only one component is placed at a given placement site and the  $p_i$ 's are numbered consecutively with the part type. Thus  $N_1 = \{p_1, p_2, \dots, p_{k_1}\}$ ,  $N_2 = \{p_{k_1+1}, p_{k_1+2}, \dots, p_{k_1+k_2}\}$ , .... Define  $n = \sum_{i=1}^m k_i$ . The calculation of  $c_{ij}$  and  $t_{ij}$  depends on the architecture of the machine. The pickup and placement problem is to find a tour through  $p_1, p_2, \dots, p_n$  which minimizes the total time to complete all placements, including the time required to start and return the machine to the resting position.

Although this problem is not exactly a TSP, its close relationship with TSP allows us to develop efficient heuristics by using TSP as a subroutine or by appropriate modifications of the TSP heuristics itself. Ahmadi and Mamer [4] developed a two stage heuristic for this problem using a TSP solver (exact or heuristic) as a subroutine. For details of other research works on printed circuit board assembly that are relevant to the TSP, we refer to [83].

### 3.2. Cellular Manufacturing

In a cellular manufacturing system families of parts (products) that require similar processing are grouped and processed together in a specialized cell to achieve efficiency and cost reductions. In such systems robots are often used in handling material to improve the efficacy of the cell. Aneja and Kamoun [23] considered the problem of sequencing the robot activities in a two machine robotic cell and showed that the problem can be formulated as a TSP. For some fundamental results on this problem, we refer to [758].

Let  $p_1, p_2, \dots, p_n$  be  $n$  part-types to be produced. Each part type  $p_i$  needs to be processed on two machines M1 and M2 with M1 followed by M2. Let  $a_i$  be the processing time of  $p_i$  on M1 and  $b_i$  be the processing time of  $p_i$  on M2. A robot picks up a part from an input storage buffer, loads it on to machine M1, collects from M1 and then loads to M2, collects from M2 and places the product in the output storage buffer. The storage buffers and machines are arranged on a straight line so that

the robot travels only along a straight line. The time taken by the robot for movement between input storage buffer and each machine, between each machine and output storage buffer, and between the machines is given by  $t$ . Let  $l$  be the time taken by the robot in loading or unloading.

Two robotic movement cycles  $C1$  and  $C2$  with a point of intersection are considered [758]. Let  $\pi$  be a permutation of  $\{1, 2, \dots, n\}$  representing the sequence in which the parts are selected for processing. In cycle  $C1$ , starting from an initial state, the robot loads a part, say  $p_{\pi(i)}$  on  $M2$ , waits for it to complete, picks up the processed part from  $M2$  and places it in the output storage buffer, goes back to the input storage buffer, picks up part  $p_{\pi(i+1)}$  and moves to  $M2$ . The cycle time in this case is given by

$$T_{\pi(i)\pi(i+1)}^1 = 6(l + t) + b_{\pi(i)} + a_{\pi(i+1)}.$$

For cycle  $C2$ , starting from an initial state, the robot loads  $p_{\pi(i)}$  on  $M2$ , moves to the input storage buffer, picks up part  $p_{\pi(i+1)}$  and loads on to  $M1$ , goes to  $M2$  and waits (if necessary) to complete the processing of  $p_{\pi(i)}$  on  $M2$ , picks up part  $p_{\pi(i)}$  from  $M2$  and moves to the output storage buffer to unload the part, moves back to  $M1$  and wait (if necessary) to complete the processing of  $p_{\pi(i+1)}$  on  $M1$ , picks up  $p_{\pi(i+1)}$  and moves to  $M2$ . The cycle time in this case can be calculated as

$$T_{\pi(i)\pi(i+1)}^2 = 8t + 6l + w_1(i + 1) + w_2(i)$$

where  $w_1(i+1)$ , the waiting time at  $M1$ , is given by  $\max\{0, a_{\pi(i+1)} - (4t + 2l + w_2(i))\}$  and  $w_2(i)$ , the waiting time at  $M2$ , is given by  $\max\{0, b_{\pi(i)} - (4t + 2l)\}$ . Let  $X_i = b_i + 2(t + l)$  and  $Y_i = a_i + 2(t + l)$ . Then  $T_{ij}^1$  and  $T_{ij}^2$  can be written as

$$T_{ij}^1 = 2(t + l) + X_i + Y_j$$

and

$$T_{ij}^2 = 2(t + l) + \max\{X_i + Y_i, 6t + 2l\}.$$

Note that each cycle  $C1$  or  $C2$  involves two parts  $p_{\pi(k)}$  and  $p_{\pi(k+1)}$  for some  $k$ , under the schedule  $\pi$ . For the cycles  $C1$  and  $C2$ , the state in which the robot completed loading a part on  $M2$  is common. Thus at such a state the robot needs to make a decision which cycle to be used. Clearly, if  $T_{\pi(i)\pi(i+1)}^1 < T_{\pi(i)\pi(i+1)}^2$ , cycle  $C1$  is selected, otherwise  $C2$  is selected. Now consider the cost matrix  $C = (c_{ij})_{n \times n}$  where  $c_{ij} = \min\{T_{ij}^1, T_{ij}^2\}$ . The optimal TSP tour using cost matrix  $C$  provides an optimal part sequencing. This TSP can be solved in  $O(n \log n)$  time exploiting the special structure the matrix  $C$  (see [23] and Chapter 11).

Instead of the straight line topology for robotic movements, other structures could be considered. For example, M1, M2, the input storage buffer, and the output storage buffer are placed at 4 corners of a rectangle and the robot moves along the perimeter of this rectangle choosing shortest path distances, the resulting part sequencing problem can also be formulated as a TSP.

### 3.3. Arc Routing Problems

A general arc routing problem, known as mixed windy rural postman problem (MWRPP), can be stated as follows. Let  $G = (V, A \cup E)$  be a mixed graph where elements of  $A$  are arcs (directed) and elements of  $E$  are edges (undirected). Let  $A' \subset A$  and  $E' \subset E$ . The arc and edge costs are assumed to be non-negative. The MWRPP is to find a minimum cost closed walk on  $G$  containing all arcs in  $A'$  and all edges in  $E'$ . Several problems such as mixed Chinese postman problem, windy Chinese postman problem, windy rural postman problem, stacker crane problem, etc. are special cases of MWRPP. Applications of MWRPP also include street sweeping, snow plowing, etc. and the problem is known to be NP-hard. We observe that MWRPP can be solved as an asymmetric TSP. This transformation is taken from Laporte [534].

In  $G$ , replace each undirected edge  $(i, j)$  by two arcs  $(i, j)$  and  $(j, i)$  with cost equal to that of  $(i, j)$ . Let  $\bar{G} = (V, \bar{A})$  be the resulting digraph. Define  $A' = A' \cup \{(i, j), (j, i) \in \bar{A} : (i, j) \in E'\}$ . We now formulate a generalized TSP on the digraph  $D = (U, Z)$ . For each arc  $(i, j) \in A'$  there is a node  $u_{ij}$  in  $U$ . The cost of arc  $(u_{ij}, u_{pq}) \in Z$  is defined as the length of the shortest path from node  $i$  to node  $q$  in  $\bar{G}$ . Now the MWRPP is equivalent to a GTSP on  $D$  where the node set partition  $\{U_1, U_2, \dots, U_m\}$  is given by  $U_l = \{u_{ij}\}$  if  $(i, j) \in A'$ ,  $l = 1, 2, \dots, |A'|$  and  $U_l = \{u_{ij}, u_{ji}\}$  if  $(i, j) \in E'$ ,  $l = |A'| + 1, \dots, |A'| + |E'|$ . This GTSP can be reduced to a TSP as discussed earlier. Laporte [534] reports experimental results on several arc routing problems using such transformations.

### 3.4. Frequency Assignment Problem

In a communication network with a collection of transmitters, the frequency assignment problem is to assign a frequency to each transmitter from a given set of available frequencies satisfying some interference constraints. These constraints can be represented by a graph  $G = (V, E)$  where each node  $i$  represents a transmitter. A non-negative integer weight  $c_{ij}$  is prescribed for each arc  $(i, j)$  representing the tolerance. Let  $F = \{0, 1, 2, \dots, R\}$  be a collection of allowable frequencies.



A frequency assignment is to assign number  $f(i) \in F$  to node  $i \in V$  such that  $|f(i) - f(j)| > c_{ij}$  for all  $(i, j) \in E$ . If such an assignment exists, it is called a feasible assignment. If  $R$  is sufficiently large a feasible assignment is always possible. The minimum value of  $R$  for which a feasible assignment for  $G$  exists is called the span of  $G$  and is denoted by  $\text{Span}(G)$ .

Let  $G^*$  be the complete graph obtained from  $G$  by adding (if necessary) edges of zero weight. Let  $c'_{ij}$  be the weight of edge  $(i, j)$  in  $G^*$  such that  $c'_{ij} = c_{ij} + 1$ . Let  $C'(H^*)$  be the sum of the weights of edges in a minimum cost Hamiltonian path in  $G^*$ . Smith and Hurly [765] showed that  $\text{Span}(G) \geq C'(H^*)$ . Thus TSP can be used to compute a lower bound for the frequency assignment problem. Successful applications of the TSP based bounds and its variations in solving frequency assignment problem are discussed in [17].

### 3.5. Structuring of Matrices

Let  $A$  be a finite set and  $f : A \times A \rightarrow \mathbb{R}$ . Consider the matrix  $X = (x_{ij})_{m \times n}$  where  $x_{ij} \in A$ . For each row  $i$  of  $X$ , let  $L_i(X) = \sum_{j=1}^n f(x_{ij}, x_{i(j+1)})$  where  $x_{i(n+1)} = x_{i1}$ . Similarly for each column  $j$ , let  $C_j(X) = \sum_{i=1}^m f(x_{ij}, x_{(i+1)j})$  where  $x_{(m+1)j} = x_{1j}$ . Define  $L(X) = \sum_{i=1}^m L_i(X)$  and  $C(X) = \sum_{j=1}^n C_j(X)$ . Let  $S(X)$  be the family of matrices obtained by permuting rows of  $X$ . For  $Y, Z \in S(X)$ ,  $Y$  is preferable to  $X$  if  $C(Y) < C(Z)$ . The set  $B = \{Y \in S(X) : C(Y) = \min_{Z \in S(X)} C(Z)\}$  is called matrices with best row structure with respect to  $X$ . Structuring of rows of a matrix  $X$  corresponds to identifying an element of  $B$ . This problem can be solved by solving a TSP on the complete graph  $K_m = (V, E)$  where  $V = \{1, 2, \dots, m\}$  with the weight of edge  $(i, j)$  is given by  $c_{ik} = \sum_{j=1}^n f(x_{ij}, x_{kj})$  [516, 555]. In a similar way, structuring of columns of a matrix can be solved as a TSP. Applications structuring of rows/columns include clustering of data [555], phsyotgenology, and theory of group decisions [723].

### 3.6. Additional Applications

We now give some additional references to more applications of TSP. Korostensky et al [513] used the traveling salesman problem to compute a near optimal *multiple sequence alignment* with guaranteed performance bound. In a related work Korostensky et al [514] used TSP based methods in constructing an *evolutionary tree* of optimal 'score'. The traveling salesman model is applicable in a variety of other situations including data analysis in psychology [454], X-Ray crystallography [111], overhauling gas turbine engines [670], warehouse order-picking problems [699],

and wall paper cutting [348], to cite a few. Marchand et al [580] formulated an optimal control problem as a TSP. Potential application areas of their model include problems that arise in routing in telecommunication networks, test objective generation, and artificial intelligence [580].

#### 4. Alternative representations of the TSP

We have introduced TSP as the problem of finding a least cost Hamiltonian cycle in a graph (graphical definition). We also discussed quadratic and linear permutation representations of TSP. In this section we study representations of the TSP as a mathematical programming problem.

##### 4.1. Linear programming representation

This is perhaps the most well studied representation of the TSP. Let  $\mathbb{F}$  be the family of all tours in  $G$  represented as a collection of incidence vectors in  $\mathbb{R}^{|E|}$  and  $P(\mathbb{F})$  be the convex hull of  $\mathbb{F}$ . Then the traveling salesman problem can be stated as the following mathematical programming problem:

$$\begin{array}{ll} \text{Minimize} & \sum_{j=1}^m c_j x_j \\ \text{Subject to} & X \in P(\mathbb{F}), \end{array}$$

where  $X = (x_1, x_2, \dots, x_m)$ ,  $m$  being the number of edges in  $G$ .

A complete linear inequality description of  $P(\mathbb{F})$  is not known which somewhat limits application of linear programming techniques to solve TSP. However, it may be noted that linear programming based methods are among the best known approaches for solving TSP which exploits partial linear inequality description of  $P(\mathbb{F})$ . For the state of the art research in this area we refer to chapters 2 and 3.

##### 4.2. Integer programming formulations

Let us first consider the case of the ATSP. It may be noted that the linear programming relaxation of the integer programming formulations discussed below could be strengthened by adding ‘valid’ inequalities corresponding to  $P(\mathbb{F})$ , especially those defining facets or higher dimensional faces of  $P(\mathbb{F})$ . For a discussion of such inequalities we refer to chapters 2 and 3. Most of the integer programming formulations of the ATSP are based on the assignment problem. A 0-1 programming

formulation of the assignment problem can be described as follows:

$$\begin{aligned}
 & \text{Minimize } \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\
 & \text{Subject to} \\
 & \quad \sum_{i=1}^n x_{ij} = 1, \quad j = 1, \dots, n, \\
 & \quad \sum_{j=1}^n x_{ij} = 1, \quad i = 1, \dots, n, \\
 & \quad x_{ij} = 0 \text{ or } 1
 \end{aligned}$$

For any solution  $X = (x_{ij})_{n \times n}$  of the above assignment problem, consider the set  $S = \{(ij) : x_{ij} = 1\}$ . Clearly,  $S$  represents a tour or a collection of subtours in  $G$ . In order to model the ATSP as an integer program, we must include additional restrictions to make sure that  $S$  does not contain any subtours. These restrictions are called *subtour elimination constraints*.

**4.2.1 Clique Packing Constraints.** Introduced by Dantzig, Fulkerson, and Johnson [239], these are the most popular form of subtour elimination constraints. It states that from any clique  $Q$  of the subgraph  $G^*$  of  $G$  induced by the node set  $V^* = \{2, 3, \dots, n\}$ , select at most  $|Q| - 1$  arcs. There are an exponential number of such constraints which are given by

$$\sum_{i \in Q} \sum_{j \in Q} x_{ij} \leq |Q| - 1 \text{ for all } \emptyset \neq Q \subset \{2, 3, \dots, n\}$$

The clique packing constraints can be written in an equivalent form as

$$\sum_{i \in Q} \sum_{j \in \bar{Q}} x_{ij} \geq 1 \text{ for all } \emptyset \neq Q \subset \{2, 3, \dots, n\}$$

where  $\bar{Q} = V - Q$ .

**4.2.2 Circuit Packing Constraints.** This class of subtour elimination constraints were introduced by Grotschel and Padberg [401]. (See also [392].) These constraints can be described by replacing cliques  $Q$  in the clique packing constraints by circuits in  $G^*$ . Again there are an exponential number of circuit packing constraints.

The assignment problem together with clique packing or circuit packing constraints leads to a valid 0-1 programming formulation of the

ATSP. These formulations use only variables that correspond to the arcs and self-loops of the complete graph  $G$  and arc called *natural formulations* [676]. It may be noted that the self-loop variables  $x_{ii}$  could be discarded from the formulation to reduce the problem size. In this case one needs to consider only cliques (circuits) of size two or more in the clique (circuit) packing constraints, further reducing the problem size. We have used the variables  $x_{ii}$  in our formulation in order to be consistent with the definition of the classical assignment problem. One could also choose  $c_{ii} = M$ , where  $M$  is a large positive number, to insure they will not appear any optimal solution to the assignment problem. In this case also it is sufficient to consider clique (circuits) of size two or more within the clique (circuit) packing constraints.

By using polynomial (in  $n$ ) number of additional constraints and variables it is possible to ensure that  $S$  is cycle free. These are called *compact representations* of the subtour elimination constraints.

**4.2.3 MTZ constraints.** Miller, Tucker and Zemlin [595] showed that by using  $(n - 1)^2$  additional constraints and  $n - 1$  additional variables, it is possible to ensure that  $S$  is free of subtours. These constraints are given by

$$(n - 1)x_{ij} + u_i - u_j \leq (n - 2), \quad i, j = 2, 3, \dots, n \quad (1)$$

where  $u_i$ ,  $i = 2, 3, \dots, n$  are unrestricted real variables. If  $S$  contains any subtour (a cycle with less than  $n$  arcs) then  $S$  will contain a subtour  $\tilde{T}$  that does not contain node 1. If  $\tilde{T}$  is a single node, say  $\{k\}$ , then inequality (1) is violated for  $i = j = k$ . If  $\tilde{T}$  contains more than one node, then adding the above inequalities for arcs in  $\tilde{T}$  leads to a contradiction. This establishes that the MTZ constraints force  $S$  to be free of subtours.

**4.2.4 Network flow constraints.** Network flow based formulations can also be used to obtain compact representations of subtour elimination constraints. We first consider a general form of subtour elimination constraints and observe that most of the network flow based subtour elimination constraints studied in literature fall within this general framework. Consider the inequality system:

$$AY \leq b \quad (2)$$

$$Y \geq 0 \quad (3)$$

$$BX + DY \leq g, \quad (4)$$

where  $X = (x_{11}, x_{12}, \dots, x_{1n}, x_{21}, \dots, x_{nn})$  is a solution to the assignment problem,  $Y = (y_1, y_2, \dots, y_p) \in \mathbb{R}^p$ ,  $A, B, D$  are matrices, and  $b$

and  $g$  are column vectors. Each  $y_k$  is associated with an arc  $(i, j)$  of  $G$ . It is possible that  $y_k$  and  $y_r$  are associated with the same arc  $(i, j)$  for  $k \neq r$ , but no  $y_k$  is associated with two arcs. The inequalities within the system (2) and (3) are called *flow constraints* and the inequalities within the system (4) are called *coupling constraints*. For any feasible solution  $Y^*$  of the flow constraints, consider the subgraph  $G_{Y^*}$  induced by the arcs of  $G$  associated with positive components of  $Y^*$ . Then  $G_{Y^*}$  is called the *flow graph* of  $Y^*$ .

**Theorem 2** *The assignment problem together with the flow and coupling constraints gives a valid mixed 0-1 programming formulation of the TSP if the following conditions are satisfied.*

- (i) *If the assignment solution  $X^0 = (x_{ij}^0)$  represents a Hamiltonian cycle, then there exists  $Y^0 \geq 0$  such that  $(X^0, Y^0)$  satisfies (2) and (4).*
- (ii) *For any feasible solution  $(X^*, Y^*)$ , the flow graph  $G_{Y^*}$  is connected.*
- (iii) *For any feasible solution  $(X^*, Y^*)$ , if  $x_{ij}^* = 0$  then all  $y_k^*$ 's associated with arc  $(i, j)$  are zeros.*

**Proof.** The assignment constraints insure that  $S = \{(i, j) : x_{ij} = 1\}$  defines either a tour or a collection of subtours. In view of (iii), if (ii) is satisfied, then  $S$  is free of subtours. The result follows from (i). ■

Let us now consider a single commodity flow formulation suggested by Gavish and Graves [350] which satisfies the conditions of Theorem 2. Let  $y_{ij}$  be the 'flow' of commodity  $y$  along arc  $(i, j)$  in  $G$ . Consider the following constraints:

Flow constraints:

$$\sum_{j=1}^n y_{ij} - \sum_{j=1}^n y_{ji} = 1, \quad i = 2, 3, \dots, n \quad (5)$$

$$\sum_{j=1}^n y_{1j} = n - 1 \quad (6)$$

$$y_{ij} \geq 0 \quad (7)$$

Coupling constraints:

$$(n - 1)x_{ij} - y_{ij} \geq 0 \text{ for } i, j = 1, 2, \dots, n \quad (8)$$

It is easy to verify that these flow constraints and coupling constraints satisfy the conditions of Theorem 2 and hence are valid subtour elimination constraints. The coupling constraints (8) could be strengthened

as

$$(n-1)x_{1j} - y_{1j} \geq 0 \text{ for } j = 2, \dots, n \quad (9)$$

$$(n-2)x_{ij} - y_{ij} \geq 0 \text{ for } i, j = 2, 3, \dots, n \quad (10)$$

Another slightly strengthened variation of the coupling constraints can be described as follows:

$$y_{j1} = 0, \text{ for } j = 2, 3, \dots, n \quad (11)$$

$$y_{1j} = (n-1)x_{1j} \text{ for } j = 2, 3, \dots, n \quad (12)$$

$$x_{ij} \leq y_{ij} \leq (n-2)x_{ij} \text{ for } i, j = 2, 3, \dots, n \quad (13)$$

This variation is equivalent to the single commodity flow formulation proposed by Gouveia and Voß [392].

A two-commodity flow formulation of the subtour elimination constraints was proposed by Finke, Claus, and Gunn [291]. Let  $y$  and  $z$  be two commodities and  $y_{ij}$  be the flow of commodity  $y$  along arc  $(i, j)$  and  $z_{ij}$  be the flow of commodity  $z$  along arc  $(i, j)$ . The resulting flow constraints are given by:

$$\sum_{j=1}^n y_{1j} - \sum_{j=1}^n y_{j1} = n-1 \quad (14)$$

$$\sum_{j=1}^n y_{ij} - \sum_{j=1}^n y_{ji} = -1, \quad i = 2, 3, \dots, n \quad (15)$$

$$\sum_{j=1}^n z_{1j} - \sum_{j=1}^n z_{j1} = -(n-1) \quad (16)$$

$$\sum_{j=1}^n z_{ij} - \sum_{j=1}^n z_{ji} = 1, \quad i = 2, 3, \dots, n \quad (17)$$

$$\sum_{j=1}^n y_{ij} + \sum_{j=1}^n z_{ij} = n-1, \quad i = 1, 2, \dots, n \quad (18)$$

$$y_{ij} \geq 0 \text{ for } i, j = 1, 2, \dots, n \quad (19)$$

$$z_{ij} \geq 0 \text{ for } i, j = 1, 2, \dots, n \quad (20)$$

with coupling constraints:

$$y_{ij} + z_{ij} = (n-1)x_{ij} \text{ for all } (ij). \quad (21)$$

It is not difficult to verify that these constraints satisfy the conditions of Theorem 2 and hence are valid subtour elimination constraints.

Using multicommodity flows, Wong [827] presented another compact representation of the subtour elimination constraints. He used  $2(n-1)$  commodities  $y^k$  and  $z^k$ ,  $k = 2, \dots, n$ . Let  $y_{ij}^k$  be the flow of commodity  $y^k$  along arc  $(i, j)$  and  $z_{ij}^k$  be the flow of commodity  $z^k$  along arc  $(i, j)$ ,  $i, j = 1, 2, \dots, n$  and  $k = 2, 3, \dots, n$ . The flow constraints of Wong can be stated as follows.

$$\sum_{j=1}^n (y_{ij}^k - y_{ji}^k) = \begin{cases} 1 & \text{if } i = 1, k = 2, 3, \dots, n, \\ -1 & \text{if } i = k, k = 2, 3, \dots, n, \\ 0 & \text{if } i \neq 1 \text{ and } k, k = 2, 3, \dots, n \end{cases} \quad (22)$$

$$\sum_{j=1}^n (z_{ij}^k - z_{ji}^k) = \begin{cases} 1 & \text{if } i = 1, k = 2, 3, \dots, n, \\ -1 & \text{if } i = k, k = 2, 3, \dots, n, \\ 0 & \text{if } i \neq 1 \text{ and } k, k = 2, 3, \dots, n \end{cases} \quad (23)$$

with coupling constraints:

$$0 \leq y_{ij}^k \leq x_{ij} \quad \text{for all } i, j, k \quad (24)$$

$$0 \leq z_{ij}^k \leq x_{ij} \quad \text{for all } i, j, k. \quad (25)$$

Note that the source node for commodity  $y^k$  is 1 for all  $k$  and for commodity  $z^k$ , it is node  $k$ . Similarly, the sink node of commodity  $y^k$  is node  $k$  and for commodity  $z^k$  it is node 1 for all  $k$ . Constraints (22) and (23) insures that one unit of commodity  $y^k$  travels from node 1 to node  $k$  while one unit of commodity  $z^k$  travels from node  $k$  to node 1. Thus the corresponding flow graph is strongly connected, establishing condition (i) of Theorem 2. Conditions (ii) and (iii) of Theorem 2 are easy to verify. Thus constraints (22) to (25) represent valid subtour elimination constraints.

Eliminating the  $z_{ij}$  variables from the above formulation, Claus [202] presented a multicommodity flow formulation of the subtour elimination constraints as described below:

Flow constraints:

$$\sum_{i=1}^n y_{1i}^k = 1 \text{ for } k = 2, 3, \dots, n \quad (26)$$

$$\sum_{i=1}^n y_{i1}^k = 0 \text{ for } k = 2, 3, \dots, n \quad (27)$$

$$\sum_{i=1}^n y_{ik}^k = 1 \text{ for } k = 2, 3, \dots, n \quad (28)$$

$$\sum_{i=1}^n y_{ki}^k = 0 \text{ for } k = 2, 3, \dots, n \quad (29)$$

$$\sum_{i=1}^n y_{ij}^k - \sum_{i=1}^n y_{ji}^k = 0 \text{ for } j, k = 2, 3, \dots, n; j \neq k \quad (30)$$

$$y_{ij}^k \geq 0 \quad (31)$$

Coupling constraints:

$$y_{ij}^k \leq x_{ij} \text{ for } i, j = 1, 2, \dots, n; k = 2, 3, \dots, n \quad (32)$$

Other related multicommodity flow formulations of the TSP which are variations of Wong [827] are proposed by Langevin [531] and Loulou [567]. The linear programming (LP) relaxations of all these multicommodity formulations yield the same objective function value as that of the LP relaxation of Dantzig et al [239]. However, LP relaxations of the single and two commodity formulations yield weaker bounds. Other well studied formulations of TSP include time-stage dependent formulations [637]. For a comparative study of LP relaxations of various (mixed) integer programming formulations of ATSP we refer to [392, 532, 637, 643].

### 4.3. Symmetric TSP

Since the symmetric traveling salesman problem is a special case of ATSP (as discussed in Section 1), the above formulations are valid for STSP also. However, exploiting symmetry, we could replace the assignment constraints by 2-matching constraints. Let  $\delta(v)$  be the set of edges incident on node  $v$  of  $G$ . Then a generic integer programming formulation of STSP is given by



$$\begin{aligned}
& \text{Minimize } \sum_{e \in E} c_e x_e \\
& \text{Subject to} \\
& \quad \sum_{e \in \delta(v)} x_e = 2, \quad v \in V \\
& \quad x_e = 0 \text{ or } 1, \quad e \in E \\
& \quad \{e \in E : x_e = 1\} \text{ contains no subtours.}
\end{aligned}$$

As discussed earlier, additional constraints can be used to make sure that  $\{e \in E : x_e = 1\}$  is free of subtours. For other integer programming formulations of the STSP we refer to [39, 41, 169, 42, 40]

#### 4.4. Binary quadratic programming formulation

We have seen that by adding new constraints to the assignment problem (2-matching problem), various mixed integer linear programming formulation for the ATSP (STSP) can be obtained. While the assignment constraints represent all permutations of  $\{1, 2, \dots, n\}$ , the additional constraints (subtour elimination constraints) make sure that only cyclic permutations are selected. We have seen in Section 1 that under the quadratic permutation representation of TSP, any permutation is feasible and hence the assignment constraints are good enough in this case. However the objective function now is no longer linear. The quadratic permutation representation of TSP yields the following binary quadratic programming representation of TSP:

$$\text{Minimize } \sum_{i,j,k=1}^n d_{ij} x_{ik} x_{j(k+1)}$$

subject to the assignment constraints, where  $x_{ij} = 1$  is interpreted as ‘the  $j^{\text{th}}$  city visited is  $i$ ’ and the index  $n + 1 \equiv 1$ . This problem is represented sometimes using a cyclic permutation matrix. Let  $T$  be a cyclic permutation matrix of order  $n$ , i.e. its  $(i, j)^{\text{th}}$  element  $t_{ij}$  is given by  $t_{i,i+1} = 1$ , for  $i = 1, \dots, n-1$ ,  $t_{n1} = 1$ , and all other entries zero. Now define  $d_{ijkl} = c_{ij} t_{kl}$ . Then TSP is equivalent to the 0-1 quadratic programming problem [545]

$$\text{Minimize } \sum_{i,j,k,l=1}^n d_{ijkl} x_{ik} x_{jl}$$

subject to the assignment constraints.

#### 4.5. Three Matroid Intersection

Let  $E$  be a finite set and  $F$  be a family of subsets of  $E$ .  $M = (E, F)$  is a *matroid* if (1)  $Y \in F$ ,  $X \subset Y$  implies  $X \in F$ . (2) if  $|X| < |Y|$  and  $X, Y \in F$  implies there exists an  $e \in Y - X$  such that  $X \cup \{e\} \in F$ . Let  $S_1, S_2, \dots, S_p$  be a partition of  $E$  and  $F$  be the collection of all subsets of  $E$  with at most one element from each  $S_i$ ,  $i = 1, 2, \dots, p$ . Then  $M = (E, F)$  is called a *partition matroid*. If  $E$  is the arc set of a directed graph  $G$  and  $S_i$  contains all arcs coming into node  $i$  of  $G$  generates a partition matroid. Similarly if  $S_i$  is the collection of all arcs going out of node  $i$  in  $G$  we have another partition matroid. If  $F$  contains the collection of all 1-forests (a forest with one additional arc) in  $G$ , then  $M = (E, F)$  is a matroid called the *1-graphic* matroid. Any collection of  $n$  edges of  $G$  that is common to the two partition matroids defined above and the 1-graphic matroid defines a Hamiltonian cycle in  $G$ . Thus the ATSP can be formulated as a minimum cost three matroid intersection problem. Since the STSP is a special case of the ATSP, it is also a three matroid intersection problem.

### 5. Matrix Transformations

Let us now discuss some useful transformations of distance matrices associated with a TSP that leaves an optimal solution invariant. Let  $C = (c_{ij})$  and  $D = (d_{ij})$  be two  $n \times n$  matrices associated with a TSP on  $G$ . For any Hamiltonian cycle  $H$  of  $G$ , for  $\alpha > 0$  and  $\beta \in \mathbb{R}$ , if

$$\sum_{ij \in H} d_{ij} = \alpha \left( \sum_{ij \in H} c_{ij} \right) + \beta$$

then we say that  $C$  and  $D$  are *equivalent* with respect to TSP. Let  $\phi : \mathbb{R} \rightarrow \mathbb{R}$  such that  $\phi(c_{ij}) = d_{ij}$ . If  $C$  and  $D$  are equivalent with respect to TSP, then  $\phi$  is called an *equivalent matrix transformation* (EMT). Thus under an equivalent matrix transformation, the optimal solution of the TSP remains unchanged. For any  $\gamma > 0$  and  $\delta \in \mathbb{R}$ ,  $d_{ij} = \gamma c_{ij} + \delta$  is an EMT. Similarly for  $a_i, b_i \in \mathbb{R}$  and  $\gamma > 0$ ,  $d_{ij} = \gamma c_{ij} + a_i + b_j$  is an EMT which is perhaps the most popular EMT studied for TSP. However, under this EMT, the matrix  $D$  need not be symmetric even if  $C$  is symmetric. By choosing  $a_i = b_i$  the EMT leaves symmetry of a matrix invariant. Another important structural property of a matrix useful in the study of TSP is the *parameterized triangle inequality* ( $\tau$ -triangle inequality) [22, 117]. We say that the matrix  $C$  satisfies the  $\tau$ -triangle inequality [22, 117] if

$$c_{ij} \leq \tau(c_{ik} + c_{kj}), \text{ for all } i \neq j \neq k$$

If  $\tau = 1$ , then we say that the matrix  $C$  satisfies the *triangle inequality*. If  $\tau = 1/2$  all elements of  $C$  are equal. Thus for  $1/2 \leq \tau < 1$ ,  $\tau$ -triangle inequality is a restriction on the triangle inequality while for  $\tau > 1$  it is a relaxation on the triangle inequality. Under the EMT  $d_{ij} = c_{ij} + a_i + b_j$  the matrix  $D$  need not satisfy  $\tau$ -triangle inequality even if  $C$  satisfies the  $\tau$ -triangle inequality for moderate values of  $\tau$ . However, for any  $1/2 < \tau$  it is possible to choose values of  $a_i$  and  $b_i$  such that  $D$  satisfies the  $\tau$ -triangle inequality even if  $C$  does not satisfy the  $\tau$ -triangle inequality. To achieve this choose  $a_i$ 's and  $b_i$ 's such that

$$(\tau - 1)(a_i + b_j) + \tau(a_k + b_k) \geq c_{ij} - \tau(c_{ik} + c_{kj}) \quad \forall i \neq j \neq k \quad (33)$$

For  $\tau > 1/2$  the above inequality is always feasible since  $a_i = b_i = M$  is a solution for large enough  $M$ . If one prefers smaller values of  $a_i$ 's and  $b_i$ 's, a minimization linear program with objective function  $\sum(a_i + b_i)$  subject to (33) as constraints, can be solved. Thus using an EMT, any cost matrix can be reduced to an equivalent cost matrix satisfying  $\tau$ -triangle inequality for any  $\tau > 1/2$ .

A cost matrix  $C$  for a TSP is said to be in *standard reduced form* if  $c_{nj} = c_{jn} = 0$ ,  $j = 1, 2, \dots, n-1$  and  $c_{12} = 0$

**Theorem 3** [489] *For any cost matrix  $C$ , there exists an EMT which produces a standard reduced matrix.*

**Proof.** For  $i = 1, 2, \dots, n-1$  define  $a_i = c_{in} + x/2$  and  $b_i = c_{ni} + x/2$  where  $x = c_{12} - c_{1n} - c_{n2}$ . Let  $a_n = b_n = -x/2$ . Now the matrix  $D$  obtained by the EMT  $d_{ij} = c_{ij} - a_i - b_j$  will be a standard reduced matrix. If  $C$  is symmetric, it can be verified that  $D$  is also symmetric. ■

Using cost matrix of a TSP in standard reduced form allows simplification of some proofs [489]. (See also Chapter 11). As a consequence of the above theorem, we could reduce the cost of all arcs (incoming and outgoing) incident on a given node  $i$ , to zero as well as the cost of another arc not incident on  $i$  leaving the optimal solution invariant.

## 6. More Variations of the TSP

We now discuss additional variations of TSP studied in the literature. Our discussion on these variations are confined to their definitions only. For more details on these problems, see the corresponding references cited. Further, the references we cite need not be the paper in which the problem was originally introduced and the reference list is not exhaustive. Through out this section  $G$  is a complete graph (digraph) with  $c_e$  as the cost of edge  $e$  in  $G$ .

**The time dependent TSP:** For each arc  $(i, j)$  of  $G$ ,  $n$  different costs  $c_{ij}^t$ ,  $t = 1, 2, \dots, n$  are given. The cost  $c_{ij}^t$  corresponds to the ‘cost’ of going from city  $i$  to city  $j$  in time period  $t$ . The objective is to find a tour  $(\pi(1), \pi(2), \dots, \pi(n), \pi(1))$ , where  $\pi(1) = 1$  corresponds to the home location which is in time period zero, in  $G$  such that  $\sum_{i=1}^n c_{\pi(i)\pi(i+1)}^i$  is minimized. The index  $n+1$  is equivalent to 1. For all  $(i, j)$ , if  $c_{ij}^1 = c_{ij}^2 = \dots = c_{ij}^n$  then the time dependent TSP reduces to the traveling salesman problem. For details on this problem, we refer to [319, 392, 669].

**Period TSP:** This generalization of TSP considers a  $k$ -day planning period, for given  $k$ . We want to find  $k$  cycles in  $G$  such that each node of  $G$  is visited a prescribed number of times during the  $k$ -day period and the total travel cost for the entire  $k$ -day period is minimized [177].

**The delivery man problem:** This problem is also known as the *minimum latency problem* and the *traveling repairman problem*. Let  $H$  be a tour in  $G$  and  $v_1$  be a starting node. For each vertex  $v_i$  of  $G$ , define the *latency* of  $v_i$  with respect to  $H$ , denoted by  $L_i(H)$ , is the total distance in  $H$  from  $v_1$  to  $v_i$ . The delivery man problem is to find a tour  $H^*$  in  $G$  such that  $\sum_{i=1}^n L_i(H^*)$  is as small as possible. The DMP is strongly NP-complete. For details on this problem, we refer to [1, 385, 599, 293, 572]. It can be verified that this problem is a special case of the time dependent TSP.

**Black and White TSP:** This problem is a generalization of the TSP. Here, the node set of  $G$  is partitioned in to two sets,  $B$  and  $W$ . The elements of  $B$  are called *black nodes* and the elements of  $W$  are called *white nodes*. A tour in  $G$  is said to be feasible if the following two conditions are satisfied. (i) The number of white nodes between any two consecutive black nodes should not exceed a positive integer  $I$  and the distance between any two consecutive black nodes should not exceed a positive real number  $R$ . The black and white TSP is to find a minimum cost feasible tour in  $G$ . Applications of this problem include design of ring networks in the telecommunication industry [133]. A variation of the black and white TSP with applications in the air-line industry, known as TSP with replenishment arcs, has been discussed in [577].

**Angle TSP:** Let  $v_1, v_2, \dots, v_n$  be  $n$  points of the Euclidean plane. Consider a two edge path from  $v_i$  to  $v_k$  passing through  $v_j$ . The ‘cost’ of passing through  $v_j$  is defined as the angle between the vectors  $\overrightarrow{v_i v_j}$  and  $\overrightarrow{v_j v_k}$ . For a tour  $H$  through  $v_1, v_2, \dots, v_n$ , let  $v_i(H)$  be the cost of pass-

ing through  $v_i$ . Then the angle TSP is to find a tour  $H$  that minimizes  $\sum_{i=1}^n v_i(H)$ . The angle TSP is NP-hard.

**Film-copy Deliverer Problem:** For each node  $i$  of  $G$ , a non-negative integer  $p_i$  is prescribed. We want to find a routing of a film-copy deliverer, starting at node 1, visit each node exactly  $p_i$  times and come back to node 1 in such a way that the total distance travelled is minimized [182]. This problem generalizes TSP and some of its variations. A generalization of the Film-Copy Deliverer problem has been studied in [549]. In this case, a minimum cost cycle in  $G$  is sought that passes through each node  $i$  of  $G$  at least  $p_i$  times and at most  $q_i$  times.

**The selective TSP:** [355] This problem is also known as the orienteering problem (see Chapter 13). For each node  $i$  of  $G$ , a weight  $w_i$  is given. Then the selective traveling salesman problem is to find a cycle  $Y$  in  $G$  such that the sum of the weights of nodes in  $Y$  is maximized while keeping  $\sum_{ij \in Y} c_{ij} \leq L$ , where  $L$  is a given number. This problem is closely related to the prize collecting traveling salesman problem studied in Chapter 14.

**Resource constrained TSP:** For each edge  $e$  of  $G$  a requirement  $r_{ij}$  is also given in addition to its cost. Then the resource constrained traveling salesman problem is to find a minimum cost tour  $H$  in  $G$  such that  $\sum_{ij \in H} c_{ij} \leq K$  where  $K$  is a given constant [663].

**Serdyukov TSP:** Let  $X_1, X_2, \dots, X_n$  be a collection of subsets of the node set of  $G$  such that  $|X_i| \leq k$  for a given constant  $k$ . Starting from a city in  $X_1$ , we want to find a minimum cost tour in  $G$  such that the  $i^{th}$  city visited must be from  $X_i$  [752]. For this problem we assume that  $G$  need not be complete. If  $d$  is the maximum vertex degree in  $G$ , then we call the resulting problem a  $(k, d)$ -problem. Serdyukov [752] showed that the  $(k, d)$ -problem is NP-hard for all  $k \geq 2, d \geq 3$ .

**Ordered Cluster TSP:** [25] This problem is a simplified version of the clustered TSP and is defined as follows. The node set of  $G$  is partitioned into  $k$  clusters  $X_1, X_2, \dots, X_k$  where  $X_1$  is singleton representing the home location. The salesman starting and ending at the home location must visit all nodes in cluster  $X_2$ , followed by all nodes in cluster  $X_3$ , and so on such that the sum of the edge costs in the resulting tour is minimized.

**Precedence Constrained TSP:** Let  $B$  be a subset of  $V \times V$  defined by  $B = \{(i, j) : i \in V, j \in V, i, j \neq 1, \text{ and } i \text{ must be visited before } j\}$  represents the precedence constraints. Then the precedence constrained TSP is find a least cost TSP tour starting at node 1, visits each node of  $G$  exactly once and returns back to node 1 in such a way that the precedence constraints  $B$  are satisfied [71]. A special case of this problem is the dial-a-ride problem [674].

**$k$ -Peripatetic Salesman Problem:** [240] A  $k$ -peripatetic salesman tour in  $G$  is the collection of  $k$  edge disjoint Hamiltonian cycles in  $G$ . The objective is to find a least cost  $k$ -peripatetic salesman tour in  $G$ . This problem has applications in network design.

**Covering Salesman Problem:** [233] In this case, we want to find a minimum cost cycle  $L$  in  $G$  such that the distance between  $L$  and any node  $i$  of  $G$  not on  $L$ , denoted by  $d(L, i)$ , is within a prescribed limit, where  $d(L, i) = \min\{c_{ij} : j \text{ is a node of } L\}$ .

**TSP with time windows:** [44, 45] For each arc  $(i, j)$  of  $G$ , let  $c_{ij}$  be the cost and  $t_{ij}$  be the traversal time. For each node  $i$  of  $G$  a triplet  $\{a_i, b_i, s_i\}$  is given where  $a_i \leq b_i$  and  $0 \leq s_i \leq b_i - a_i$ . The quantity  $s_i$  represents the service time at node  $i$  and the job at node  $i$  must be completed in the time interval  $[a_i, b_i]$ . If the salesman arrives at node  $i$  before  $a_i$ , he will have to wait until  $a_i$ . Then TSP with time windows is to find a least cost tour in  $G$  satisfying the restriction discussed above.

**Moving Target TSP:** A set  $X = \{x_1, x_2, \dots, x_n\}$  of  $n$  objects placed at points  $\{p_1, p_2, \dots, p_n\}$ . Each object  $x_i$  is moving from  $p_i \in \mathbb{R}^n$  at velocity  $v_i$ . A pursuer starting at origin moving with speed  $v$  wants to intercept all points  $x_1, x_2, \dots, x_n$  in the fastest possible time [447]. This problem is related to the time dependent TSP.

**Remote TSP:** For a given integer  $k$ , we are interested in finding a subset  $S$  of  $V$  with  $|S| = k$  such that the minimum cost of a tour in the subgraph of  $G$  induced by  $S$  is maximized [430].

There are several other variations that one could discuss depending on various application scenarios. These variations include traveling salesman location problem, traveling salesman problem with backhauls [353],  $k$ -best TSP [800], minmax TSP [767], multi-criteria TSP [271], stochastic TSP (different variations),  $k$ -MST [113, 601], minimum cost biconnected spanning subgraph problem [604], graph searching problem [519],

$k$ -delivery TSP [175], quadratic assignment problem [545] etc., and combinations of one or more of the variations discussed here. Vehicle routing problems form another very important class of generalizations of TSP which are not discussed here.

**Acknowledgement:** I am thankful to G.Gutin and K.P.K. Nair for their comments on an earlier version of this chapter. Special thanks are due to S.N. Kabadi and H. Marchand for several discussions. This work was partially supported by the NSERC grant OPG0170381.

## Chapter 2

# POLYHEDRAL THEORY AND BRANCH-AND-CUT ALGORITHMS FOR THE SYMMETRIC TSP

Denis Naddef

*Laboratoire Informatique et Distribution, Institut National Polytechnique de Grenoble,  
France*

Denis.Naddef@imag.fr

## 1. Introduction

In this chapter we deal with the problem of solving symmetric TSP (STSP) instances to optimality. Of course, STSP instances are particular cases of asymmetric TSP (ATSP) instances, those for which the distance between any two cities is irrelevant of the direction. Therefore we could transform any instance of the STSP to an asymmetric one and use the results of Chapter 4 to solve it. In fact the techniques of Chapter 4 do not perform well when the costs of the arcs  $(i, j)$  and  $(j, i)$  only slightly differ. Progress in the solution techniques for the STSP is such that it is common to transform an ATSP into a symmetric one to solve it to optimality (see [474]).

Various methods have been proposed, among which we can cite, for historical reasons, a Lagrangian relaxation algorithm based on *1-trees* by Held and Karp (see [444] and [445]) and a dynamic programming approach that can be found in any text book dealing with dynamic programming. Both methods are very limited, even today, in the size of problems they can solve to optimality. Belloni and Lucena have revisited the Lagrangian approach to the TSP in [98] and report promising results at least for instances that are not too large. The only method that has given good, not to say impressive, results is the Branch-and-Cut method using the double index formulation of the problem. This method finds its foundations in the seminal work of Dantzig, Fulkerson and Johnson all the way back in 1954 [239].



The next section contains the double index integer linear programming model we will use. We will also mention other possible formulations of the problem as integer linear programs. We will explain why a substantial amount of theoretical work has to be carried out in order to solve large instances of the problem. Those theoretical developments will be the subject of sections 3 to 12. The following three sections will be devoted to the use of these developments to solve TSP instances to optimality. From now on, in this chapter, *solve* will be understood as *solve to optimality*, which includes a proof of optimality, even if this proof is not easily checkable. Numerical results will be given to show that the method indeed works very well on a large set of instances.

We assume the reader knows the basic concepts of linear algebra dealing with polyhedra.

Except when specified, we assume that we deal with a complete graph  $K_n = (V, E)$ , with  $n = |V|$  representing the number of vertices. Let  $S \subset V$ , then  $\delta(S)$  (resp.  $\gamma(S)$ ) represents the set of edges with exactly one endnode in  $S$  (resp. both endnodes in  $S$ ), i.e.  $\delta(S) = \{(u, v) \in E : u \in S, v \notin S\}$  (resp.  $\gamma(S) = \{(u, v) \in E : u \in S, v \in S\}$ ). The edge set  $\delta(S)$  is in general called the *coboundary* of  $S$  (some authors say *cocycle* of  $S$  or *cut* defined by  $S$ ). We write  $\delta(v)$  instead of  $\delta(\{v\})$  for  $v \in V$ . For  $S \subset V$  and  $T \subset V \setminus S$  we denote by  $(S : T) = (T : S)$  the set of edges with one endnode in  $S$  and the other in  $T$ . We denote by  $\mathbb{R}^E$  the set of vectors indexed by  $E$ , that is the components of a vector of  $\mathbb{R}^E$  are in one to one correspondence with the elements of  $E$ . For  $E^* \subset E$  and  $x \in \mathbb{R}^E$  we let  $x(E^*)$  represent  $\sum_{e \in E^*} x_e$ . Let  $G(S)$  denote the induced subgraph on  $S$ , i.e.  $G(S) = (S, \gamma(S))$ . Let  $x^* \in \mathbb{R}^E$ , with  $x_e^*$  viewed as the *capacity* of edge  $e$ ; for  $S \subset V$  we call  $x^*(\delta(S))$ , the *value* of the cut defined by  $S$ . In the following we will abusively say cut  $S$  for cut defined by  $S$ , that is for  $\delta(S)$ .

The STSP instances we will mention are taken from the TSPLIB [709]. The number in the name of the instance represents the number of cities.

## 2. Integer linear programming models

There are various ways of modelling the STSP as an integer linear program. Some contain a polynomial (in  $n$ ) number of variables and constraints, they are said to be *compact*, some others only a polynomial number of variables but an exponential number of constraints.

Since a STSP instance can be transformed into an asymmetric one by replacing each edge by two oppositely directed arcs of same cost, all the integer formulations for the ATSP yield formulations for the STSP. There are various known compact formulations for the ATSP, see for

example [650] and [291]. We will mention two compact formulations for the STSP that do not come from the ATSP.

For computational reasons, which will become clear in this chapter, among two integer models, the best is the one whose linear relaxation has the highest objective function value. In order for this to make sense one must assume that the integer linear programming model is minimal in terms of inequalities, since any integer linear programming model can be strengthened by the use of valid inequalities for the underlying polyhedron, which will be the subject of most of this chapter. This strengthening, at least theoretically, could achieve a value equal to the optimal integral value.

Among all the integer linear programming models known, in this respect, three models emerge and attain the same value for their linear relaxations. None is known that attains a higher value. These are the *multistage insertion* formulation of Arthanari [39] (see also [41]), the *cycle shrink* of Carr [169], and finally what is known as the *double index* or *subtour elimination* formulation. This latter formulation will be extensively described here since it is the only one that has been used so far in computational studies. The multistage-insertion is inspired by dynamic programming recursion, that is building up a tour step by step. Cycle shrink does the opposite, that is going from a tour to a node. It is not surprising then that these two formulations are equivalent, see Arthanari and Usha [42] (see also [40] for the equivalence with the double index formulation). For the ATSP, Padberg and Sung [650] show that the double index formulation dominates all the other known ones.

We now describe the *double index* formulation. To every edge  $e \in E$  we associate a variable  $x_e$  which will take value 1 if  $e$  is in the resulting optimal tour and 0 else. The STSP can then be formulated as the following integer linear program:

$$\begin{aligned} (\text{IP}(\text{STSP})) \quad & \min \sum_{e \in E} c_e x_e \end{aligned} \tag{1}$$

subject to

$$x(\delta(v)) = 2 \quad \text{for } v \in V \tag{2}$$

$$x(\delta(S)) \geq 2 \quad \text{for } 3 \leq |S| \leq |V|/2 \tag{3}$$

$$0 \leq x_e \leq 1 \quad \text{for } e \in E \tag{4}$$

$$x_e \text{ integer} \quad \text{for } e \in E \tag{5}$$

Equations (2) just say that two edges have to be chosen incident to any vertex. Inequalities (3) forbid cycles which do not contain all the vertices and are called *subtour elimination inequalities*. If  $S = \{v\}$ , i.e.  $|S| = 1$ , the corresponding Inequality (3) is implied by the Equation (2)

corresponding to  $v$ . Since  $\delta(S) = \delta(V \setminus S)$ , we can restrict the subtour elimination inequalities to sets  $S$  with at most half the vertices. The number of subtour elimination inequalities is still in  $\Omega(2^n)$ , that is, exponential in the size of the problem. This may seem to be a serious problem, we will discuss this point later on.

The name *double index* formulation comes from the ATSP in which both extremities of the arcs play different roles and therefore  $x_{ij}$  is used instead of  $x_e$ , creating two indices. When dealing with the STSP this is not the case, so we stick to variables  $x_e$  and will avoid using the misleading terminology “double index” and rather use the other usual name of *subtour elimination* formulation which comes from the Inequalities (3).

Why deal with an exponentially large integer model while the other two equivalent, in terms of strength, formulations only have a polynomial number of them? This has to do with the fact that separating these inequalities is not a problem. We will make this concept of separation more precise shortly.

Why do we prefer formulations with high optimal linear relaxation value? This has to do with the solution technique which goes as follows.

The linear relaxation  $LP(STSP)$  of  $IP(STSP)$  is the linear program obtained by dropping the integrality conditions (5). The most obvious way to solve  $IP(STSP)$  is via Branch-and-Bound using as lower bound the value of the linear relaxation of the subproblems (see for example [209], [826]). The method goes as follows (assuming we are minimizing, which is our case) and that the term feasible relates to the *integer* linear program:

### A Branch-and-Bound algorithm for integer linear programming

- **Initialization:** The set of subproblems to solve is initialized to the linear program obtained by dropping the integrality conditions on the variables. This problem is referred to as the root subproblem.
- **Choose a subproblem:** If the list of open problems is empty, the best known feasible solution is optimal. Else choose an open subproblem and delete it from the list.
- **Treat subproblem:** Solve the associated linear program. If the solution is integer, go back to *Choose a subproblem* after eventually updating the best known integer solution and the best known solution value.
- If the value of the objective function exceeds that of the best known feasible solution, go back to *Choose a subproblem*.

- Else, using some linear inequality, partition the current subproblem into two new subproblems. The union of the feasible (integer) solutions to each of these two subproblems contains all the feasible solutions of the problem that has been partitioned. This is commonly done by choosing a variable with a current fractional value  $\bar{x}_e$ , and by creating two subproblems which are added to the list of open problems, one in which we impose  $x_e \geq 1$  and another in which we impose  $x_e \leq 0$ . (There are other ways of doing this, which will be exposed in due time.) These two subproblems are called the *sons* of the current subproblem. Go to *Choose a subproblem*.

The reader may realize that, as described here, the algorithm is not of any use for our problem since none of the linear programs could fit on any computer for even relatively small size instances. This is due to the number of constraints which is exponential in the number of cities of the TSP instance. One way of getting around this is known as *constraint* or *row generation*. One call to the solution of a linear program, in the previous algorithm, will now be replaced by a series of calls. In our case it goes as follows.

Assume we have an algorithm that for a given vector  $\bar{x} : E \rightarrow \mathbb{R}$ , seen as capacities on the edges of  $G$ , returns a *minimum capacity cut* of  $G$ . Such an algorithm returns a set  $S \subset V$  such that  $\bar{x}(\delta(S))$  is minimum. The Branch-and-Bound algorithm described above is modified in the initialization and the treatment of each subproblem as follows:

- **Initialization:** The set of subproblems to solve is initialized to the linear program consisting of Constraints (2) and (4).
- **Treat subproblem:** Until the subproblem is not declared examined, repeat: Solve the current linear program which yields the optimal solution  $\bar{x}$ . Search for  $S \subset V$  such that  $\bar{x}(\delta(S)) < 2$ . If no such  $S$  exists, the subproblem is **examined**, else, add  $x(\delta(S)) \geq 2$  to the set of constraints of the current linear program.

When one has finished treating the first subproblem, which we refer to as the *rootnode problem*, one has *optimized over the STSP subtour elimination polytope*.

We are now concerned with two questions:

- Is there an efficient algorithm to find a minimum capacity cut in a weighted undirected graph?
- Does the number of calls to the minimum capacity cut algorithm remain reasonable in practice?

To answer the first question, Nagamochi and Ibaraki in [622] and Nagamochi, Ono and Ibaraki in [623] give efficient minimum cut algorithms in the case of undirected graphs. Padberg and Rinaldi in [646] give very efficient preprocessing procedures that significantly accelerate any minimum cut algorithm. One can find a study of performances of several minimum cut algorithms in Jünger, Rinaldi and Thienel [476] and in Chandra, Chekuri, Goldberg, Karger, Levine and Stein [181].

As for the second question, since finding a minimum capacity cut is polynomial, using the ellipsoid algorithm, one can, theoretically, finish in a polynomial number of iterations (see Grötschel, Lovász and Schrijver [399]). If we use the procedure we just described, almost all problems of the TSPLIB require very few iterations.

The value of the linear relaxation obtained from  $IP(STSP)$  by dropping the integrality conditions on the variables (i.e. the root problem), is known as the *Held and Karp bound* or *value on the subtour elimination polytope*.

Although the Held and Karp bound is very good, it is not good enough to be able to solve even average-size STSP instances by the procedure just described since the number of subproblems created is far too high. We mention here a conjecture, which seems difficult to trace back to its origin, but that can be found in Goemans [383] .

**Conjecture 1** *If the edge costs satisfy the triangular inequality, then the optimal value of a tour is at most  $4/3$  the value of the Held and Karp bound.*

Consider the partition of the vertex set  $V$  into  $V_i^j$ ,  $i = 0, \dots, k$ ,  $j = 1, \dots, 3$ , with  $V_0^j = A$  and  $V_k^j = Z$  for  $j = 1, \dots, 3$ , and  $k \geq 3$ . In Section 6 this will be called a  $(k-1)$ - path configuration. Let  $c_e = 0$  if  $e \in \gamma(V_i^j)$  for all  $i$  and  $j$ ,  $c_{uv} = 1$  if  $u \in V_i^j$  and  $v \in V_{i+1}^j$  for all  $i$  and  $j$ ,  $c_{uv} = k-2$  if  $u \in A$  and  $v \in Z$ . For all the other edges  $e = (u, v)$ , let  $c_{uv}$  equal the length of a shortest path linking  $u$  to  $v$  using only edges for which the cost has been defined in the preceding lines. For such an instance of STSP, the optimal value of a tour is  $4k-2$  and the Held and Karp bound is  $3k$ , therefore the ratio can be made as close as one desires from the bound of the conjecture.

For all the examples of the TSPLIB, the gap is much less than the one we would expect from the conjecture.

The more general procedure known as Branch-and-Cut differs from the algorithm described to optimize on the subtour elimination polytope in the way a subproblem is treated. One not only tries to generate constraints coming from the integer linear formulation but also any linear inequality that separates the current fractional solution from the feasible

solutions. Such an inequality is called a *cutting plane* or just a *cut*. Using the term *cut* would be confusing since it is also a set of edges that disconnects an undirected graph. In the following, except for the name Branch-and-Cut, we will avoid saying cut for an inequality. Remember that we will use cut  $S$  in place of cut  $\delta(S)$ .

The next sections will show the existence of a finite set of linear inequalities, which can be added to  $IP(STSP)$  in such a way that the linear relaxation of the newly obtained integer program is always integral and therefore corresponds to an optimal tour. Unfortunately, as we will see, this is only an existence theorem and it is unlikely that we will discover that complete set of linear inequalities in a near future even for medium size instances. Table 3.1, which will be commented in the next section, gives the number of such inequalities for instances up to 10 nodes. Fortunately, even a partial knowledge of that description is of precious help in solving STSP instances, and that will be the topic of the next sections.

In view of all this the **Treat subproblem** phase of a Branch-and-Cut looks like this:

**Treat subproblem.** Repeat until a stopping criterion is attained:

- Solve the current lp, let  $\bar{x}$  be its fractional optimal solution  $\bar{x}$ .
- Find a linear inequality  $fx \geq f_0$  satisfied by all incidence vectors (see definition in next section) of tours and such that  $f\bar{x} < f_0$ .

Two stopping criteria are given in Section 19.4.

### 3. Introducing the symmetric TSP polytope and its various relaxations

We let  $\mathcal{H}_n$  (resp.  $\mathcal{H}_G$ ) represent the set of tours of the complete graph  $K_n$  (resp. of graph  $G$ ). To every tour  $\Gamma \in \mathcal{H}_n$  (resp.  $\in \mathcal{H}_G$ ), we associate a vector  $x^\Gamma \in \mathbb{R}^E$  such that  $x_e^\Gamma = 1$  if  $e \in \Gamma$  and  $x_e^\Gamma = 0$ , else. This vector is called indifferently *incidence* or *representative* vector of  $\Gamma$ . Some authors use *characteristic* vector of  $\Gamma$ .

#### 3.1. The symmetric TSP polytope

The *symmetric TSP polytope* of  $G$ ,  $STSPP(G)$ , is the convex hull of all the vectors  $x^\Gamma$  when  $\Gamma$  ranges over all Hamiltonian cycles of  $G$ . When  $G = K_n$ , we let  $STSPP(n)$  stand for  $STSPP(K_n)$ , i.e.  $STSPP(n) = \text{conv}\{x^\Gamma : \Gamma \in \mathcal{H}_n\}$ . When not specified, we assume the underlying graph is  $K_n$ .

| n  | # tours | # different facets       | # facet classes |
|----|---------|--------------------------|-----------------|
| 3  | 1       | 0                        | 0               |
| 4  | 3       | 3                        | 1               |
| 5  | 12      | 20                       | 2               |
| 6  | 60      | 100                      | 4               |
| 7  | 360     | 3 437                    | 6               |
| 8  | 2 520   | 194 187                  | 24              |
| 9  | 20 160  | 42 104 442               | 192             |
| 10 | 181 440 | $\geq 51\,043\,900\,866$ | $\geq 15\,379$  |

Table 2.1. Some statistics on  $STSP(n)$ 

It is well known from a theorem of Weyl [823], that  $STSP(n)$  can be described by a *finite* set of linear equations and inequalities. The study of the symmetric TSP Polytope consists in finding such linear equations and inequalities. All the linear *equations* are known in the case of  $STSP(n)$ , this is the basis of the proof of Theorem 2. This is not the case for  $STSP(G)$  if  $G$  is an arbitrary connected graph.

Before going further on we give an idea of the richness of the facial structure of  $STSP(n)$ . Table 3.1 is taken from Christof and Reinelt [186]. This table gives, for small values of  $n$ , the number of tours, the number of different facets and the number of classes in which they can be put. Two facet inducing inequalities for  $STSP(n)$  are said to belong to the same class if a renumbering of the vertices transforms one into the other. One can observe that this polytope is highly degenerate, that is the number of facets incident to a vertex greatly exceeds the minimum number of facets needed to define that vertex. As can be seen from this table, the complexity of the polytope increases very fast and so does its degeneracy. The numbers in the last line are conjectured to be exact.

The first thing to do, when studying a polyhedron, is to determine its dimension. That dimension is not known for  $STSP(G)$  when  $G$  is a connected arbitrary graph. Note that  $STSP(G)$  is non empty if and only if  $G$  is Hamiltonian.

For complete graphs, the polytope dimension is given by the following theorem.

**Theorem 2** *The dimension of  $STSP(n)$  is  $|E| - |V|$ .*

**Proof:** (sketch): For  $n = 3$  there is only one tour, so the theorem is true in that case. Equations (2) are linearly independent, so the dimension cannot be more than what is announced in the theorem. It is therefore enough to exhibit  $|E| - |V| + 1$  affinely independent tours. For this we use a theorem on the partition of the edges of  $K_{n-1}$ . If  $n = 2k + 1$ , then the edges of  $K_{n-1}$  can be partitioned into  $k$  edge disjoint Hamiltonian

cycles. If  $n = 2k$ , then the edges of  $K_{n-1}$  can be partitioned into  $k - 1$  edge disjoint Hamiltonian cycles and a perfect matching. For  $n$  odd, for each of the  $(n - 1)/2$  disjoint Hamiltonian cycles that partition the edges of  $K_{n-1}$  and each edge of that cycle, we create one Hamiltonian cycle of  $K_n$  by inserting node  $n$  between the endnodes of that edge, i.e. if  $e = (i, j)$ , we remove edge  $e$  and add the two edges  $(i, n)$  and  $(j, n)$ . In the other case, for the Hamiltonian cycles of the partition, we do the same thing. As for the perfect matching, we complete it arbitrarily to a Hamiltonian cycle, and do as previously only with the edges of the perfect matching. We leave it to the reader to check that in both case we have the right number of cycles and that the corresponding vectors are affinely independent. ■

**Remark 3** *Another proof not using the decomposition of the edges of  $K_n$  is given by Queyranne and Wang in [687].*

**Corollary 4** *Any Equation in the description of  $STSPP(n)$  is a linear combination of the Equations (2). Therefore Equations (2) are the only necessary equations in a minimal description.*

**Remark 5** *In Chapter 11, the problem of characterizing the cost functions such that all Hamiltonian cycles have the same cost (constant TSP) is raised. If one knows a maximal linearly independent set of linear equations  $c^i x = c_0^i$  for  $i = 1, \dots, p$ , satisfied by all the incidence vectors of Hamiltonian cycles, one has the dimension of  $STSPP(G)$  and also a solution to the constant STSP problem. Let  $c$  be a cost function on the edges such that all Hamiltonian cycles of  $G$  have the same cost  $c_0$ , then  $cx = c_0$  is a linear equations satisfied by all Hamiltonian cycles of  $G$  and is therefore a linear combination of the equations  $c^i x = c_0^i$  for  $i = 1, \dots, p$ .*

In the case of complete graphs this yields:

**Corollary 6** *(see also Chapter 11) The only cost functions on the edges that have the property that all Hamiltonian cycles of the complete graph have the same cost, are those obtained from any function  $\pi : V \rightarrow \mathbb{R}$  and by setting  $c_e = \pi(i) + \pi(j)$  for all  $e = (i, j)$ .*

**Proof:** Let  $c : E \rightarrow \mathbb{R}$  be such that all the tours have a cost of  $c_0$ . Then  $cx = c_0$  is an equation satisfied by all tours, and therefore, by Theorem 2, must be a linear combination of the equations (2). The coefficients of that linear combination are the  $\pi(i)$ 's. ■

Finding the dimension for general graphs is much more complex and only very few results are known. In fact deciding whether  $STSPP(G) \neq$



$\emptyset$  is NP-complete. The reason is that the set of equations is difficult to find, since some constraints written as inequalities turn out to be equations. Take for example the case of Figure 2.1. There is an implicit equation, linearly independent from the degree constraints, namely that all tours contain exactly two of the three horizontal edges, therefore the subtour inequality defined by one of the triangles is in fact an equation.

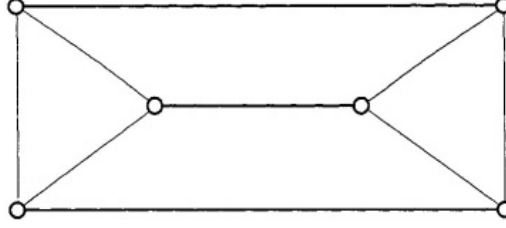


Figure 2.1. The diamond graph

For  $G$  a Halin graphs (see Cornuéjols, Naddef and Pulleyblank [220]), a complete description of  $STSP(G)$  is known. This case was generalized to classes of graphs which have the property to “decompose”, in some way, using three-edge cutsets (see Cornuéjols, Naddef and Pulleyblank [221]). It would be a nice result to characterize those graphs for which Constraints (2) to (4) of the linear relaxation of  $IP(STSP)$  describe  $STSP(G)$ . We will mention, later on, such a result but for the convex hull of *spanning closed walks*, a *spanning closed walk* being a cycle visiting each vertex **at least** once (any edge can be used an arbitrary number of times).

From now on we restrict to the case of complete graphs. Non full dimensional polyhedra have the undesirable property that an infinite number of linear inequalities define the same facet, unlike the case of full dimensional ones where a facet is described by a unique inequality up to scaling by a strictly positive factor. The following theorem, which in fact is a corollary of Theorem 2, deals with two representations of the same facet of  $STSP(n)$ , where  $A$  is the node-edge incidence matrix of  $K_n$ .

**Theorem 7** *Let  $fx \geq f_0$  be a facet inducing inequality of  $STSP(n)$ . The valid inequality  $gx \geq g_0$  defines the same facet if and only if there exists  $\pi : V \rightarrow \mathbb{R}$  and  $\pi_0 \in \mathbb{R}$ ,  $\pi_0 > 0$ , such that:  $g = \pi_0 f + \pi A$  and  $g_0 = \pi_0 f_0 + 2 \sum_{i \in V} \pi_i$*

Testing whether two facet inducing linear inequalities define or not the same facet can be solved in  $O(n^2)$  using the following algorithm of F.Margot [582]. Choose a spanning tree  $T$  and add an edge  $e^*$  that

creates an odd cycle when added to the edges of  $T$ . Solve for the variables  $\pi_i, i \geq 1$  as a function of  $\pi_0$  using the set of  $n$  equations relative to each edge of  $T + \{e^*\}$ . This can be performed in  $O(n)$  time. Solve for  $\pi_0$  using the right hand sides. In  $O(n^2)$  time check whether the solution satisfies the equations relative to all the edges not in  $T + \{e^*\}$ .

The usual way to tackle the problem of multiple linear descriptions of the same facet, is to embed the non full dimensional polyhedron in a full dimensional one. In general we require the original polyhedron to be a face of the full dimensional one. The larger polyhedron is called a *relaxation* of the smaller one. A first step is to study a minimal linear description of the relaxation, which is unique (up to scaling by a positive integer), and then try to see which facets of the relaxation define facets of the original polyhedron.

We now turn to the various relaxations that have been used to study  $STSP(n)$ .

### 3.2. The monotone Relaxation

Historically this has been the first relaxation used in the study of  $STSP(n)$  [195], [402], [401], [403], [404], [642]. The *monotone STSP polytope* is the convex hull of the incidence vectors of the tours and all edge subsets of tours of  $K_n$ . It is trivially of full dimension since the empty set is a subset of a tour, and the sets consisting of a single edge are also subsets of tours, since, in  $K_n$ , every edge belongs to some tour (in fact each edge appears in  $(n-2)!$  different tours). Balas and Fischetti [74] give some very simple conditions under which facet inducing inequalities of the monotone STSP polytope yield facets of  $STSP(n)$ .

We will not detail this relaxation since it seems to have attained its limits and is nowadays only very seldom used. We will pass over relaxations of  $STSP(n)$  using the *2-matching polytope* which has received very little attention (see Cornuéjols and Pulleyblank [224], [225], [223]) and turn to two much more interesting relaxations.

### 3.3. The Hamiltonian path relaxation

This relaxation is due to Queyranne and Wang [687]. They observed that there is a bijection between the tours of  $K_{n+1}$  and the Hamiltonian paths of  $K_n$ . Let  $HP(n)$  denote the convex hull of the representative vectors of all the Hamiltonian paths (no fixed extremities) of  $K_n$ . The following theorem states that this polytope is near full dimensional.

**Theorem 8**  $Dim(HP(n)) = \frac{n(n+1)}{2} - 1$

**Proof:** Note that  $x(E_n) = n - 1$  holds for all representative vectors of Hamiltonian paths of  $K_n$ . Hence the dimension cannot be more than stated. Let  $fx = f_0$  be an equation satisfied by all Hamiltonian paths of  $K_n$ . Let  $e_1$  and  $e_2$  be any two different edges of  $K_n$  and let  $\Gamma$  be a Hamiltonian cycle of  $K_n$  containing these two edges, which always exists. Let  $P_i = \Gamma \setminus \{e_i\}$ , for  $i = 1, 2$ , be two Hamiltonian paths obtained from  $\Gamma$  by removing one of the edges  $e_1$  and  $e_2$  respectively. We have  $fx^{P_1} = f_0$  and  $fx^{P_2} = f_0$  and therefore  $fx^{P_1} - fx^{P_2} = 0 = f_{e_1} - f_{e_2}$ . Therefore  $f_{e_1} = f_{e_2}$ , and since  $e_1$  and  $e_2$  are arbitrary,  $f_e$  is a constant for  $e \in E_n$ , that is  $fx = f_0$  is a multiple of  $x(E_n) = n - 1$ . ■

The main interest of this approach is that it provides a normalized form for the linear inequalities defining facets of  $STSP(n)$ . A facet defining inequality  $fx \leq f_0$  for  $STSP(n)$  is in *normalized form* if:

- $f_e = 0$  for all  $e \in \delta(\{1\})$
- $f_e \geq 0$  for all  $e \in E \setminus \delta(\{1\})$
- $\exists e \in E \setminus \delta(\{1\})$  such that  $f_e = 0$
- $\min\{f_e : e \in E \setminus \delta(\{1\}) \text{ and } f_e > 0\} = 1$

**Theorem 9** *Every facet defining inequality of  $STSP(n)$  has a unique normalized form*

**Proof:** See [687]. ■

How difficult is it to find the normalized form of the facet defined by a facet inducing inequality  $fx \leq f_0$ ? Queyranne and Wang give a  $O(n^2)$  algorithm to do so. This yields another  $O(n^2)$  algorithm to check whether or not two facet defining inequalities define the same facet of  $STSP(n)$ . To do so, find the normalized form of the facet defined by each. If they are identical, the facets are the same, else they are different. We will give later on another standard form for  $STSP(n)$  facet defining inequalities, which will only lead to a  $O(n^3)$  algorithm for the problem of recognizing whether or not two inequalities define the same facet of  $STSP(n)$ .

### 3.4. The graphical relaxation

Let  $G = (V, E)$  be a connected, not necessarily complete, graph. A *spanning closed walk*  $W$  of  $G$  is a family of edges of  $G$  (in a family we assume a same element can appear more than once) such that the graph  $(V, W^*)$  is *eulerian*, where  $W^*$  is a set of edges obtained from  $W$  by replicating an edge as many times as it appears in  $W$ . An *eulerian*

graph is a connected multigraph in which each vertex is incident to an even number of edges. Figure 2.2 shows a graph and one of its closed walks. We will **always** assume closed walks to be spanning and therefore omit the word spanning from now on.

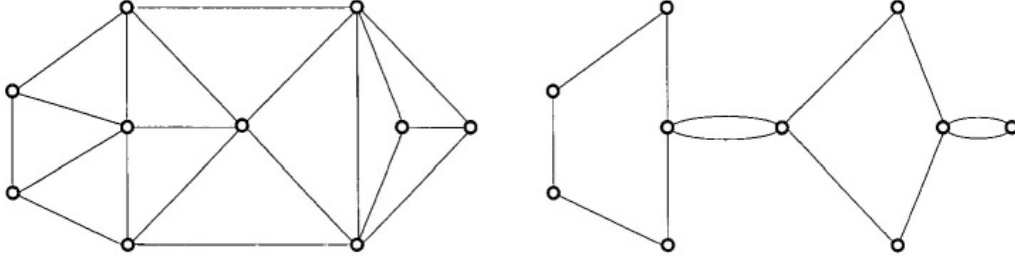


Figure 2.2. A graph and one of its closed walks

We can assign a vector  $x^W \in \mathbb{R}^E$  to a closed walk  $W$  such that  $x_e^W$  represents the number of times  $e$  appears in  $W$ .

The *Graphical Traveling Salesman Polyhedron of  $G$* ,  $GTSP(G)$ , is the convex hull of all the representative vectors of closed walks of  $G$ . As we will see,  $GTSP(n) = GTSP(K_n)$  has very strong ties with  $STSP(n)$ , which explains why today this is the most used relaxation to study the latter polytope. Note that  $STSP(n)$  is the face of  $GTSP(n)$  defined by  $x(E) = n$ .

If  $W$  is a closed walk of  $G$ , then so is  $W + (2k)\{e\}$  (the walk obtained from  $W$  by going  $k$  times back and forth along edge  $e$ ) for any positive integer  $k$ , and therefore  $GTSP(G)$  is an *unbounded* polyhedron if  $G$  is connected and has at least two vertices.

The problem of finding a closed walk of minimum cost in  $G$  is called the *Graphical Traveling Salesman Problem on  $G$* ,  $GTSP(G)$ . The term “graphical” may not be the most suitable, but at the time it was defined in [219], the idea was to reflect the fact that this version of the traveling salesman problem has an optimal solution as long as the graph is connected. Note that if some edge  $e$  is such that  $c_e < 0$ , then the problem has no finite optimal solution since adding to any solution two copies of edge  $e$  strictly decreases its value. If all edge costs are non-negative, then there is an optimal solution in which no edge will appear more than twice. Finally note that if in addition the edge costs satisfy the triangular inequality and  $G = K_n$ , then there is an optimal solution to  $GTSP(n)$  which is also a solution to  $STSP(n)$ .

**Theorem 10** *If  $G$  is connected, then  $GTSP(G)$  is of full dimension, otherwise it is empty.*

**Proof:** If  $G$  is not connected, then no closed walk exists. Else let  $W$  be any given closed walk. Consider the  $|E|$  closed walks  $W_e = W + 2\{e\}$  for all  $e \in E$ . The representative vectors of those  $|E|$  closed walks and that of  $W$  are affinely independent. ■

The following theorem shows that if  $G$  has no *bridge*, that is an edge the removal of which disconnects the graph, then the convex hull of the extreme points of  $GTSP(G)$  is a polytope of full dimension. If edge  $e$  is a bridge, then each closed walk which corresponds to an extreme point uses exactly twice that edge, and therefore  $x_e = 2$  is an equation satisfied by all such walks.

**Theorem 11** *If  $G$  is connected with  $k$  bridges, then the convex hull of the extreme points of  $GTSP(G)$  is a polytope of dimension  $|E| - k$ .*

**Proof:** See [219]. ■

**Theorem 12** *The inequality  $x_e \geq 0$  defines a facet of  $GTSP(G)$  if and only if the edge  $e$  is not a bridge.*

**Proof:** If  $e$  is a bridge, then  $x_e \geq 2$  holds for every closed walk of  $G$ . If  $e$  is not a bridge, then there exists a closed walk  $W$  of  $G \setminus \{e\}$ . Consider the  $|E| - 1$  closed walks  $W_f = W + 2\{f\}$  for every  $f \in E \setminus \{e\}$ . The representative vectors of these closed walks together with that of  $W$  are affinely independent. ■

**Theorem 13** *Let  $S \subset V$ , then  $x(\delta(S)) \geq 2$  is a facet of  $GTSP(G)$  if and only if the induced subgraphs  $G(S)$  and  $G(V \setminus S)$  are connected.*

**Proof:** If, say,  $G(S)$  is not connected, then  $x(\delta(S)) \geq 4$  holds for all closed walks and therefore the inequality  $x(\delta(S)) \geq 2$  is not even supporting for  $GTSP(G)$ . Conversely, for each  $e \in \delta(S)$ , let  $W_e$  be a closed walk that contains twice  $e$  and no other edge of  $\delta(S)$ . Let  $W_{e^*}$  be one of the just defined closed walks. For all  $e \notin \delta(S)$ , let  $W_e = W_{e^*} + 2\{e\}$ . The  $|E|$  representative vectors of these closed walks are affinely independent and all satisfy the inequality with equality. ■

The reader may be surprised at this point that we did not give a formulation of  $GTSP(G)$  as an integer linear program. The problem is that no such formulation is known. All we can write is the following:

$$\min cx \tag{6}$$

subject to

$$x(\delta(S)) \geq 2 \text{ and even for } 1 \leq |S| \leq |V|/2 \tag{7}$$

$$x_e \geq 0 \text{ and integer for } e \in E \tag{8}$$

**Research Question 14** Find an integer linear formulation for the graphical traveling salesman problem using only the variables of the double index formulation.

The previous question needs to be made more precise. It is known that there does not exist a set of linear inequalities with the property that all points with integral components inside the polyhedron defined by these inequalities correspond to closed walks. The question is to find a set of linear inequalities such that all *integral extreme points* of the polyhedron they define correspond to spanning closed walks.

As suggested by M. Fischetti, one could change the space of variables, associating an integer variable  $y_u \in \mathbb{N}$  to each node  $u$ , and add the constraints  $x(\delta(u)) = 2y_u$ .

Fonlupt and Naddef [317] characterized those graphs  $G$  for which the following set of linear inequalities defines  $GTSP(G)$

$$x(\delta(S)) \geq 2 \quad \text{for } 1 \leq |S| \leq |V|/2 \quad (9)$$

$$x_e \geq 0 \quad \text{for } e \in E \quad (10)$$

A *minor* of a graph  $G$  is any graph  $H$  that can be obtained from  $G$  by recursively performing the following operations in some order.

- 1 (edge deletion) Remove an edge from the current graph
- 2 (edge contraction) Remove an edge from the current graph and identify its two extremities.

**Theorem 15** Inequalities (9) and (10) describe  $GTSP(G)$  if and only if the graph  $G$  does not contain, as a minor, one of the three graphs shown in Figure 2.3.

**Proof:** The proof is very long and technical. See [317]. ■

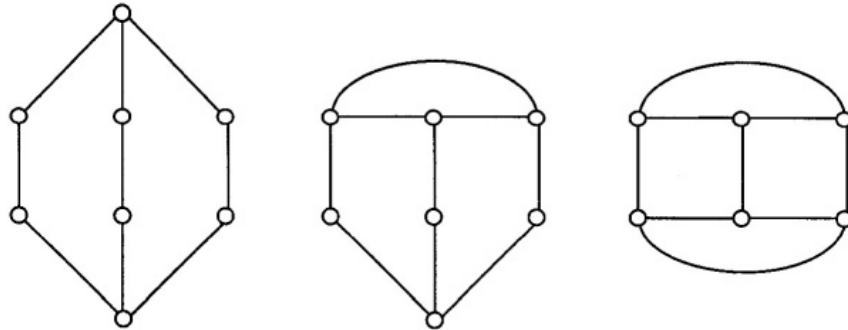


Figure 2.3. Excluded minors for Theorem 15

Graphs without such minors are well characterized, in the sense that they are easy to recognize. There is a small set of *minimal* graphs of the family called *bricks*. Given two graphs of the family, one can compose them in two ways to obtain a new graph of the family. One is by identifying a node of each, the second by identifying two nodes of each. Each non minimal graph of the family is obtained from two smaller ones of the family. Conversely, using disconnecting sets of one or two nodes, we can decompose the graph into smaller graphs recursively. At the end, if all obtained graphs belong to the list of possible starting bricks, then the graph belongs to the family. For more details see [317].

A similar characterization for the convex hull of Hamiltonian cycles is not known. Cornuéjols, Naddef and Pulleyblank, in [220] and [221], give families of graphs, which includes the Halin graphs, for which the convex hull of the Hamiltonian cycles is given by the degree constraints and the subtour elimination inequalities.

There are other known families of TSP instances that can be solved to optimality using only these inequalities in the formulation. But in these families, this is due to the objective function. Papadimitriou and Steiglitz [654] gave a family of TSP instances known as the *Papadimitriou and Steiglitz's traps* designed to defeat any attempt to solve them by a  $k - OPT$  procedure, that is a procedure in which at each iteration one tries to replace a tour by a better one differing from it by at most  $k$  edges. Padberg and T.Y Sung [649] have shown that these graphs always yield an integral solution to the linear program obtained by Constraints (2), (3) and (4). The polytope on these constraints is in general called the *subtour elimination polytope*. The same has been observed by Althaus and Mehlhorn [20] for TSP problems arising in curve reconstruction. It yields a polynomial reconstruction algorithm. These two last examples are studied in Chapter 11. Moreover, in some experimental results, G. Rinaldi has observed that if the distance between cities is independently and uniformly randomly generated in some large interval, optimizing over the subtour elimination polytope almost always yields integer optimal solutions.

We now study more in depth the strong relationship between  $GTSP(n)$  and  $STSP(n)$ .

#### 4. The graphical relaxation Framework

Except when otherwise specified, graphs are assumed to be complete.

### 4.1. General theorems

We introduce first some notation. The same way that we represented the set of all tours of  $K_n$  by  $\mathcal{H}_n$ , we represent the set of all closed walks of  $K_n$  by  $\mathcal{W}_n^*$ . A *minimal closed walk* is a closed walk that does not contain a subfamily of edges which is also a closed walk. In particular, a minimal closed walk never contains an edge more than twice. The converse is of course false. The set of minimal closed walks is represented by  $\mathcal{W}_n$ .

From now on we do not differentiate sets of edges and the vectors that represent them. Therefore when we talk of a tour or a closed walk it is either a set of edges or its representative vector.

Given an inequality  $fx \geq f_0$ , valid for  $STSP(n)$  (resp.  $GTSP(n)$ ), we call *extremal* those tours (resp. minimal closed walks) which satisfy that inequality with equality. We let  $\mathcal{H}_f^-$  (resp.  $\mathcal{W}_f^-$ ) represent the set of extremal tours (resp. minimal walks) with respect to  $fx \geq f_0$ . We will also often say that a tour of  $\mathcal{H}_f^-$  is *tight* for  $fx \geq f_0$ , the same for the closed walks.

A trivial remark is that for any valid inequality  $fx \geq f_0$  of  $GTSP(n)$  we have  $f_e \geq 0$  for all  $e \in E$ . This comes from the fact that if  $f_{e^*} < 0$  for some  $e^* \in E$ , then adding enough copies of  $e^*$  to any closed walk will bring its value below  $f_0$ , contradicting that it is a valid inequality.

For any inequality  $fx \geq f_0$  defined on  $\mathbb{R}^E$  and for each node  $u \in V$ , we define the set  $\Delta_f(u)$  by :

$$\Delta_f(u) = \{(v, w) \in E : v \neq u, w \neq u, f_{vw} = f_{uv} + f_{uw}\}. \quad (11)$$

The set  $\Delta_f(u)$  plays a central role in our study of  $STSP(n)$  and  $GTSP(n)$ .

Another central notion in the study of  $GTSP(n)$  is that of *tight triangular inequality*.

**Definition 16** *An inequality  $fx \geq f_0$  defined on  $\mathbb{R}^E$  is said to be tight triangular or in tight triangular form if the following conditions are satisfied:*

- (a) *The coefficients  $f_e$  satisfy the triangular inequality, i.e.  $f_{uv} \leq f_{uw} + f_{vw}$  for each triplet  $\langle u, v, w \rangle$  of distinct nodes of  $V$ .*
- (b)  *$\Delta_f(u) \neq \emptyset$  for all  $u \in V$*

We will abbreviate “tight triangular” by “TT”. The following theorem shows that almost all facet defining inequalities for  $GTSP(n)$  are tight triangular.

**Theorem 17** *A facet defining inequality  $fx \geq f_0$  for  $GTSP(n)$  falls in one of the following three categories:*



- (i) trivial inequalities  $x_e \geq 0$  for all  $e \in E$
- (ii) degree inequality  $x(\delta(v)) \geq 2$  for all  $v \in V$
- (iii) tight triangular inequalities

**Proof:** Let  $fx \geq f_0$  be a facet defining inequality for  $GTSP(n)$ . Suppose it does not satisfy condition (a) of Definition 16. Then there is a triplet of vertices  $u, v, w$  such that  $f_{uv} > f_{uw} + f_{vw}$ . If there is a walk  $W_{uv} \in \mathcal{W}_f^-$  containing the edge  $(u, v)$ , then the closed walk  $W' = W_{uv} + \{(u, w)\} + \{(w, v)\} - \{(u, v)\}$  is such that  $fx^{W'} < f_0$ , which contradicts the fact that the inequality is valid. Therefore no closed walk of  $\mathcal{W}_f^-$  contains the edge  $(u, v)$  and the facet is that defined by  $x_{uv} \geq 0$ . Now assume that the inequality satisfies (a) but not (b) of Definition 16. Therefore there exists  $u \in V$  such that for all pairs of distinct nodes  $v$  and  $w$  of  $V \setminus \{u\}$ , we have  $f_{vw} < f_{uv} + f_{uw}$ . If there is a closed walk  $W \in \mathcal{W}_f^-$  such that the degree of  $u$  in  $W$  is at least 4, then there always exists two neighbors  $t$  and  $z$  (see Figure 2.4) of  $u$  on  $W$  such that  $W' = W + \{(t, z)\} - \{(u, t), (u, z)\}$  is also a closed walk (if  $(u, t)$  or  $(u, z)$  appears more than once in  $W$ , we only remove one copy). We have by hypothesis, that  $fx^{W'} < f_0$ , contradicting the validity of the inequality. Therefore all closed walks  $W \in \mathcal{W}_f^-$  which do satisfy condition (a) and not condition (b) satisfy  $x(\delta(u)) = 2$  for some  $u \in V$ . ■

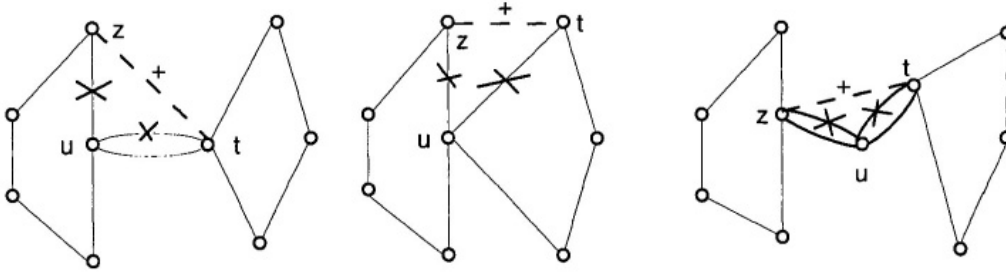


Figure 2.4. Examples of "shortcuts"

**Corollary 18** Let  $fx \geq f_0$  be a facet defining TT inequality for  $GTSP(n)$ . Then

- (a) for every edge  $e \in E$  there exists a closed walk  $W \in \mathcal{W}_f^-$  such that  $e \in W$
- (b) for every node  $v \in V$  there exists a closed walk  $W \in \mathcal{W}_f^-$  such that  $v$  has degree at least 4 in  $W$ .

**Remark 19** All facet defining TT inequalities  $fx \geq f_0$  for  $GTSP(n)$  have the following structural property. There exists a unique partition  $V_1, \dots, V_j, \dots, V_k$  of the vertex set  $V$  such that  $f_e = f_{ij} > 0$  for all  $e \in (V_i : V_j)$ ,  $i \neq j$ , and  $f_e = 0$  if and only if  $e \in E(V_j)$  for some  $j$ ,  $1 \leq j \leq k$ .

**Proof:** Let  $V_1, \dots, V_j, \dots, V_k$  be the vertex sets of the connected components of the graph  $G^0 = (V, E^0)$ , where  $E^0 = \{e \in E : f_e = 0\}$ . Let  $e \in E(V_j)$  and  $W_e \in \mathcal{W}_f^-$  such that  $e \in W_e$ . Assume  $f_e > 0$ . There is a path in  $V_j$  linking the extremities of  $e$  and for which all the edges have a coefficient of 0. The closed walk  $W^*$  obtained from  $W_e$  by removing  $e$  and by adding the edges of that path is such that  $fx^{W^*} < f_0$ , which contradicts the validity of  $fx \geq f_0$ . Now let  $e$  and  $e'$  be two edges of  $(V_i : V_j)$  and assume  $f_{e'} < f_e$ . Let  $W_e \in \mathcal{W}_f^-$  such that  $e \in W_e$ . The closed walk  $W^*$  obtained from  $W_e$  by removing  $e$  and adding the edges of a path in  $V_i$  from the extremity of  $e$  to the one of  $e'$  it contains, the edge  $e'$  and the edges of a path in  $V_j$  from the extremity of  $e'$  to the one of  $e$  it contains, does not satisfy the inequality  $fx \geq f_0$ . Note that the existence of  $W_e$  in all cases is guaranteed since otherwise the inequality would be  $x_e \geq 0$ , which is not tight triangular. ■

If, in the partition of Remark 19, all the sets have cardinality 1, we say that the inequality  $fx \geq f_0$  is *simple*. As we will see, it is often convenient to study, first, simple inequalities and then to obtain the more general inequalities via the *node lifting* procedures described in Section 4.3.

**Definition 20** For every ordered triplet  $\langle u, t, z \rangle$  of distinct nodes in  $V$ , we call shortcut on  $\langle u, t, z \rangle$  the vector  $s_{utz} \in \mathbb{R}^E$  defined by:

$$s_{utz}(e) = \begin{cases} 1 & \text{if } e = (t, z) \\ -1 & \text{if } e \in \{(u, t), (u, z)\} \\ 0 & \text{otherwise} \end{cases}$$

Let  $W \in \mathcal{W}_f^-$  with strictly more than  $n$  edges. The following lemma will be very useful:

**Lemma 21** Let  $fx \geq f_0$  be a TT inequality supporting for  $GTSP(n)$  and let  $W \in \mathcal{W}_f^-$  be a closed walk with  $t > n$  edges containing a certain edge  $e^*$ . For every vertex  $u$  with degree  $k \geq 4$  in  $W$ , there exists a shortcut  $s_{uvw}$  such that  $x^W + s_{uvw} \in \mathcal{W}_f^-$ , contains  $e^*$  and has  $t - 1$  edges.

**Proof:** Easy, see [618]. ■

**Corollary 22** *Let  $fx \geq f_0$  be a facet defining TT inequality for  $GTSP(n)$ . Then, for each edge  $e \in E$ , there exists  $H \in \mathcal{H}_f^-$  that contains  $e$ .*

The following lemma shows how to transform a facet inducing inequality for  $STSP(n)$  into an equivalent one in TT form.

**Lemma 23** *Let  $hx \geq h_0$  be a facet defining inequality for  $STSP(n)$ . An inequality  $fx \geq f_0$  equivalent to  $hx \geq h_0$  is tight triangular if and only if  $f = \pi A + \pi_0 h$  and  $f_0 = 2 \sum_{u \in W} \pi_u + \pi_0 h_0$  where  $(\pi, \pi_0)$  satisfy*

$$\pi_u = \frac{1}{2} \pi_0 \max\{h(v, w) - h(u, v) - h(u, w) : u, v, w \in V, u \neq v \neq w\} \quad (12)$$

**Proof:** It is straightforward to check that if  $(\pi, \pi_0)$  satisfies Equation (12), the inequality  $fx \geq f_0$  is tight triangular. Suppose that  $fx \geq f_0$  is tight triangular. Condition (a) of the definition of tight triangularity imposes that

$$\pi_u \geq \frac{1}{2} \pi_0 \max\{h(v, w) - h(u, v) - h(u, w) : u, v, w \in V, u \neq v \neq w\} \quad (13)$$

The second condition of that definition implies the existence of a triple of distinct vertices  $u, v, w$  for which (13) holds with equality. ■

A basis of a facet defining inequality  $fx \geq f_0$  for  $GTSP(n)$  is a set  $\mathcal{B}_f$  of  $|E|$  closed walks in  $\mathcal{W}_f^-$  whose incidence vectors are linearly independent. A closed walk  $W$  is *almost Hamiltonian* in  $u$  if  $u$  has degree 4 in  $W$  and all other vertices have degree 2.

**Definition 24** *A basis  $\mathcal{B}_f$  of a facet defining inequality  $fx \geq f_0$  of  $GTSP(n)$  is called canonical if it contains  $|E| - n$  Hamiltonian cycles and  $n$  almost Hamiltonian walks.*

**Theorem 25** *A non-trivial TT inequality  $fx \geq f_0$  which is facet defining for  $STSP(n)$  defines a facet of  $GTSP(n)$ .*

**Proof:** We just show how to build a basis  $\mathcal{B}_f$ . Since the inequality is facet defining for  $STSP(n)$ , there exists  $|E| - n$  linearly independent Hamiltonian cycles in  $\mathcal{H}_f^-$ . We now construct  $n$  almost Hamiltonian walks, one for each  $u \in V$ . Let  $e = (v, w) \in \Delta_f(u)$  and  $\Gamma_e \in \mathcal{H}_f^-$  a Hamiltonian cycle containing  $e$ , which exists as the inequality is not equivalent to the trivial inequality  $x_e \geq 0$ . Let  $W_u = \Gamma_e - \{e\} + \{(u, v), (u, w)\}$ . All we have to prove is that the previously mentioned  $|E| - n$  Hamiltonian cycles together with these  $n$  almost Hamiltonian walks are linearly independent. This can be found in [618]. ■

Note that the facet inducing inequality  $x_e \leq 1$  for  $STSP(n)$  has a TT-form which is  $x(\delta(\{u, v\})) \geq 2$  where  $e = (u, v)$ , which we already know is facet inducing for  $GTSP(n)$ . The following theorem shows the strong relationship between  $STSP(n)$  and  $GTSP(n)$ .

**Theorem 26** *Every non-trivial facet of  $STSP(n)$  is contained in exactly  $n + 1$  facets of  $GTSP(n)$ ,  $n$  of which are defined by the degree inequalities.*

**Proof:** Let  $F$  be a non-trivial facet of  $STSP(n)$ . Then  $F$  belongs to the  $n$  facets of  $GTSP(n)$  defined by the degree inequalities  $x(\delta(\{v\})) \geq 2$ , for all  $v \in V$ . Every other facet of  $GTSP(n)$  containing  $F$  is defined by a TT-inequality. By Lemma 23, there is only one such facet and the theorem follows. ■

This theorem shows that the polyhedral structure of  $STSP(n)$  is very closely related to that of  $GTSP(n)$ .

A corollary of this theorem is that the TT-form is a canonical way of expressing the facets of  $STSP(n)$ . Therefore, together with Lemma 23, it leads to a  $O(n^3)$  algorithm to recognize whether or not two facet inducing inequalities of  $STSP(n)$  define the same facet which, as already mentioned, it is not as good as Margot's or Queyranne and Wang's  $O(n^2)$  algorithms.

We will study the polyhedral structure of  $GTSP(n)$  in more detail in the next sections. At this point, the reader may want to skip the rest of this section which addresses very technical aspects of the study of  $STSP(n)$ . In particular, we address the problem of finding sufficient conditions for a facet defining inequality of  $GTSP(n)$  to define a facet of  $STSP(n)$ . Note that, at the time of writing this chapter, no tight triangular facet inducing inequality for  $GTSP(n)$  is known that is not also one for  $STSP(n)$  (except of course the degree inequalities).

From Definition 24, a facet inducing TT inequality for  $GTSP(n)$  defines a facet for  $STSP(n)$  if and only if it has a canonical basis. In the following we investigate sufficient conditions for the existence of a canonical basis. The following remark is central in the approach to such conditions.

**Remark 27** *Let  $fx \geq f_0$  be a TT facet defining inequality for  $GTSP(n)$  and  $\mathcal{B}_f$  be one of its bases. Let  $\{W_u : u \in V\}$  be a set of  $n$  almost Hamiltonian walks of  $\mathcal{W}_f^-$ , where  $W_u$  is almost Hamiltonian in  $u$ . If every closed walk of  $\mathcal{B}_f$  can be reduced to a cycle of  $\mathcal{H}_f^-$  by using only shortcuts obtained by a linear combination of the incidence vectors of elements of  $\mathcal{A}_f = \mathcal{H}_f^- \cup \{W_u : u \in V\}$ , then  $\mathcal{A}_f$  contains a canonical basis of  $fx \geq f_0$ . That is, all the incidence vectors of the closed walks  $T$  of  $\mathcal{B}_f$  can be expressed as a linear combination of the incidence vectors*

of elements in  $\mathcal{A}_f$ . In general, it is sufficient that every shortcut can be obtained as a linear combination of the incidence vectors of elements in  $\mathcal{A}_f$  with coefficients in  $\mathbb{R}$ . To obtain simple sufficient conditions we will restrict the coefficients to  $\{-1, 0, 1\}$ .

Let  $e = (u, v)$  and  $e' = (w, y)$  be two distinct edges of  $E$ . We say that  $e$  and  $e'$  are  $f$ -adjacent if there exists a Hamiltonian cycle in  $\mathcal{H}_f^-$  containing both  $e$  and  $e'$ . Let  $z$  be a vertex in  $V$ , we say that  $e$  and  $e'$  are  $f$ -adjacent in  $z$  if:

- (i)  $e$  and  $e'$  belong to  $\Delta_f(z)$ ;
- (ii) There exists a closed walk  $W_z \in \mathcal{W}_f^-$  almost Hamiltonian in  $z$  that contains  $(z, u)$ ,  $(z, v)$ ,  $(z, w)$  and  $(z, y)$ ;
- (iii)  $W_z - \{(z, u), (z, v)\} + \{e\}$  is a Hamiltonian cycle (and therefore so is  $W_z - \{(z, w), (z, y)\} + \{e'\}$ ).

A set of edges  $J \subset E$  is said to be  $f$ -connected if for every pair of distinct edges  $e_1, e_2 \in J$ , there exists a sequence of edges  $e'_1 = e_1, e'_2, \dots, e'_{k-1}, e'_k = e_2$  in  $J$ , such that for  $i = 1, \dots, k-1$ ,  $e'_i$  is  $f$ -adjacent to  $e'_{i+1}$ . A set of edges  $J \subset E$  is said to be  $f$ -connected in  $z$  if for every pair of distinct edges,  $e_1$  and  $e_2 \in J$ , there exists a sequence of edges  $e'_1 = e_1, e'_2, \dots, e'_{k-1}, e'_k = e_2$  in  $E$  (not necessarily in  $J$ ), such that for  $i = 1, \dots, k-1$ ,  $e'_i$  is  $f$ -adjacent in  $z$  to  $e'_{i+1}$ . Observe that the notion of  $f$ -connectivity in  $z$  is weaker than that of  $f$ -connectivity. Any subset of a  $f$ -connected set in  $z$  is also  $f$ -connected in  $z$ .

**Lemma 28** *Let  $fx \geq f_0$  be a TT facet defining inequality of  $GTSP(n)$ . If  $\Delta_f(u)$  is  $f$ -connected in  $u$  for every  $u \in V$ , then  $fx \geq f_0$  has a canonical basis, hence it is facet defining for  $STSP(n)$ .*

**Proof:** Let  $u \in V$  and  $\{W_u : u \in V\}$  be any set of  $n$  almost Hamiltonian walks of  $\mathcal{W}_f^-$ . This set always exists, by Corollary 22, since  $fx \geq f_0$  is tight triangular it can be constructed as in the proof of Theorem 25. By Lemma 21 there exists a shortcut  $s_{u\bar{y}\bar{z}}$  with  $(\bar{y}, \bar{z}) \in \Delta_f(u)$  that can be used to reduce  $W_u$  to a Hamiltonian cycle  $\Gamma \in \mathcal{H}_f^-$ . We therefore have  $s_{u\bar{y}\bar{z}} = x^\Gamma - x^{W_u}$ , hence  $s_{u\bar{y}\bar{z}}$  is a linear combination with coefficients in  $\{-1, 1\}$  of incidence vectors of elements from  $\mathcal{A}_f = \mathcal{H}_f^- \cup \{W_u : u \in V\}$ . If  $|\Delta_f(u)| = 1$ , we are done. Else let  $e = (v, w)$  and  $e' = (y, z)$  be two distinct edges of  $\Delta_f(u)$  which are  $f$ -adjacent in  $u$ . Let us assume that the shortcut  $s_{uyz}$  is a linear combination with coefficients in  $\{-1, 1\}$  of incidence vectors of elements from  $\mathcal{A}_f$ . Since  $e$  and  $e'$  are  $f$ -adjacent in  $u$ , there exist a closed walk  $W'_u \in \mathcal{W}_f^-$  almost Hamiltonian in  $u$  which contains  $(u, v)$ ,  $(u, w)$ ,  $(u, y)$  and  $(u, z)$ . We have  $\Gamma_1 =$

$W'_u - \{(u, y), (u, z)\} + \{(y, z)\} \in \mathcal{H}_f^-$  and  $\Gamma_2 = W'_u - \{(u, v), (u, w)\} + \{(v, w)\} \in \mathcal{H}_f^-$ , by triangular inequality. The shortcut  $s_{uvw}$  can be expressed as:  $s_{uvw} = x^{\Gamma_2} - x^{\Gamma_1} + s_{uyz}$ , hence it is a linear combination with coefficients in  $\{-1, 1\}$  of incidence vectors of closed walks from  $\mathcal{A}_f$ . By the assumption that  $\Delta_f(u)$  is  $f$ -connected in  $u$ , it follows that for all  $u \in V$  and all  $(v, w) \in \Delta_f(u)$ , the shortcuts  $s_{uvw}$  can be expressed as a linear combination of the incidence vectors of elements in  $\mathcal{A}_f$ , and by Remark 27 the lemma follows. ■

The conditions of this lemma are too restrictive in general. We give now a weaker version, which is the one that is in general used in proving facet inducing results for  $STSPP(n)$  from results for  $GTSP(n)$ .

**Lemma 29** *Let  $fx \geq f_0$  be a TT facet defining inequality of  $GTSP(n)$ . If there exists a basis  $\mathcal{B}_f$  of  $fx \geq f_0$  and for every  $u \in V$  there exists a nonempty set of edges  $J_u \subseteq \Delta_f(u)$ ,  $f$ -connected in  $u$ , such that every closed walk  $W \in \mathcal{B}_f$  can be reduced to a tour of  $\mathcal{H}_f^-$  by using only shortcuts from the set  $\{s_{uvw} : (v, w) \in J_u, u \in V\}$ , then  $fx \geq f_0$  has a canonical basis, hence it is facet defining for  $STSPP(n)$ .*

**Proof:** For every  $u \in V$ , let  $(v, w)$  be any edge in  $J_u$ . By Corollary 22 there exists  $\Gamma \in \mathcal{H}_f^-$  containing the edge  $(v, w)$ . Let  $W_u$  be the almost Hamiltonian walk defined by  $W_u = \Gamma - (v, w) + (u, v) + (u, w)$ . By Remark 27 and a process analogous to that of the proof of the preceding lemma, it follows that the set  $\mathcal{A}_f = \mathcal{H}_f^- \cup \{W_u : u \in V\}$  contains a canonical basis for  $fx \geq f_0$  and the lemma follows. ■

We now turn to the problem of deriving facet inducing inequalities for  $GTSP(n)$  and for  $STSPP(n)$  from other such inequalities. The first way we will study is via *composition* of two facet inducing inequalities, the second will be via what we will call *node lifting*.

## 4.2. Composition of inequalities

In [616], a composition of valid inequalities known as the *s-sum* is defined. We will focus here on the case of *2-sums*. For sake of simplicity we will assume that the inequalities  $fx \geq f_0$  we deal with are *simple*, that is  $f_e > 0$  for all  $e \in E$ .

For the reader with some knowledge of the traveling salesman polytope, the *2-sum* composition we describe here, can be used to build clique tree inequalities from comb inequalities.

We say that two edge weighted graphs  $G^1 = (V^1, E^1, f^1)$  and  $G^2 = (V^2, E^2, f^2)$  are *isomorphic* if there exists a one to one correspondence between their vertex sets that preserves the weights on the edges.

**Definition 30** Let  $f^1x \geq f_0^1$  and  $f^2x \geq f_0^2$  be two TT inequalities valid for  $GTSP(n_1) = (V^1, E^1)$  and  $GTSP(n_2) = (V^2, E^2)$ , respectively. Let  $e_1 = (u_1, v_1) \in E_1$  and  $e_2 = (u_2, v_2) \in E_2$  be two edges such that  $f_{e_1}^1 = f_{e_2}^2 = \epsilon > 0$ . Then a 2 – sum inequality obtained by identifying  $u_1$  with  $u_2$  (called now  $u$ ) and  $v_1$  with  $v_2$  (called now  $v$ ) is the inequality  $fx \geq f_0^1 + f_0^2 - 2\epsilon$  defined on  $G = K_n$  with  $n = n_1 + n_2 - 2$  as follows:

- 1  $V_n = (V^1 \setminus \{u_1, v_1\}) \cup (V^2 \setminus \{u_2, v_2\}) \cup \{u, v\}$
- 2 The weighted subgraph  $G(V^i)$  is isomorphic to the weighted graph  $G^i$ , with  $u$  and  $v$  corresponding to  $u_i$  and  $v_i$  respectively, for  $i = 1, 2$ .
- 3 The coefficients of the edges with one endpoint in  $V^1$  and the other in  $V^2$ , that we call the crossing edges of the 2 – sum, are computed in the following way
  - (a) Order the crossing edges  $e_1, \dots, e_j, \dots, e_k$
  - (b) For  $j = 1$  to  $k$ , let  $T^j$  be a minimum  $f$ -length closed walk among all closed walks of  $G$  containing  $e_j$  that uses only edges from  $E_1 \cup E_2 \cup \{e_1, \dots, e_j\}$ . Let  $f_{e_j}$  be such that the  $f$ -length of  $T^j$  is  $f_0^1 + f_0^2 - 2\epsilon$ .

The condition  $f_{e_1}^1 = f_{e_2}^2$  is not restrictive since we can always scale one of the inequalities in order to obtain this condition.

The procedure described at point 3 of Definition 30 is called *sequential lifting* in the literature (see Padberg [640]). In general the coefficients of the crossing edges depend on the lifting sequence. If it is not the case we say that the 2 – sum is *stable*. Most of the known inequalities that come from 2 – sums are stable.

If, in the sequential lifting, the coefficients of the crossing edges are such that the minimum over all closed walks is the same as the minimum over all Hamiltonian cycles, then the 2 – sum inequality is called *h-liftable*.

For the sake of simplicity, when obvious, we do not mention the correspondence between the two composing graphs and the corresponding elements of the isomorphic subgraphs of the 2 – sum.

Let  $fx \geq f_0$  be an inequality supporting for  $GTSP(n)$ . A vertex  $v \in V$  is said to be *k-critical* for that inequality if the  $f$ -length of a minimum  $f$ -length closed walk of  $K_n \setminus \{v\}$  is  $f_0 - k$ .

**Remark 31** In the previous definition  $k$  must satisfy  $k \leq 2 \cdot \min\{f_e : e \in \delta(v)\}$ . We will only consider two types of vertices, the 0-critical and the  $\bar{k}$ -critical with  $\bar{k} = 2 \cdot \min\{f_e : e \in \delta(v)\}$

We assume the 2 – sum is performed as described in Definition 30 and the notation is the one used in that definition. The following theorem, the proof of which can be found in [618], states conditions under which two facet inducing inequalities yield, by 2 – sum, another facet inducing inequality.

**Theorem 32** *Let  $f^1x \geq f_0^1$  and  $f^2x \geq f_0^2$  be two TT facet inducing inequalities for  $GTSP(n_1)$  and  $GTSP(n_2)$ , respectively. The 2–sum inequality  $fx \geq f_0$  is facet defining for  $GTSP(n)$  if  $v_1$  is  $2\epsilon$ -critical for  $f^1x \geq f_0^1$  and at least one of the two vertices  $u_2$  and  $v_2$  is  $2\epsilon$ -critical for  $f^2x \geq f_0^2$ .*

The next theorem is the corresponding theorem for  $STSP(n)$ .

**Theorem 33** *Let  $f^1x \geq f_0^1$  and  $f^2x \geq f_0^2$  be two TT facet inducing inequalities of  $STSP(n_1)$  and  $STSP(n_2)$ , respectively. The 2 – sum inequality  $fx \geq f_0$  is facet defining for  $STSP(n)$  if it is  $h$ -liftable and:*

(a)  $v_1$  is  $2\epsilon$ -critical for  $f^1x \geq f_0^1$ ,

(b)  $\delta(u_2)$  is  $f^2$ -connected,

and either (Case (A))

(c')  $u_2$  is  $2\epsilon$ -critical for  $f^2x \geq f_0^2$ ,

(d')  $\delta(v_1)$  is  $f^1$ -connected,

or (Case (B))

(c'')  $v_2$  is  $2\epsilon$ -critical for  $f^2x \geq f_0^2$ ,

(d'')  $\delta(u_1)$  is  $f^1$ -connected,

(e'') there exists a Hamiltonian cycle  $H_1 \in \mathcal{H}_{f^1}^{\equiv}$  containing edge  $(u_1, v_1)$  and any edge  $e_1 \in \Delta_{f^1}(v_1)$ ,

(f'') there exists a Hamiltonian cycle  $H_2 \in \mathcal{H}_{f^2}^{\equiv}$  containing edge  $(u_2, v_2)$  and any edge  $e_2 \in \Delta_{f^2}(v_2)$ ,

**Proof:** See [618] ■

The 2 – sum procedure can be carried out recursively. The only difficulty is to keep track of the “criticality” of the vertices after the composition. Various technical lemmas dealing with this aspect are given in [618].



### 4.3. Node lifting

As mentioned earlier, when studying a given family of facet inducing inequalities for STSPP(n), it is easier to first study the simple inequalities of the class. Then the result is generalized (*lifted*) to the non simple inequalities of that family. The tools to obtain these results are the aim of this section. Remark 19 states that a facet inducing inequality in TT form is such that the components of the subgraph on the edges with zero coefficient are cliques. This may suggest that the only *lifting* operation is the replacement of nodes by cliques. This is not the case. A comb (see next section) with no node inside the handle not contained in any tooth yields a facet inducing inequality. The same comb with a node in the handle not in any tooth also does.

Let  $fx \geq f_0$  be a simple valid inequality for  $GTSP(n)$ . An inequality  $f^*x \geq f_0$  (same right hand side) is obtained by *clique lifting* from a simple one if each node  $u \in V$  is replaced by a clique  $K_{k_u}^u = (V^u, E^u)$  of  $k_u \geq 1$  vertices and:

- 1  $f_e^* = 0$  for all  $e \in E^u$  for some  $u \in V$ ,
- 2 for all  $e \in (V^u : V^v)$ ,  $f_e^* = f_{uv}$ , for all  $u, v \in V$ .

That is all the edges of the newly formed cliques get a coefficient of zero, the edges between two newly formed cliques inherit the coefficient of the corresponding edge in the original graph.

**Theorem 34** *Let  $fx \geq f_0$  be a TT facet inducing inequality of  $GTSP(n)$ . Any inequality obtained from  $fx \geq f_0$  by clique lifting is also facet inducing for  $GTSP(n^*)$ , where  $n^* > n$  is the number of vertices of the resulting graph.*

**Proof:** The inequality is obviously valid since any closed walk on the “extended” graph induces a closed walk on  $K_n$ . It is supporting since any closed walk  $T$  of  $K_n$  can be extended to a closed walk  $T'$  on the extended graph with  $f(T) = f^*(T')$ . Assume  $f^*x \geq f_0$  is not facet inducing. Since it is valid and supporting, there is a facet that contains the face it induces. Let  $gx \geq f_0$  be a linear representation of that facet (by scaling we can always assume the right hand side to be  $f_0$ ). Note that we have  $\mathcal{W}_{f^*}^- \subset \mathcal{W}_g^-$ . Let  $e \in \gamma(V_u)$  for some  $u \in V_n$  and  $T' \in \mathcal{W}_{f^*}^-$ , so  $T' \in \mathcal{W}_g^-$ . The closed walk  $T' + 2\{e\}$  is also in  $\mathcal{W}_{f^*}^-$  and therefore in  $\mathcal{W}_g^-$ , which implies that  $g_e = f_e^* = 0$ . We are therefore left with the edges which correspond to some edge of the simple inequality, and proving that  $g_e = f_e^* = f_e$  for these amounts to proving that the simple inequality is facet inducing. ■

Things are not that easy for  $STSPP(n)$ . The clique lifting can be seen as a recursive application of an operation we will call *zero-lifting of a node*. We define now what we mean by *node lifting*.

Let  $K_n = (V_n, E_n)$  and  $K_{n^*} = (V_{n^*}, E_{n^*})$ ,  $n^* \geq n$  be two complete graphs. Let  $V_n = \{u_1, u_2, \dots, u_n\}$  and  $V_{n^*} = V_n \cup \{u_{n+1}, \dots, u_{n^*}\}$ . We say that the inequality  $f^*x \geq f_0^*$  defined on  $\mathbb{R}^{E_{n^*}}$  has been obtained by *node-lifting* of  $fx \geq f_0$  defined on  $\mathbb{R}^{E_n}$  if

$$f_{(u_i u_j)}^* = f_{(u_i u_j)} \text{ for all } 1 \leq i < j \leq n.$$

The special case where  $f_{(uv)}^* = 0$  for some  $u \in V_n$  and all  $v \in \{u_{n+1}, \dots, u_{n^*}\}$  is called *zero-lifting of node u*. Note that by triangularity one then have  $f_{(vw)}^* = 0$  for all  $v, w \in \{u_{n+1}, \dots, u_{n^*}\}$ . For a TT inequality obtained by zero-lifting, the following proposition is a direct consequence of Remark 19.

**Proposition 35** *For any facet inducing TT inequality  $f^*x \geq f_0$  obtained from  $fx \geq f_0$  by zero-lifting of node  $u$ , the following holds:*

- 1  $f_{(vw)}^* = f_{(vu)}$  for all  $v \in V_n - \{u\}$  and  $w \in \{u_{n+1}, \dots, u_{n^*}\}$ ,
- 2  $\Delta_{f^*}(w) = \Delta_f(u) \cup \bigcup \{\delta(w') \setminus \{(w', w)\} : w' \in \{u, u_{n+1}, \dots, u_{n^*}\}, w' \neq w\}$  for all  $w \in \{u_{n+1}, \dots, u_{n^*}\}$ , where  $\delta(w')$  is a taken in  $E_{n^*}$

There are node-liftings which are not zero-liftings. For example the inequality of Figure 2.5 (b) is what is called in [618] a *1-node lifting* of the inequality of Figure 2.5 (a), and that of Figure 2.5 (c) a *1-node-lifting* of that of Figure 2.5 (b). In that figure the coefficient of the shown edges is the one shown; for the others the coefficient is that of a shortest path, in term of coefficient of the shown edges, between their end point. For example, in Figure 2.5 (a) a non shown edge linking a node of the top cycle to one of the bottom cycle and not shown has a coefficient of 3, since a shortest path between its extremities consists of an horizontal edge and a vertical one. All right hand sides are 10. Note that these three inequalities are what we will call later on *comb inequalities*.

A *1-node lifting* of an inequality  $fx \geq f_0$  defined on  $\mathbb{R}^{E_n}$  is an inequality  $f^*x \geq f_0$  (same right hand side) defined on  $\mathbb{R}^{E_{n+1}}$  and which is not a zero-lifting. The coefficients  $f_{vu_{n+1}}^*$  for  $v \in V_n$ ,  $u_{n+1}$  being the new vertex, are not defined by this definition. They must satisfy the following necessary conditions.

**Theorem 36** *Let  $f^*x \geq f_0^*$  be a facet defining inequality for  $GTSP(n+1)$  which is obtained by 1-node lifting of a TT facet inducing inequality for  $GTSP(n)$ ; then the following conditions hold:*

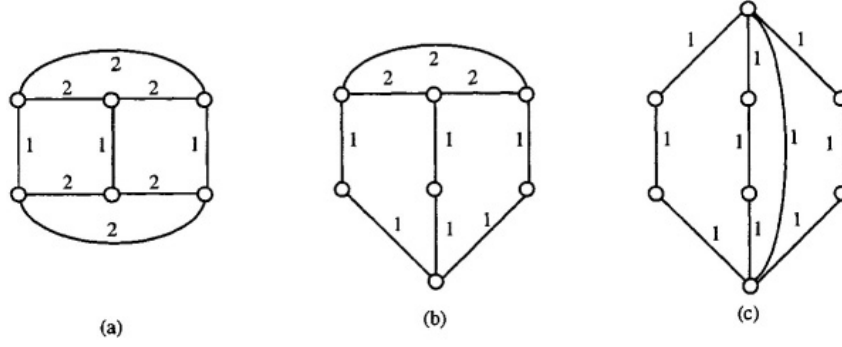


Figure 2.5. Examples of 1-node liftings

- 1  $f^*x \geq f_0^*$  is tight triangular,
- 2  $f_0^* = f_0$
- 3 for all  $e \in \Delta_{f^*}(u_{n+1})$  there exist  $e' \neq e, e' \in \Delta_{f^*}(u_{n+1})$  and  $\Gamma \in \mathcal{H}_f^-$  such that both  $e$  and  $e'$  belong to  $\Gamma$ .
- 4 every connected component of the graph  $(V_n, \Delta_{f^*}(u_{n+1}))$  contains at least one odd cycle.

**Proof:** By Theorem 17, the only non TT facet inducing inequalities for  $GTSP(n+1)$  are the non negativity constraints or the degree constraints, and none can be obtained by 1-node lifting, so Statement 1 follows. We can only have  $f_0^* \geq f_0$ . Since the inequality is TT, let  $e \in \Delta_{f^*}(u_{n+1})$ . There is a closed walk  $W$  of  $\mathcal{W}_f^-$  that contains  $e = (u, v)$ , and  $W' = W - \{e\} + \{(u, u_{n+1}), (v, u_{n+1})\}$  is such that  $f^*(W') = f_0$ , which implies  $f_0^* = f_0$ . For the rest see [618]. ■

Facet inducing results on 1-node lifting for  $GTSP(n)$  and  $STSP(n)$  can be found in [618]. One can also find there some results on zero-liftings for  $STSP(n)$  (we have settled here the case of  $GTSP(n)$ )

We now give a 2-node lifting procedure, known as *edge cloning*, which concerns two vertices and that cannot be obtained by successive 1-node liftings.

Let  $fx \geq f_0$  be a TT inequality defined on  $\mathbb{R}_n^E$  and  $e \in E_n$ . We say that the inequality  $f^*x \geq f_0^*$  defined on  $\mathbb{R}_{n+2h}^E$ , with  $h \geq 1$ , is obtained from  $fx \geq f_0$  by cloning  $h$  times the edge  $e$  if it is obtained by node lifting of  $fx \geq f_0$  and, assuming without loss of generality that  $e = (u_{n-1}, u_n)$ :

$$f_0^* = f_0 + 2hf_e,$$

$$f_{u_i u_{n+j}}^* = \begin{cases} f_{u_i u_{n-1}} & \text{for } 1 \leq i \leq n-2, 1 \leq j \leq 2h-1 \text{ and } j \text{ odd,} \\ f_{u_i u_n} & \text{for } 1 \leq i \leq n-2, 1 \leq j \leq 2h-1 \text{ and } j \text{ even,} \end{cases}$$

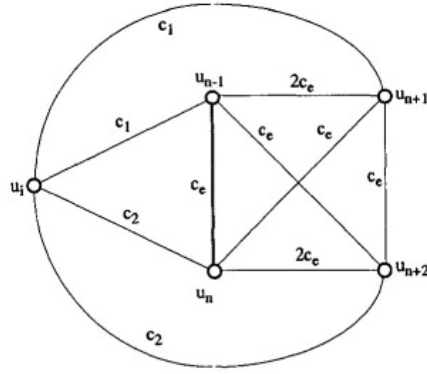


Figure 2.6. Examples of edge cloning

$$f_{u_{n+i}u_{n+j}}^* = \begin{cases} 2f_e & \text{for } -1 \leq i < j \leq 2h \text{ and } j-i \text{ even,} \\ f_e & \text{for } -1 \leq i < j \leq 2h \text{ and } j-i \text{ odd,} \end{cases}$$

Figure 2.6 shows the coefficients of the inequality with edge  $e$  cloned and  $h = 1$ . Let  $fx \geq f_0$  be a TT inequality valid for  $GTSP(n)$ . We say that  $e = (u, v)$  is  $f$ -clonable if:

- 1  $u$  and  $v$  are  $2f_e$ -critical for  $fx \geq f_0$ ,
- 2 the  $f$ -length of any closed walk  $W$  of  $K_n$  is at least  $f_0 + (d-2)f_e$  where  $d = \min(|(\delta(u) \cap T)|, |(\delta(v) \cap T)|)$

This definition is more restrictive than that given in [618]. The first condition is not required there, but is required in all the subsequent theorems. In Figure 2.7 the coefficients of the edges are shown next to them, if not equal to 1. The edges not shown have a coefficient equal to that of a shortest path, with edges among those shown, between their extremities. The first inequality is a comb inequality, the right hand side is 10, only the three edges in bold are  $f$ -clonable. The other edges in that example are not  $f$ -clonable because the six vertices incident to the bold edges are 2-critical and the other two are 0-critical, therefore no other edge has its two extremities  $2f_e$ -critical. An example of violation of the second condition is given in the second example (the right hand side is 14). The second inequality is what we will call, later on, a path inequality. None of the edges is  $f$ -clonable. All the vertices except the top and bottom one are 2-critical, the other two vertices being 0-critical. In particular, vertices  $u$  and  $v$  (in black) are 2-critical. The shown closed walk  $W$  has  $f$ -value  $14 = f_0$  and both extremities of  $e$  have degree 4 in  $W$ , which shows that edge  $e$  is not clonable. The case of the other edges follows by symmetry.

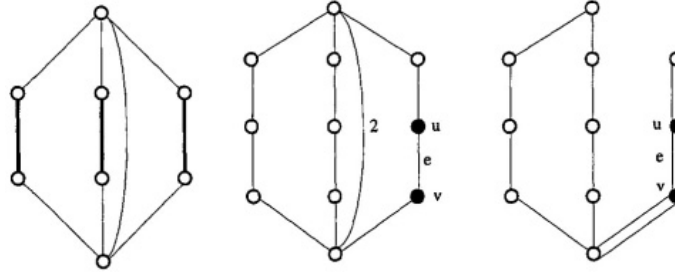


Figure 2.7. Examples of clonable and non clonable edges

**Theorem 37** Let  $fx \geq f_0$  be a TT facet inducing inequality for  $GTSP(n)$  and  $e = (u_{n-1}, u_n)$  be a  $f$ -clonable edge. Then the following holds:

- 1 the inequality  $f^*x \geq f_0^*$ , obtained from  $fx \geq f_0$  by cloning  $h > 1$  times edge  $e$ , is facet inducing for  $GTSP(n + 2h)$ ,
- 2 the edges of  $(\{u_{n-1}, u_{n+1}, \dots, u_{n+2h-1}\} : \{u_n, u_{n+2}, \dots, u_{n+2h}\})$  are  $f^*$ -clonable,
- 3 if  $e' = (z_1, z_2) \neq e$  is an edge in  $E_n$  such that  $z_1$  and  $z_2$  are  $2f_{e'}$ -critical for  $fx \geq f_0$ , then  $z_1$  and  $z_2$  are  $2f_{e'}^*$ -critical for  $f^*x \geq f_0^*$ .

**Proof:** See the proofs of Lemma 4.11 and Theorem 4.12 in [618]. ■

Note that Statement 2 implies that the “new” vertices are  $2f_e^*$ -critical. The purpose of Statements 2 and 3 is to fix the status of the various vertices and edges in view of further edge clonings. The corresponding theorem for  $STSP(n)$  is:

**Theorem 38** Let  $fx \geq f_0$  be a non-trivial TT facet inducing inequality for  $STSP(n)$  and let  $e = (u_{n-1}, u_n)$  be a  $f$ -clonable edge. Then the inequality  $f^*x \geq f_0^*$  obtained from  $fx \geq f_0$  by cloning  $h > 1$  times edge  $e$  is facet inducing for  $STSP(n + 2h)$

**Proof:** See the proof of Theorem 4.13 in [618]. ■

The family of chain inequalities defined by Padberg and Hong [643] can be obtained from the comb inequalities by edge cloning.

We now turn to the study of specific classes of facet inducing inequalities of  $GTSP(n)$  and of  $STSP(n)$ .

## 5. The Comb inequalities

Comb inequalities were first discovered, in a more restrictive form, by Chvátal [195] who derived them from the 2-matching constraints of

Edmonds [265]. They were generalized to the current form and shown to be facet inducing for  $STSP(n)$  by Grötschel and Padberg [403].

A *comb inequality* is usually defined by a set  $H \subset V$ , called *handle*, and an odd number  $t \geq 3$  of vertex subsets  $\{T_1, T_2, \dots, T_t\}$ , called *teeth*, such that:

$$H \cap T_i \neq \emptyset \quad \text{for } i = 1, \dots, t \quad (14)$$

$$T_i \setminus H \neq \emptyset \quad \text{for } i = 1, \dots, t \quad (15)$$

$$T_i \cap T_j = \emptyset \quad \text{for } 1 \leq i < j \leq t \quad (16)$$

Conditions (14) and (15) say that every tooth  $T_i$  intersects properly the handle  $H$ , Condition (16) states that no two teeth intersect.

The corresponding *comb inequality* is:

$$x(\delta(H)) + \sum_{j=1}^t x(\delta(T_j)) \geq 3t + 1 \quad (17)$$

**Remark 39** *Comb inequalities are better known in the following form:*

$$x(\gamma(H)) + \sum_{i=1}^t x(\gamma(T_i)) \leq |H| + \sum_{i=1}^t (|T_i| - 1) - (t + 1)/2 \quad (18)$$

*This form still has a few adepts because:*

- *It is a rank type inequality for the independence system associated with the monotonization of the polytope we mentioned earlier.*
- *It is a mod-2 cut, fact which is used in some separation routines (see Caprara, Fischetti and Letchford [161], Letchford [559]).*
- *It is sometimes sparser than the other form, fact which is useful when solving large linear programs.*

*On the other hand it is not in TT form. We will never use this form.*

Note that Inequality (17) is closed under taking complements of sets since  $x(\delta(S)) = x(\delta(V \setminus S))$  for all proper subset  $S$  of  $V$ . The definition of a comb inequality is not very satisfactory since Condition (16) is not closed under taking complements. There is no real way around this problem as long as we do not deal directly with the edge sets. The most satisfactory one is to require that the given sets satisfy the conditions after eventually replacing some of them by their complements. Figure 2.8 gives two sets  $S$  that give the same comb inequality, the first is a set as described in (14) to (16), the other with  $T_3$  replaced by its complement.

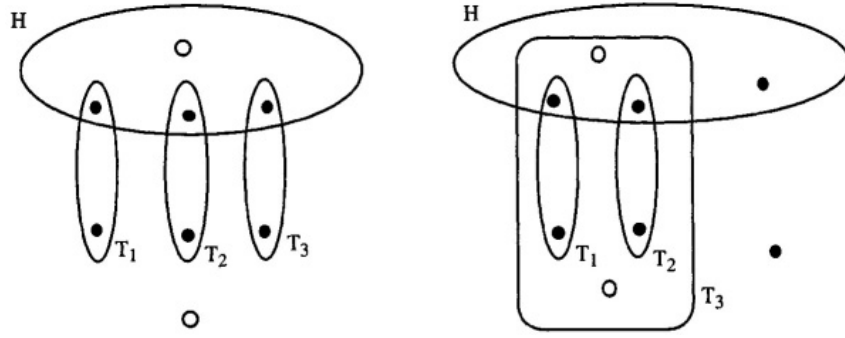


Figure 2.8. Example of a comb ( $t = 3$ )

Figure 2.8 enables us to define our convention for figures that will hold throughout this chapter. Sets with a black point have to be nonempty, those with a white filled point may or may not have nodes in them, those with no points must be empty. A point does not represent a unique node but a set of nodes in that position.

We will give three proofs of validity of comb inequalities, an algebraic (classical) one and two others that will give more insight into what these inequalities say in term of closed walks. We have so far used the term *tight* in various ways. We now use it for sets and will use it frequently. A subset  $S \subset V$  is said to be *tight* for a Hamiltonian cycle or a closed walk  $\Gamma$  if  $|\Gamma \cap \delta(S)| = 2$ . In other terms, if  $x^\Gamma$  is the incidence vector of  $\Gamma$ , the corresponding subtour elimination inequality is tight, i.e.  $x^\Gamma(\delta(S)) = 2$ .

**Theorem 40** *Comb inequalities are valid for  $GTSP(n)$  (and therefore for  $STSP(n)$ ).*

**Proof:** (1) It is easy to check that we have:

$$x(\delta(H)) + \sum_{j=1}^t x(\delta(T_j)) \geq \quad (19)$$

$$1/2 \left[ \sum_{j=1}^t x(\delta(H \cap T_j)) + \sum_{j=1}^t x(\delta(T_j \setminus H)) + \sum_{j=1}^t x(\delta(T_j)) \right] \geq 3t$$

Now the left part of (19),  $x(\delta(H)) + \sum_{j=1}^t x(\delta(T_j))$ , is an even integer as sum of even integers, when  $x$  represents a closed walk. Since  $3t$  is odd,  $3t$  can be replaced by  $3t + 1$ . ■

This easy proof does not really explain why any Hamiltonian cycle or any closed walk, intersects these  $t + 1$  coboundaries in at least  $3t + 1$  edges and not in  $2t + 2$  obtained by adding up the  $t + 1$  corresponding subtour elimination inequalities. The following proof, taken from Naddef and Pochet [615], will give more insights into the validity of comb inequalities.

**Proof: (2) A non-algebraic and inductive proof of validity of comb inequalities.** The comb inequality is assumed to be described by sets that satisfy Conditions (14) to (16). Assume first that  $t=3$ . Take a Hamiltonian cycle  $\Gamma$ , if  $|\Gamma \cap \delta(T_j)| = 2$  for  $j=1, 2$  and  $3$ , then  $\Gamma \cap (T_i \setminus H : T_i \cap H) \neq \emptyset$ , so  $|\Gamma \cap \delta(H)| \geq 3$ . Since  $|\Gamma \cap \delta(H)|$  must be even we have  $|\Gamma \cap \delta(H)| \geq 4$  and Inequality (17) is satisfied by all  $x$  representing such a cycle. If  $|\Gamma \cap \delta(T_{j^*})| \neq 2$ , for some  $j^*$ ,  $|\Gamma \cap \delta(T_{j^*})| \geq 4$ , the 3 other sets having at least 2 edges of their coboundaries in  $\Gamma$ , again Inequality (17) is satisfied by all  $x$  representing such a cycle. Proving validity for the general case can be done by induction on the number of teeth  $t \geq 5$ . Assume validity for combs with  $t-2$  teeth. Take a Hamiltonian cycle  $\Gamma$ . If  $|\Gamma \cap \delta(T_j)| = 2$  for  $j = 1, \dots, t$ , then  $|\Gamma \cap \delta(H)| \geq t$  and by parity one must have  $|\Gamma \cap \delta(H)| \geq t+1$  and the result follows for those cycles. Else assume  $|\Gamma \cap \delta(T_{j^*})| \geq 4$ , for some  $j^*$ , let us assume for simplicity  $j^* = t$ . By induction hypothesis we have that:  $|\Gamma \cap \delta(H)| + \sum_{j=1}^{t-2} |\Gamma \cap \delta(T_j)| \geq 3t - 6 + 1$ , adding  $|\Gamma \cap \delta(T_t)| \geq 4$  and  $|\Gamma \cap \delta(T_{t-1})| \geq 2$  we prove again that the inequality is valid. ■

This second proof shows why the teeth and the handle play a different role. The teeth, in some sense, “force” a certain number of edges to be present in the coboundary of the handle.

The last proof is given because it may be adapted to prove validity of most of the inequalities of the coming sections. It is similar to a proof that can also be found in Applegate et al. [29].

**Proof: (3) A non-algebraic and non-inductive proof of validity of comb inequalities.** Let  $\Gamma$  be any Hamiltonian cycle, and  $x^\Gamma$  its associated characteristic vector. Inequality (19) is satisfied by  $x^\Gamma$  if all teeth are tight for  $\Gamma$  using exactly the same argument as in the previous proof. It is also trivially satisfied whenever  $x^\Gamma(\delta(H)) \geq t+1$ . In the other cases, i.e.  $x^\Gamma(\delta(H)) < t+1$ , define  $n$  by  $x^\Gamma(\delta(H)) = t+1 - 2\eta$ . Note that  $\eta > 0$  is integer because  $t$  is odd. and  $x^\Gamma(\delta(H))$  even. As  $(t+1)/2$  represents the minimum number of paths traversing  $H$  when all teeth are tight,  $\eta$  represents the reduction in number of paths traversing  $H$  with respect to the tight teeth case. We prove basically, and the comb inequality tells us essentially, that the number of non tight teeth has to be at least as large as  $\eta$ .

First, we consider each tooth  $T_j$  separately, and define  $\theta_j = 1$  if  $x^\Gamma(T_j \cap H : T_j \setminus H) = 0$ ,  $\theta_j = 0$  otherwise. It is readily seen that  $x^\Gamma(\delta(T_j))/2 \geq 1 + \theta_j$  because  $x^\Gamma(\delta(T_j))/2$  is the number of paths traversing  $T_j$  in the tour  $\Gamma$ , and the tooth  $T_j$  is traversed by at least two paths (i.e. is not tight) when  $\theta_j = 1$ . Thus,  $x^\Gamma(\delta(T_j)) - 2 \geq 2\theta_j$ .



Next, we consider the handle for which it is easy to check that  $x^\Gamma(\delta(H)) \geq \sum_{j=1}^t x^\Gamma(T_j \cap H : T_j \setminus H) \geq \sum_{j=1}^t (1 - \theta_j) = t - \sum_{j=1}^t \theta_j$ . Together with  $x^\Gamma(\delta(H)) = t + 1 - 2\eta$ , this gives  $\sum_{j=1}^t \theta_j \geq 2\eta - 1 \geq \eta$  where the last inequality holds because  $\eta$  is a positive integer.

Hence, we have shown that the reduction  $\eta$  in the number of paths traversing  $H$  is at most  $\sum_{j=1}^t \theta_j$ , which is at most, by definition of  $\theta_j$ , the number of non-tight teeth. This suffices to prove validity because  $x^\Gamma(\delta(H)) = t + 1 - 2\eta \geq t + 1 - 2 \sum_{j=1}^t \theta_j \geq t + 1 - \sum_{j=1}^t [x^\Gamma(\delta(T_j)) - 2]$ . ■

**Theorem 41** (Grötschel and Padberg [403], [404]) *The comb inequalities are facet inducing for  $STSP(n)$  and therefore for  $GTSP(n)$  ( $n \geq 6$ ).*

**Proof:** The original proof is long and technical. It can be rewritten by proving that the simple comb inequalities are facet inducing for  $GTSP(n)$  (very easy), then for  $STSP(n)$  by using Lemma 29, and finally using the node lifting theorems of [618] to obtain the result for general comb inequalities. ■

Comb inequalities together with subtour elimination inequalities and non negativity inequalities, completely describe  $STSP(6)$ , but not  $STSP(7)$  (see end of next section). Comb inequalities, as we will see, are central in solving large TSP instances to optimality.

We now turn to more classes of inequalities. For most of them, comb inequalities can be seen as a subfamily. For the others, the comb inequalities can be seen as building blocks.

## 6. The Star and Path inequalities

*Star inequalities* (Fleischman [311]) are defined by two sets of subsets of  $V$ ,  $\mathcal{K} = \{H_1, \dots, H_h\}$ , called *handles* and  $\mathcal{T} = \{T_1, \dots, T_t\}$ , called *teeth*, with  $t \geq 3$  and odd, together with non-zero integers  $\alpha_1, \dots, \alpha_h$  associated with the handles, and non-zero integers  $\beta_1, \dots, \beta_t$  associated with the teeth, such that:

$$H_1 \subset H_2 \subset \dots \subset H_i \subset \dots \subset H_h \quad (20)$$

$$T_i \cap T_j = \emptyset \quad \text{for } 1 \leq i < j \leq t \quad (21)$$

$$H_1 \cap T_j \neq \emptyset \quad \text{for } j = 1, \dots, t \quad (22)$$

$$T_j \setminus H_h \neq \emptyset \quad \text{for } j = 1, \dots, t \quad (23)$$

$$(H_{i+1} \setminus H_i) \setminus \bigcup_{j=1}^t T_j = \emptyset \quad \forall i, 1 \leq i \leq h-1 \quad (24)$$

And a condition that relates the integers  $\beta_j$  to the integers  $\alpha_i$ , which we call the *Interval Property*.

To define that property we will need a few definitions but we give first the corresponding Star inequality:

$$\sum_{i=1}^h \alpha_i x(\delta(H_i)) + \sum_{j=1}^t \beta_j x(\delta(T_j)) \geq (t+1) \sum_{i=1}^h \alpha_i + 2 \sum_{j=1}^t \beta_j \quad (25)$$

Given a tooth  $T_j$ , a maximal index set of (successive) handles which have the same intersection with  $T_j$  is called *interval* relative to  $T_j$ . We call  $\sum_{i=\ell}^{\ell+r} \alpha_i$  the *weight* of the interval  $I = \{\ell, \ell+1, \dots, \ell+r\}$ .

**Definition 42** *The Interval Property:* for each tooth  $T_j$ , we have  $\beta_j$  at least equal to the maximum weight of an interval relative to  $T_j$ .

Figure 2.9 shows a tooth and the traces of the different handles on it. In the tooth, where a node is shown, there is at least one node, where none is shown there are no nodes. The handles, as suggested by the drawing, are numbered from top to bottom. There are 4 intervals,  $\{1\}, \{2, 3\}, \{4, 5\}, \{6, 7\}$ , of weight 3, 4, 5 and 4, and therefore the  $\beta$  coefficient of that tooth must be at least 5.

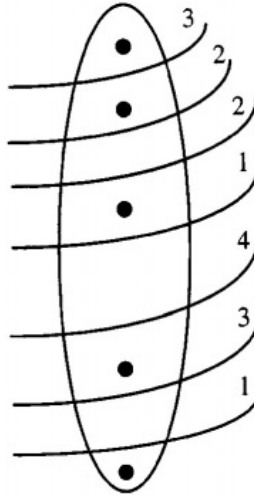


Figure 2.9. Example of intervals

*Path inequalities* (see [219]) are the special case where all the intervals relative to a same tooth have the same weight and the coefficient of that tooth is exactly that value. Figure 2.10 gives an example of a Path inequality. The coefficients associated with the handles and teeth are beside each set.

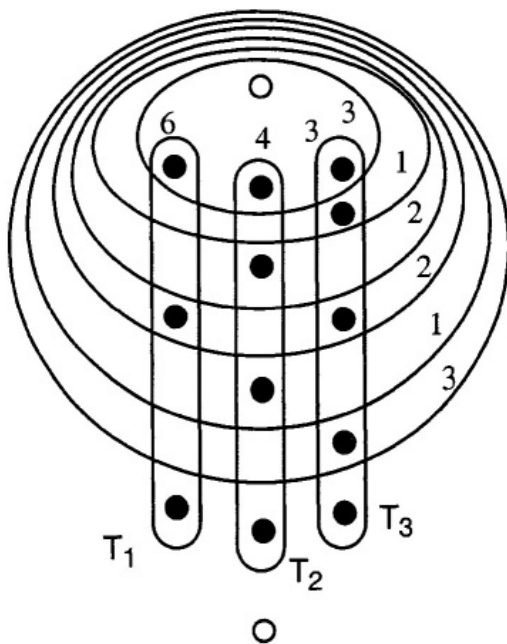


Figure 2.10. Example of a path ( $h = 6, t = 3$ )

In [219] a distinction is made between the case in which there is at least one vertex in the position of each of the two white filled points (path inequalities), only in one of these positions (wheelbarrow inequalities) or in none of these two positions (bicycle inequalities). We will here represent all three case under the same name: *path inequality*. Comb inequalities are exactly the path inequalities with a single handle.

The rationale behind the Interval Property is far from being obvious. We will try to explain it.

The RHS of Inequality (25) can easily be understood by the following argument. Consider Hamiltonian cycles for which all teeth are tight. Each handle must then have at least  $t + 1$  edges of those cycles in its coboundary because  $t$  is odd (see comb inequalities), which leads to the first part of the RHS. The second part is due to the coboundaries of the teeth. This shows that for all such tours the inequality is valid. As one may suspect, the Interval Property is designed to make it valid for all the other tours. This property can be understood by looking at the tours  $\Gamma$  that are tight on all teeth except  $T_j$  and  $|\Gamma \cap \delta(T_{j^*})| = 4$ . Let  $\{r, \dots, s-1\}$  be an interval of  $T_{j^*}$ . Assume  $\Gamma$  uses edges of the coboundaries of the handles  $H_r$  to  $H_{s-1}$  only inside the teeth  $T_i$  for  $i \neq j^*$ . In other words,  $x(T_{j^*} \cap H_i : T_{j^*} \setminus H_i) = 0$  for all  $i \in \{r, \dots, s-1\}$  (see Figure 2.11 where  $j^* = 2$  from Figure 2.10). Then the coboundaries of the handles  $H_r$  to  $H_{s-1}$  may contain only  $t-1$  edges, the coboundaries

of the other handles only  $t+1$ . Therefore a loss to the LHS, relatively to the former case, of  $2 \sum_{i=r}^{s-1} \alpha_i$  is compensated by an increase of  $2\beta_{j^*}$  due to the tooth  $T_{j^*}$  having now four edges in its coboundary. Therefore, to have validity of the inequality one must have  $\beta_{j^*} \geq \sum_{i=r}^{s-1} \alpha_i$ . This argument is valid for all intervals of  $T_{j^*}$  and therefore  $\beta_{j^*}$  must be greater or equal to the largest weight of an interval.

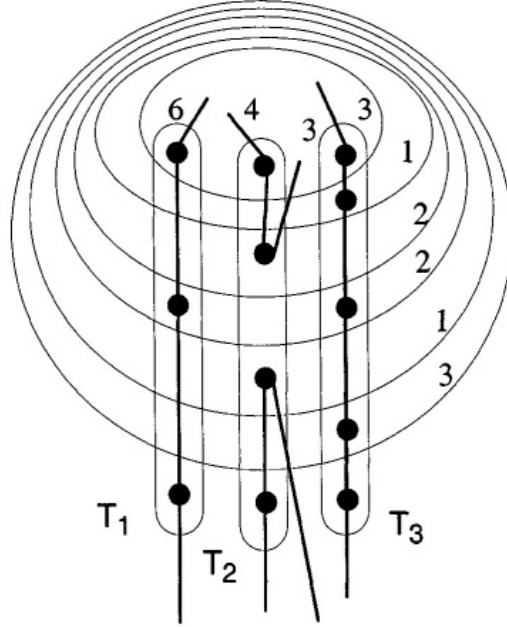


Figure 2.11. Example of minimal trace with  $T_2$  not tight ( $r = 3$ ,  $s - 1 = 4$ )

Of course, all this does not constitute a proof of validity. One can adapt the third proof of validity of combs to obtain one for the stars (see [615]). See [311] for a proof of:

**Theorem 43** *Star inequalities are valid for  $GTSP(n)$  and therefore for  $STSP(n)$ .*

Most star inequalities are not facet inducing even for  $GTSP(n)$ , but we have:

**Theorem 44** *Path inequalities are facet inducing for  $GTSP(n)$ .*

**Proof:** See [219] ■

In Naddef and Rinaldi [619], and unpublished work of Queyranne and Wang the corresponding theorem for  $STSP(n)$  can be found.

In the theoretical study of Goemans [383] on the comparison of valid inequalities for the TSP, the path inequalities turn out to be, at least in theory, the most powerful known inequalities in term of potential to

reduce the gap between the Held and Karp bound and the optimal value. We will see that they do turn out to be also useful in practice.

We end this section with an example of a facet inducing star inequality for  $STSP(n)$  which is not a path inequality. It is shown in Figure 2.12.

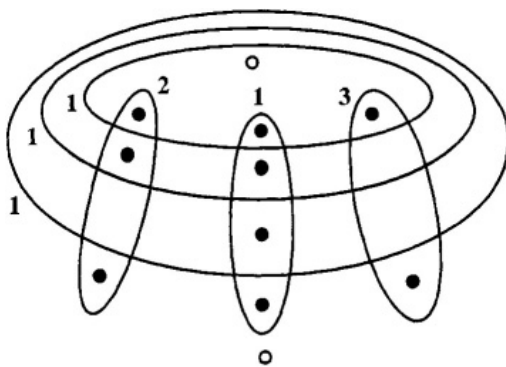


Figure 2.12. A facet inducing star (non-path) inequality;  $RHS=24$

**Research Question 45** Find a necessary and sufficient condition for a star inequality to be facet inducing for  $GTSP(n)$ .

Chances are that this necessary and sufficient condition will also hold for  $STSP(n)$ . It is easy to find necessary conditions. One is that the interval property must be satisfied with equality, else the inequality is the sum of another star inequality and of a subtour elimination inequality. Another necessary condition is that one cannot decompose the coefficients  $\alpha_i$  and  $\beta_j$  in such a way that the star inequality is the sum of two other star inequalities.

To give an idea of the difficulty of the question, the first star inequality of Figure 2.13 is not facet inducing for  $GTSP(n)$ ,  $n \geq 8$ , while the second is for  $n \geq 10$ . These two inequalities look pretty much like that of Figure 2.12. The RHS are 26 and 22, respectively.

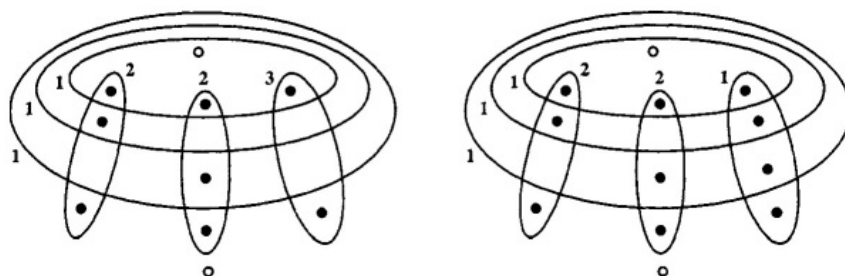


Figure 2.13. Examples of non facet and facet inducing star inequalities

For  $n = 7$  path inequalities together with comb inequalities, subtour elimination inequalities and non-negativity constraints define entirely  $STSP(7)$ . For  $n = 8$ , to obtain such a description, one must add the crown inequalities ([617]) and four classes known as NEW1, NEW2, NEW3 and NEW4 that can be found in [473].

## 7. The Clique Tree and Bipartition inequalities

*Clique tree inequalities* were defined by Grötschel and Pulleyblank [406]. A *Clique tree* is defined by two sets of subsets of  $V$ ,  $\mathcal{K} = \{H_1, \dots, H_h\}$  and  $\mathcal{T} = \{T_1, \dots, T_t\}$ , such that:

$$H_i \cap H_j = \emptyset \quad \text{for } 1 \leq i < j \leq h \quad (26)$$

$$T_i \cap T_j = \emptyset \quad \text{for } 1 \leq i < j \leq t \quad (27)$$

$$T_j \setminus \bigcup_{i=1}^h H_i \neq \emptyset \quad \text{for } 1 \leq j \leq t \quad (28)$$

$$t_j = |\{i : T_j \cap H_i \neq \emptyset\}| \geq 1 \quad \text{for } 1 \leq j \leq t \quad (29)$$

$$h_i = |\{j : H_i \cap T_j \neq \emptyset\}| \geq 3 \text{ and odd} \quad \text{for } 1 \leq i \leq h \quad (30)$$

$$\text{If } T_j \cap H_i \neq \emptyset, \text{ it is a disconnecting} \quad (31)$$

set of the hypergraph  $(V, \mathcal{K} \cup \mathcal{T})$ .

In other words, handles are pairwise disjoint, and so are the teeth. No tooth is contained in a handle, every tooth has at least a vertex not in any handle and every handle intersects an odd number ( $\geq 3$ ) of teeth. The *tree-like* structure is given by Condition (31). Further, in this section, we will drop this last condition together with Condition (28) to obtain a larger family of inequalities, the *bipartition inequalities* of Boyd and Cunningham [135]. When studying these inequalities, we will see the role of Condition (28) in the definition of a clique tree.

Figure 2.14 gives an example of a clique tree. Handles are in bold lines. If there are no vertices in position “Z”, then the clique tree spans the whole graph.

The corresponding *clique tree inequality* is:

$$\sum_{i=1}^h x(\delta(H_i)) + \sum_{j=1}^t x(\delta(T_j)) \geq \sum_{i=1}^h (h_i + 1) + 2t = 2h + 3t - 1 \quad (32)$$

in which  $h_i$  is defined by Condition (30).

As for all the inequalities that we will propose in this chapter, the RHS can easily be figured out by looking at tours for which all teeth are tight. For these tours, the handles must have at least  $h_i + 1$  edges in their coboundaries, and this leads naturally to the RHS. To prove validity, one must prove that every time we augment the number of

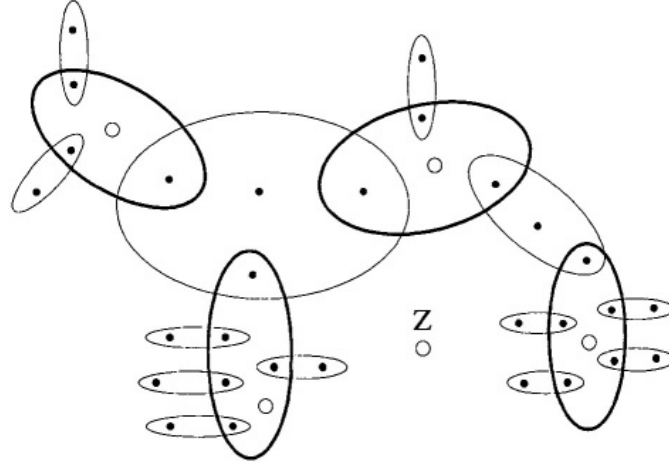


Figure 2.14. Example of a Clique Tree

edges in the coboundary of some teeth, we do not “save” more on the total number of edges in the coboundary of the handles. This is done in proofs of Applegate et al. [29] and in Naddef and Pochet [615]. An algebraic proof of validity is given in [406]. Moreover one can find there the following theorem:

**Theorem 46** (Grötschel-Pulleyblank [406]) *Clique tree inequalities are facet inducing for  $STSP(n)$ .*

A clique tree with a single handle is a comb. Some authors admit  $\mathcal{K} = \emptyset$  (which, by the way, does not satisfy Condition (29)), if so, subtour elimination inequalities are also clique tree inequalities. In [616], Naddef and Rinaldi show that the clique trees, which have at least a vertex in position “Z” of Figure 2.14, can be obtained by repeated 2-sums starting with two combs.

We now turn to the *bipartition inequalities* of Boyd and Cunningham [135]. A *bipartition* is defined just like a clique tree but without Conditions (28) and (31). There are coefficients  $\beta_1, \dots, \beta_j, \dots, \beta_t$  associated with the teeth. Let  $D$  be the set of indices of teeth that contain no vertex outside any handle, i.e.

$$T_j \setminus \bigcup_{i=1}^n H_i = \emptyset \text{ if and only if } j \in D \quad (33)$$

A tooth in the set  $\{T_j : j \in D\}$  is called *degenerate* by Boyd and Cunningham. We will define, later on, a more restrictive notion of degeneracy which will not consider as degenerate those teeth with index in  $D$  which intersect only two handles. The intuitive explanation of the co-

efficients, further down, will explain why we will not qualify these teeth as degenerate.

Let  $t_j$  be as defined in Condition (29), that is the number of handles that tooth  $T_j$  intersects, then the coefficients  $\beta_j$  are defined by:

$$\beta_j = 1 \quad \text{if } j \notin D \quad (34)$$

$$\beta_j = \frac{t_j}{t_j - 1} \quad \text{if } j \in D \quad (35)$$

The corresponding *bipartition inequality* is:

$$\sum_{i=1}^h x(\delta(H_i)) + \sum_{j=1}^t \beta_j x(\delta(T_j)) \geq \sum_{i=1}^h (h_i + 1) + 2 \sum_{j=1}^t \beta_j \quad (36)$$

In Figure 2.15, in which handles are in bold lines, the two teeth  $T^*$  and  $T$  are degenerate. The first one intersects three handles, therefore its coefficient is  $3/2$ , the second only two and therefore its coefficient is 2.

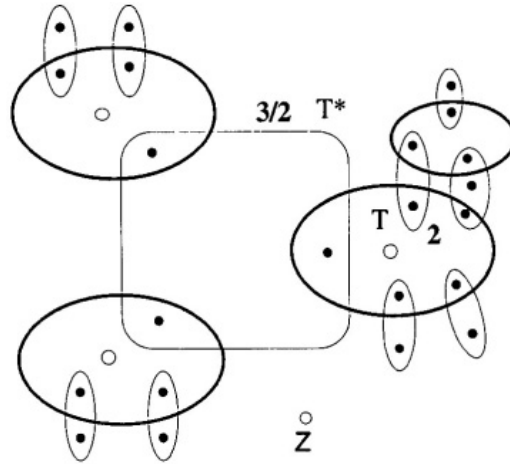


Figure 2.15. Example of a Bipartition

The right hand side is again understood by looking at the tours for which all teeth are tight. An example of the trace of such a tour on the various coboundaries, involved in the inequality, is given in the first drawing of Figure 2.16, which minimizes the total number of edges in these coboundaries. In such a tour each handle must have at least  $h_i + 1$  edges, where  $h_i$  is the number of teeth that handle  $H_i$  intersects.

Assume that we look at tours which intersect tooth  $T$  in four edges. The second drawing of Figure 2.16 shows the trace of such a tour. Note that the increase of two edges in the coboundary of  $T$  is compensated for by a decrease of two edges in each of the coboundaries of the two only



handles it intersects. Enabling tours to use more edges in the coboundary of  $T$  does not yield any further gain in the minimum number of edges of the coboundaries of the handles. This explains the coefficient of 2, in order to maintain validity of the inequality.

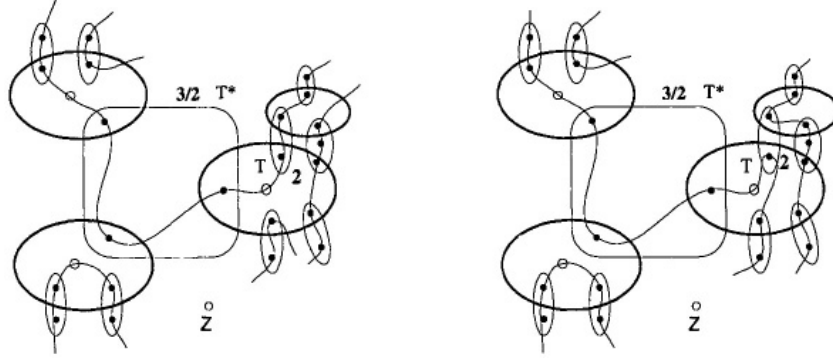


Figure 2.16. Traces of tight tours with 2 and 4 edges in  $\delta(T)$

Now we turn to tooth  $T^*$ . The first drawing of Figure 2.17 shows again a minimal trace on the handles of a tour tight for all teeth. In the second, the trace corresponds to tours that intersect the coboundary of tooth  $T^*$  in four edges. We also gain two edges on the coboundaries of the handles. If we go one step further, and enable six edges in the coboundary of tooth  $T^*$ , that is four more than the minimum, this time we gain *four* edges in the coboundaries of the handles from the previous tour and *six* from the first one. If the tours, the traces of which are shown in the first and third drawing of Figure 2.17, are to be tight for the inequality, then the loss of 4 edges on the coboundary of  $T^*$  cannot be more than compensated by the gain of 6 edges on the coboundaries of the handles, therefore  $\beta_{T^*} \geq 6/4 = 3/2$ . Since enabling yet more edges in the coboundary of  $T^*$  does not decrease further the minimum number of edges in the coboundaries of the handles, if  $\beta_{T^*} > 3/2$ , then the only tight tours would be those which are tight on  $T^*$ , and therefore the inequality would be contained in the facet  $x(\delta(T^*)) \geq 2$ . This explains the coefficient  $t_j/(t_j - 1)$  of some teeth.

In Section 9, the technique that we just used in order to define the coefficients of  $T$  and  $T^*$  will be developed into a general procedure. In some sense, the coefficient of teeth should *normally* be defined by the case of four edges in their coboundary, so we will call degenerate only those teeth for which the coefficient is defined by tours having six or more edges in their coboundaries. Therefore tooth  $T$  of our example will no longer be considered as degenerate. We make this more precise in Section 9.

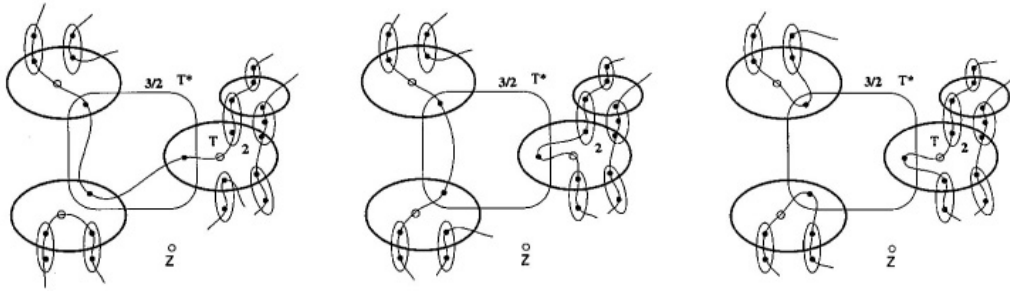


Figure 2.17. Traces of tight tours with 2, 4 and 6 edges in  $\delta(T^*)$

Of course, since all we use is that a cycle intersects a coboundary in an even number of edges, so do closed walks, in all the preceding material, tours can be replaced by closed walks. In no case have we given a proof of validity of the bipartition inequality. This is done by:

**Theorem 47** (Boyd- Cunningham [135]) *Bipartition inequalities are valid for STSP( $n$ ).*

In [232] W. Cunningham and Y. Wang give two necessary conditions for such an inequality to be facet defining. The first is that there be no subset of degenerate teeth which is disconnecting for the underlying hypergraph (as a consequence, the bipartition of our example is not facet inducing). The second is that for each edge there exists a tight tour that contains it, which as already mentioned is necessary for any non-trivial inequality to be facet inducing. The next section gives an example of a bipartition which does not yield a facet inducing inequality, but for which there is a way around, via a correcting term, to get a facet inducing inequality. Cunningham and Wang conjecture that together these two conditions are sufficient.

## 8. The Ladder inequalities

A *ladder inequality* is defined by two handles  $H_1$  and  $H_2$  an even number  $t \geq 4$  of teeth  $\{T_1, \dots, T_j, \dots, T_t\}$ , with integer coefficients  $\beta_1, \dots, \beta_j, \dots, \beta_t$  associated with them, such that (see Figure 2.18 in

which the coefficients  $\beta$  are written next to the corresponding teeth):

$$H_1 \cap H_2 = \emptyset \quad (37)$$

$$T_j \cap T_k = \emptyset \quad 1 \leq j < k \leq t \quad (38)$$

$$H_1 \cap T_1 \neq \emptyset, H_2 \cap T_1 = \emptyset \quad (39)$$

$$H_2 \cap T_2 \neq \emptyset, H_1 \cap T_2 = \emptyset \quad (40)$$

$$T_1 \setminus H_1 \neq \emptyset, T_2 \setminus H_2 \neq \emptyset \quad (41)$$

$$T_j \cap H_i \neq \emptyset \quad \text{for } i = 1, 2 \text{ and } j \geq 3 \quad (42)$$

$$\text{if } T_j \setminus (H_1 \cup H_2) \neq \emptyset, \text{ then } \beta_j = 1, \quad \text{else } \beta_j = 2; \text{ for } j \geq 3 \quad (43)$$

$$\beta_1 = \beta_2 = 1 \quad (44)$$

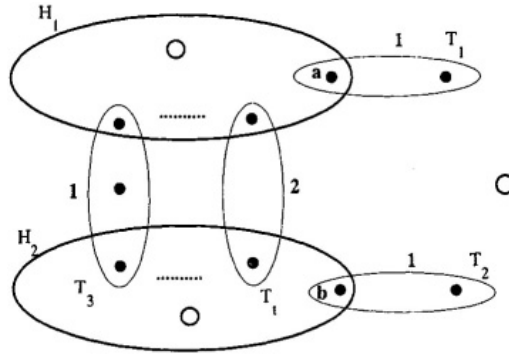


Figure 2.18. Example of a ladder

Teeth  $T_1$  and  $T_2$  play a special role. The first one only intersects  $H_1$  and the second only  $H_2$ , all other teeth intersect both handles. The ladder inequality is:

$$\sum_{i=1}^2 x(\delta(H_i)) + \sum_{j=1}^t \beta_j x(\delta(T_j)) - 2x(H_1 \cap T_1 : H_2 \cap T_2) \geq 2t + 2 \sum_{j=1}^t \beta_j \quad (45)$$

The coefficients on the teeth can be explained just as in the case of the bipartition inequalities. Note the special correcting (or lifting) term  $-2x(H_1 \cap T_1 : H_2 \cap T_2)$  on the left hand side of Inequality (45) without which this would only be a particular case of bipartition inequalities.

Again we try to give the reader some insight into the validity of this inequality. Observe first that no tour that is tight for the corresponding bipartition (not ladder) inequality contains an edge of  $(H_1 \cap T_1 : H_2 \cap T_2)$  (we leave it to the reader to convince himself/herself of this fact or consult [615]), and therefore the corresponding face is contained in the intersection of the facets  $x_e \geq 0$  for  $e \in (H_1 \cap T_1 : H_2 \cap T_2)$ . Figure 2.19

gives the trace of a tight tour for the ladder inequality, that contains such an edge and with all teeth tight. Extension of this tour to the case  $t \geq 6$  is straightforward.

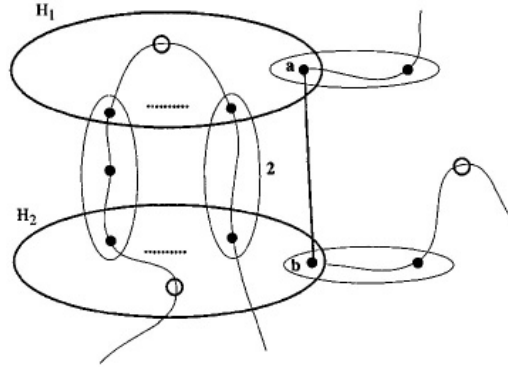


Figure 2.19. Trace of a tight tour containing  $e \in (H_1 \cap T_1 : H_2 \cap T_2)$

**Theorem 48** (Boyd, Cunningham, Queyranne and Wang [136]) *The ladder inequalities are facet defining for  $STSP(n)$ .*

## 9. A general approach to some TSP valid inequalities

Let  $\mathcal{S} = \{S_1, \dots, S_i, \dots, S_p\}$  be a set of distinct (possibly overlapping) subsets of  $V$ . Let  $\alpha_1, \dots, \alpha_i, \dots, \alpha_p$  be strictly positive integers. An inequality is said to be in *closed-set form* if it can be written as:

$$\sum_{i=1}^p \alpha_i x(\delta(S_i)) \geq r(\mathcal{S}) \quad (46)$$

where  $r(\mathcal{S})$  is the minimum of the LHS on all tours.

Path, star, bipartition inequalities are inequalities in closed-set form, and therefore so are the comb and clique tree inequalities. Inequalities containing path, star and clique trees, and named *binested inequalities* in [613], are also inequalities in closed-set form. Ladder inequalities are not in closed-set form because of the correcting term on the LHS.

In most of the known closed-set form inequalities for  $STSP(n)$ , the set  $\mathcal{S}$  of subsets of  $V$  is partitioned into two sets  $\mathcal{K}$  and  $\mathcal{T}$  where  $\mathcal{K} = \{H_1, \dots, H_h\}$  is the set of so called *handles*,  $\mathcal{T} = \{T_1, \dots, T_t\}$  is the set of so called *teeth*.

We also have non-zero integers  $\alpha_1, \dots, \alpha_h$ , associated with the handles, and (not necessarily integer) rationals  $\beta_1, \dots, \beta_t$ , associated with the teeth.

These inequalities in closed-set form read:

$$\sum_{i=1}^h \alpha_i x(\delta(H_i)) + \sum_{j=1}^t \beta_j x(\delta(T_j)) \geq A + 2 \sum_{j=1}^t \beta_j \quad (47)$$

or equivalently

$$\sum_{i=1}^h \alpha_i x(\delta(H_i)) \geq A - \sum_{j=1}^t \beta_j (x(\delta(T_j)) - 2). \quad (48)$$

Expression (48) suggests the following interpretation, which can be used in order to define a general procedure for computing the coefficients  $A$  and  $\beta_j$ ,  $j = 1, \dots, t$ , of the valid inequality.

When all teeth are tight for a given Hamiltonian cycle  $\Gamma$  that is  $x^\Gamma(\delta(T_j)) = 2$  for all  $j = 1, \dots, t$ , Inequality (48) then reduces to  $\sum_{i=1}^h \alpha_i x(\delta(H_i)) \geq A$ . Hence, for validity, the largest possible value of  $A$  is the minimum value of the left hand side of (48) over all Hamiltonian cycles  $\Gamma$  with all teeth tight. Formally,

$$A = \min_{\Gamma} \left\{ \sum_{i=1}^h \alpha_i |\Gamma \cap H_i| : |\Gamma \cap \delta(T_j)| = 2 \text{ for all } j = 1, \dots, t \right\} \quad (49)$$

and does not depend on the coefficients  $\beta_j$ .

Now we describe a general procedure to compute the coefficients  $\beta_j$  in the valid Inequality (48) when the coefficients of the handles are assumed to be fixed. First we order the teeth. We assume without loss of generality that the teeth have been renumbered in such a way that their order is  $T_1, \dots, T_j, \dots, T_t$ . Next, we compute the coefficients  $\beta_j$  from  $j = 1$  to  $j = t$  using a sequential lifting procedure. The lifted variables are the non-negative integer slack variables  $s_j$  associated to the subtour elimination inequalities defined on the teeth, namely  $s_j = x(\delta(T_j)) - 2$  for  $j = 1, \dots, t$ . Initially, to compute the coefficient  $A$  in Expression (49), all slacks  $s_j$  are projected to zero and we obtain the inequality  $\sum_{i=1}^h \alpha_i x(\delta(H_i)) \geq A$  which is valid for the subspace where  $s_j = 0$  for  $j = 1, \dots, t$ , that is when all teeth are tight. Then, sequentially, from  $j = 1$  to  $j = t$ , all slacks  $s_j$  are lifted.

For  $k \in \{1, \dots, t\}$ , assuming coefficients  $\beta_1, \dots, \beta_{k-1}$  have been already computed, the lifting coefficient  $\beta_k$  is taken as the minimum value  $\beta$  for which the inequality

$$\sum_{i=1}^h \alpha_i x^\Gamma(\delta(H_i)) + \sum_{j=1}^{k-1} \beta_j [x^\Gamma(\delta(T_j)) - 2] + \beta [x^\Gamma(\delta(T_k)) - 2] \geq A \quad (50)$$

is satisfied by all Hamiltonian cycles  $\Gamma$  with  $s_j = x^\Gamma(\delta(T_j)) - 2 = 0$ , for all  $j > k$ .

We let  $\mathcal{B}_\ell^k$  denote the set of Hamiltonian cycles for which all the teeth  $T_{k+1}, \dots, T_t$  are tight and intersect the coboundary of  $T_k$  in  $2\ell$  edges. Then the value of  $\beta_k$  is defined by

$$\beta_k = \max_{\ell > 1} \frac{b_k^1 - b_k^\ell}{2\ell - 2} \quad (51)$$

where

$$b_k^l = \min_{\Gamma \in \mathcal{B}_l^k} \left[ \sum_{i=1}^h \alpha_i x^\Gamma(\delta(H_i)) + \sum_{j < k} \beta_j [x^\Gamma(\delta(T_j)) - 2] \right] \quad (52)$$

We call *degenerate* those teeth  $T_k$  for which the coefficient  $\beta_k$  cannot be obtained for  $\ell = 2$  in Equation (51).

Note that by construction of  $\beta_k$ , we must have  $b_k^1 = A$  for all  $k \in \{1, \dots, t\}$ . Also note that the  $\beta_k$  are not necessarily integer. The construction is summarized in the following theorem.

**Theorem 49** *Let  $\mathcal{K} = \{H_1, \dots, H_h\}$ , let  $\mathcal{T} = \{T_1, \dots, T_t\}$ , be an ordered set of teeth, and let  $\alpha_1, \dots, \alpha_h$  be nonzero integers associated with the handles. If  $A$  is defined by Expression (49), and, for  $k \in \{1, \dots, t\}$ , if  $\beta_k$  is the coefficient of tooth  $T_k$  determined by Expressions (52) and (51), then the following inequality is valid:*

$$\sum_{i=1}^h \alpha_i x(\delta(H_i)) \geq A - \sum_{j=1}^t \beta_j [x(\delta(T_j)) - 2] \quad (53)$$

**Proof:** Trivial by the way we constructed the coefficients of the teeth. ■

**Remark 50** *The above procedure provides a way of defining valid inequalities for the TSP. The difficulty in using Theorem 49 to prove validity of inequalities is the exponential growth in the number of cases to study when the number of teeth increases.*

**Remark 51** *In all the inequalities seen so far in this chapter, the coefficients of the teeth are order independent, that is they can be computed as if they are the first in the sequential lifting. This is not always the case. We give now an example.*

Figure 2.20, which has been provided by M. Queyranne, illustrates this. Here, the lifting coefficients are defined as in Theorem 49, but we

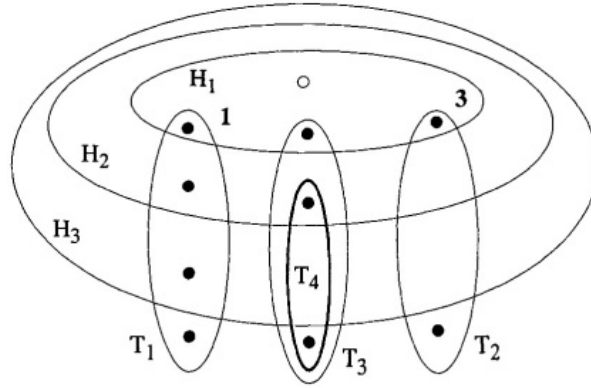


Figure 2.20. Example of a facet defining inequality

will see that their values are sequence dependent, and in fact only one order will lead to an inequality. Handles have all coefficients of 1.

The support in Figure 2.20 would give an inequality of the form

$$\sum_{i=1}^3 x(\delta(H_i)) \geq 12 - \sum_{j=1}^4 \beta_j [x(\delta(T_j)) - 2]. \quad (54)$$

If you authorize a Hamiltonian cycle to intersect the coboundary of  $T_4$  in 4 or more edges restricting all the other teeth to be tight, we cannot compensate by “saving” anything on the coboundaries of the handles. Therefore, this procedure will always produce an inequality which is just the addition of the inequality obtained without tooth  $T_4$  and the subtour elimination inequality associated to that tooth. In other words, lifting  $T_4$  first gives  $\beta_4 = 0$ .

The coefficients of  $T_1$  and  $T_2$  can be defined in the manner used so far. This gives  $\beta_1 = 1$  and  $\beta_2 = 3$ . Let us authorize a Hamiltonian cycle to intersect the coboundary of  $T_3$  in 4 edges, imposing  $T_4$  to be tight. We can only save 2 edges on the coboundaries of the handles (whether or not we impose  $T_1$  and  $T_2$  to be tight); see Figure 2.21(a). These 2 edges we save are 2 of the 4 that crossed the border of  $H_1$  when we imposed all teeth to be tight. This yields a coefficient of  $\beta_3 = 1$  for  $T_3$ , instead of a coefficient of 2 in the star inequality made up of the same handles and teeth  $T_1$ ,  $T_2$  and  $T_3$ . Now that the coefficients of  $T_1$ ,  $T_2$  and  $T_3$  are known we can define the coefficient of  $T_4$ .

The trace of a tour shown in Figure 2.21(b), shows that we can save 4 edges on the coboundaries of the handles by enabling a cycle to intersect the coboundary of  $T_4$  in 4 edges. Two of these 4 edges are compensated for, by the extra 2 edges in the coboundary of  $T_3$ , the 2 others by the

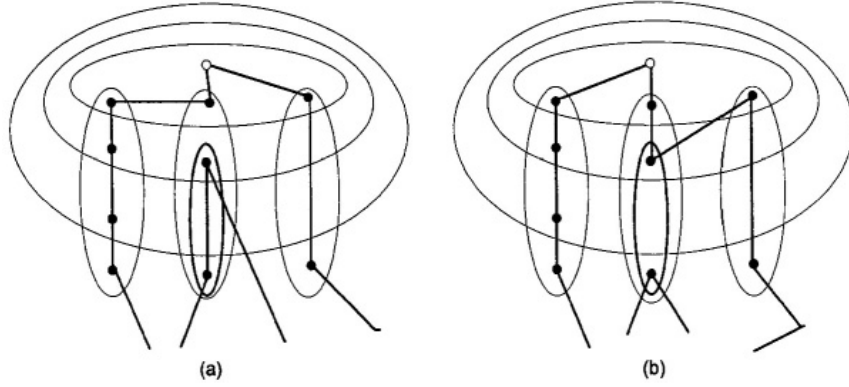


Figure 2.21. Some traces of Hamiltonian cycles

extra 2 edges in the coboundary of  $T_4$ , which yields a coefficient of  $\beta_4 = 1$  for tooth  $T_4$ . This inequality is facet defining for  $STSP(n)$ ,  $n \geq 9$ .

We use the developments of this section to describe a family of inequalities that contains the inequalities described in the previous sections.

## 10. A unifying family of inequalities

This section may be skipped by most readers. It would be too long to expose here the binested inequalities (see [613]). They contain the star (and therefore path), the clique tree, and the bipartition inequalities without degenerate teeth (in the sense of the previous section). They do not contain the ladder inequalities. One way of making them contain all remaining bipartition inequalities and the ladder inequalities would be as follows:

- 1 Recompute the coefficient of the teeth using the method of the previous section,
- 2 Arbitrarily order all edges which do not appear in a tight tour with the newly computed coefficients and compute sequentially the coefficient of the correcting term corresponding to them. The correcting coefficients will be subtracted to the LHS just like in the ladder inequalities.

We conjecture that the first calculation (the one for the teeth) is order independent. The second one is not. One can find an example in [615] in relation with generalizations of the ladder inequalities, for which this lifting is not order independent.



## 11. Domino inequalities

In the previous sections, teeth in some sense forced edges to cross the border of the handles or had to be no longer tight. This was obtained either because of parity or by the situation of nodes in key positions. There is another way of forcing edges in the border of the handles, that is by penalizing some edges, which must be taken in some circumstances if one does not cross the border of the handle. This is the rationale behind the *Domino Parity* inequalities defined by Letchford [559], although this is not explicit in his exposition. We describe here a large subclass of these inequalities which we will call *Domino* inequalities.

Let  $H \subset V$ , referred to as the *handle*,  $\mathcal{T} = \{T_1, \dots, T_i, \dots, T_t\}$ ,  $t \geq 3$  and odd, where  $T_i = A_i \cup B_i \subset V$ , referred to as the set of *teeth*, such that:

$$A_i \cap B_i = \emptyset \quad \text{for } i = 1, \dots, t \quad (55)$$

$$A_i \cup B_i \neq V \quad \text{for } i = 1, \dots, t \quad (56)$$

$$(A_i : B_i) \cap (A_j : B_j) = \emptyset \quad \text{for } 1 \leq i < j \leq t \quad (57)$$

We say that  $A_i$  and  $B_i$  are the *two half dominoes* of the *domino* (or tooth)  $T_i$ . Note that there are no further restrictions on the elements of  $\mathcal{T}$ , i.e. they can for example intersect.

The main difference with the Domino Parity configurations of Letchford is that we added the non crossing Condition 57. Naddef proves in [614] that the only facet inducing Domino Parity configurations that do not satisfy this condition have two teeth, say  $T_1$  and  $T_2$  such that  $T_1 \cup T_2 = V$  with  $T_1 \cap T_2 \neq \emptyset$ , but in that case there exists a domino inequality satisfying condition 57, which includes the same facet. Let  $C = (\bigcup_{i=1}^t (A_i : B_i)) \setminus \delta(H)$  be the set of edges inside the teeth and crossing from one half domino to the other without crossing the boundary of the handle. The *domino inequality* is defined by:

$$x(\delta(H)) + \sum_{j=1}^t x(\delta(T_j)) + 2x(C) \geq 3t + 1 \quad (58)$$

**Theorem 52** *Domino inequalities are valid for STSP( $n$ ).*

**Proof:** Call *LHS* the left hand sides of these inequalities. it is easy to check that:

$$LHS \geq \frac{1}{2} \sum_{i=1}^t x(\delta(A_i)) + \frac{1}{2} \sum_{i=1}^t x(\delta(B_i)) + \frac{1}{2} \sum_{i=1}^t x(\delta(T_i)) = 3t \quad (59)$$

Since LHS is even for all tours, we can raise  $3t$  to  $3t + 1$ . ■

Note the similarity with the algebraic proof of validity of comb inequalities. To understand the role of the partition  $\{A_i, B_i\}$  of each tooth, consider the example of Figure 2.22, where, except for  $T_7$ ,  $A_i = T_i \cap H$  and  $B_i = T_i \setminus H$ . For  $T_7$ , we have  $A_7 \supseteq T_3 \cup T_4$ ,  $B_7 \supseteq T_5 \cup T_6$ . All teeth  $T_1$  to  $T_6$  are pairwise node disjoint. A tour for which all teeth are tight does not necessarily force more than six edges in the coboundary of  $H$  as seen in Figure 2.23(a), but then it necessarily uses an edge of  $(A_i : B_i) \setminus \delta(H)$  which is *penalized* by two units. So either such a tour goes from  $A_i$  to  $B_i$  by using an edge of the coboundary of  $H$  as in Figure 2.23(b), or it uses a penalized edge to do so. In this example none of the other domino partitions play a role. The domino inequality corresponding to this example is facet inducing since it is a twisted comb inequality (definition below, see also [161]).

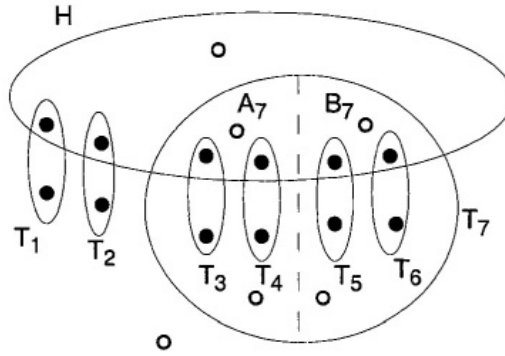


Figure 2.22. Example of domino inequality

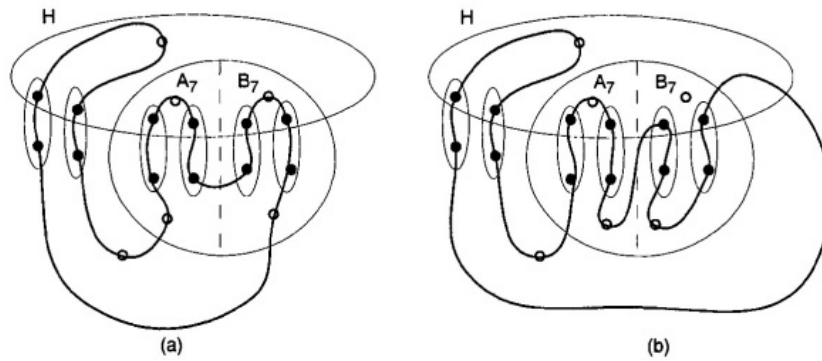


Figure 2.23. Tight tours for the example of Figure 2.22

We now turn to necessary conditions in order for a domino inequality to be facet inducing. A *minimal* domino configuration is one such that the number of non minimal (by inclusion) teeth is minimum, among all domino configurations that yield the same domino inequality. A comb

can be seen as a domino with only minimal teeth, but also as a domino configuration with one tooth containing all the others and such that one of its half dominoes contains all the teeth, the other half domino contains none, is non empty and has an empty intersection with the handle. Finally there is a node not in any tooth nor in the handle. Naddef in [614], proves the following propositions:

**Proposition 53** *The teeth of a facet inducing minimal domino inequality are nested.*

**Proposition 54** *If  $T_i$  is a minimal tooth in a facet defining domino inequality, then  $\{A_i, B_i\} = \{T_i \cap H, T_i \setminus H\}$ .*

**Corollary 55** *Only the domino partition for non-minimal teeth has some interest. We will omit, from now on, giving the domino partition of minimal teeth.*

Minimal teeth are similar to comb teeth. For non minimal teeth, each tooth they contain must be contained in one of the half domino of that tooth. For minimal domino configurations it is shown in [614] that in a non minimal tooth each half domino must contain at least two *odd* teeth of immediate lower rank in the nesting. A tooth is odd if either it is minimal or it contains, including itself, an odd number of teeth.

**Conjecture 56** *Minimal Domino inequalities with at least three odd maximal teeth and such that each half domino of a non minimal tooth contains at least two odd teeth, are facet inducing for STSPP( $n$ ).*

One can find in [614] the proof that a large subfamily of these inequalities is facet inducing for the graphical relaxation.

We end this section with a family of facet inducing domino inequalities. Boyd, Cockburn and Viella define in [134] the following *twisted combs*. A twisted comb inequality is a domino inequality satisfying all the previous necessary conditions and which has a unique maximal tooth which is not also minimal. Note that the example of Figure 2.22 refers to a twisted comb.

**Theorem 57** (Boyd, Cockburn, Vella [134]) *The twisted comb inequalities are facet inducing for STSPP( $n$ ).*

For more on the domino inequalities, besides the papers already mentioned, one can also read Cockburn [204].

**Research Question 58** *How can one generalize the domino partition for non minimal teeth to inequalities with more than one handle?*

A first answer to this question can be found in [614] where a generalization of path inequalities can be found.

## 12. Other inequalities

There are many other inequalities which do not fall in one of the classes defined so far. The best known example are the *hypohamiltonian inequalities* (see [195]). A graph is said to be *hypohamiltonian* if it is *not* Hamiltonian but the deletion of *any* vertex yields a Hamiltonian graph. A hypohamiltonian graph is said to be *edge maximal* if it is hypohamiltonian and the addition of *any* non existing edge yields a Hamiltonian graph. The Petersen graph is an edge maximal hypohamiltonian graph.

Let  $G = (V, E^*)$  be an edge maximal hypohamiltonian graph and  $K_n = (V, E)$  be the complete graph on the same vertex set. The *simple hypohamiltonian* inequality  $fx \geq f_0$  is given by:

- 1  $f_e = 1$  if  $e \in E^*$
- 2  $f_e = 2$  if  $e \in E \setminus E^*$
- 3  $f_0 = |V| + 1$

A general hypohamiltonian inequality is obtained by clique lifting of a simple hypohamiltonian inequality, that is replacing each vertex by a clique of some size, the coefficients of the inequality on the edges linking two nodes of the same clique is 0, the other coefficients are inherited from the ones of the simple inequality. In other words a hypohamiltonian inequality  $fx \geq f_0$  is such that the partition of  $V$  into  $V_1, \dots, V_i, \dots, V_p$ , the vertex sets of the connected components of the graph  $G_0 = (V, E_0)$  with  $E_0 = \{e \in E : f_e = 0\}$  induces an edge maximal hypohamiltonian graph when one shrinks each  $V_i$  to a single node in  $G_1 = (V, E_1)$ , with  $E_1 = \{e \in E : f_e = 1\}$ .

**Theorem 59** *Hypohamiltonian inequalities are facet inducing for  $GTSP(n)$ .*

**Proof:** Let  $fx \geq f_0 = p + 1$  be a hypohamiltonian inequality. Let the sets  $V_1, \dots, V_i, \dots, V_p$  be the the partition of  $V$  induced by the connected components of  $G_0 = (V, E_0)$ . Inequality  $fx \geq f_0$  is trivially valid since any closed walk must either contain an edge with coefficient 2, or two edges of coefficient 1 in some  $(V_i : V_j)$ ,  $i \neq j$ . Therefore the coefficients sum up to at least  $p + 1$  on the closed walk. Assume  $fx \geq f_0$  is not facet inducing, i.e. the face it defines is strictly contained in some facet. Let  $hx \geq f_0$  (same RHS), be the corresponding linear inequality. Every closed walk  $W \in \mathcal{W}_f^-$  is also in  $\mathcal{W}_h^-$ . Let  $W \in \mathcal{W}_f^-$ . For  $e \in E(V_i)$  for some  $i$ , we have  $W + 2k\{e\} \in \mathcal{W}_f^-$  for all integer  $k$ , and therefore also in  $\mathcal{W}_h^-$ , which proves that  $h_e = 0$  for these edges. As mentioned earlier, necessarily  $h_e = \gamma_{ij}$  for all  $e \in (V_i : V_j)$ . Let  $W_i$  be a Hamiltonian cycle

of  $G \setminus V_i$  that contains only edges of  $f$  value 0 or 1. Such a cycle exists since picking a single vertex in each  $V_i$  and the edges of  $f$ -value 1 linking them yields an hypohamiltonian graph. Let  $e \in \delta(V_i)$  be such that  $f_e = 1$ . We have that  $W_i + 2\{e\} \in \mathcal{W}_f^-$ , so also in  $\mathcal{W}_h^-$ . this proves that all these edges have the same value  $h_e = \gamma_i$ . Since a hypohamiltonian graph is necessarily connected, we have that for  $h_e = \gamma$  for all  $e \in E$  such that  $f_e = 1$ , and therefore  $\gamma = 1$ , since we chose the same RHS. Now for any edge  $e \in (V_i : V_j)$  such that  $f_e = 2$ , there is a Hamiltonian cycle containing it and only edges of  $f$ -value 0 and 1, which proves that  $h_e = 2$  for these edges, and we have proved that  $h_e = f_e$  for all  $e \in E$ . ■

We have described most of the known facet inducing inequalities that can be found in the literature, with the exception of the *crown* inequalities of Naddef and Rinaldi [617]. Most of the other known inequalities come from clique lifting of the inequalities obtained from the complete description of polytopes for small instances by Christof and Reinelt and which are available at: [www.informatik.uni-heidelberg.de/groups/comopt/software/SMAPO/tsp/tsp.html](http://www.informatik.uni-heidelberg.de/groups/comopt/software/SMAPO/tsp/tsp.html).

### 13. The separation problem

The separation problem for the STSP is the following: *Given a fractional solution  $x^*$ , find a valid inequality, preferably facet inducing for  $STSP(n)$ , say  $fx \geq f_0$ , such that  $fx^* < f_0$  and such that  $f_0 - fx^*$  is not too small.*

We may search for a violated inequality in two ways. Either by searching among known classes of inequalities, for example among those we described in the previous sections, or in a very general form. Applegate et al. in [31] refer to the first type of search as a template paradigm separation. This has been for a long time the only type of STSP separation framework (see Padberg and Rinaldi [645], [647] and [648], Grötschel [397], Padberg and Crowder [229]). More recently Christof, Reinelt and Wenger have tried to use the complete knowledge of the  $STSP(n)$  for  $n \leq 10$ . We will give a brief description of their method in Section 15. Applegate, Bixby, Chvátal and Cook [31] uses a more elaborate method which we will sketch in Section 18.

We end this section with the few complexity results known so far. Separating subtour elimination constraints is polynomial because it amounts to finding a minimum cut in an undirected weighted graph— we already addressed this point in Section 2. There also exists a polynomial time separation algorithm for *2-matching* constraints i.e. simple comb inequalities (see Padberg and Rao [644]).

Another result deals with bipartition inequalities with a fixed number of handles and teeth. Carr [170] and [171] showed that there is a polynomial separation algorithm in this case. We give the nice and elegant argument in the particular case of a comb with  $t$  teeth.

- Choose  $2t$  vertices and, among these, choose  $t$  vertices that will belong to the handle. Match each of these  $t$  vertices of the handle to one of the  $t$  others to form  $t$  disjoint pairs.
- Find a minimum cut that separates the  $t$  nodes chosen to be in the handle from the  $t$  others. This can be done in polynomial time.
- For each pair of matched vertices, find a minimum cut that separates the pair from the other  $t - 2$  vertices. Choose one shore of the minimum cut to be a tooth.
- If the teeth intersect, there is an uncrossing argument that enables to obtain non intersecting teeth with the same coboundary values in  $x^*$ .
- Check for violation.

Since there is only a polynomial number of choices for the  $2t$  vertices, and for each such choice a polynomial number of choices thereafter, the whole procedure is polynomial for any fixed  $t$ . Of course this does not lead to a practical separation routine, but some of the separation routines that we will describe can be seen as trying to guess clever choices of some of these  $2t$  vertices.

Fleischer and Tardos [310], designed a polynomial algorithm to separate maximally violated comb inequalities as long as the support graph of  $x^*$  (the graph induced by the edges  $e$  with  $x_e^* > 0$ ) is planar. A comb is said to be maximally violated if the difference between the RHS and the LHS is 1 (.5 if in the standard " $\leq$ " form).

In case of planar support graphs, Letchford [559] gives a polynomial algorithm that separates combs without the maximal violation requirement. This same algorithm also separates the domino inequalities under the same conditions.

The following has never been implemented and is so far of only a theoretical interest. Caprara, Fischetti and Letchford [161] address the separation of maximally violated cuts in the general context of ILP's of the form  $\min\{cx : Ax \leq b, x \text{ integer}\}$ , where  $A$  is an  $m \times n$  integer matrix and  $b$  an  $m$ -dimensional integer vector. For any given integer  $k$  they study *mod- $k$*  cuts of the form  $\lambda Ax \leq \lfloor \lambda b \rfloor$  for any  $\lambda \in \{0, 1/k, \dots, (k-1)/k\}^m$  such that  $\lambda A$  is integer. A *mod- $k$*  cut is called maximally violated if it is violated by  $(k-1)/k$  by the given fractional point  $x^*$ . It is

shown that, for any given  $k$ , separation of maximally violated mod- $k$  cuts requires  $O(mn \min\{m, n\})$  time. This result has applications to both the symmetric and asymmetric TSP. Indeed, for any given  $k$ , Caprara et al. propose an  $O(|V|^2|E^*|)$ -time exact separation algorithm for mod- $k$  cuts which are maximally violated by a given fractional (symmetric or asymmetric) TSP solution with support graph  $G^* = (V, E^*)$ . This implies that one can identify in  $O(|V|^2|E^*|)$  time a maximally violated mod-2 cut for the symmetric TSP whenever a maximally violated comb inequality exists. The reader is referred to [161] for more details.

We now turn to the heuristic separation of specific classes of inequalities.

## 14. Greedy heuristic for minimum cut

Finding a minimum cut that contains a given set  $S_0$  is polynomial. In some of the following separation heuristics this search has to be performed so many times that we will instead use the following *greedy* heuristic – also known as *max-back*.

Consider a graph  $G = (V, E)$  and a weight function  $x^*$  on the edges such that  $x^*(\delta(v)) = 2$  for all  $v \in V$ . Let  $S \subset V$ , we say that  $v \notin S$  sees  $S$  by an amount of  $b(v) = \sum_{u \in S} x_{uv}^*$ . We call  $b(v)$  the *max-back* value of  $v$  relatively to  $S$ . A vertex not linked to  $S$  by any edge has a max-back value of zero. Note that this definition still makes sense if  $v \in S$ , and we will sometimes use it also in this case.

**The max-back heuristic:** Input  $S_0 \subset V$  and  $x^*$ .

- $Cutmin = \sum_{e \in \delta(S_0)} x_e^*$ , for all  $v \notin S_0$ ,  $b(v) = \text{max-back value of } v \text{ relative to } S_0$ . Set  $S = Smin = S_0$ ,  $Cutval = Cutmin$
- While  $S \neq V$ : Choose  $v \notin S$  of maximum max-back value.  $S = S + \{v\}$ ,  $Cutval = Cutval + 2 - 2 * b(v)$ . For all  $t \notin S$ , set  $b(t) = b(t) + x_{vt}^*$ . If  $Cutval < Cutmin$  then  $Cutmin = Cutval$  and  $Smin = S$ .

The idea behind this heuristic is that if one wants to keep the cut small then one should take the vertices that see the current set by the largest amount. Note that the max-back idea is the key notion of the Nagamoshi-Ibaraki minimum cut algorithm (see [622],[623]). The terminology comes from A. Frank.

In general we will not explore the whole graph and will stop the previous loop after a certain number of iterations. We will also store the nodes not in  $S$  with a non zero max-back value in three lists:

- A list `supto1` which contains the nodes that see  $S$  by more than 1

- A list `infto1` which contains the nodes that see  $S$  by strictly less than 1.
- A list `equalto1` which contains the nodes that see  $S$  by exactly 1

These definitions may vary depending on the use we make of them. At this point we will just note that the insertion into  $S$  of nodes of the list `supto1` strictly decreases the value of the cut, and if not empty, the next chosen vertex is in that list. Those of `equalto1` will leave the value of the cut unchanged, whereas those of the last list will increase the value of the cut.

Some separation procedures use another greedy heuristic to find minimum cuts which we will describe later. In that heuristic a path is added at each iteration.

## 15. Graph associated to a vector $x^* \in \mathbb{R}^E$

The *support graph* of  $x^* \in \mathbb{R}^E$  is the graph  $G^* = (V, E^*)$  where  $E^* = \{e \in E : x_e^* > 0\}$ . Before performing separation it is useful to do some transformations on this graph.

By shrinking a set  $S \subset V$ , we mean replacing all vertices in  $S$  by a single one  $s$ , called a *pseudo-node*, and deleting all edges with both extremities in  $S$ . All the edges having exactly one extremity in  $S$  are replaced by edges whose extremity in  $S$  is replaced by  $s$ , the other is unchanged. This may lead to a multigraph, that is there may be more than one edge between a pair of vertices. It is convenient in that case to only deal with a single edge  $e$  whose associated weight  $x_e^*$  is the sum of all the weights of these edges. As the reader has observed, we will use the same notation, that is  $x^*$ , for solution induced by  $x^*$  on the shrunk graph.

If one desires the degree inequality to be satisfied on the new node  $s$  only sets  $S$  with  $x^*(\delta(S)) = 2$  can be shrunk.

We say that the shrinking of a set  $S \subset V$  is *legal* if every violated inequality in the the original graph yields such a inequality in the shrunk graph. This concept is very restrictive. Checking whether there is a violated inequality amounts to checking whether or not vector  $x^*$  is a convex combination of representative vectors of tours. We will come back to this later. Finally we define a shrinking *legal* for a class of inequalities, as a shrinking such that if the original graph contains a violated inequality of that class, then so does the shrunk graph.

Some easy recognizable cases of shrinkings are given in [647], two of them are given below. It is very important to perform these shrinkings before starting any separation heuristic, since they very often consid-



erably simplify the solution and reduce the size of the graph we work on.

A *path of 1-edges* is a maximal path  $P$  in the support graph  $G^*$  such that  $x_e^* = 1$  for all  $e \in P$ .

Most authors advocate replacing a path of 1-edges by a single edge, that is shrinking all the nodes except one extremity of the path. This shrinking is not legal in general for a given class of inequalities, but it is for comb separation. Naddef and Thienel [620] found useful to reduce only those paths with more than 4 edges and then shrink all nodes except the 4 vertices of the two extreme edges. This will leave a path of 4 edges, the middle vertex being the one to which all nodes have been shrunk into. The advantage is that this is legal for most classes of known inequalities and that if one uses the two extremities of one of the extreme edges as a tooth, then we know exactly what vertices we are dealing with.

The second shrinking concerns a path of 1-edges, the extremities  $u$  and  $v$  of which are adjacent to a common node  $w$  with  $x_{uw}^* + x_{vw}^* = 1$ . In this case we can shrink the node set  $S$  of the path. Note that this yields a new edge  $(s, w)$  with  $x_{sw}^* = 1$ , which could yield another shrinking, therefore these shrinking operations have to be done recursively. Padberg and Rinaldi [647] show that this is a legal shrinking.

From now on we assume that these operations have been performed. An example of a graph obtained by such shrinkings is given in Figure 2.25 from the solution of Figure 2.24.

Christof, Reinelt and Wenger use the list of all minimum cuts to shrink the graph down to a graph on  $k \leq 10$  nodes, in all possible ways, and look for a violated inequality in the list of inequalities describing  $STSP(n)$  for  $n \leq 10$ , see [184], [185], [187], [822] for more details.

## 16. Heuristics for Comb Separation

Comb separation has received a lot of attention. The reason is that comb inequalities are the first discovered and the simplest inequalities that do not appear in the integer formulation of the traveling salesman problem. We have already mentioned that the exact separation of the special case of 2-matching inequalities is polynomial.

It is useful to understand how a comb inequality can be violated by  $x^*$ . Assuming that all subtour inequalities are satisfied by  $x^*$ , each tooth has a coboundary of value at least 2. If each tooth has coboundary value 2, the previously mentioned proof of validity, shows that the coboundary of the handle cannot be less than  $2k + 1$  and therefore maximum violation is 1 if all subtour elimination inequalities are satisfied. Therefore if the handle has a coboundary of  $2k + 2$  or more, no violated comb with  $2k + 1$

teeth can be violated. In general to have violation we must have that the sum of the excesses of the values of the coboundaries over their minimum values be less than 1. That is:

$$x^*(\delta(H)) - (2k + 1) + \sum_{i=1}^{2k+1} (x^*(\delta(T_i)) - 2) < 1 \quad (60)$$

### 16.1. The biconnected component heuristic

A *cutnode* of a graph is a vertex whose removal disconnects the graph. If a graph does not contain any cutnode it is said to be *biconnected*. A *block* of a graph is a maximal biconnected induced subgraph. Finding all blocks is easy and can be done in linear time (see [787]).

Let  $G_{-1}^*$  be obtained from  $G^*$  by deleting all edges with  $x_e^* = 1$ . Padberg and Rinaldi [647] use a heuristic in which each block or union of adjacent blocks of  $G_{-1}^*$  is considered as a potential handle  $H$  of a comb. The sets of extremities of an edge  $e = (u, v)$  with  $v \in H$ ,  $u \notin H$ ,  $x_e^* = 1$  are used as teeth. The other teeth are chosen among the adjacent blocks, that is blocks which share a vertex with  $H$ . We advocate to try to grow by the max-back procedure a tooth from each cutnode of  $G_{-1}^*$  contained in  $H$ . This procedure is quite effective in the beginning of the Branch-and-Cut procedure. Moreover, it is very fast.

Note that if the paths of edges of value 1 have not been contracted to a unique edge, then one must add to each block the nodes of such paths linking two of its nodes.

Figure 2.24 gives an example of a fractional point of kroA100, the dotted edges correspond to value .5, the others to value 1. Figure 2.25 gives that same graph after contraction following the rules described earlier. The blocks happen here to be exactly the connected components of the graph from which the edges  $e$  with  $x_e^* = 1$  have been removed and correspond to the nodes of the 6 triangles. Note that in the original graph there was one more block. Several violated 2-matching inequalities are easily found, some in the original graph are comb inequalities which are not 2-matching inequalities. Figure 2.26 shows two of these inequalities and one of the four 2-matching inequalities.

### 16.2. The max-back Heuristic

In this heuristic, starting from a given node chosen in a certain subset  $B$ , we grow a handle using a modified max-back procedure. Once in a while we stop growing the handle and try to grow a number of teeth using again a max-back procedure. If a violated comb has been obtained we stop, else we go back to grow the handle, and so on as long as no

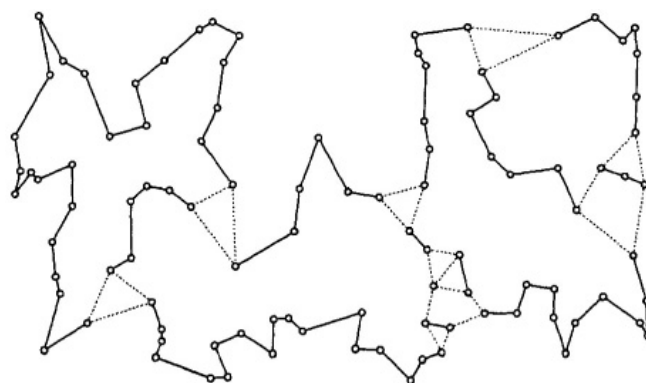


Figure 2.24. Example of a fractional point

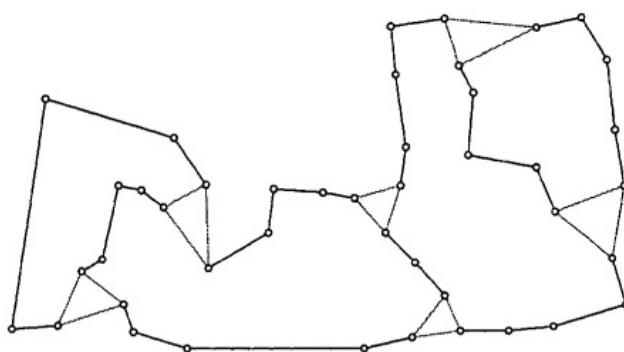


Figure 2.25. The graph of Figure 2.24 after shrinking

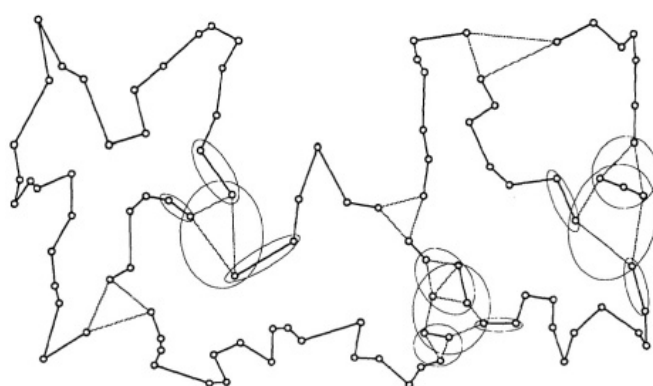


Figure 2.26. Examples of violated combs from Figure 2.24

violated comb has been found and the maximum number of iterations

has not been exceeded. We then choose another node from  $B$  until we have tried with all nodes in  $B$ . We make this more precise now.

In our case we chose  $B$  to be all extremities of paths of 1-edges. Let  $u \in B$ , we grow  $S$  by max-back starting with  $S_0 = \{u\}$ . Let  $v$  be such that  $x_{uv}^* = 1$ , the idea is that  $\{u, v\}$  will be the first tooth. One therefore forbids the max-back procedure to pick up any other node of the path of 1-edges one extremity of which is  $u$ . Nodes of max-back value at least one and not incident to a 1-edge linking it to the current growing handle are included in the handle under construction. Those nodes incident to a 1-edge are chosen if their max-back value is greater than a prescribed value (say 1.7). The idea is that we lose a potential “nice” tooth, but on the other hand the coboundary of the handle will decrease in value. Whenever we introduce a node  $v$  in the handle, we also update the max-back value of all the nodes adjacent to it, including those which are already in the handle under construction.

Every time the previous procedure would choose a node of max-back value strictly less than 1 and the coboundary value is not too close to an even number, we first try to find a violated comb by searching for the right number of teeth. This number is easily computed knowing the value of the coboundary of the current handle. If that value is strictly between  $2k$  and  $2k + 2$ , we need  $2k + 1$  teeth. One tooth is already known, as already mentioned. Nodes of the handle which are incident to a 1-edge the other extremity of which is not in the handle, give us additional teeth. For the remaining teeth, we greedily choose the vertex in the handle with minimum max-back value among those not already in a tooth as the starting node for the next tooth. The rationale behind this is that if a node is poorly linked to the other nodes of the handle, it is likely that we will find a set with small coboundary value containing it that will yield a tooth. The tooth is grown by pure max-back from that node, leaving it grow freely inside or outside the handle, except that it cannot intersect already existing teeth. We do so until either we reach a value very close to 2, or a maximum number of iteration has been exceeded. We then return the best set found. In fact, in view of the search for other inequalities, we return more sets than just the best one, especially if that set contains only one node outside the handle. For other inequalities we will need the information on the best tooth having at least two vertices outside the handle. The weakness of this procedure is that we never change a previously found tooth, hence depending on the order in which we consider the starting nodes, we may or may not succeed in finding a violated comb. This is however unavoidable due to the nature of the greedy heuristics. In the code of Naddef and Thienel [620], the minimum max-back value for the choice of the starting node

of a tooth suffers many exceptions, the main one is as follows, If a path of 1-edges linking say  $u$  and  $v$  is entirely inside the handle, it is very often the case that a useful tooth will contain the whole path. In this case we consider that  $u$  and  $v$  as having a max-back value equal to the sum of their real max-back value minus two.

This heuristic is fast, pretty effective and easy to implement. It suffers from the fact that in many instances many max-back values are identical and one has to be lucky to choose the right node. This is especially true at the beginning of the cutting phase procedure. This is why Naddef and Thienel prefer the much slower but much more successful heuristic described in the next section.

Note that, in some way, this relates to Carr's comb separation algorithm. Once the handle is chosen, we choose the nodes in the handle which will be the seeds, as Carr calls them, of the teeth. The difference is that Carr's method also chooses one seed outside the handle for each tooth.

### 16.3. The Ear Heuristic

We now give another greedy way of growing sets. Given  $S \subset V$ , an *ear* relative to  $S$  is a (possibly closed) path in  $G$  with intermediate nodes not in  $S$ . In most graph theoretic domains where ears are used, i.e. connectivity or matching theory, a single edge between two nodes of  $S$  is considered as an ear, and this path need not be minimal in terms of vertices outside  $S$ . Here we will assume that there is at least one node not in  $S$  in the path and that, except for the first and last node of the path outside  $S$ , none of the nodes of that path are incident to nodes of  $S$ .

Given  $S \subset V$ , by "increasing  $S$  by an ear", we mean adding to  $S$  the ear that yields a set of minimum coboundary value among all sets that can be obtained this way. Of course, like in the case of the max-back procedure, when growing handles we are faced with the dilemma of choosing or not nodes linked to  $S$  by a 1-edge. We use the same rule as in the previous case. In fact the only difference with the previous section is the way sets are grown. In some way we control better the way the set grows and therefore this method typically gives much better results. The only problem is that one has not yet found an efficient method to find the best ear: if we could improve this part substantially, the running times reported below using the Naddef and Thienel separation heuristics would be drastically decreased.

#### 16.4. About minimum cuts

The two comb separation heuristics we will describe later on, as well as the exact separation algorithms for violated combs and domino inequalities in planar graphs we have mentioned, make extensive use of the structure of minimum cuts. Let us assume that all subtour elimination are satisfied by  $x^*$ , that is  $x^*(\delta(S)) \geq 2$  for all  $S \subset V$ . Therefore the value of a minimum cut is 2. since each node defines such a cut. Finding all cuts of value 2 is relatively easy. There are several ways of storing all these cuts in a compact form, either by a *cactus* representation of Lemonosov, Karzanov and Timofeev (see for example [310], [309], [241], [822]), by a PQ-tree of Booth and Lueker [131] or by a poset representation (see [342]).

The main property of minimum cuts which is used is that if the sets  $S_1$  and  $S_2$  define minimum *crossing cuts*, that is  $S_1 \cup S_2 \neq V$ ,  $S_1 \cap S_2 \neq \emptyset$ ,  $S_1 \setminus S_2 \neq \emptyset$  and  $S_2 \setminus S_1 \neq \emptyset$ , then  $S_1 \cup S_2$ ,  $S_1 \cap S_2$ ,  $S_1 \setminus S_2$  and  $S_2 \setminus S_1$  all define minimum cuts. In particular,  $x^*$  representing the capacities of the edges, we have  $x^*(S_1 : V \setminus (S_1 \cup S_2)) = x^*(S_1 \setminus S_2 : S_1 \cap S_2) = x^*(S_2 : V \setminus (S_1 \cup S_2)) = x^*(S_2 \setminus S_1 : S_1 \cap S_2) = 1$ ,  $x^*(S_1 \cap S_2 : V \setminus (S_1 \cup S_2)) = 0$  and  $x^*(S_1 \setminus S_2 : S_2 \setminus S_1) = 0$ . Figure 2.27 summarizes this.

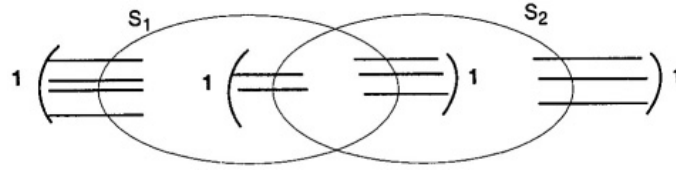


Figure 2.27. Two crossing minimum cuts

We only briefly describe here the PQ-tree structure to store minimum cuts. For each cut the shore that does not contain a prescribed node  $a$  is stored. A *PQ-tree* is a rooted tree with each internal node having at least two children and labelled either *P*-node or *Q*-node. Following [28], for a node  $u$  of a *PQ*-tree, we let  $D(u)$  be the set of leaves of that tree that are descendants of  $u$ .

A *PQ*-tree represents all the shores of the minimum cuts not containing node  $a$  if and only if each shore is either:

- (i)  $D(u)$  for some node  $u$  of the *PQ*-tree
- (ii) The union of  $D(u)$ s for *consecutive* sons of a *Q*-node
- (iii) The union of  $D(u)$ s for any subset of sons of a *P*-node

and each such set is the shore of a minimum cut. Note that the leaves are exactly the nodes of  $G \setminus \{a\}$ . Without going into the details of the

$PQ$ -tree structure, from the preceding lines it is obvious that the sons of a  $Q$ -node are not placed in an arbitrary order.

### 16.5. The Domino Heuristic

Assuming all subtour elimination inequalities are satisfied, a  $2k + 1$  tooth comb is maximally violated if the handle has a coboundary value of  $2k + 1$  and each tooth is tight. That is, all teeth define minimum cuts, but also so do the intersection of each one with the handle and its intersection with the complement of the handle. Moreover for each tooth  $T_i$  we have:  $x^*(T_i \cap H : H) = x^*(T_i \cap H : T_i \setminus H) = x^*(T_i \setminus H : V \setminus H) = 1$  and therefore  $x^*(H : V \setminus (H \cup_{i=1}^{2k+1} T_i)) = 0$ ; see Figure 2.28 where the only edges that can cross the boundary of one of the sets that define the comb are shown.

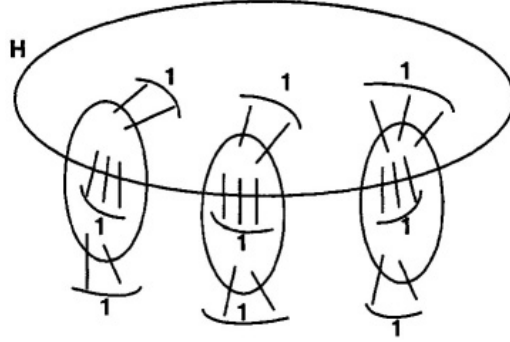


Figure 2.28. Edges in maximally violated comb

Applegate et al. [28] call a set  $T = A \cup B$ , such that  $A \cap B = \emptyset$  and  $x^*(\delta(T)) = x^*(\delta(A)) = x^*(\delta(B)) = 2$  a *domino*. We call an edge  $e \in (A : B)$  such that  $x_e^* > 0$  a *crossing edge* of the domino  $T = A \cup B$ .

If we find a set of  $2k + 1$  pairwise node disjoint dominoes such that the union of their crossing edges form a cut of the support graph of  $x^*$ , then we have found a maximally violated comb. The handle is one of the shores of that cut. Such a set is called a *cutter* in [28]. This is the idea behind the heuristic proposed by Applegate et al. [28].

A *necklace* is a partition  $V_0, \dots, V_i, \dots, V_k$  of  $V$  into tight sets such that  $S = (V_i \cup V_{i+1})$  forms a domino for all  $i$ , where subscripts are taken modulo  $k$ . Each  $Q$ -node  $v$  of a  $PQ$ -tree compatible with  $x^*$  defines a necklace with  $V_0 = V \setminus D(v)$  and  $V_i = D(v_i)$  where  $v_1, \dots, v_i, \dots, v_k$  are the sons of  $v$  in that order.

Given a  $PQ$ -tree compatible with  $x^*$ , let  $\mathcal{D}$  be the set of all dominoes of all the necklaces obtained from the  $Q$ -node of that tree. Applegate et al. in [28] search  $\mathcal{D}$  in a clever way in order to find a set of dominoes

which defines a cutter and therefore yields a violated comb. We refer the reader to [28] for more details. It very often happens that the support graph of a fractional solution is planar and therefore, one may now rather try to use the newly available algorithms of Fleischer and Tardos [310], of Letchford [559] or of Caprara, Fischetti and Letchford [161], although this latter has, so far, not yet been implemented.

## 16.6. The Consecutive Ones Heuristic

We just give here a flavor of the consecutive ones heuristic of Applegate et al [28]. It is easier to understand if one assumes that the minimum cut has a value of two, and that we have a  $PQ$ -tree that represents the set of all minimum cuts. This is not really a restriction since some TSP codes as the one of Naddef and Thienel [620] and [621], only separate on other constraints when all subtour elimination inequalities are satisfied.

A set of subsets of a given ground set is said to have the *consecutive ones property* if one can number the elements of the ground set in such a way that all the elements of any subset have consecutive numbers. Given  $S_i \subset V \setminus \{a\}$ ,  $i = 1, \dots, k$ , where  $a$  is as prescribed node as in Section 16.4, if all these subsets are tight for a certain tour  $\Gamma$ , then they satisfy the consecutive ones property since numbering consecutively the nodes in the order induced by  $\Gamma$  yields the desired property.

Let  $H$  be a vertex set such that  $x^*(\delta(H)) < 4$ . Any tour  $\Gamma$  either intersects the coboundary of  $H$  in two or at least four edges. Let's try and see if it is possible in two edges, that is we want to test whether  $H$  can be tight, when all the currently tight sets remain tight. There is an algorithm that enables to refine the current  $PQ$ -tree to incorporate the cut defined by  $H$  or returns a failure message together with a certificate explaining why this is not possible. This certificate has the form of three tight sets  $T_1, T_2, T_3$  which are incompatible with the fact that  $H$  could be added to our set of subsets preserving the consecutive ones property. The sets  $H, T_1, T_2, T_3$  define a violated comb inequality since the three sets  $T_1, T_2, T_3$  are either pairwise disjoint or one contains the two others, which do not intersect.

The problem is the choice of the set  $H$ . It could be the handle of a former violated comb, plus or minus some nodes. That is. one starts with the handle of a former comb, and possibly increase it by the max-back procedure described earlier.

**Research Question 60** Assume the set of minimum cuts is stored in a cactus instead of a  $PQ$ -tree. Can we obtain a certificate that a given



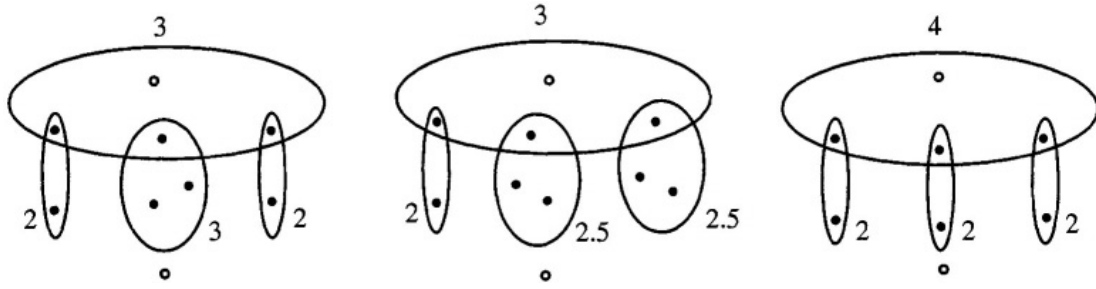


Figure 2.29. Examples of non violated comb inequalities

*set cannot define a minimum cut if all the current minimum cuts remain minimum? Same question if one uses a poset representation.*

To make the question more precise: Take a set  $S$  which does not define a minimum cut. Assume the solution  $x^*$  has changed in such a way that all the minimum cuts remain minimum. Can  $S$  now also define a minimum cut? In case of a NO answer a good certificate is asked for.

## 17. Separation of multi-handle inequalities

### 17.1. How do multiple handles help?

Most of the non-comb inequalities for  $STSP(n)$  we have described have combs as building blocks and therefore have more than one handle. If one wants to design specific separation heuristics (in a template paradigm) it is important to understand how the addition of a handle can yield a violation when none of the constituting comb inequalities is violated. This is explained in details in Naddef and Thienel [621]. Figure 2.29 shows non violated (in fact, tight) comb inequalities. These could have been found in our search for violated combs.

All the figures of this section will follow the following convention as far as numbers are concerned: cut values will be given in floating point format, i.e. 2.0, 3.2, to distinguish them from set coefficients which are integers. In some of the forthcoming figures we also have drawn in bold the non violated comb inequality from which we have started the search for a multi-handle violated inequality.

The key remark in understanding how one of these inequalities can be violated, is to see that if a tooth with large cut value intersects *properly* enough handles with cut value close to an odd integer, then it is likely to be part of a violated inequality. We illustrate this in the following figures.

Figure 2.30 shows how violated path inequalities can be found on the two first cases of Figure 2.29, as long as one can find a second handle that

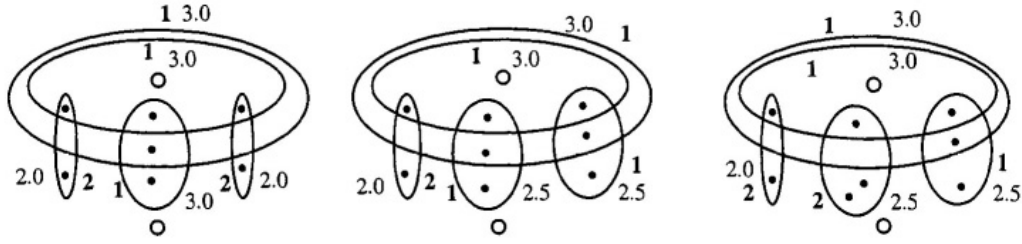


Figure 2.30. Examples of violated path inequalities with 2 handles

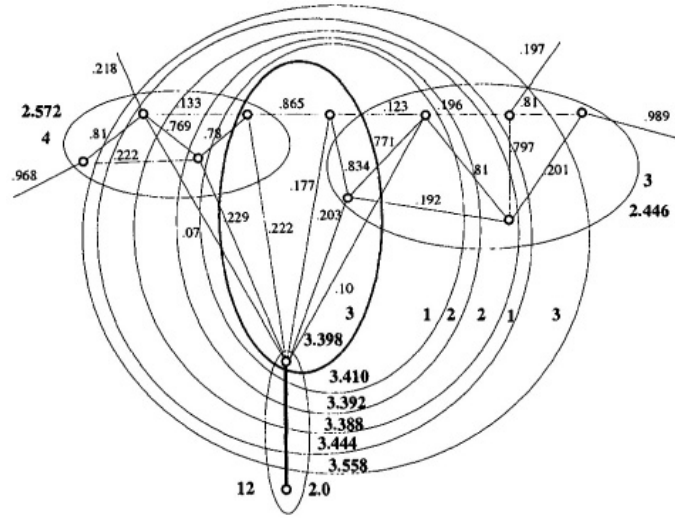


Figure 2.31. Example of a violated path inequality in ts225

intersects the teeth as shown (this is what we meant by *properly*). The last of these drawings shows a case in which the second handle intersects one of the teeth in such a way that the tooth gets a higher coefficient, yielding a weaker inequality. In the two first drawings, the inequalities are violated by 1, it is violated by only .5 in the last one. Note that teeth with cut value close to 2.0 are penalized only marginally if they receive higher coefficients. Figure 2.31 shows an example on the instance ts225, of a non violated comb inequality (central handle in bold) and a violated path inequality with the same set of teeth. The RHS is 76 and the LHS 70.3.

The same argument on teeth with high cut value holds if the handles are disjoint instead of being nested. This yields either bipartition or clique tree inequalities. Figure 2.32 refers to the case of clique trees if only one tooth has a large cut value, and Figure 2.33 to the case in which more than a tooth has such a cut value.

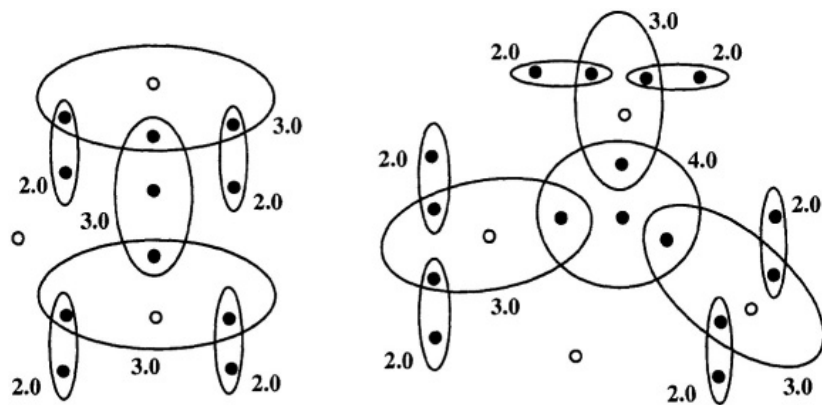


Figure 2.32. Examples of violated clique trees

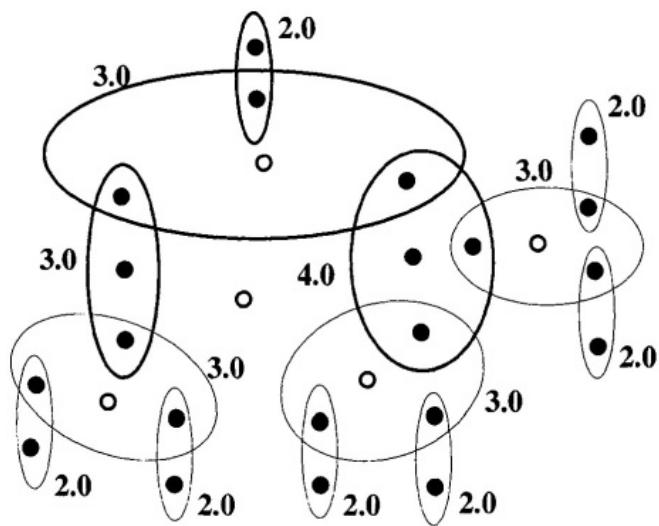


Figure 2.33. Example of violated clique tree

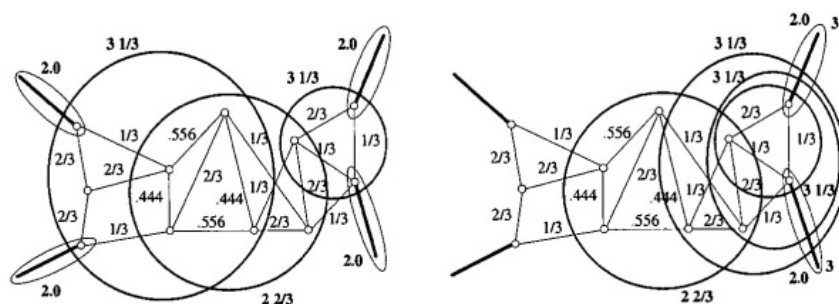


Figure 2.34. Violated clique tree and path inequalities from pr299

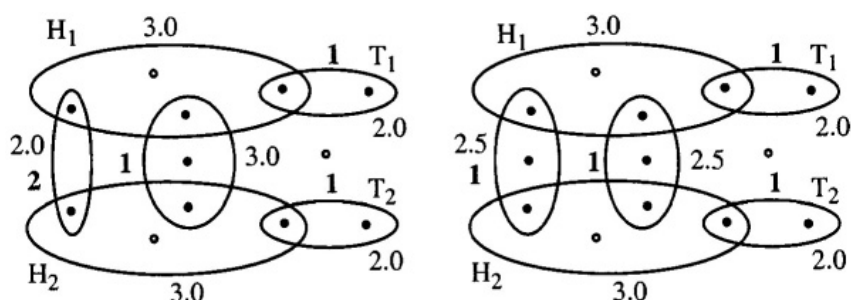


Figure 2.35. Example how ladders can be violated

Figure 2.34 gives an example of a clique tree inequality that can be found this way. It is taken from pr299. The RHS is 18 and the LHS 17.333. In this case there is also a violated path inequality, as shown in the second part of the figure; the two teeth made up by an edge have coefficient 3, the RHS 26 is and the LHS is 24.667. The path inequalities defined by any two of these three handles is also violated.

Ladder inequalities may also help in the same way, but since there can only have two handles, one cannot expect them to accept too high cut values for the teeth. Using the handles to compensate for high cut values for the teeth adds up in using a ladder inequality as a bipartition inequality on two handles (see Figure 2.35). We will see later on how ladders can help to compensate for bad cut values of handles, that is values too close to an even number. The correcting term will then be the key factor. Figure 2.36 gives an example of ladder violation in gil262, the RHS is 16, the LHS is 15. Figure 2.37 gives two violated ladders in a fractional solution to pr439, one with disjoint handles, the other with nested handles, in both cases the RHS is 18, the LHS is 17.667.

Note that so far we have shown how to deal with cases related to combs of the two first types of Figure 2.29, that is in the case in which non-violation is due to teeth of too high cut value. In the last case of that figure, the handle has a too high cut value and finding extra handles

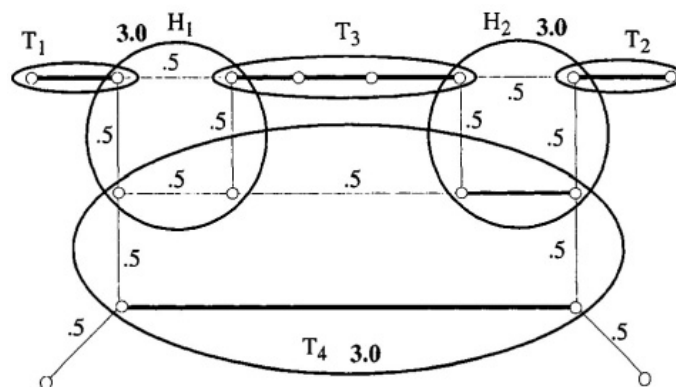


Figure 2.36. Example of ladder violation in gil262

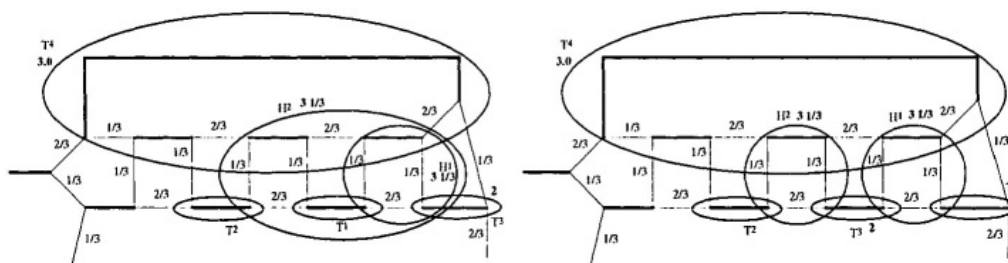


Figure 2.37. Example of violated ladders from pr439

in a way or another would not help. The only known possibility is to use the correcting term of the ladder inequalities. Figure 2.38 shows how this can happen on an example from a fractional solution to pr136. The first drawing shows a tight comb with a handle of cut value 4, the second shows a violated ladder in which the violation comes from the correcting term induced by the edge with black filled extremities. Figure 2.39 gives two examples taken from two fractional solutions to pr439 in which the handles are nested. In both cases the violation comes from the correcting term given by the edge, the two extremities of which are filled in black. All combs that can be built using one of the handles and the three teeth it intersects are not violated. The violation in the first case is  $2/3$  and  $0.518$  in the second.

How can we hope to find such violated multi-handle inequalities? Naddef and Thienel [621] suggest the following strategy.

When searching for teeth in combs, if the best found tooth only has one node outside the handle and its cut value is not close to 2, also return the best tooth with at least two nodes outside the handle. For example, in Figure 2.31 the edge of value .78 in the coboundary of the inner handle yields a tooth of cut value 2.44 which is better than the 2.57

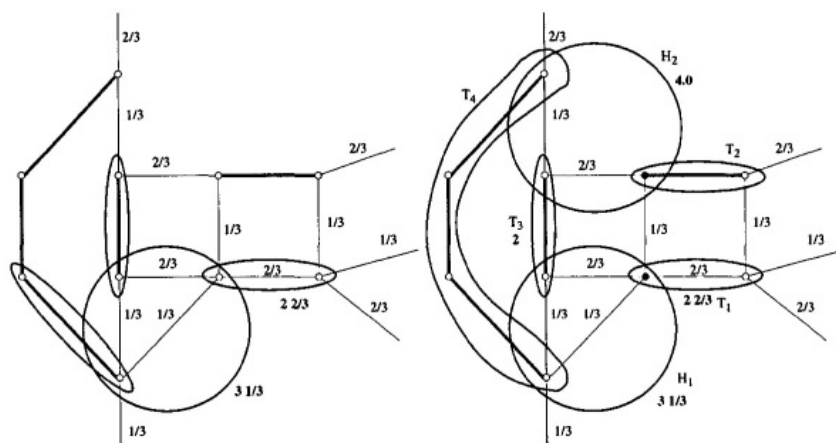


Figure 2.38. Violated ladder from pr136 with correcting term

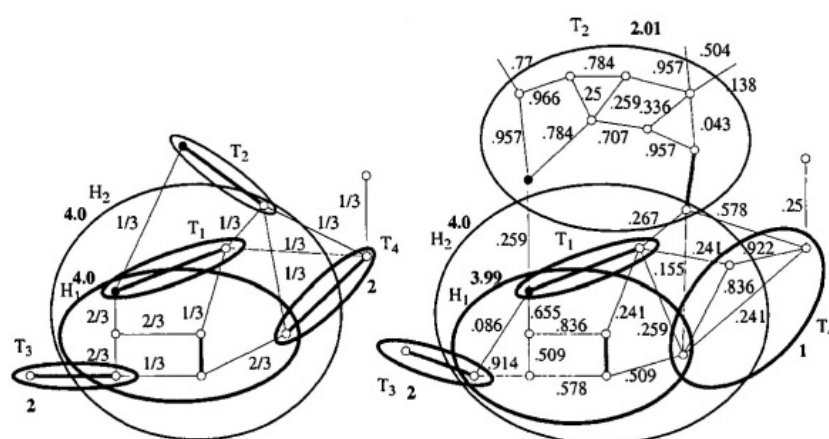


Figure 2.39. Violated ladders from pr439 with correcting term

cut value of the chosen tooth. This is not computationally expensive, since in the search for a tooth one goes anyway beyond the set which is finally returned. The idea is that a tooth with large cut value having only one node outside the handle will be a handicap towards violation that cannot be compensated by finding extra handles as shown in the previous section.

The general strategy is based on a non-violated comb. If we have large enough teeth, in terms of number of nodes outside the handle, we search for a violated path inequality. We try to grow handles, starting with the current one such that their cut values are low enough and that they yield small coefficients for the teeth with large cut value. We can also try to find handles included in the previous one. All this is done by the max-back or the ear procedure described earlier.

If one does not succeed, then one tries to find another type of violated inequality, either in a way similar to that for paths (ladder inequalities with nested handle) or by first growing a new handle which intersects one or several teeth of large cut value, and then searching for new teeth and so on. Everything is done in a greedy way. These simple greedy procedures give surprisingly good results as will be reported in the section on computational results. The interested reader is referred to [620] and [621] for more details.

## 18. Separation outside the template paradigm

So far we have developed separation heuristics that try to find a violated inequality from a prescribed family of valid inequalities. Some other paths have been explored. Applegate, Bixby, Chvátal and Cook in [30] and in [31] have explored another direction which is the topic of this section.

Figure 2.40 shows a portion of a fractional point of the instance fnl4461. When developing a STSPsolver, one very often looks at such drawing of a fractional solution. The fact that one can quite easily visualize fractional solutions, is certainly one of the keys to the progress that has been possible for this particular problem.

When doing so, one looks at small parts of the drawing to try and figure out what one has missed in terms of violated inequalities. In other words, one locally searches for violated inequalities. Therefore we will refer to the following procedure as the *local cut* procedure (which in fact was the first name their authors gave it). For example one may wonder if there is a violated inequality involving the black filled nodes of Figure 2.40. We will see that the answer “yes” will be very easy to

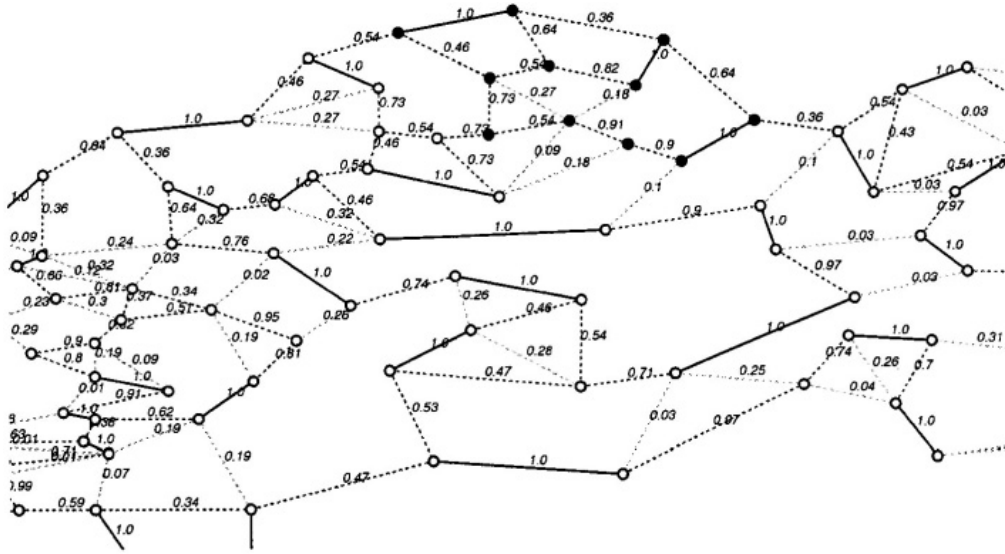


Figure 2.40. Part of a fractional solution of fnl4461

find. Much harder will be the answer to the question of knowing which valid inequality is violated.

Let us contract all the other nodes into a single one to obtain the graph of Figure 2.41 in which the larger circled node represents the node obtained from the shrinking. We will refer to that node as the *pseudo-node*. Note that in both figures, all nodes may in fact represent more than a single node in the original problem because we have applied the traditional legal graph reductions described earlier. This will only matter at the end of our computations.

It happens here that the sum of the  $x_e^*$  on the star of the pseudo-node is two. If we impose this condition, then the *local search* for violated inequalities may turn short for lack of interesting sets. This is why in their development, Applegate et al. [31], do not require this condition. If that value is less than two we have a violated subtour elimination inequality, else it could be anything greater or equal to two. To make things easier to expose, we will assume that this sum is two and try to adapt their method to this case. Let  $\tilde{G} = (\tilde{V}, \tilde{E})$  be the small graph obtained after shrinking a very large set of nodes into a single pseudo node. We continue calling  $x^*$  the fractional solution induced on it by our fractional vector  $x^*$ .

If in  $\tilde{G}$  the solution  $x^*$  (see Figure 2.41) is a convex combination of incidence vectors of Hamiltonian cycles, then  $x^*$  violates no valid inequality in  $\tilde{G}$ . Else there is necessarily at least one such inequality.



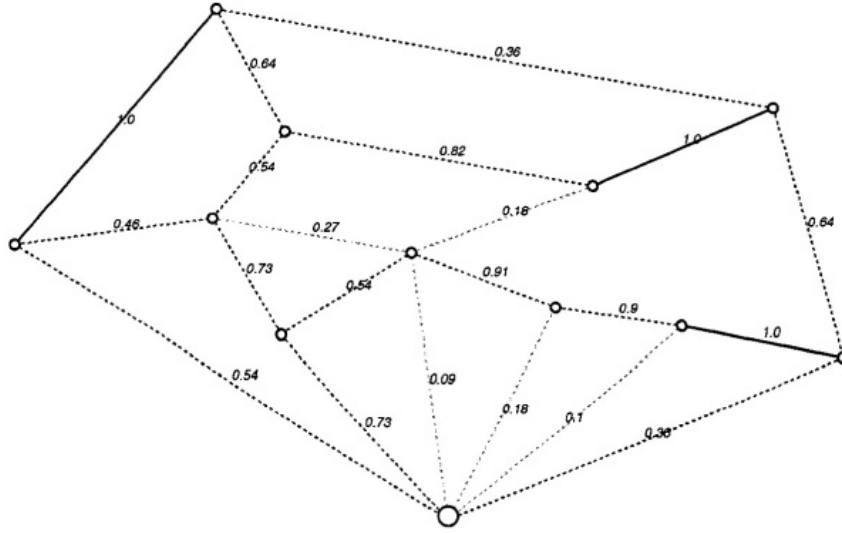


Figure 2.41. The same solution after shrinking

Note that any Hamiltonian cycle candidate to be used in such a convex combination must contain all edges  $e$  such that  $x_e^* = 1$  and no edge for which  $x_e^* = 0$ . We call such tours *compatible* with  $x^*$ . In our example there are only four possible such Hamiltonian cycles. None uses the edge of value .18 not incident to the pseudo node, and therefore this solution is not a convex combination of incidence vectors of Hamiltonian cycles and it is worthwhile to search for a violated inequality in this graph.

To make things more precise, one can solve the following linear program which tries to write  $x^*$  as a convex combination of incidence vectors of Hamiltonian cycles. The summations are on all compatible Hamiltonian cycles  $\Gamma$  except when expressed differently:

$$\max \sum_{\Gamma} \lambda_{\Gamma} \quad (61)$$

$$\sum_{\{\Gamma: e \in \Gamma\}} \lambda_{\Gamma} = x_e^* \text{ for all } e \in \tilde{E} \quad (62)$$

$$\sum_{\Gamma} \lambda_{\Gamma} = 1 \quad (63)$$

$$\lambda_{\Gamma} \geq 0 \text{ for all } \Gamma \quad (64)$$

This problem can be solved by generating first all compatible Hamiltonian cycles, which in general are in small number if the number of nodes is small and we have some edges  $e$  with  $x_e^* = 1$ , since these restrict a lot the number of choices. Note that  $\tilde{G}$  is in general not dense since,

as we will see in the next section, we work on a very restricted set of edges (variables). This problem can be also solved by dynamic column generation. Each column generation is a TSP problem using the dual variables as costs, but on a small instance with many fixed variables. We will comment on this point later on.

If the above LP has a solution then we are out of business, else by the Farkas lemma there exists an inequality  $fx \leq f_0$  which explains this infeasibility, i.e. such that  $fx \leq f_0$  for all compatible tours and  $fx^* > f_0$ . Some commercial linear programming software, such as CPLEX, provide a function that return the coefficients  $f_e$  for  $e \in \tilde{E}$  of such an inequality.

In [31] the following slightly different linear program is solved in order to get the coefficients  $f_e$  directly from the dual without having to use an ad hoc procedure. Here again  $\Gamma$  is a generic compatible tour.

$$\max \quad s \quad (65)$$

$$-\sum_{\{\Gamma: e \in \Gamma\}} \lambda_\Gamma + sx_e^* + w_e = 0 \text{ for all } e \in \tilde{E} \quad (66)$$

$$\sum_{\Gamma} \lambda_\Gamma - s = 0 \quad (67)$$

$$-1 \leq w_e \leq 1 \quad \text{for all } e \in \tilde{E} \quad (68)$$

$$\lambda_\Gamma \geq 0 \text{ for all } \Gamma \quad (69)$$

This linear program is unbounded if and only if  $x^*$  is a convex combination of incidence vectors of tours of  $\tilde{G}$ . Else letting  $f_e$ , for  $e \in \tilde{E}$  be the dual variables associated to constraints (66),  $f_0$  that associated to constraint (67),  $u_e$  and  $v_e$  those associated to the bounds on the components of  $w$ , the dual writes:

$$\min \quad \sum_{e \in \tilde{E}} (u_e + v_e) \quad (70)$$

$$-\sum_{e \in \Gamma} f_e + f_0 \geq 0 \quad \text{for all compatible tour } \Gamma \quad (71)$$

$$\sum_{e \in \tilde{E}} f_e x_e^* - f_0 = 1 \quad (72)$$

$$f_e + u_e - v_e = 0 \quad \text{for all } e \in \tilde{E} \quad (73)$$

$$u_e \geq 0, v_e \geq 0 \quad \text{for all } e \in \tilde{E} \quad (74)$$

Any feasible solution to this dual problem yields an inequality  $fx \leq f_0$  which is valid for all compatible tours as shown by constraints (71) and such that  $fx^* = f_0 + 1 > f_0$  by Constraint (72). Note that due to the Objective Function (70), if one finds a violated inequality, one finds

one which minimizes  $\sum_{e \in \tilde{E}} |f_e|$ , so it is in some sense a most violated inequality.

The steps in [31] (slightly modified for sake of simplicity to the case of a pseudo-node of “degree” two) are:

- 1 Turn the inequality  $fx \leq f_0$  into one with integer coefficients.
- 2 Transform it into an inequality valid for all tours which only use edges in  $\tilde{E}$ .
- 3 Transform it into a facet inducing inequality on  $\tilde{G}$ .
- 4 Perform a sequential lifting to compute the coefficients on the edges not in  $\tilde{E}$ . Assume one has transformed the inequality into one in “greater or equal” form,  $gx \geq g_0$ , which can easily be done by multiplying by  $-1$  both sides and then adding some degree equalities in order to bring back all coefficients non negative. Then each lifting operation refers again to a STSP on a very small graph, which does not mean that things will be necessarily easy. Experiments carried out by Jünger, Reinelt and Rinaldi [474] show that when the coefficients of the objective function are those of weird facet inducing inequalities, then the TSP may be extremely difficult to solve by Branch and Cut even for very small instances. For this last reason it is advisable to put a time limit on this phase and return a failure message if it exceeds that time.
- 5 Transform the inequality into a facet inducing inequality using a tilting procedure inspired by Minkowski. Procedure also used in Step (2). Note that Applegate et al. do not need this step, but when dealing with Hamiltonian cycleslifting an edge may increase the dimension of the underlying polytope by more than one unit, which is not the case when one uses closed walks. On the other hand, Applegate et al. need a more sophisticated routine for the column generations and the lifting procedures which are no longer TSPs.
- 6 Put in TT form. For technical reasons due to their Branch-and-Cut code, Applegate et al. only keep those TT inequalities that can be put in closed set form.
- 7 Finally the shrunk nodes are expanded using 0-liftings.

Applegate et al. report excellent results with their procedure. We will report some of their results in the section devoted to computational results.

## 19. Branch-and-Cut implementation of the STSP

The Branch-and-Cut method is very simple in its principle, but once we want to use it there are many technical points to address. Figure 2.42, taken from Jünger, Reinelt and Rinaldi [474], shows the flowchart of a Branch-and-Cut implementation. For a detailed study on Branch-and-Cut we refer the reader to Jünger and Thienel [477], [478], Thienel [790], Jünger, Reinelt and Rinaldi [474]. From now on  $lp$  stands for linear program. We will only address the following points:

- Management of variables
  - active variables:** What variables should one work with?
  - pricing:** How often should we apply variable pricing, and how should one do it?
  - fixing:** How to perform variable fixing?
- Management of constraints
  - separation:** What separation strategy should one choose?
  - storing:** How should one store the constraints?
  - cleaning:** How to keep the  $lp$  of reasonable size?
- How to obtain a good feasible solution?
- Branching
  - When** should one decide to stop separating and start branching?
  - How** should we perform the branching and how much effort should one invest in it?
  - Which** subproblem has to be solved next?

We will have to refer very often to two implementations of the Branch-and-Cut algorithm for the STSP, that of Applegate, Bixby, Chvátal and Cook, and that of Jünger, Naddef, Reinelt and Thienel. We will refer to them by ABCC and JNRT.

### 19.1. Variables

Since we are working on complete graphs it is out of question to have all the variables present in the  $lps$ . One starts with a reasonably sized set of variables. The set of corresponding edges is usually referred to as the edge set of the *sparse graph*. There are various ways of choosing the



that the  $k$  nearest neighbors is not in general a good choice since some instances have optimal solutions containing edges linking some nodes to far neighbors. For example he points out that in the optimal solution to att532 there is an edge that links a node to its 22nd nearest neighbor. Helsgaun also noted that an optimal tour contains between 70% and 80% of the edges of the minimum cost 1-tree. A minimum cost 1-tree is obtained by taking a minimum cost spanning tree of  $G \setminus \{1\}$  to which one adds the two least cost edges incident to node 1. The *marginal value*  $\mu_e$  of an edge  $e$  not in the minimum cost 1-tree  $T$ , is defined as the minimum cost of a 1-tree containing edge  $e$  minus the cost of  $T$ . A marginal cost of zero means that there exists a minimum cost 1-tree that contains  $e$ , else this cost is positive. A good choice of the sparse graph is to take all edges of  $T$  and all the edges with marginal value less than a given value  $\alpha$ . Helsgaun shows in [446] how this can be done efficiently.

Of course we have no guarantee that the optimal solution is among those edges. Therefore one must regularly perform what is called a *pricing*, complete or partial. If one started with the  $k$  nearest neighbor graph, it seems quite logical to first search among the edges in the  $k + t$  nearest neighbor graph, with  $t$  small (e.g.,  $t=2$  or 4). The edges of the distance 2 closure of the Delaunay triangulation, that is the edges linking nodes at distance 2 on the Delaunay triangulation, seems also a good choice in the other case. These edges are often referred to as those of the *reserve graph*. We will see that this pricing operation, in some sense, guides many decisions in the design of the Branch-and-Cut code. The pricing is done using the optimal solution to the dual problem. How often should it be performed is not an easy question to answer. For separation heuristics based on template paradigms, too many edges seem to be a handicap because the lp solutions seem to become too fractional. For example starting directly with the five nearest neighbor graph (instead of the three), very often yields a worse running time and more branch and cut nodes in the enumeration tree. Conversely, it happens that trying to avoid a complete pricing, one will spend quite some time separating while the lower bound is over the (yet unknown) optimal value. In the JNRT code, a reserve pricing is performed every 10 iterations, a complete pricing is performed if less than 2 variables were priced in from the reserve graph. A complete pricing is necessary in all cases when the solution to the lp is feasible (integral), or when the lower bounds exceeds the upper bound. Even in this case, only a restricted number of variables is generated before one reoptimizes. In all cases of the TSPLIB only a very small percentage of the variables are used.

Variable fixing is an important point in Branch-and-Cut. A variable can be fixed to 0 (resp. 1) if one does not cut off all optimal solutions when we never (resp. always) take that edge. This can be done in various ways. The first is by *reduced cost*. Let  $GUB$  (Global Upper Bound) be the best known value of a tour,  $LB$  (Lower Bound) be the optimal value of our current lp and assume that a complete pricing has shown that the current lp solution  $x^*$  is globally optimal. Let  $e$  be such that either  $x_e^* = 0$  or  $x_e$  is not a variable of the lp. Let  $\bar{c}_e$  be its reduced cost and assume  $\bar{c}_e > 0$ . If  $\bar{c}_e + LB \geq GUB$  one can fix the corresponding variable to zero, delete it from the lp, if necessary, and definitively forget about it. One advantage is that pricing will not have to be performed on it again. Note that the case “=” in the preceding formula may eliminate all optimal solutions, but in this case it is not important since one already has at hand such an optimal solution, namely the solution that yields the current  $GUB$ . With the same argument one can fix a variable  $x_e$  to 1 which is such that  $x_e^* = 1$  and  $LB - \bar{c}_e \geq GUB$  (this variable is non basic and at its upper bound value).

Variables can be fixed also by *logical implication*. For example, if two incident edges have been fixed to 1, then all other edges incident to the common node can be fixed to 0. If all edges incident to a node except two are fixed to 0, then the two remaining ones can be fixed to 1. The edge linking the extremities of a path of edges fixed to 1 can be fixed to 0. These variable fixings can be done at any node of the search tree, but are only valid in the corresponding subtree. One usually talks of variable *setting* in case the node is not the *current* rootnode of the search tree, where by *current* rootnode we mean the lowest node in the search tree which is a common ancestor of all active nodes.

## 19.2. Constraints

As one would expect from the name of the method, constraint generation is the key point in a Branch-and-Cut code. As seen in the previous sections there are a variety of different constraint types. Should one give priority to some over the others? The strategies of the two latest codes devoted to the TSP diverge quite a bit in the separation strategies. The JNRT code will first search for violated subtour inequalities and only search for other violated inequalities once no subtour inequality is violated. It was generally taught that it was too costly to do so. The computational results that will be reported in the next section show that it is not the case. The ABCC code does not do so, following the strategy of all the preceding codes of Hong and Padberg, Grötschel and Holland, and Padberg and Rinaldi. The JNRT code only relies on

template paradigms, the ABCC code also uses more general separation procedures.

There are many theoretical ways of measuring the strength of a facet inducing inequality (see [618]).

- The number of tours that are on the facet.
- The volume of the subtour elimination polytope it cuts off.
- Its distance to the closest vertex of the subtour elimination polytope it cuts off.

Practically, the only measure that counts is the help in solving an instance? In this respect subtour elimination inequalities and comb inequalities are the strongest inequalities. For the other inequalities it depends a lot on the instance. Path inequalities seem to be the next useful family.

Before the work of Applegate, Bixby, Chvátal and Cook, several groups have thought of using general cutting planes for integer programming such as the Gomory cuts or the lift and project cuts of Balas, Ceria and Cornuéjols [68] and [69]. The major difficulty with this approach deals with the management of the variables. Remember that we are working with a sparse graph, that is not all variables are considered at a given time, and that one must be able to do pricing efficiently. To do so one must be able to retrieve very easily the coefficient of any variable in a given inequality without storing it explicitly. This can easily be done if the inequality is in closed-set form: one stores each set that defines the inequality, together with its coefficient, then the coefficient of an edge is just the weighted sum of the coboundaries it belongs to. This is why the ABCC code only keeps closed-set form inequalities in its general separation procedure.

Linear programs tend to grow very big, therefore one has to remove regularly those constraints that are no longer active. A good idea is also to remove those that have been very little active in the past  $k$  iterations, that is have had an optimal associated dual variable close to zero. This has to be done with caution in order not to destroy the basis. Note that we never delete variables except when fixing to 0. This may be necessary for some large instances in which variables would flow in. Removed subtour elimination inequalities are discarded since they can easily be recovered, the others are put into a pool which is regularly searched through. A too costly strategy for large and difficult instances and which has been in use in the first TSP codes, is to search first the pool for violated inequalities, and then call the separation heuristics in case of failure. This has the advantage of guaranteeing that no inequality



is duplicated in the pool. This strategy is no longer used because the pool of constraints grows very big and searching it is very time consuming. One must then make sure not to store the same inequality twice.

### 19.3. Upperbounding

We now address the problem of finding a good tour. The aim is on one hand to enable efficient variable fixing, and on the other not to explore useless branches of the search tree. Moreover, a very good upper bound makes the choice of the tree exploration strategy less critical.

Traditionally a heuristic is called to yield a first value and a tour, the edges of which are used in the sparse graph. Most codes try to exploit the lp solution, i.e., they try to take advantage of the information the lp solution  $x^*$  carries. It seems more probable that an edge  $e$  with  $x^*$  close to 1 is in an optimal solution than one with such a value close to 0. Based on this, an initial tour is built and a Lin-Kernighan ([563],[446]) type heuristic is called to try and improve it. It is important not to try and improve twice a same tour. This can be done using a hash function, see [475], [446] and [587].

Any integer feasible solution of the lp encountered on the way also can yield a better tour.

### 19.4. Branching

Branching occurs when one has renounced to solve the instance with the pure cutting plane method. This may occur either because one does not find any more violated inequalities or because the lower bound has not progressed in a significant manner in the past iterations. This last phenomenon is known as *tailing off*. Before branching a complete pricing is performed in order to be able to fix as many variables as possible, since these fixings will be definitive. The value of the objective function of the current lp, that is the lower bound, is known as the *root value* or *root bound*. We assume the reader familiar with the Branch and Bound terminology.

Since, in some sense, resorting to branching is felt as a failure, not much attention has been dedicated for a long time to this operation. Unfortunately powers of 2 grow very fast and it has become obvious that one must do that operation carefully.

The objective of branching is to split a certain subset of tours into two smaller subsets, to each subset corresponds a lp relaxation which does not contain the current fractional solution as a feasible solution. This can be done in various ways. The classical way is to choose some edge  $e$  such that the corresponding variable  $x_e^*$  is fractional. The tours

are split into those that contain  $e$  and those that do not. This is done by modifying the bounds of the corresponding variable in the current lp, moving the lower-bound to 1 in one case, the upper-bound to 0 in the other. This creates two new problems that are added to the list of problems to solve. To choose among all fractional variables, the mostly used criterion is to choose one of value close .5, and among those one with highest cost. Clochard and Naddef [203] advocate using subtour inequalities to perform branching. Let  $S \subset V$ , with  $x^*(\delta(S))$  not too close to an integer value, say  $2k < x^*(\delta(S)) < 2k + 2$ . Then one can split the tours into those that cross the boundary of  $S$  at most  $2k$  times and those that do so at least  $2k + 2$  times. This is done by adding the appropriate inequality to the current lp. The difficulty is in finding a convenient set  $S$ . In the JNRT code this is done by using handles of previously found combs, star or clique trees, but there is a lot of room for improvement in this choice.

The branching inequality could be anything else. Another possibility is, given two disjoint sets  $S, T \subset V$ , split the tours between those which use at least an edge from  $(S : T)$  and those which don't. This is efficient if the shortest edge of  $(S : T)$ , in terms of cost (not  $x^*$  value) is large and a fractional amount of these edges are used in the solution  $x^*$ , an amount close to .5 also seems a good choice. Figure 2.43 illustrates this last branching rule on a fractional solution to fl1400. We have not put in the edge values. Each cluster of nodes  $A$ ,  $B$  and  $C$  contains several hundreds of nodes. We have  $x^*(A : B) = x^*(A : C) = x^*(B : C) = 0.5$ . There is only one edge  $e$  between  $B$  and  $C$  with non-zero  $x^*$  value, and it is likely to be selected by the standard criterion (close to .5 and high cost). What will happen on the side of the search tree in which one will have set  $x_e = 0$ ? Since many edges linking  $B$  and  $C$  have costs very close to that of  $e$ , it is very likely that one will appear with that same value (or several adding up to that value). The result will be a problem that will have a lower bound almost equal to that of its father and no progress will have been made. This is typical of a case in which one should branch by saying that  $x(B : C) = 0$  on one side and  $x(B : C) \geq 1$  on the other side. Two other candidates are of course  $x(A : C)$  and  $x(A : B)$ . Note that this fractional point contains some maximally violated combs which most certainly could be found using the domino separation procedure, but not by the heuristics of Naddef and Thienel, which is certainly why some codes such as the ABCC code do very well on this instance and the JNRT code does very poorly. One of these combs is shown in Figure 2.43, the set  $C$  which is one of the teeth, the other sets defining the comb are shown in thin lines.

In general branching on variables does pretty well, but it is disastrous on difficult instances. Trying to perform better on these instances unfortunately has a price on the easier instances. But this is not specific to the TSP. In fact, NP-hardness means that there are very difficult instances which may be rare. One has at some point to decide how much one is willing to sacrifice on the solution time for the easy instances in order to perform not too badly on those rare instances.

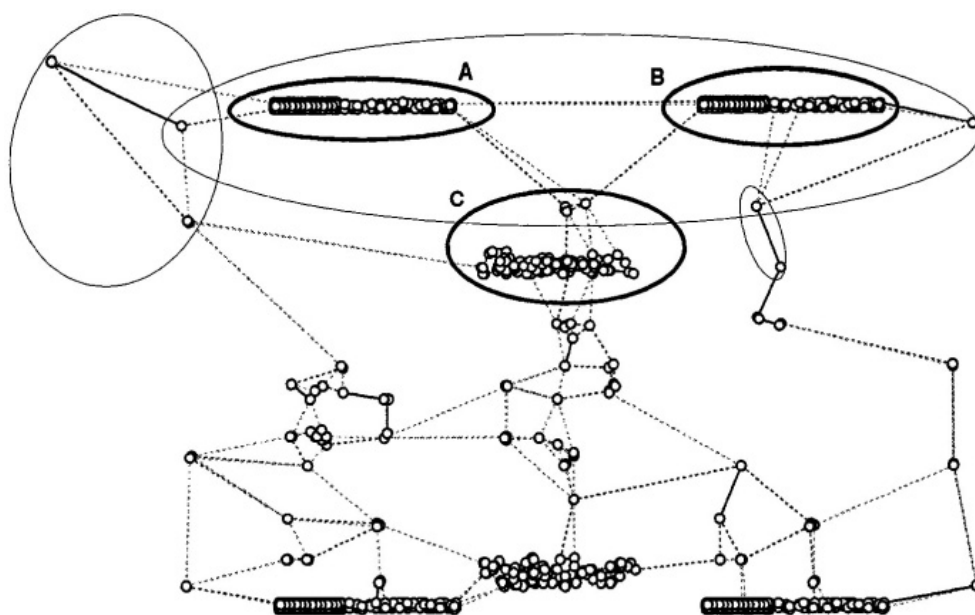


Figure 2.43. Illustration for the third branching rule

Applegate et al. introduced *strong* branching in an effort to optimize the probability of performing a good choice. It roughly goes as follows: choose a large enough set of candidates for branching (variables or inequalities). Test each of them by solving the corresponding lp relaxation only. The standard criterion is to choose the candidate for which the smallest of the two lower bounds of its sons, is the largest. Jünger et al. adapted this criterion a little by choosing among those with the same rounded smallest lower bound, the one with largest other lower bound, that is between a candidate yielding two bounds of 125.85 and 125.99 and one with 125.45 and 127.36, they choose the second. There is no reason, other than simplicity, to restrict to the rounded value; a better choice would be to choose among those which are within a certain interval, the one which has a larger upperbound. Computational experiments show that the time spent in choosing a good candidate definitively pays off, at least on difficult instances.

While performing this strong branching, one may encounter a bound which exceeds the upper bound. One has to take advantage of this by either setting the corresponding variable to the other value, or by adding the converse inequality to the lp.

We are left with the problem of choosing which open problem to solve next. It is well known in Branch-and-Bound that if the initial upper-bound equals the optimal value, whatever exploration strategy one uses, the same tree will be explored. This is no longer true in Branch-and-Cut. This is due to the fact that the inequalities one generates are globally valid, that is wherever they have been generated in the search tree, they are valid at any node of the search tree. The non-trivial inequalities, as already mentioned, are stored in a pool through which one searches regularly. The content of that pool depends on the previously visited nodes of the search tree, and since Branch-and-Cut is highly unrobust, the whole search may be considerably changed. Nevertheless it seems a good idea to go depth first if one believes that the current feasible solution is optimal. If not it seems that branching on the mostpromising open problem (the one with lowest lower bound) is the best choice.

## 20. Computational results

The two latest implementations of the Branch-and-Cut method for the TSP are that of Applegate, Bixby, Chvátal and Cook (ABCC) known as *CONCORDE*, and that of Jünger, Naddef, Reinelt and Thienel (JNRT). The latter relies on a general purpose Branch-and-Cut system known as *ABACUS* and uses the separation heuristics of Padberg and Rinaldi for subtours, and the ones of Naddef and Thienel for the other inequalities. The initial upperbounding is done using Andre Rohe's TSP implementation of Lin Kernighan. *ABACUS* is a system that manages all parts of Branch-and-Cut which are problem independent, making the development of optimization software based on Branch-and-Cut much easier since one may concentrate on problem-specific technicalities. Of course there is a price, which for the TSP ranges somewhere between 10% and 25% on CPU time.

Another main difference between these two codes is the separation strategies. JNRT only calls for separation of violated inequalities which are not subtour elimination inequalities when none of the latter are violated. They also restrict to separation into known classes of facet inducing inequalities. This is not the case for ABCC, as already seen.

Table 20 gives a comparison of times and number of nodes of the search tree between the two codes. The times reported below are for JNRT on a Sun Ultra-60 with a 295 MHz processor. For CONCORDE (ABCC) on

a 500 MHz Compaq XP1000 workstation. Following some benchmarking of ABCC, and due to the fact that JNRT use a general purpose Branch-and-Cut code, a factor of 4 or 5 seems to be reasonable to compare CPU times. These times, like all other figures should be taken with caution. Branch-and-Cut is a highly unrobust method: a slight change in some parameter setting may change considerably the number of nodes of the Branch-and-Cut tree and the CPU time. Times also fluctuate considerably depending on whether or not the upper-bound has been found early enough to save useless computations. For the JNRT code we also give, for some relevant instances, the results of runs in which the optimum value was entered as the initial upper bound. The interest of such data is to differentiate those instances for which the separation of valid inequalities is the difficulty from those for which it is the upperbounding. The data for the instance fnl4461 for the ABCC code also corresponds to the initial upperbound set to the optimal value and for pcb3038, in the ABCC code the initial tour happens to be the optimal one. JNRT chose the option to privilege separation over branching, in the aim of testing how good the separation heuristics are. It may have been faster sometimes to branch much earlier.

A first conclusion is that separation using template paradigms does pretty well in most cases. The local cut separation procedures helps in some difficult instances. Another conclusion is that the size of a problem is not really what makes it difficult. Finally, if one looks into the recent history of TSP solving, one realizes that good separation is the first clue to success, the second being clever branching. The instance ts225 was solved by Applegate et al. for the first time in over 5000 BC nodes in about two years of CPU time, then in a little over an hour and about 40 BC nodes by Naddef and Thienel using their separation routines, and now is solved in only a few seconds by CONCORDE. The same is illustrated by the instances pcb3038 and fnl4461 which were first solved in a few years of CPU time and now in less than one day of CPU. Table 2.2 gives the results of CONCORDE on very large instances. The parameter settings are in general different from the ones used for the previous table. Sometimes the optimal tour value was an input. See the web page of CONCORDE for more information.

## **21. Conclusion**

In this chapter we have seen an example of the efforts that must be carried out in order to achieve good performance in the search for optimal solutions of large instances of the symmetric TSP via Branch-and-Cut. This effort is two-fold. First a deep theoretical research in

| Name    | BC nodes |      |      | time in mn:ss |        |        |
|---------|----------|------|------|---------------|--------|--------|
|         | JNRTopt  | JNRT | ABCC | JNRTopt       | JNRT   | ABCC   |
| pr76    | -        | 1    | 1    | -             | 0:35   | 0:02   |
| gr120   | -        | 1    | 1    | -             | 0:01   | 0:02   |
| pr136   | -        | 1    | 1    | -             | 0:02   | 0:04   |
| pr144   | -        | 1    | 1    | -             | 0:01   | 0:02   |
| pr152   | -        | 1    | 1    | -             | 0:02   | 0:08   |
| u159    | -        | 1    | 1    | -             | 0:01   | 0:01   |
| rat195  | -        | 1    | 1    | -             | 1:41   | 0:22   |
| d198    | -        | 3    | 3    | -             | 0:20   | 0:12   |
| kroA200 | -        | 1    | 1    | -             | 0:18   | 0:07   |
| kroB200 | -        | 1    | 1    | -             | 0:11   | 0:04   |
| gr202   | -        | 1    | 1    | -             | 0:09   | 0:05   |
| ts225   | -        | 57   | 1    | -             | 86:26  | 0:21   |
| pr226   | -        | 1    | 1    | -             | 0:03   | 0:04   |
| gr229   | -        | 3    | 3    | -             | 3:31   | 0:39   |
| gil262  | -        | 1    | 1    | -             | 0:43   | 0:13   |
| pr264   | -        | 1    | 1    | -             | 0:03   | 0:03   |
| pr299   | -        | 1    | 3    | -             | 1:06   | 0:17   |
| lin318  | -        | 1    | 1    | -             | 0:40   | 0:10   |
| rd400   | 13       | 15   | 15   | 2:39          | 4:38   | 2:28   |
| fl417   | -        | 1    | 5    | -             | 0:55   | 0:58   |
| gr431   | 9        | 9    | 13   | 7:02          | 10:40  | 2:13   |
| pr439   | -        | 5    | 15   | -             | 8:39   | 3:36   |
| pcb442  | -        | 9    | 9    | -             | 1:33   | 0:50   |
| d493    | -        | 1    | 5    | -             | 10:29  | 1:53   |
| att532  | 5        | 5    | 7    | 6:23          | 9:47   | 1:50   |
| ali535  | -        | 1    | 3    | -             | 2:18   | 0:53   |
| pa561   | 7        | 9    | 17   | 27:16         | 43:20  | 4:07   |
| u574    | -        | 1    | 1    | -             | 0:35   | 0:23   |
| rat575  | 7        | 11   | 25   | 4:47          | 20:38  | 6:03   |
| p654    | -        | 1    | 3    | -             | 0:15   | 0:15   |
| gr666   | -        | 1    | 3    | -             | 8:18   | 0:50   |
| u724    | 9        | 9    | 11   | 10:05         | 20:45  | 3:45   |
| rat783  | -        | 1    | 1    | -             | 2:13   | 0:38   |
| dsj1000 | -        | 1    | 7    | -             | 75:27  | 6:50   |
| pr1002  | -        | 1    | 1    | -             | 0:59   | 0:34   |
| u1060   | 7        | 29   | 21   | 9:11          | 55:09  | 9:31   |
| vm1084  | 7        | 7    | 11   | 66:25         | 81:21  | 10:04  |
| pcb1173 | 13       | 25   | 19   | 21:49         | 77:23  | 7:48   |
| rl1304  | -        | 1    | 1    | -             | 12:01  | 3:09   |
| rl1323  | 33       | 55   | 25   | 327:35        | 407:01 | 62:20  |
| nrv1379 | -        | 7    | 19   | -             | 29:09  | 9:38   |
| d1655   | -        | 1    | 5    | -             | 25:33  | 4:23   |
| vm1748  | 9        | 19   | 17   | 155:56        | 318:57 | 37:03  |
| pr2392  | -        | 1    | 1    | -             | 7:44   | 1:47   |
| pcb3038 | 265      | -    | 313  | 5771mn        | -      | 1347mn |
| fnl4461 | 409      | -    | 213  | 9410mn        | -      | 890mn  |

Table 2.2. Computational results

| Name     | BC nodes | time                |
|----------|----------|---------------------|
| rl5915   | 161      | 644h 20mn           |
| rl5934   | 205      | 163h 35mn           |
| pla7397  | 101      | 119h 10mn           |
| rl11849  | 431      | $\simeq$ 155 days   |
| usa13509 | 9539     | $\simeq$ 4 years    |
| d15112   | 164569   | $\simeq$ 22.6 years |

Table 2.3. Very large instances with Concorde

order to study the convex hull of the solutions. Note that even if one uses separation routine outside the template paradigm, this study is necessary, since some steps of the algorithm need these theoretical results to justify them. Following that theoretical study, one must invest in a relevant algorithmic and implementation effort. The implementation effort is unfortunately now far too high for a newcomer. If this very exciting field is not to die, the only hope is that large pieces of existing source code be made available to anybody interested in developing a new challenging program. We hope that this will be the case in a near future.

## Chapter 3

# POLYHEDRAL THEORY FOR THE ASYMMETRIC TRAVELING SALESMAN PROBLEM

Egon Balas

*Graduate School of Industrial Administration*

*Carnegie-Mellon University*

*Pittsburgh, PA 15213, USA*

*eb17+@andrew.cmu.edu*

Matteo Fischetti

*D.E.I., University of Padova*

*Via Gradenigo, 6/A*

*35131 Padova, Italy*

*fisch@dei.unipd.it*

### 1. Introduction

The application of polyhedral methods to the TSP started in the mid-1970's (see [396] and [402] for the first major breakthroughs). For more than a decade, until the late 1980's, the main emphasis was on the symmetric TSP (STSP). There were several reasons for this: the traveling salesman paradigm suggests a geometric interpretation in which the costs are symmetric; some of the important real world applications, like in chip manufacturing, are symmetric; the polyhedral formulation of the symmetric TSP connects nicely to matching theory and borrows from the latter the family of facet defining 2-matching inequalities; finally, the asymmetric TSP (ATSP) can be reduced to a STSP on an undirected graph with twice as many nodes.

Nevertheless, there are good reasons to study the asymmetric TSP on its own. First, the asymmetric TSP is the more general one, which subsumes the symmetric TSP as a special case.



Second, the structure of the ATS polytope – i.e. the convex hull of incidence vectors of TS tours in a digraph – is much richer than that of the STS polytope. Every facet  $F$  of the STS polytope defined on a complete undirected graph  $G$  corresponds (trivially) to a face  $F'$  of the ATS polytope defined on the directed graph  $G'$  obtained from  $G$  by replacing every edge with a pair of antiparallel arcs. But the symmetric inequalities obtained this way define only a tiny fraction of the multitude of facets of the ATS polytope, the vast majority of the latter being defined by asymmetric inequalities that have no counterpart for the STS polytope defined on  $G$ .

Third, there are many important real-world problems that are naturally modeled as asymmetric TSP's. In industrial scheduling, the optimal sequencing of jobs on machines with setup times is an ATSP: more generally, the optimal ordering of any set of tasks or operations with sequence dependent changeover costs is an ATSP or one of its generalizations.

Fourth, the study of the ATS polyhedron provides certain insights into the structure of the STS polyhedron. While it is true that an asymmetric facet inducing inequality  $\alpha'x \leq \alpha_0$  for the ATS polytope defined on the digraph  $G'$  typically has no counterpart for the STS polytope defined on  $G$ , it is also true that such an inequality gives rise to a family of facet inducing inequalities for a symmetric TS polytope defined on another, larger undirected graph, as will be discussed later in this survey.

Finally, while it is true that an asymmetric TSP can be reduced to a symmetric one, this symmetric TSP has a very special structure and is defined on an undirected graph with twice as many nodes as the digraph of the ATSP, and so this transformation is not without a price.

We next introduce the main notation used in the sequel.

Let  $G = (V, A)$  be the complete (loop-free) digraph on  $n$  nodes, with  $n \geq 5$ . We associate a variable  $x_{ij}$  with every arc  $(i, j) \in A$ . For  $S, T \subseteq V$ , we denote  $x(S, T) := \sum (x_{ij} : i \in S, j \in T, (i, j) \in A)$  and write  $x(i, S)$  and  $x(S, j)$  for  $x(\{i\}, S)$  and  $x(S, \{j\})$  respectively. Further, for any  $S \subseteq V$  we denote by  $\delta^+(S)$  the set of arcs with their tail in  $S$  and their head in  $V \setminus S$ , and by  $\delta^-(S)$  the set of arcs with their tail in  $V \setminus S$  and their head in  $S$ . Also, we write  $\delta^+(v), \delta^-(v)$  for  $\delta^+(\{v\}), \delta^-(\{v\})$ , respectively. Finally, we denote  $\delta(S) := \delta^+(S) \cup \delta^-(S)$ .

When  $G$  has costs (usually restricted to nonnegative values) on its arcs, the traveling salesman problem defined on  $G$  is the problem of finding a minimum-cost directed Hamiltonian cycle in  $G$ . It is also called the *Asymmetric Traveling Salesman Problem* (ATSP), to distinguish it from its *symmetric* counterpart (STSP) defined on an undirected graph.

There are several known formulations of the ATSP. We will use the standard one, due to Dantzig, Fulkerson and Johnson [239]:

$$\begin{aligned}
 & \text{minimize } cx \\
 & \text{s.t.} \\
 & \quad x(\delta^+(v)) = 1 \quad v \in V \quad (1) \\
 & \quad x(\delta^-(v)) = 1 \quad v \in V \quad (2) \\
 & \quad x(S, S) \leq |S| - 1 \quad S \subset V, \quad |S| \geq 2 \quad (3) \\
 & \quad x \in \{0, 1\}^A \quad (4)
 \end{aligned}$$

Here  $c = (c_{ij})$  is the vector of costs, the equations (1) and (2) are the *outdegree* and *indegree* constraints, respectively (briefly, the degree constraints), while (3) are the *subtour elimination constraints* (SEC, for short). This formulation has  $n^2 - n$  variables and  $O(2^n)$  constraints. There are more compact formulations, but this one has proved so far the most useful.

The ATS polytope is then the convex hull of points satisfying (1)-(4).

The *monotone ATS polytope*  $\tilde{P}$  is obtained from  $P$  by replacing  $=$  with  $\leq$  in all the degree constraints. It is well known [402] that  $P$  has dimension  $n(n-1) - 2n + 1$  whereas  $\tilde{P}$  is of full dimension.

Whenever there is a need to specify the graph  $G$  on which  $P$  or  $\tilde{P}$  is defined, we will write  $P(G)$  and  $\tilde{P}(G)$ , respectively. Moreover, we sometimes use the notation  $P_n$  for  $P$  defined on the complete digraph with  $n$  nodes.

The polyhedral approach to combinatorial optimization consists in trying to describe the solution set to the problem studied through a system of linear inequalities (and possibly equations) that define its convex hull. Of particular importance in this attempt is the identification of inequalities that define facets of the convex hull, because these form a minimal system. If the attempt is successful in that all the facets of the convex hull are identified, then the problem becomes a linear program. If the attempt is only partly successful, in that it fails to completely characterize the convex hull but it succeeds in describing large families of facets of the latter, then the partial description of the convex hull obtained this way usually makes it much easier to solve the problem by enumerative methods like branch and bound, branch and cut, etc.

While a complete characterization of the ATS polytope – i.e. a listing of all the inequalities defining it – is only available for  $n \leq 6$  [283], several families of valid inequalities have been identified recently, and

many of them have been shown to be facet defining for the corresponding polytope. In this survey we focus on ATSP-specific results (asymmetric inequalities, lifting theorems, etc.), and refer the reader to Chapter 2 of the present book for a thorough description of the properties shared with the STSP. Also, the separation problem for ATSP inequalities is not addressed in the present chapter: the interested reader is referred to Chapter 2 for the separation of symmetric inequalities, and to Chapter 4 and to [75] and [161] for ATSP-specific separation procedures.

Finally, we assume the reader is familiar with the fundamentals of polyhedral theory, as given e.g. in the excellent survey of polyhedral theory for the TSP by Grötschel and Padberg [405].

The present chapter is organized as follows. In Section 2 we review the main classes of inequalities for the ATS polytope (for short: ATS inequalities) known at the time of the writing of [405]. Section 3 addresses the monotone ATS polytope, a widely used relaxation of the ATS polytope. Facet-lifting procedures are analyzed in Section 4. The important question of whether two different facet defining inequalities define the same facet of  $P$ , is addressed in Section 5. Sections 6 and 7 are devoted to the study of large classes of ATS-specific facet-defining inequalities introduced recently, namely the odd CAT and the SD inequalities, respectively. Lifted cycle inequalities are finally investigated in Section 8, where a characterization of this class is provided, and some specific subclasses with interesting properties are analyzed.

## 2. Basic ATS inequalities

Several classes of valid inequalities for both  $P$  and  $\tilde{P}$  were known at the time of the writing of the Grötschel and Padberg survey [405]. Among them, we will outline next the so-called comb, clique tree,  $D_k^+$ ,  $D_k^-$ ,  $T_k$ , C2 and C3 inequalities.

### 2.1. Symmetric Inequalities

Since the STSP is a special case of the ATSP, any given inequality  $\bar{\alpha}y := \sum_{e \in E} \bar{\alpha}_e y_e \leq \alpha_0$  defined for the STS polytope associated with a complete undirected graph  $G_E = (V, E)$ , has an obvious ATS counterpart  $\alpha x := \sum_{(i,j) \in A} \alpha_{ij} x_{ij} \leq \alpha_0$  obtained by defining  $\alpha_{ij} := \alpha_{ji} := \bar{\alpha}_e$  for each  $e = [i, j] \in E$ . Conversely, every ATS inequality  $\alpha x \leq \alpha_0$  which is *symmetric* (in the sense that  $\alpha_{ij} = \alpha_{ji}$  for all  $(i, j) \in A$ ,  $i < j$ ) has an STS counterpart  $\bar{\alpha}y := \sum_{e \in E} \bar{\alpha}_e y_e \leq \alpha_0$  obtained by setting  $\alpha_e := \alpha_{ij}$  ( $= \alpha_{ji}$ ) for each  $e = [i, j] \in E$ . It can easily be shown that the above mapping between STS and symmetric ATS inequalities preserves validity (but not necessarily the facet-defining property, as discussed later in

this subsection). As a consequence, the ATS polytope inherits from its undirected version all classes of valid STS inequalities.

Note however that symmetric inequalities are just a small fraction of the whole set of (facet-defining) ATS inequalities. For example, Euler and Le Verge [283] showed that only 2 out of the 287 classes of inequalities representing a complete and irredundant description of the ATS polytope on 6 nodes, are symmetric (namely, the classes of SEC's (3) with  $|S| = 2, 3$ ). This implies that only 35 out of the 319,015 facets of  $P_6$  are defined by symmetric inequalities.

We next present the two most basic families of symmetric inequalities, the comb and clique-tree inequalities, and refer the reader to Chapter 2 of the present book for other large classes of STS inequalities.

**Proposition 1 ([403, 404])** *Let  $H \subset V$  be a “handle” and  $T_1, \dots, T_s \subset V$  be pairwise disjoint “teeth” satisfying: (i)  $|H \cap T_i| \geq 1$  and  $|T_i \setminus H| \geq 1$  for all  $i = 1, \dots, s$ , and (ii)  $s \geq 3$  and odd. The following comb inequality is valid for both  $P$  and  $\tilde{P}$ :*

$$x(H, H) + \sum_{i=1}^s x(T_i, T_i) \leq |H| + \sum_{i=1}^s (|T_i| - 1) - (s + 1)/2.$$

**Theorem 2 ([295])** *Comb inequalities are facet inducing for  $P_n$  for  $n \geq 7$ , and for  $\tilde{P}_n$  for  $n \geq 6$ .*

Note that, unlike in the symmetric case, a comb inequality does not define a facet of  $P_n$  when  $n = 6$ , as in this case the corresponding face turns out to be the intersection of two distinct facets of  $P_n$  defined by asymmetric inequalities having no counterpart in the symmetric case – a pathological situation that will be discussed later in this subsection.

**Proposition 3 ([406])** *Let  $C := \{H_1, \dots, H_r, T_1, \dots, T_s\}$ , with  $s \geq 1$  and odd, be a family of nonempty subsets of  $V$  where the  $H_i$  ( $i = 1, \dots, r$ ) are called handles and the  $T_j$  ( $j = 1, \dots, s$ ) are called teeth, which satisfy (see Figure 3.1 for an illustration): (i)  $T_i \cap T_j = \emptyset$ , for each  $i, j \in \{1, \dots, s\}$ ,  $i \neq j$ ; (ii)  $H_i \cap H_j = \emptyset$ , for each  $i, j \in \{1, \dots, r\}$ ,  $i \neq j$ ; (iii)  $2 \leq |T_j| \leq n - 2$  and  $T_j \setminus (\bigcup_{i=1}^r H_i) \neq \emptyset$ , for  $j = 1, \dots, s$ ; (iv) the number,  $h_i$ , of teeth overlapping  $H_i$  is odd and at least 3, for  $i = 1, \dots, r$ ; (v) the intersection graph of  $C$  (i.e., the undirected graph having one node for each subset belonging to  $C$  and one edge for each pair of overlapping subsets) is a tree.*

*Then the following clique tree inequality is valid for both  $P$  and  $\tilde{P}$ :*

$$\sum_{i=1}^r x(H_i, H_i) + \sum_{j=1}^s x(T_j, T_j) \leq s(C),$$

where  $t_j$  denotes the number of handles intersecting tooth  $T_j$  ( $j = 1, \dots, s$ ), and  $s(C) := \sum_{i=1}^r |H_i| + \sum_{j=1}^s (|T_j| - t_j) - (s + 1)/2$  is the size of  $C$ .

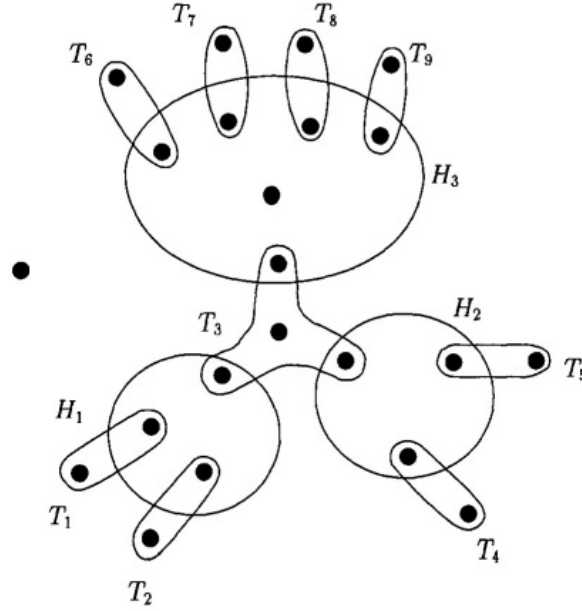


Figure 3.1. A clique tree with  $r = 3$  handles and  $s = 9$  teeth

Observe that clique tree inequalities with just one handle coincide with the comb inequalities introduced previously.

Clique tree inequalities have been proved by Grötschel and Pulleyblank [406] to be valid and facet-inducing (in their undirected form) for the STS counterparts,  $Q$  and  $\tilde{Q}$ , of the ATS polytopes  $P$  and  $\tilde{P}$ . As already observed, this implies that they are valid (but not necessarily facet-defining) for  $P$  and  $\tilde{P}$ . Later, Fischetti [297] proved the following:

**Theorem 4** *All clique tree inequalities (except for combs when  $n = 6$ ) define facets of both  $P$  and  $\tilde{P}$ .*

We next address the correspondence between facets of the STS and ATS polytopes. It was observed in [295] that the STS counterpart of every symmetric ATS inequality, say  $\alpha x \leq \alpha_0$ , defining a facet of  $P$  (resp.,  $\tilde{P}$ ) necessarily defines a facet of  $Q$  (resp.,  $\tilde{Q}$ ). Indeed, consider the polytope  $P$  (our reasoning trivially applies to  $\tilde{P}$  as well). Let  $\bar{\alpha}y \leq \alpha_0$  be the undirected counterpart of  $\alpha x \leq \alpha_0$ . This inequality clearly defines a proper face of  $Q$ . Assume now that it is not facet defining for  $Q$ . Then there must exist a facet defining inequality  $\bar{\beta}y \leq \beta_0$ , valid for  $Q$  and satisfying  $\{y \in Q : \bar{\alpha}y = \alpha_0\} \subset \{y \in Q : \bar{\beta}y = \beta_0\} \subset Q$ .

But this implies that the directed counterpart of is valid for  $P$  and induces a proper face which strictly contains the face impossible since  $\alpha x \leq \alpha_0$  is facet-defining by assumption.

An important question raised in [405] is whether or not the directed version of a facet-defining inequality for the STS polytope is also facet defining for the ATS polytope. This issue has been investigated by Queyranne and Wang [689], who showed that several known classes of facet-defining inequalities for the STS polytope, including the path, wheelbarrow, chain, and ladder inequalities (see Chapter 2 on these inequalities), do define facets of  $P$  (with the exception of some pathological cases). Moreover, [689] proposed an operation called tree-composition and used it to produce a new large class of symmetric inequalities that define facets of the ATS (and hence of the STS) polytope.

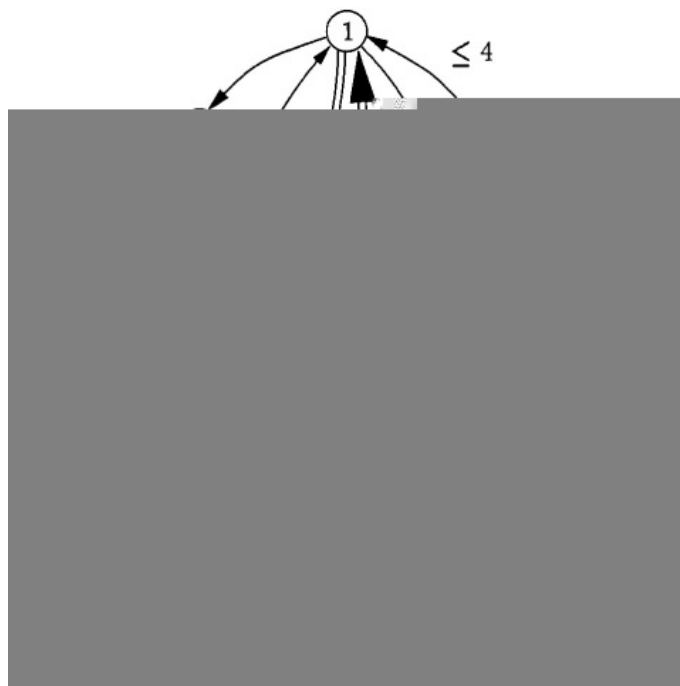


Figure 3.2. Curtain inequality for the ATSP and corresponding inequality for the STSP.

A more intriguing correspondence between STS and ATS polytopes (defined on graphs with a different number of nodes) derives from the known fact [495] that every ATSP defined on the complete digraph  $G = (V, A)$  can be restated as a STSP defined on an undirected bipartite graph  $G_B^* := (V^*, E_B^*)$ , where  $V^*$  has a pair of nodes  $i^+, i^-$  for



























































































































































































*This page intentionally left blank*



































*This page intentionally left blank*















































































































$\sum_{i \in V} w_i = W$  and  $\sum_{i \in V} w_i^2 \leq W$ .  
 To prove the lemma, we define a non-negative weight function  $w_i$  for each node  $i \in V$  such that

**Lemma 38** There is a non-negative weight function  $w_i$  such that for all  $i \in V$  and  $j \in V$  the TSP functional satisfies

where  $w_i$  is uniformly in  $[0, 1]$ .

We define  $w_i$  as follows. Let  $T$  be a TSP tour. For each node  $i \in V$ , let  $d_i$  be the degree of node  $i$  in  $T$ . Let  $w_i = 1/d_i$ . Since  $T$  is a TSP tour,  $d_i = 2$  for all  $i \in V$ . Therefore,  $w_i = 1/2$  for all  $i \in V$ . This satisfies the conditions of the lemma.

**Corollary 39** (Concentration for the TSP) We have

**Proof:** Let  $T$  be a TSP tour. For each node  $i \in V$ , let  $d_i$  be the degree of node  $i$  in  $T$ . Let  $w_i = 1/d_i$ . Since  $T$  is a TSP tour,  $d_i = 2$  for all  $i \in V$ . Therefore,  $w_i = 1/2$  for all  $i \in V$ . This satisfies the conditions of the lemma.

Let  $w_i = 1/d_i$ . Since  $T$  is a TSP tour,  $d_i = 2$  for all  $i \in V$ . Therefore,  $w_i = 1/2$  for all  $i \in V$ . This satisfies the conditions of the lemma.

Let  $w_i = 1/d_i$ . Since  $T$  is a TSP tour,  $d_i = 2$  for all  $i \in V$ . Therefore,  $w_i = 1/2$  for all  $i \in V$ . This satisfies the conditions of the lemma.















































































*This page intentionally left blank*

















































































































































































































































































































*This page intentionally left blank*



































































































*This page intentionally left blank*























































































































































































































*This page intentionally left blank*





















































*This page intentionally left blank*





































































































































































































*This page intentionally left blank*

























































































*This page intentionally left blank*





















































































- [231] V. D. Cung, T. Mautor, P. Michelon, and A. Tavares. Scatter Search for the Quadratic Assignment Problem. In *Proc. 1996 IEEE International Confer. Evolutionary Comput.*, pages 165–169, 1996.
- [232] W.H. Cunningham and Y. Wang. Restricted 2 factor polytopes. *Math. Program. Ser. A*, 87:87–111, 2000.
- [233] J.R. Current and D.A. Schilling. The covering salesman problem. *Transp. Sci.*, 23:208–213, 1989.
- [234] J.R. Current and D.A. Schilling. The median tour and maximal covering problems. *Eur. J. Oper. Res.*, 73:114–126, 1994.
- [235] M. Cutler. Efficient special case algorithms for the  $n$ -line planar traveling salesman problem. *Networks*, 10:183–195, 1980.
- [236] D. Cvetkrovi, V. Dimitrijevi, and M. Milosavljevi. A survey of some non standard traveling salesman problems. *Yugoslav J. Oper. Res.*, 2:163–185, 1992.
- [237] A. Czumaj and A. Lingas. A polynomial time approximation scheme for Euclidean minimum cost  $k$ -connectivity. In *Proc. 25th Internat. Colloq. Automata Languages Programming*, pages 682–694, 1998.
- [238] M. Dam and M. Zachariasen. Tabu search on the geometric traveling salesman problem. In I.H. Osman and J.P. Kelly, editors, *Meta heuristics: Theory and Applications*, pages 571–587. Kluwer Academic Publishers, Boston, 1996.
- [239] G.B. Dantzig, D.R. Fulkerson, and S.M. Johnson. Solution of a large scale traveling salesman problem. *Oper. Res.*, 2:393–410, 1954.
- [240] J.B.J.M. De Kort. Bounds for the symmetric 2 peripatetic salesman problem. *Optimization*, 23:357–367, 1992.
- [241] A. de Vitis. The cactus representation of all minimum cuts in a weighted graph. Technical report, Istituto di Analisi dei Sistemi ed Informatica del CNR, Viale Manzoni 30, 00185 Roma, Italy, 1997.
- [242] V.G. Deineko and V.L. Filonenko. On the reconstruction of specially structured matrices. In *Aktualnyje Problemy EVM i programmirovaniye*, Dnepropetrovsk, DGU, 1979. (in Russian).
- [243] V.G. Deineko, R. Rudolf, and G.J. Woeginger. On the recognition of permuted Supnick and incomplete Monge matrices. *Acta Inform.*, 33:559–569, 1996.
- [244] V.G. Deineko, R. Rudolf, and G.J. Woeginger. Sometimes traveling is easy: the master tour problem. *SIAM J. Discrete Math.*, 11:81–93, 1998.
- [245] V.G. Deineko and V.N. Trofimov. On possible values of the function in the special traveling salesman problem. *Kibernetika*, 6:135–136, 1981. (in Russian).
- [246] V.G. Deineko, R. van Dal, and G. Rote. The convex hull and line traveling salesman problem: a solvable case. *Inform. Process. Lett.*, 51:141–148, 1994.
- [247] V.G. Deineko and G.J. Woeginger. The **convex-hull-and- $k$ -line** travelling salesman problem. *Inform. Process. Lett.*, 59:295–301, 1996.
- [248] V.G. Deineko and G.J. Woeginger. A study of exponential neighbourhoods for the traveling salesman problem and the quadratic assignment problem. *Math. Program., Ser. A*, 87:519–542, 2000.
- [249] M. Dell’Amico and P. Toth. Algorithms and codes for dense assignment problems: the state of the art. *Discrete Appl. Math.*, 100:17–48, 2000.

- [250] V.M. Demidenko. A special case of traveling salesman problems. *Izv. Akad. Navuk BSSR, Ser. Fiz. mat. Navuk*, no. 5:28–32, 1976. (in Russian).
- [251] V.M. Demidenko. The traveling salesman problem with asymmetric matrices. *Izv. Akad. Navuk BSSR, Ser. Fiz. Mat. Navuk*, 1:29–35, 1979. (in Russian).
- [252] V.M. Demidenko. An extention of Supnick conditions for the Asymmetric Traveling Salesman Problem and its application. Manuscript, Belorussian Academy of Sciences, 1995.
- [253] U. Derigs and U. Zimmerman. An augmenting path method for solving linear bottleneck assignment problems. *Computing*, 19:285–295, 1978.
- [254] R. Diestel. *Graph Theory*. Springer Verlag, New York, 2000. 2nd edition, first edition 1997.
- [255] V. Dimitrijević and Z. Sarić. An efficient transformation of the generalized traveling salesman problem into the traveling salesman problem on digraphs. *Inf. Sci.*, 102:105–110, 1997.
- [256] G.A. Dirac. Some theorems on abstract graphs. *Proc. London Math. Soc.*, 2:69–81, 1952.
- [257] V. Dobric and J.E. Yukich. Exact asymptotics for transportation cost in high dimensions. *J. Theor. Probab.*, 8:97–118, 1995.
- [258] J.J. Dongarra. Performance of various computers using standard linear equations software. Report CS 89 85, University of Tennessee, Knoxville, 1996.
- [259] U. Dorndorf and E. Pesch. Fast clustering algorithms. *ORSA J. Comput.*, 6:141–153, 1994.
- [260] N.N. Doroshko and V.I. Sarvanov. A minmax traveling salesman problem and Hamiltonian cycles in powers of graphs. *Vestsi Akad. Navuk BSSR, Ser. Fiz. Mat. Navuk*, no. 6, 1981. (in Russian).
- [261] R.G. Downey and M.R. Fellows. *Parametrized Complexity*. Springer Verlag, Berlin, 1999.
- [262] M.E. Dyer and A.M. Frieze. On patching algorithms for random asymmetric travelling salesman problems. *Math. Program., Ser. A*, 46:361–378, 1990.
- [263] M.E. Dyer, A.M. Frieze, and C.J.H. McDiarmid. Partitioning heuristics for two geometric maximization problems. *Oper. Res. Lett.*, 3(5):267–270, 1984.
- [264] H. Edelsbrunner, G. Rote, and E. Welzl. Testing the necklace condition for shortest tours and optimal factors in the plane. *Theoret. Comput. Sci.*, 66:157–180, 1989.
- [265] J. Edmonds. Maximum matching and a polyhedron with 0,1 vertices. *J. Res. Nat. Bur. Stand B*, 59:125–130, 1965.
- [266] J. Edmonds. Paths, trees, and flowers. *Canad. J. Math.*, 17:449–467, 1965.
- [267] J. Edmonds. Optimum branchings. *J. Res. Natl. Bur. Stand., Sect. B*, 71B:233–240, 1967.
- [268] J. Edmonds. Matroid intersection. *Ann. Discrete Math.*, 4:39–49, 1979.
- [269] J. Edmonds and D.R. Fulkerson. Bottleneck extrema. *J. Comb. Theory*, 8:299–306, 1970.

- [270] J. Edmonds and E.L. Johnson. Matching: a Well-solved Class of Integer Linear Programs. In *Combinatorial Structures and their Applications (Proc. Calgary Internat. Conf., Calgary, Alta., 1969)*. Gordon and Breach, New York, 1970.
- [271] M. Ehrgott. Approximation algorithms for combinatorial multicriteria optimization problems. *Internat. Trans. Oper. Res.*, 7:5–31, 2000.
- [272] H. Emmons and K. Matur. Lot sizing in a no-wait flow shop. *Oper. Res. Lett.*, 17:159–164, 1995.
- [273] L. Engebresten. An explicit lower bound for TSP with distances one and two. Technical Report 46, Electronic Colloquium on Computational Complexity, 1998.
- [274] H. Enomoto, Y. Oda, and K. Ota. Pyramidal tours with step-backs and the asymmetric traveling salesman problem. *Discrete Appl. Math.*, 87:57–65, 1998.
- [275] D. Eppstein. Sequence comparison with mixed convex and concave costs. *J. Algorithms*, 11:85–101, 1990.
- [276] P. Erdős. Remarks on a paper by Posa. *Magyar Tud. Akad. Mat. Kut. Int. Kozl.*, 7:227–229, 1962.
- [277] P. Erdős and A. Rényi. On random graphs I. *Publ. Math. Debrecen*, 6:290–297, 1959.
- [278] P. Erdős and A. Rényi. On the evolution of random graphs. *Publ. Math. Inst. Hungar. Acad. Sci.*, 5:17–61, 1960.
- [279] P. Erdős and A. Rényi. On the strength of connectedness of a random graph. *Acta. Math. Acad. Sci. Hungar.*, 12:261–267, 1963.
- [280] P. Erdős and A. Rényi. On random matrices. *Publ. Math. Hungar. Acad. Inst. Sci.*, 8:455–461, 1964.
- [281] P. Erdős and J. Spencer. Evolution of the  $n$ -cube. *Comput. Math. Appl.*, 5:33–39, 1979.
- [282] J. Ernvall, J. Katajainen, and M. Penttonen. NP-completeness of the Hamming salesman problem. *BIT*, 25:289–292, 1985.
- [283] R. Euler and H. Le Verge. Complete linear description of small asymmetric traveling salesman polytopes. *Discrete Appl. Math.*, 62:193–208, 1995.
- [284] S.P. Fekete. Simplicity and hardness of the maximum traveling salesman problem under geometric distances. Technical Report 98.329, Center for Applied Computer Science, Universitat zu Koln, 1998.
- [285] S.P. Fekete. Simplicity and hardness of the maximum traveling salesman problem under geometric distances. In *Proceedings of the Tenth Annual ACM-SIAM Symposium on Discrete Algorithms (Baltimore, MD, 1999)*, pages 337–345. ACM, New York, 1999.
- [286] S.P. Fekete and H. Meijer. On minimum stars and maximum matchings. *Discrete Comput. Geom.*, 23:389–407, 2000.
- [287] S.P. Fekete, H. Meijer, A. Rohe, and W. Tietze. Good and fast heuristics for large geometric maximum matching and maximum traveling salesman problems. Manuscript, 2000.
- [288] Z. Fencel. Algorithm 456 - routing problem [H]. *Commun. ACM*, 16:572–574, 1973.

- [289] T.I. Fenner and A.M. Frieze. Hamiltonian cycles in random regular graphs. *J. Comb. Theory, Ser. B*, 38:103–112, 1984.
- [290] A. Fink, G. Schneidereit, and S. Voss. Solving general ring network design problems by meta-heuristics. In M. Laguna and J.L. González Velarde, editors, *Computing Tools for Modeling, Optimization and Simulation*. Kluwer, Boston, 2000.
- [291] G. Finke, A. Claus, and E. Gunn. A two commodity network flow approach to the traveling salesman problem. *Congres. Numer.*, 41:167–178, 1984.
- [292] S.T. Fischer. A note on the complexity of local search problems. *Inf. Process. Lett.*, 53:69–75, 1995.
- [293] F. Fischetti, G. Laporte, and S. Martello. The delivery man problem and cumulative matroids. *Oper. Res.*, 41:1055–1064, 1993.
- [294] M. Fischetti. An improved bound for the asymmetric travelling salesman problem. *Ricerca Operativa*, 37:71–85, 1986.
- [295] M. Fischetti. Facets of the asymmetric traveling salesman polytope. *Math. Oper. Res.*, 16:42–56, 1991.
- [296] M. Fischetti. Three facet lifting theorems for the asymmetric traveling salesman polytope. In E. Balas, G. Cornuéjols, and R. Kannan, editors, *Proc. Integer Program. Comb. Optimization (IPCO) 2*, pages 260–273. GSIA, Carnegie Mellon University, 1992.
- [297] M. Fischetti. Clique tree inequalities define facets of the asymmetric traveling salesman problem. *Discrete Appl. Math.*, 56:9–18, 1995.
- [298] M. Fischetti, A. Lodi, S. Martello, and P. Toth. A polyhedral approach to simplified crew scheduling and vehicle scheduling problems. *Manage. Sci.*, 47:833–850, 2001.
- [299] M. Fischetti, J.J. Salazar, and P. Toth. The symmetric generalized traveling salesman polytope. *Networks*, 26:113–123, 1995.
- [300] M. Fischetti, J.J. Salazar, and P. Toth. A branch-and-cut algorithm for the symmetric generalized traveling salesman problem. *Oper. Res.*, 45:378–394, 1997.
- [301] M. Fischetti, J.J. Salazar, and P. Toth. Solving the orienteering problem through branch-and-cut. *INFORMS J. Comput.*, 10:133–148, 1998.
- [302] M. Fischetti and P. Toth. An additive approach for the optimal solution of the prize-collecting travelling salesman problem. In B.L. Golden and A.A. Assad, editors, *Vehicle Routing: Methods and Studies*, pages 319–343. North-Holland, 1988.
- [303] M. Fischetti and P. Toth. An additive bounding procedure for combinatorial optimization problems. *Oper. Res.*, 37:319–328, 1989.
- [304] M. Fischetti and P. Toth. An additive bounding procedure for the asymmetric travelling salesman problem. *Math. Programm., Ser. A*, 53:173–197, 1992.
- [305] M. Fischetti and P. Toth. An efficient algorithm for the min-sum arborescence problem. *ORSA J. Comput.*, 5:325–352, 1993.
- [306] M. Fischetti and P. Toth. A polyhedral approach to the asymmetric traveling salesman problem. *Manage. Sci.*, 43:1520–1536, 1997.

- [307] M. Fischetti and D. Vigo. A branch and cut algorithm for the resource-constrained min-sum arborescence problem. *Networks*, 29:55–67, 1997.
- [308] M.L. Fisher, G.L. Nemhauser, and L.A. Wolsey. An analysis of approximations for finding a maximum weight hamiltonian circuit. *Oper. Res.*, 27(4):799–809, 1979.
- [309] L. Fleischer. Building Chain and Cactus Representations of All Minimum Cuts from Hao-Orlin in the Same Asymptotic Run Time. *J. Algorithms*, 33(1):51–72, 1999.
- [310] L. Fleischer and É. Tardos. Separating maximally violated comb inequalities in planar graphs. *Math. Oper. Res.*, 24:130–148, 1999.
- [311] B. Fleischmann. A new class of cutting planes of the symmetric traveling salesman problem. *Math. Program. Ser. A*, 40:225–246, 1988.
- [312] H. Fleischner. The square of every 2-connected graph is Hamiltonian. *J. Comb. Theory, Ser. B*, 16:29–34, 1974.
- [313] C. Fleurent, F. Glover, P. Michelon, and Z. Valli. A Scatter Search approach for unconstrained continuous optimization. In *Proc. 1996 IEEE International Confer. Evolutionary Comput.*, pages 643–648, 1996.
- [314] M.M. Flood. The traveling-salesman problem. *Oper. Res.*, 4:61–75, 1956.
- [315] F. Focacci, A. Lodi, and M. Milano. A hybrid exact algorithm for the TSPTW. Technical Report OR/01/2, D.E.I.S., University of Bologna, 2001.
- [316] J. Fonlupt and A. Nacheff. Dynamic programming and the graphical traveling salesman problem. *J. Assoc. Comput. Mach.*, 40(5):1165–1187, 1993.
- [317] J. Fonlupt and D. Naddef. The traveling salesman problem in graphs with excluded minors. *Math. Program. Ser. A*, 53:147–172, 1992.
- [318] R. Fourer. Software survey: Linear programming. *OR/MS Today*, 26:64–71, 1999.
- [319] K.R. Fox. *Production scheduling on parallel lines with dependencies*. PhD thesis, John Hopkins University, 1973.
- [320] A. Frank and É. Tardos. An application of simultaneous approximation in combinatorial optimization. In *Annual Symposium on Foundations of Computer Science*, pages 459–463, New York, 1985. IEEE.
- [321] G.N. Frederickson and D.J. Guan. Nonpreemptive ensemble motion planning on a tree. *J. Algorithms*, 15(1):29–60, 1993.
- [322] M.L. Fredman, D.S. Johnson, L.A. McGeoch, and G. Ostheimer. Data structures for traveling salesmen. *J. Algorithms*, 18:432–479, 1995.
- [323] A. Frick. TSPGA - an evolution program for the symmetric traveling salesman problem. In H.J. Zimmermann, editor, *EUFIT'98 - 6th European Congress on Intelligent Techniques and Soft Computing*, pages 513–517. Mainz Verlag, Aachen, 1998.
- [324] A. Frieze, G. Galbiati, and F. Maffioli. On the worst-case performance of some algorithms for the asymmetric traveling salesman problem. *Networks*, 12:23–39, 1982.
- [325] A.M. Frieze. Worst-case analysis of algorithms for travelling salesman problems. *Oper. Res.-Verf.*, 32:93–112, 1979.

- [326] A.M. Frieze. On the exact solution of random travelling salesman problems with medium-sized integer costs. *SIAM J. Comput.*, 16:1052–1072, 1987.
- [327] A.M. Frieze. Parallel algorithms for finding Hamilton cycles in random graphs. *Inf. Process. Lett.*, 25:111–117, 1987.
- [328] A.M. Frieze. An algorithm for finding Hamilton cycles in random digraphs. *J. Algorithms*, 9:181–204, 1988.
- [329] A.M. Frieze. Finding Hamilton cycles in sparse random graphs. *J. Comb. Theory, Ser. B*, 44:230–250, 1988.
- [330] A.M. Frieze. Partitioning random graphs into large cycles. *Discrete Math.*, 70:149–158, 1988.
- [331] A.M. Frieze. On matchings and Hamilton cycles in random graphs. In J. Siemens, editor, *Surveys in Combinatorics, Proc. 12th British Combinatorial Conference*, pages 84–114. London Math. Soc., 1989.
- [332] A.M. Frieze, M.R. Jerrum, M. Molloy, R.W. Robinson, and N.C. Wormald. Generating and counting Hamilton cycles in random regular graphs. *J. Algorithms*, 21:176–198, 1996.
- [333] A.M. Frieze, R.M. Karp, and B. Reed. When is the assignment bound asymptotically tight for the asymmetric traveling-salesman problem? *SIAM J. Comput.*, 24:484–493, 1995.
- [334] A.M. Frieze and C. McDiarmid. Algorithmic theory of random graphs. *Random Struct. Algorithms*, 10:5–42, 1997.
- [335] A.M. Frieze and B.D. McKay. Multicoloured trees in random graphs. *Random Struct. Algorithms*, 5:45–56, 1994.
- [336] A.M. Frieze and G. Sorkin. The probabilistic relationship between the assignment and asymmetric traveling salesman problems. In *Proc. 12th Ann. ACM-SIAM Symp. Discrete Algorithms*, pages 652–660. ACM-SIAM, 2001.
- [337] A.M. Frieze and S. Suen. Counting Hamilton cycles in random directed graphs. *Random Struct. Algorithms*, 3:235–242, 1992.
- [338] D.R. Fulkerson. Packing rooted directed cuts in a weighted directed graph. *Math. Program., Ser. A*, 6:1–13, 1974.
- [339] S.H. Fuller. An optimal drum scheduling algorithm. *IEEE Trans. Comput.*, C-21(11):1153–1165, 1972.
- [340] E.Y. Gabovich. The small traveling salesman problem. *Trudy Vychisl. Centra Tartu. Gos. Univ.*, 19:17–51, 1970. (in Russian).
- [341] E.Y. Gabovich. Constant discrete programming problems on substitution sets. *Kibernetika*, no. 5:128–134, 1976.
- [342] H. Gabow. Applications of a poset representation to edge connectivity and graph rigidity. In *Proc. 32th IEEE Symp. Found. Comp.*, pages 812–821, 1991.
- [343] H.N. Gabow and R.E. Tarjan. Algorithms for two bottleneck optimization problems. *J. Algorithms*, 9:411–417, 1988.
- [344] N.E. Gaikov. On the minimization of a linear forms on cycles. Manuscript deposited in VINITI (AISTI) (in Russian), 1980.

- [345] A. Garcia, P. Jodra, and J. Tejel. An efficient algorithm for on-line searching of minima in Monge path-decomposable tridimensional arrays. *Inform. Process. Lett.*, 68(1):3–9, 1998.
- [346] M.R. Garey, R.L. Graham, and D.S. Johnson. Some NP-complete geometric problems. In *Proc. 8th ACM Symp. Theory Comput.*, pages 10–22, 1976.
- [347] M.R. Garey and D.S. Johnson. *Computers and Intractability*. W.H. Freeman, San Francisco, CA, 1979.
- [348] R.S. Garfinkel. Minimizing wallpaper waste part I: a class of traveling salesman problems. *Oper. Res.*, 25:741–751, 1977.
- [349] R.S. Garfinkel and K.C. Gilbert. The bottleneck traveling salesman problem: Algorithms and probabilistic analysis. *J. Assoc. Comput. Mach.*, 25:435–448, 1978.
- [350] B. Gavish and S.C. Graves. The traveling salesman problem and related problems. Working Paper GR-078-78, Operations Research Center, MIT, 1978.
- [351] M. Gendreau, A. Hertz, and G. Laporte. New insertion and postoptimization procedures for the traveling salesman problem. *Oper. Res.*, 40:1086–1094, 1992.
- [352] M. Gendreau, A. Hertz, and G. Laporte. A tabu search heuristic for the vehicle routing problem. *Manage. Sci.*, 40(10):1276–1290, 1994.
- [353] M. Gendreau, A. Hertz, and G. Laporte. The traveling salesman problem with backhauls. *Comput. Oper. Res.*, 23:501–508, 1996.
- [354] M. Gendreau, G. Laporte, and F. Semet. The covering tour problem. *Oper. Res.*, 45:568–576, 1997.
- [355] M. Gendreau, G. Laporte, and F. Semet. The selective traveling salesman problem. *Networks*, 32:263–273, 1998.
- [356] G.V. Gens and E.V. Levner. Effective approximate algorithms for combinatorial problems. In *TsEMI*. Moscow, 1981.
- [357] A.M.H. Gerards and A. Schrijver. Matrices with the Edmonds-Johnson property. *Combinatorica*, 6:365–379, 1986.
- [358] H.W. Ghosh. Expected travel among random points. *Bulletin Culcatta Stat. Assoc.*, 2:83–87, 1948.
- [359] J. Giesen. Curve reconstruction, the TSP and Menger’s Theorem on length. In *Proc. 15th Annual ACM Symposium on Computational Geometry*, pages 207–216, 1999.
- [360] P.C. Gilmore and R.E. Gomory. Sequencing a one state-variable machine: A solvable case of the traveling salesman problem. *Oper. Res.*, 12:655–679, 1964.
- [361] P.C. Gilmore, E.L. Lawler, and D.B. Shmoys. Well-solved special cases. In E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan, and D.B. Shmoys, editors, *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*, pages 87–143. Wiley, Chichester, 1985.
- [362] E.Kh. Gimadi. The traveling salesman problem on a maximum: conditions for the asymptotic accuracy of the algorithm “go to the most remote city”. *Upravlyaemye Sistemy*, no. 27:11–15, 1989. (in Russian).
- [363] E.Kh. Gimadi, N.I. Glebov, and A.I. Serdyukov. An approximation algorithm for the traveling salesman problem and its probabilistic analysis. *Diskretnui*

- Analiz i Issledovanie Operatsii*, 1(2):8–17, 1994. in Russian. Translated in *Operations Research and Discrete Analysis. Translation of Discret. Anal. Issled. Oper. I (1994), N 1-4*, Mathematics and its Applications, Kluwer, Dordrecht, 1996; 35–43.
- [364] E.Kh. Gimadi and N.K. Maksishko. Justification of conditions for the asymptotic exactness of an approximate algorithm for solving the traveling salesman problem on a maximum in the case of a discrete distribution. *Upravlyaemye Sistemy*, no. 30:25–29, 1990. (in Russian).
- [365] E.Kh. Gimadi and V.A. Perepelitsa. On the problem of finding the minimal hamiltonian circuit on a graph with weighted arcs. *Diskretnui Analiz*, no. 15:57–65, 1969. (in Russian).
- [366] E.Kh. Gimadi and V.A. Perepelitsa. An asymptotical approach to the solution of the traveling salesman problem. *Upravlyaemye Sistemy*, 12:35–45, 1974. (in Russian).
- [367] F. Glover. A multiphase-dual algorithm for zero-one integer programming problems. *Oper. Res.*, 13:879–919, 1965.
- [368] F. Glover. Heuristics for Integer Programming Using Surrogate Constraints. *Decision Sci.*, 8(1):156–166, 1977.
- [369] F. Glover. Tabu search – Part I. *ORSA J. Comput.*, 1:190–206, 1989.
- [370] F. Glover. Tabu search – Part II. *ORSA J. Comput.*, 2:4–32, 1990.
- [371] F. Glover. Multilevel Tabu Search and Embedded Search Neighborhoods for the Traveling Salesman Problem. Technical report, University of Colorado, Boulder, 1991.
- [372] F. Glover. Ejection chains, reference structures, and alternating path algorithms for traveling salesman problem. University of Colorado-Boulder, April 1992.
- [373] F. Glover. New ejection chain and alternating path methods for traveling salesman problems. In O. Balci, R. Sharda, and S. Zenios, editors, *Computer Science and Operations Research: New Developments in Their Interfaces*, pages 491–507. Pergamon, Oxford, 1992.
- [374] F. Glover. Ejection chains, reference structures and alternating path methods for traveling salesman problems. *Discrete Appl. Math.*, 65:223–253, 1996.
- [375] F. Glover. Finding a best travelingsalesman 4-opt move in the same time as a best 2-opt move. *J. Heuristics*, 2(2):169–179, 1996.
- [376] F. Glover. A template for Scatter Search and Path Relinking. In *Lect. Notes Comput. Sci.*, volume 1363, pages 13–54. Springer, Heidelberg, 1997.
- [377] F. Glover, G. Gutin, A. Yeo, and A. Zverovich. Construction heuristics for the asymmetric TSP. *Eur. J. Oper. Res.*, 129:555–568, 2001.
- [378] F. Glover and G. Kochenberger. *State-of-the-Art Handbook in Metaheuristics*. Kluwer, Boston, to appear.
- [379] F. Glover and M. Laguna. *Tabu Search*. Kluwer, Boston, 1997.
- [380] F. Glover, M. Laguna, and R. Martí. Scatter Search. In *Theory and Applications of Evolutionary Computation: Recent Trends*. Springer-Verlag, Heidelberg, to appear.



- [381] F. Glover, A. Løkketangen, and D. Woodruff. Scatter Search to Generate Diverse MIP Solutions. Technical report, University of Colorado, Boulder, 1999.
- [382] F. Glover and A.P. Punnen. The travelling salesman problem: new solvable cases and linkages with the development of approximation algorithms. *J. Oper. Res. Soc.*, 48:502–510, 1997.
- [383] M. Goemans. Worst-case comparison of valid inequalities for the TSP. *Math. Program. Ser. A*, 69:335–349, 1995.
- [384] M. Goemans and D. Bertsimas. Probabilistic analysis of the Held-Karp relaxation for the Euclidean traveling salesman problem. *Math. Oper. Res.*, 16:72–89, 1991.
- [385] M. Goemans and J. Kleinberg. An improved approximation ratio for the minimum latency problem. *Math. Program. Ser. A*, 82:111–124, 1998.
- [386] B.L. Golden, L. Levy, and R. Dahl. Two generalizations of the traveling salesman problem. *Omega*, 9:439–441, 1981.
- [387] B.L. Golden, L. Levy, and R. Vohra. The orienteering problem. *Nav. Res. Logist.*, 34:307–318, 1987.
- [388] B.L. Golden and W.R. Stewart. Empirical analysis of heuristics. In E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan, and D.B. Shmoys, editors, *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley, Chichester, 1985.
- [389] B.L. Golden, Q. Wang, and L. Liu. A multifaceted heuristic for the orienteering problem. *Nav. Res. Logist.*, 35:359–366, 1988.
- [390] R.E. Gomory and T.C. Hu. Multi-terminal network flows. *J. Soc. Ind. Appl. Math.*, 9:551–570, 1961.
- [391] M. Goossens, F. Mittelbach, and A. Samarin. *The LATEX Companion*. Addison-Wesley, Reading, MA, 1998. <http://www.latex-project.org/>.
- [392] L. Gouveia and S. Voss. A classification of formulations for the (time-dependent) traveling salesman problem. *Eur. J. Oper. Res.*, 83:69–82, 1995.
- [393] R.L. Graham. Personal communication with S. Arora. 1996.
- [394] R.L. Graham. An efficient algorithm for determining the convex hull of a finite planar set. *Inform. Process. Lett.*, 1:132–133, 1972.
- [395] M. Grigni, E. Koutsoupias, and C.H. Papadimitriou. An approximation scheme for planar graph TSP. In *Proc. 36th Symp. Found. Comput. Sci.*, pages 640–645, 1995.
- [396] M. Grötschel. *Polyedrische Charakterisierungen Kombinatorischer Optimierungsprobleme*. Hain, Maisenheim am Glen, 1977.
- [397] M. Grötschel. On the symmetric traveling salesman problem: solution of 120-city problem. *Math. Program. Studies*, 12:61–77, 1980.
- [398] M. Grötschel and O. Holland. Solution of large-scale symmetric travelling salesman problems. *Math. Program., Ser. A*, 51:141–202, 1991.
- [399] M. Grötschel, L. Lovász, and A. Schrijver. The ellipsoid method and its consequences in combinatorial optimization. *Combinatorica*, 1:169–197, 1981.
- [400] M. Grötschel, L. Lovász, and A. Schrijver. *Geometric Algorithms and Combinatorial Optimization*. Springer-Verlag, Berlin, 1988.

- [401] M. Grötschel and M.W. M. Padberg. On the symmetric traveling salesman problem: theory and computations. In R. Henn et al., editor, *Optimization and Operations Research*, Lect. Notes Econom. Math. Systems, Vol. 157, pages 105–115. Springer, Berlin, 1978.
- [402] M. Grötschel and M.W. Padberg. Lineare charakterisierungen von traveling salesman problemen. *Zeitschrift für Operations Research*, 21:33–64, 1977.
- [403] M. Grötschel and M.W. Padberg. On the symmetric traveling salesman problem I: inequalities. *Math. Program.*, 16:265–280, 1979.
- [404] M. Grötschel and M.W. Padberg. On the symmetric traveling salesman problem II: lifting theorems and facets. *Math. Program.*, 16:281–302, 1979.
- [405] M. Grötschel and M.W. Padberg. Polyhedral theory. In E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan, and Shmoys D.B., editors, *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*, pages 251–305. Wiley, Chichester, 1985.
- [406] M. Grötschel and W. Pulleyblank. Clique tree inequalities and the Symmetric Traveling Salesman Problem. *Math. Oper. Res.*, 11:537–569, 1986.
- [407] M. Grötschel and Y. Wakabayashi. On the structure of the monotone asymmetric traveling salesman polytope, I: Hypohamiltonian facets. *Discrete Math.*, 34:43–59, 1981.
- [408] M. Grötschel and Y. Wakabayashi. On the structure of the monotone asymmetric traveling salesman polytope, II: Hypotractable facets. *Math. Program. Study*, 14:77–97, 1981.
- [409] J. Gu and X. Huang. Efficient Local Search with Search Space Smoothing: A Case Study of the Traveling Salesman Problem. *IEEE Trans. Systems Man Cybernet.*, 24(5):728–735, 1994.
- [410] S.K. Gupta and A.K. Mittal. A minmax problem as a linear programming problem. *Opsearch*, 19:49–53, 1982.
- [411] S.K. Gupta and A.P. Punnen.  $k$ -sum optimization problems. *Oper. Res. Lett.*, 9:121–126, 1990.
- [412] Y. Gurevich and S. Shelah. Expected computation time for Hamiltonian path problem. *SIAM J. Comput.*, 16:486–502, 1987.
- [413] D. Gusfield, R. Karp, L. Wang, and P. Stelling. Graph traversals, genes and matroids: an efficient case of the traveling salesman problem. *Discrete Appl. Math.*, 88:167–180, 1998.
- [414] G. Gutin. On an approach to solving the traveling salesman problem. In *Proceedings of the USSR Conference on System Research*, pages 184–185. Nauka, Moscow, 1984. (in Russian).
- [415] G. Gutin. On the efficiency of a local algorithm for solving the travelling salesman problem. *Autom. Remote Control*, 49(11):1514–1519, 1988.
- [416] G. Gutin. Exponential neighbourhood local search for the travelling salesman problem. *Comput. Oper. Res.*, 26:313–320, 1999.
- [417] G. Gutin and A. Yeo. TSP heuristics with large domination number. Technical Report 12, Dept Math and Stats, Brunel University, 1998.

- [418] G. Gutin and A. Yeo. Small diameter neighbourhood graphs for the traveling salesman problem: at most four moves from tour to tour. *Comput. Oper. Res.*, 26:321–327, 1999.
- [419] G. Gutin and A. Yeo. Anti-matroids. Submitted, 2001.
- [420] G. Gutin and A. Yeo. TSP tour domination and Hamilton cycle decomposition of regular digraphs. *Oper. Res. Lett.*, 28:107–111, 2001.
- [421] G. Gutin and A. Yeo. Upper bounds on ATSP neighborhood size. Technical Report TR-01-01. Dept of Computer Science, Royal Holloway Univ. London. 2001.
- [422] G. Gutin and A. Yeo. Assignment Problem based algorithms are impractical for the Generalized TSP. *Australas. J. Combinatorics*, to appear, 2002.
- [423] G. Gutin and A. Yeo. Polynomial approximation algorithms for the TSP and the QAP with factorial domination number. *Discrete Appl. Math.*, to appear.
- [424] G. Gutin, A. Yeo, and A. Zverovich. Traveling salesman should not be greedy: domination analysis of TSP greedy-type heuristics. *Discrete Appl. Math.*, to appear.
- [425] G. Gutin and A. Zverovich. Evaluation of the Contract-or-Patch Heuristic for the Asymmetric TSP. Submitted.
- [426] N. Guttmann-Beck, R. Hassin, S. Khuller, and B. Raghavachari. Approximation algorithms with bounded performance guarantees for the clustered traveling salesman problem. *Algorithmica*, 28:422–437, 2000.
- [427] H. Hamacher (ed.). Operations research software exchange program. *Eur. J. Oper. Res.*, 38:118–122, 1989.
- [428] R. Häggkvist. Series of lectures on Hamilton decomposition. British Combinatorial Conference, Swansea, UK, 1981, International Conference on Convexity and Graph Theory, Israel, March 1981, and Seminar Orsey, France, 1986.
- [429] R. Häggkvist. Personal communications, 1999–2000.
- [430] M.M. Halldórsson, K. Iwano, N. Katoh, and T. Tokuyama. Finding subsets maximizing minimum structures. Research Report, Science Institute, University of Iceland, 1995.
- [431] J.H. Halton. The shoelace problem. *Math. Intell.*, 17:36–39, 1995.
- [432] P. Hansen and **N. Mladenović**. Variable neighborhood search: Principles and applications. *Eur. J. Oper. Res.*, 130:449–467, 1994.
- [433] P. Hansen and **N. Mladenović**. An Introduction to Variable Neighborhood Search. In *Meta-Heuristics: Advances and Trends in Local Search Paradigms for Optimization*. Kluwer, Boston, 1999.
- [434] G.H. Hardy and E.M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, New York, 1979.
- [435] D. Hartvigsen. *Extensions of Matching Theory*. PhD thesis, Department of Mathematics, Carnegie Mellon University, Pittsburgh, PA, USA, 1984.
- [436] D. Hartvigsen. The square-free 2-factor problem in bipartite graphs. In *Proc. IPCO VII, Lect. Notes Comput. Sci.*, volume 1610, pages 234–241. Springer-Verlag, Berlin, 1999.

- [437] D. Hartvigsen and W. Pulleyblank. Outer-facial graphs and the traveling salesman problem. *SIAM J. Optimization*, 1:676–689, 1994.
- [438] R. Hassin and S. Rubinstein. An approximation algorithm for the maximum traveling salesman problem. *Inform. Process. Lett.*, 67:125–130, 1998.
- [439] R. Hassin and S. Rubinstein. Better approximations for max TSP. *Inf. Process. Lett.*, 75:181–186, 2000.
- [440] R. Hassin and S. Rubinstein. A  $7/8$ -approximation algorithm for metric max TSP. In *Proc. 7th Workshop Algorithms Data Struct. (WADS 2001)*. Springer-Verlag, 2001. To appear in *Lect. Notes Comput. Sci.*
- [441] X. He and Z.Z. Chen. Shortest path in complete bipartite digraph problem and its applications. In *Proc. 8<sup>th</sup> Ann. ACM-SIAM Symp. Discrete Algorithms Jan 5-7 1997*, New Orleans, LA, 1997.
- [442] K.H. Helbig-Hansen and J. Krarup. Improvements of the Held-Karp algorithm for the symmetric traveling salesman problem. *Math. Program.*, 7:87–96, 1974.
- [443] M. Held and R.M. Karp. A dynamic programming approach to sequencing problems. *J. Soc. Ind. Appl. Math.*, 10:196–210, 1962.
- [444] M. Held and R.M. Karp. The traveling salesman problem and minimal spanning trees. *Oper. Res.*, 18:1138–1162, 1970.
- [445] M. Held and R.M. Karp. The Traveling Salesman Problem and Minimum Spanning Trees: Part II. *Math. Program.*, 1:6–25, 1971.
- [446] K. Helsgaun. An effective implementation of the Lin-Kernighan traveling salesman heuristic. *Eur. J. Oper. Res.*, 12:106–130, 2000. Source code currently available from the author's <http://www.dat.ruc.dk/~keld/>.
- [447] C.S. Helvig, G. Robins, and A. Zelikovsky. Moving-target TSP and related problems. In *Lect. Notes Comput. Sci., Vol. 1461*, pages 453–464. Springer-Verlag, Berlin, 1998.
- [448] A.M. Hobbs. Powers of graphs, line graphs, and total graphs. In *Lecture Notes in Mathematics*, volume 642, pages 271–285. Springer, 1978.
- [449] D.S. Hochbaum and D.B. Shmoys. A unified approach to approximation algorithms for bottleneck problems. *J. Assoc. Comput. Mach.*, 33:533–550, 1986.
- [450] A.J. Hoffman. On simple linear programming problems. In *Convexity, Proc. Symposia Pure Math., AMS, Providence, RI*, pages 317–327, 1961.
- [451] A.J. Hoffman and P. Wolfe. History. In E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan, and D.B. Shmoys, editors, *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley, Chichester, 1985.
- [452] I. Hong, A. B. Kahng, and B. Moon. Improved large-step Markov chain variants for the symmetric TSP. *J. Heuristics*, 3:63–81, 1997.
- [453] J.E. Hopcroft and R.E. Tarjan. Dividing a graph into triconnected components. *SIAM J. Comput.*, 2:135–158, 1973.
- [454] L.J. Hubert and F.B. Baker. Applications of combinatorial programming to data analysis: the traveling salesman and related problem. *Psychometrika*, 43:81–91, 1978.
- [455] C. Hurkens. Nasty TSP instances for farthest insertion. In *Integer Programming and Combinatorial Optimization*, pages 346–352. Math. Programm. Soc., 1992.

- [456] A. Itai, C.H. Papadimitriou, and J.L. Szwarcfiter. Hamilton paths in grid graphs. *SIAM J. Comput.*, 11:676–686, 1982.
- [457] M.R. Jerrum and A.J. Sinclair. Approximating the permanent. *SIAM J. Comput.*, 18:1149–1178, 1989.
- [458] R.J. Jessen. Statistical investigation of a sample farm survey. Research Bulletin 304, Iowa State College of Agriculture, 1942.
- [459] D.B. Johnson. Efficient algorithms for shortest paths in sparse networks. *J. Assoc. Comput. Much.*, 24:1–13, 1977.
- [460] D.S. Johnson. Local optimization and the traveling salesman problem. In *Proc. 17th Colloq. Automata Languages Program.*, pages 446–461, Berlin, 1990. Lect. Notes Comput. Sci., Vol.443, Springer-Verlag.
- [461] D.S. Johnson, J.L. Bentley, L.A. McGeoch, and E.E. Rothberg. Near-optimal solutions to very large traveling salesman problems. Monograph, in preparation.
- [462] D.S. Johnson, L. McGeoch, F. Glover, and C. Rego. [www.research.att.com/~dsj/chtsp](http://www.research.att.com/~dsj/chtsp).
- [463] D.S. Johnson and L.A. McGeoch. The traveling salesman problem: A case study in local optimization. In E.H.L. Aarts and J.K. Lenstra, editors, *Local Search in Combinatorial Optimization*, pages 215–310. Wiley, Chichester, 1997.
- [464] D.S. Johnson, L.A. McGeoch, F. Glover, and C. Rego. Website for the DIMACS Implementation Challenge on the Traveling Salesman Problem: <http://www.research.att.com/~dsj/chtsp/>.
- [465] D.S. Johnson, L.A. McGeoch, and E.E. Rothberg. Asymptotic experimental analysis for the Held-Karp traveling salesman bound. In *Proc. 7th Ann. ACM-SIAM Symp. Discrete Algorithms*, pages 341–350, 1996.
- [466] D.S. Johnson and C.H. Papadimitriou. Computational complexity. In E.L. Lawler, J.K. Lenstra, A.G.H. Rinnooy Kan, and D.B. Shmoys, editors, *The Traveling Salesman Problem*. Wiley, Chichester, 1985.
- [467] S.M. Johnson. Optimal two- and three-stage production schedules with set-up times included. *Naval Res. Logist. Quart.*, 1:61–68, 1954.
- [468] W.B. Johnson and J. Lindenstrauss. Extensions of Lipschitz mappings into Hilbert space. *Contemp. Math.*, 26:189–206, 1984.
- [469] K. Jongens and T. Volgenant. The symmetric clustered traveling salesman problem. *Eur. J. Oper. Res.*, 19:68–75, 1985.
- [470] R. Jonker and T. Volgenant. A branch and bound algorithm for the symmetric traveling salesman problem based on the 1-tree relaxation. *Eur. J. Oper. Res.*, 9:83–89, 1982.
- [471] R. Jonker and T. Volgenant. Transforming asymmetric into symmetric traveling salesman problems. *Oper. Res. Lett.*, 2:161–163, 1983.
- [472] C. Jorgensen and S. Powell. Solving 0-1 minmax problems. *J. Oper. Res. Soc.*, 38:515–522, 1987.
- [473] M. Jünger, T. Christof, and G. Reinelt. A complete description of the traveling salesman polytope on 8 nodes. *Oper. Res. Lett.*, 10:497–500, 1991.
- [474] M. Jünger, G. Reinelt, and G. Rinaldi. The Traveling Salesman Problem. In M. Ball, T. Magnanti, C.L. Monma, and G.L. Nemhauser, editors, *Network*

- Models*, volume 7 of *Handbook on Operations Research and Management Sciences*, pages 225–330. North Holland, Amsterdam, 1995.
- [475] M. Jünger, G. Reinelt, and S. Thienel. Provably good solutions for the traveling salesman problem. *Z. Oper. Res.*, 40:183–217, 1994.
  - [476] M. Jünger, G. Rinaldi, and S. Thienel. Practical performance of efficient minimum cut algorithms. *Algorithmica*, 26:172–195, 2000.
  - [477] M. Jünger and S. Thienel. Introduction to ABACUS—A Branch-And-Cut System. *Oper. Res. Lett.*, 22:83–95, 1998.
  - [478] M. Jünger and S. Thienel. The ABACUS system for branch-and-cut-and-price algorithms in integer programming and combinatorial optimization. *Software - Practice and Experience*, 30:1325–1352, 2000.
  - [479] S.N. Kabadi. A solvable case of traveling salesman problem. Presented at the ORSA Conference, November 1985, Atlanta.
  - [480] S.N. Kabadi. Computational complexity of some problems in graph theory. Working paper, Faculty of Administration, University of New Brunswick, 1990.
  - [481] S.N. Kabadi. A general sufficiency condition for solvability of the Traveling Salesman Problem by Gilmore-Gomory patching. Working paper, Faculty of Administration, University of New Brunswick, 2001.
  - [482] S.N. Kabadi. Generalization of the Gilmore-Gomory scheme and new polynomially solvable classes of the Traveling Salesman Problem. *Discrete Appl. Math.*, to appear, 2001.
  - [483] S.N. Kabadi and M.F. Baki. Gilmore-Gomory type traveling salesman problems. *Comput. Oper. Res.*, 26:329–351, 1999.
  - [484] S.N. Kabadi and R. Chandrasekaran. The Gilmore-Gomory Traveling Salesman Problem. Invited talk in honor of R. Gomory, President IBM Research Center, ORSA Conference, October 1989, New York, 1989.
  - [485] S.N. Kabadi and R. Chandrasekaran. Generalized traveling salesman problem on a tree. Working paper 90-027, Faculty of Administration, University of New Brunswick, 1990.
  - [486] S.N. Kabadi and R. Chandrasekaran. Solvable classes of generalized traveling salesman problem. *DIMACS Series Discrete Math. Theor. Comput. Sci.*, 1:49–59, 1990.
  - [487] S.N. Kabadi and R. Chandrasekaran. Tree traveling salesman problem (extended abstract). In *Proc. 2nd Canad. Confer. Comput. Geometry*, J.J. Urrutia (ed.), pages 367–371, 1990.
  - [488] S.N. Kabadi and A.P. Punnen. The traveling salesman problem with two distinct tour values. Working paper 95-026, Faculty of Administration, University of New Brunswick, 1995.
  - [489] S.N. Kabadi and A.P. Punnen. Weighted graphs with Hamiltonian cycles of same length. Working paper 96-021, Faculty of Administration, University of New Brunswick, 1996.
  - [490] S.N. Kabadi and A.P. Punnen. New solvable cases of generalized graphical traveling salesman problem. Under preparation, 2001.
  - [491] K. Kalmanson. Edgeconvex circuits and the traveling salesman problem. *Canad. J. Math.*, 27(5):1000–1010, 1975.

- [492] P.C. Kanellakis and C.H. Papadimitriou. Local search for the asymmetric traveling salesman problem. *Oper. Res.*, 28(5):1086–1099, 1980.
- [493] V. Kantabutra. Traveling salesman cycles are not always subgraphs of Voronoi duals. *Inform. Process. Lett.*, 16:11–12, 1983.
- [494] N. Karmarkar. A new polynomial time algorithm for linear programming. *Combinatorica*, 4:373–395, 1984.
- [495] R.M. Karp. Reducibility among combinatorial problems. In R.E. Miller and J.W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, New York, 1972.
- [496] R.M. Karp. The probabilistic analysis of some combinatorial search algorithms. In J.F. Traub, editor, *Algorithms and Complexity: New Directions and Recent Results*, pages 1 – 19. Academic Press, New York, 1976.
- [497] R.M. Karp. Probabilistic analysis of partitioning algorithms for the traveling-salesman in the plane. *Math. Oper. Res.*, 2:209–224, 1977.
- [498] R.M. Karp. A patching algorithm for the non-symmetric traveling salesman problem. *SIAM J. Comput.*, 8:561–573, 1979.
- [499] R.M. Karp and R.J. Lipton. Some connections between nonuniform and uniform complexity classes. In *Proc. 12th ACM Ann. Symp. Theory Comput. (STOC '80)*, pages 302–309. L.A., California, 1980.
- [500] R.M. Karp and J.M. Steele. Probabilistic analysis of heuristics. In E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan, and D.B. Shmoys, editors, *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley, Chichester, 1985.
- [501] S. Kataoka and S. Morito. Selection of relaxation problems for a class of asymmetric traveling salesman problem instances. *J. Oper. Res. Soc. Japan*, 34:233–249, 1991.
- [502] J. Kelly, B. Rangeswamy, and J. Xu. A Scatter Search-based learning algorithm for neural network training. *Journal of Heuristics*, 2:129–146, 1996.
- [503] H. Kesten and S. Lee. The central limit theorem for weighted minimal spanning trees on random points. *Ann. Appl. Prob.*, 6:495–527, 1996.
- [504] S. Khanna, R. Motwani, M. Sudan, and U. Vazirani. On syntactic versus computational views of approximability. Technical Report TR95-023, ECCC, 1995.
- [505] G.A.P. Kindervater, A. Volgenant, G. De Leve, and V. van Gijlswijk. On dual solutions of the linear assignment problem. *Eur. J. Oper. Res.*, 19:76–81, 1985.
- [506] P.S. Klyaus. Generation of test problems for the traveling salesman problem. Preprint 16, Inst. Mat. Akad. Navuk BSSR, Minsk, USSR, 1976. (in Russian).
- [507] P.S. Klyaus. The structure of the optimal solution of certain classes of travelling salesman problem. *Vestsi Akad. Navuk BSSR, Ser. Fiz. -Mat. Navuk*, no. 6:95–98, 1976. (in Russian).
- [508] D.E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms (2nd Edition)*. Addison-Wesley, Reading, MA, 1981.
- [509] D.E. Knuth and S. Levy. *The CWEB System of Structured Documentation*. Addison-Wesley, Reading, MA, 1993. <http://www-cs-faculty.stanford.edu/~knuth/cweb.html>.

- [510] V.F. Kolchin. *Random Mappings*. Nauka, Moscow, 1984. (in Russian). Translated in Translation Series in Mathematics and Engineering, Optimization Software, Inc., Publications Division, New York, 1986.
- [511] J.G. Kollias, Y. Manolopoulos, and C.H. Papadimitriou. The optimum execution order of queries in linear storage. *Inform. Proc. Lett.*, 36:141–145, 1990.
- [512] J. Komlós and E. Szemerédi. Limit distributions for the existence of Hamilton circuits in a random graph. *Discrete Math.*, 43:55–63, 1983.
- [513] C. Korostensky and G.H. Gonnet. Near optimal multiple sequence alignments using a TSP approach. *SPIRE*, 99:105–114, 1999.
- [514] C. Korostensky and G.H. Gonnet. Using the TSP algorithms for evolutionary tree constructions. *Bioinformatics*, 16:619–627, 2000.
- [515] S.R. Kosaraju, J.K. Park, and C. Stein. Long tours and short superstrings. In *Proc. 35th Ann. IEEE Symp. Found. Comput. Sci. (FOCS'94)*, pages 166–177. IEEE, 1994.
- [516] E.G. Kosman. Structuring of matrices. *Soviet Math. Doklady*, 41:152–155, 1990.
- [517] A.V. Kostochka and A.I. Serdyukov. Polynomial algorithms with the estimates  $3/4$  and  $5/6$  for the traveling salesman problem of the maximum. *Upravlyaemye Sistemy*, no. 26:55–59, 1985. (in Russian).
- [518] U. Kousgaard. Personal communication, 2000.
- [519] E. Koutsoupias, C.H. Papadimitriou, and M. Yannakakis. Searching a fixed graph. In *Lect. Notes Comput. Sci., Vol. 1099*, pages 280–289. Springer-Verlag, Berlin, 1996.
- [520] M.M. Kovalev and V.M. Kotov. Analysis of algorithms for the construction of a hamiltonian cycle of maximum weight. *Vestsi Akad. Navuk BSSR, Ser. Fiz-Mat. Navuk*, no. 4:45–50, 1984. (in Russian).
- [521] M.M. Kovalev and V.M. Kotov. Estimate of the error of series of approximate algorithms. *Vestnik Belorusskogo Gosudarstvennogo Universiteta, Seriya I Fizika, Matematika, Mekhanika*, no. 3:44–48, 1986. (in Russian).
- [522] M. Krivelevich, B. Sudakov, V. Vu, and N. Wormald. Random regular graphs of high degree. *Random Struct. Algorithms*, to appear.
- [523] J.B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proc. Amer. Math. Soc.*, 7:48–50, 1956.
- [524] S. Krynski. Graphs in which all Hamiltonian cycles have the same length. *Discrete Appl. Math.*, 55:87–89, 1994.
- [525] D.A. Kumar and C.P. Rangan. Approximation algorithms for the traveling salesman problem with range condition. *Theor. Inform. Appl.*, 34:173–181, 2000.
- [526] N.N. Kuzjurin. Asymptotic investigation on the problem of covering. *Problemy Kybernetiky*, 37:19–56, 1980. (in Russian).
- [527] M. Labbé, G. Laporte, I. Rodríguez, and J.J. Salazar. The median cycle problem. Working paper Université Libre de Bruxelles, 1999.
- [528] M. Laguna, J.P. Kelly, J.L. Gonzalez-Valarde, and F. Glover. Tabu Search for multilevel generalized assignment problems. *Eur. J. Oper. Res.*, 82:176–189, 1995.



- [529] M. Laguna, H. Lourenço, and R. Martí. Assigning proctors to exams with Scatter Search. In *Computing Tools for Modeling: Optimization and Simulation*, Interfaces in Computer Science and Operations Research, pages 215–227. Kluwer, Boston, 2000.
- [530] M. Laguna and R. Martí. Scatter Search for the Linear Ordering Problem. In *New Ideas in Optimization*, pages 331–339. McGraw Hill, Berkshire, 1999.
- [531] A. Langevin. *Planification de tournées de véhicules*. PhD thesis, Department of Applied Mathematics, École Polytechnique, Montreal, 1988.
- [532] A. Langevin, F. Soumis, and J. Desrosiers. Classification of traveling salesman problem formulations. *Oper. Res. Lett.*, 9:127–132, 1990.
- [533] G. Laporte. The traveling salesman problem: An overview of exact and approximate algorithms. *Eur. J. Oper. Res.*, 59:231–237, 1992.
- [534] G. Laporte. Modeling and solving several classes of arc routing problems as traveling salesman problems. *Comput. Oper. Res.*, 24:1057–1061, 1997.
- [535] G. Laporte, A. Asef-Vaziri, and C. Srisankarajah. Some applications of the generalized traveling salesman problem. *J. Oper. Res. Soc.*, 47:1461–1467, 1996.
- [536] G. Laporte and S. Martello. The selective traveling salesman problem. *Discrete Appl. Math.*, 26:193–207, 1990.
- [537] G. Laporte, H. Mercure, and Y. Nobert. Generalized Travelling Salesman Problem through  $n$  sets of nodes: the asymmetrical cases. *Discrete Appl. Math.*, 18:185–197, 1987.
- [538] G. Laporte and Y. Nobert. Generalized Travelling Salesman through  $n$  sets of nodes: an integer programming approach. *INFOR, Can. J. Oper. Res. Inf. Process.*, 21:61–75, 1983.
- [539] G. Laporte and I.H. Osman. *Routing Problems: A Bibliography*, volume 61. Annals of Operations Research, 1995.
- [540] G. Laporte, J. Riera, and J.J. Salazar. A branch-and-cut algorithm for the undirected travelling purchaser problem. Working paper, Université de Montreal, 2000.
- [541] G. Laporte and F. Semet. Computational evaluation of a transformation procedure for the symmetric generalized traveling salesman problem. *INFOR, Can. J. Oper. Res. Inf. Process.*, 37:114–120, 1999.
- [542] H.T. Lau. *Finding Hamiltonian cycle in the square of a block*. PhD thesis, McGill University, 1980.
- [543] H.T. Lau. Finding EPS-graphs. *Monatsh. Math.*, 92:37–40, 1981.
- [544] H.T. Lau. *Combinatorial heuristic algorithms with FORTRAN*. Lect. Notes Econom. Math. Systems, Vol. 280. Springer Verlag, Berlin, 1986.
- [545] E.L. Lawler. The quadratic assignment problem. *Manage. Sci.*, 9:586–599, 1963.
- [546] E.L. Lawler. A solvable case of the traveling salesman problem. *Math. Program.*, 1:267–269, 1971.
- [547] E.L. Lawler. *Combinatorial Optimization: Networks and Matroids*. Holt, Rinehart and Winston, New York, 1976.

- [548] E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan, and D.B. Shmoys, editors. *Travelling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley, Chichester, 1985.
- [549] A.I. Lazebnik and I.L. Hranovic. A solution of the generalized traveling salesman problem by the branch and bound method. *Ekonom. Mat. Metody*, 9:363–364, 1973. (in Russian).
- [550] S. Lee. The central limit theorem for Euclidean minimal spanning trees I. *Ann. Appl. Prob.*, 6:996–1020, 1997.
- [551] S. Lee. Asymptotics of power-weighted Euclidean functionals. *Stoch. Proc. Appl.*, 79:109–116, 1999.
- [552] S. Lee. Worst case asymptotics of power-weighted Euclidean functionals. Preprint, 1999.
- [553] A.C. Leifer and M.B. Rosenwein. Strong linear programming relaxations for the orienteering problem. *Eur. J. Oper. Res.*, 73:517–523, 1994.
- [554] H.W. Lenstra. Integer programming with a fixed number of variables. *Math. Oper. Res.*, 8, 1983.
- [555] J.K. Lenstra and A.H.G. Rinnooy Kan. Some simple applications of the traveling salesman problem. *Oper. Res. Quart.*, 26:717–733, 1975.
- [556] J.K. Lenstra and A.H.G. Rinnooy Kan. Complexity of vehicle routing and scheduling problems. *Networks*, 11:221–227, 1981.
- [557] V.K. Leont’ev. Investigation of an algorithm for solving the travelling salesman problem. *Zh. Vychisl. Mat. Mat. Fiz.*, no. 5, 1973. (in Russian).
- [558] V.K. Leont’ev. Stability of the traveling salesman problem. *Zh. Vychisl. Mat. Mat. Fiz.*, no. 5, 1975. (in Russian).
- [559] A. Letchford. Separating a superclass of comb inequalities in planar graphs. *Math. Oper. Res.*, 25:443–454, 2000.
- [560] L.A. Levin. Universal sorting problems. *Problems Informat. Transmission*, 9:265–266, 1973.
- [561] Y. Lien, E. Ma, and B.W.S. Wah. Transformation of the generalized traveling salesman problem into the standard traveling salesman problem. *Inf. Sci.*, 74:177–189, 1993.
- [562] S. Lin. Computer solutions of the traveling salesman problem. *Bell System Tech. J.*, 44:2245–2269, 1965.
- [563] S. Lin and B.W. Kernighan. An effective heuristic algorithm for the traveling salesman problem. *Oper. Res.*, 21:972–989, 1973.
- [564] J.D. Litke. An improved solution to the traveling salesman problem with thousands of nodes. *Commun. ACM*, 27(12):1227–1236, 1984.
- [565] J.D.C. Little, K.G. Murty, D.W. Sweeney, and C. Karel. An algorithm for traveling salesman problem. *Oper. Res.*, 11:972–989, 1963.
- [566] A. Löbel. Vehicle scheduling in public transit and Lagrangean pricing. *Manage. Sci.*, 44:1637–1649, 1998.
- [567] R.J. Loulou. On multicommodity flow formulation for the TSP. Working paper, McGill University, Montreal, 1988.

- [568] L. Lovász. Matroid matching and some applications. *J. Comb. Theory Ser. B*, 28:208–236, 1980.
- [569] L. Lovász and M.D. Plummer. On a family of planar bicritical graphs. *Proc. London Math. Soc.*, (3)30:160–176, 1975.
- [570] L. Lovász and M.D. Plummer. *Matching Theory*. North-Holland, Amsterdam, 1986.
- [571] X. Lu. A Chvátal-Erdős type condition for Hamiltonian graphs. *J. Graph Theory*, 18:791–800, 1994.
- [572] A. Lucena. The time-dependent traveling salesman problem - The deliveryman case. *Networks*, 20:753–763, 1990.
- [573] T. Luczak. Cycles in random graphs. *Discrete Math.*, 98:231–236, 1991.
- [574] P.D. MacKenzie and Q.F. Stout. Optimal parallel construction of Hamiltonian cycles and spanning trees in random graphs. In *Proc. 5th ACM Symp. Parallel Algorithms Architectures*, pages 224–229. Springer, 1993.
- [575] P.C. Mahalanobis. A sample survey of the acreage under jute in Bengal. *Sankhya*, 4:511–530, 1940.
- [576] K.-T. Mak and A.J. Morton. A modified Lin-Kernighan traveling-salesman heuristic. *Oper. Res. Lett.*, 13:127–132, 1993.
- [577] V. Mak and N. Boland. Heuristic approaches to the asymmetric traveling salesman problem with replenishment arcs. *Internat. Trans. Oper. Res.*, 7:431–447, 2000.
- [578] G.S. Manku. A linear time algorithm for the bottleneck biconnected spanning subgraph problem. *Inform. Process. Lett.*, 59:1–7, 1996.
- [579] Y. Manoussakis. Directed Hamiltonian graphs. *J. Graph Theory*, 16(1):51–59, 1992.
- [580] H. Marchand, O. Boivineau, and S. Lafortune. On the synthesis of optimal schedulers in discrete event control problems with multiple goals. *SIAM J. Control Optimization*, 39:512–532, 2000.
- [581] O. Marcotte and S. Suri. Fast matching algorithms for points on a polygon. *SIAM J. Comput.*, 20(3):405–422, 1991.
- [582] F. Margot, 2001. Private communication with D. Naddef.
- [583] E.S. Marks. A lower bound for the expected travel among  $m$  random points. *Ann. Math. Stat.*, 19:419–422, 1948.
- [584] K. Marriott and P.J. Stuckey. *Programming with Constraints*. MIT Press, Cambridge, MA, 1998.
- [585] S. Martello. An enumerative algorithm for finding Hamiltonian circuits in a directed graph. *ACM Trans. Math. Softw.*, 9:131–138, 1983.
- [586] S. Martello and P. Toth. *Knapsack Problems: Algorithms and Computer Implementations*. Wiley, Chichester, 1990.
- [587] O. Martin, S.W. Otto, and E.W. Felten. Large-step Markov chains for the traveling salesman problem. *Complex Syst.*, 5:299–326, 1991.
- [588] O. Martin, S.W. Otto, and E.W. Felten. Large-step Markov chains for the TSP incorporating local search heuristics. *Oper. Res. Lett.*, 11:219–224, 1992.

- [589] C. McDiarmid. Clutter percolation and random graphs. *Math. Program. Study*, 13:17–25, 1980.
- [590] I.I. Melamed, S.I. Sergeev, and I.Kh. Sigal. The traveling salesman problem - I. Theoretical issues. *Automat. Remote Control*, pages 1147–1173, 1989.
- [591] I.I. Melamed, S.I. Sergeev, and I.Kh. Sigal. The traveling salesman problem. Exact Algorithms. *Avtomat. Telemekh.*, 1989. Russian.
- [592] I.I. Melamed, S.I. Sergeev, and I.Kh. Sigal. The traveling salesman problem. Approximation Algorithms. *Automat. Remote Control*, pages 1459–1479, 1990.
- [593] K. Menger. Das botenproblem. *Ergebnisse Eines Mathematischen Kolloquiums*, 2:11–12, 1932.
- [594] M. Michalski. On a class of polynomially solvable travelling salesman problems. *Zastosowania Matematyki*, 19:531–539, 1987.
- [595] C.E. Miller, A.W. Tucker, and R.A. Zemlin. Integer programming formulations and traveling salesman problems. *J. Assoc. Comput. Mach.*, 7:326–329, 1960.
- [596] D.L. Miller and J.F. Pekny. Results from a parallel branch and bound algorithm for the asymmetric traveling salesman problem. *Oper. Res. Lett.*, 8:129–135, 1989.
- [597] D.L. Miller and J.F. Pekny. Exact solution of large asymmetric traveling salesman problems. *Science*, 251:754–761, 1991.
- [598] D.L. Miller and J.F. Pekny. A staged primal-dual algorithm for perfect b-matching with edge capacities. *ORSA J. Comput.*, 7:298–320, 1995.
- [599] E. Minieka. The delivery man problem on a tree network. *Ann. Oper. Res.*, 18:261–266, 1989.
- [600] M. Misiurewicz. Lacing irregular shoes. *Math. Intell*, 18:32–34, 1996.
- [601] J.S.B. Mitchell. Guillotine subdivisions approximate polygonal subdivisions: A simple polynomial-time approximation scheme for geometric TSP, , and related problems. *SIAM J. Comput.*, 28:1298–1309, 1999.
- [602] **N. Mladenović** and P. Hansen. Variable Neighborhood Search. *Comput. Oper. Res.*, 24:1097–1100, 1997.
- [603] G. Monge. Memoires sur la theorie des deblais et des remblais. In *Histoire de l'Academie Royale des Science, Annee M. DCCLXXXI, avec les Memoires de Mathematique et de Physique, pur la meme Annee, Tires des Registres de cette Academie, Paris*, pages 666–704, 1781.
- [604] C.L. Monma, B.S. Munson, and W.R. Pulleyblank. Minimum-weight two-connected spanning networks. *Math. Prog., Ser. A*, 46:153–171, 1990.
- [605] J.W. Moon. *Topics on Tournaments*. Holt, Rinehart and Winston, New York, 1968.
- [606] B. M. Moret, D. A. Bader, and T. Warnow. High-performance algorithmic engineering for computational phylogenetics. In *Proc. 2001 Int'l Conf. Comput. Sci. (ICCS 2001), San Francisco*. Lect. Notes Comput. Sci., Vol. 2073–2074, Springer-Verlag, Berlin, 2001.
- [607] B.M. Moret, S. Wyman, D.A. Bader, T. Warnow, and M. Yan. A new implementation and detailed study of breakpoint analysis. In *Proc. 6th Pacific Symp.*

- Biocomputing (PSB 2001)*, Hawaii, pages 583–594. World Scientific Pub., Singapore, 2001.
- [608] G. Morton and A.H. Land. A contribution to the ‘Travelling-Salesman’ problem. *J. R. Stat. Soc., Ser. B*, 17:185–203, 1955.
  - [609] T.E. Morton and D.W. Pentico. *Heuristic Scheduling Systems*. Wiley, Chichester, 1993.
  - [610] K.G. Murty. An algorithm for ranking all the assignments in increasing order of cost. *Oper. Res.*, 16:682–687, 1968.
  - [611] K.G. Murty. *Linear Programming*. Wiley, New York, 1986.
  - [612] K.G. Murty. *Network Programming*. Prentice Hall, New York, 1992.
  - [613] D. Naddef. The binested inequalities of the symmetric traveling salesman polytope. *Math. Oper. Res.*, 17:882–900, 1992.
  - [614] D. Naddef. On the domino inequalities for the Symmetric Traveling Salesman Polytope. Technical report, Laboratoire ID-IMAG, 2001. To appear in *The Sharpest Cuts*, MPS/SIAM Series in Optimization.
  - [615] D. Naddef and Y. Pochet. The traveling salesman polytope revisited. *Math. Oper. Res.*, to appear.
  - [616] D. Naddef and G. Rinaldi. The symmetric traveling salesman polytope and its graphical relaxation: Composition of valid inequalities. *Math. Program. Ser. A*, 51:359–400, 1991.
  - [617] D. Naddef and G. Rinaldi. The crown inequalities for the traveling salesman polytope. *Math. Oper. Res.*, 17:308–326, 1992.
  - [618] D. Naddef and G. Rinaldi. The graphical relaxation: a new framework for the symmetric traveling salesman polytope. *Math. Program. Ser. A*, 58:53–88, 1993.
  - [619] D. Naddef and G. Rinaldi. The symmetric traveling salesman polytope: New facets from the graphical relaxation. Technical report, Laboratoire ID-IMAG, 2000.
  - [620] D. Naddef and S. Thienel. Efficient separation routines for the symmetric traveling salesman problem I: general tools and comb separation. Technical report, LMC-IMAG Grenoble, 1999. To appear in *Math. Program. Ser. A*.
  - [621] D. Naddef and S. Thienel. Efficient separation routines for the symmetric traveling salesman problem II: separating multi handle inequalities. Technical report, LMC-IMAG Grenoble, 1999. To appear in *Math. Program. Ser. A*.
  - [622] H. Nagamochi and T. Ibaraki. Computing edge-connectivity in multi-graphs and capacitated graphs. *SIAM J. Discrete Math.*, 5:54–66, 1992.
  - [623] H. Nagamochi, T. Ono, and T. Ibaraki. Implementing an efficient minimum capacity cut algorithm. *Math. Program. Ser. A*, 67:325–341, 1994.
  - [624] G.L. Nemhauser and L.E. Trotter. Properties of vertex packing and independent systems polyhedra. *Math. Program.*, 6:48–61, 1974.
  - [625] G.L. Nemhauser and L.A. Wolsey. *Integer and Combinatorial Optimization*. Wiley, Chichester, 1988.
  - [626] D. Neto. *Efficient Cluster Compensation for Lin-Kernighan Heuristics*. PhD thesis, Department of Computer Science, University of Toronto, 1999. Source code currently available at <http://www.cs.toronto.edu/~neto/research/lk/>.

- [627] A. Nijenhuis and H. Wilf. *Combinatorial Algorithms*. Acad. Press, New York, 1975.
- [628] A. Nijenhuis and H.S. Wilf. *Combinatorial Algorithms for Computers and Calculators*. Academic Press, N.Y., 2 edition, 1978.
- [629] C.E. Noon. *The Generalized Traveling Salesman Problem*. PhD thesis, Dept Ind. Oper. Res., University of Tennessee, 1988.
- [630] C.E. Noon and J.C. Bean. A Lagrangian based approach for the asymmetric generalized traveling salesman problem. *Oper. Res.*, 39:623–632, 1991.
- [631] C.E. Noon and J.C. Bean. An efficient transformation of the generalized traveling salesman problem. *INFOR*, 31:39–44, 1993.
- [632] H.L. Ong. Approximate algorithms for the travelling purchaser problem. *Oper. Res. Lett.*, 1:201–205, 1982.
- [633] H.L. Ong and J.B. Moore. Worst-case analysis of two travelling salesman heuristics. *Oper. Res. Lett.*, 2:273–277, 1984.
- [634] M. Oosten. A note on sequential lifting procedures. Research Memorandum M94-08, University of Limburg, 1994.
- [635] I. Or. *Traveling Salesman-Type Combinatorial Problems and their Relation to the Logistics of Regional Blood Banking*. PhD thesis, Department of Industrial Engineering and Management Sciences, Northwestern University, Evanston, IL, 1976.
- [636] O. Ore. Note on Hamiltonian circuits. *Am. Math. Mon.*, 67:55, 1960.
- [637] A.J. Orman and H.P. Williams. A survey of different integer programming formulations of the travelling salesman problem. Preprint, University of Southampton, 1999.
- [638] P.S. Ow and T.E. Morton. Filtered Beam Search in Scheduling. *Internat. J. Product. Res.*, 26(1):35–62, 1988.
- [639] C.O. Ozgur and J.R. Brown. A two-stage traveling salesman procedure for the single machine sequence dependent scheduling problem. *Omega*, 23:205–219, 1995.
- [640] M.W. Padberg. On the facial structure of the set packing polyhedra. *Math. Program.*, 5:199–216, 1973.
- [641] M.W. Padberg. A note on zero-one programming. *Oper. Res.*, 23:833–837, 1975.
- [642] M.W. Padberg and M. Grötschel. Polyhedral computations. In E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan, and D.B. Shmoys, editors, *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley, Chichester, 1985.
- [643] M.W. Padberg and S. Hong. On the symmetric traveling salesman problem: A computational study. *Math. Program. Study*, 12:78–107, 1980.
- [644] M.W. Padberg and M.R. Rao. Odd minimum cut-sets and b-matching. *Math. Oper. Res.*, 7:67–80, 1982.
- [645] M.W. Padberg and G. Rinaldi. Optimization of a 532-city symmetric traveling salesman problem by branch-and-cut. *Oper. Res. Lett.*, 6:1–7, 1987.

- [646] M.W. Padberg and G. Rinaldi. An efficient algorithm for the minimum capacity cut problem. *Math. Program., Ser. A*, 47:19–36, 1990.
- [647] M.W. Padberg and G. Rinaldi. Facet identification for the symmetric traveling salesman polytope. *Math. Program., Ser. A*, 47:219–257, 1990.
- [648] M.W. Padberg and G. Rinaldi. A branch-and-cut algorithm for the resolution of large-scale symmetric traveling salesman problems. *SIAM Review*, 33:60–100, 1991.
- [649] M.W. Padberg and T.Y. Sung. A polynomial-time solution to Papadimitriou and Steiglitz’s ‘traps’. *Oper. Res. Lett.*, 7:117–125, 1988.
- [650] M.W. Padberg and T.Y. Sung. An analytical comparison of different formulations of the traveling salesman problem. *Math. Program. Ser. A*, 52:315–357, 1991.
- [651] C.H. Papadimitriou. The Euclidean traveling salesman problem is NP-complete. *Theoret. Comput. Sci.*, 4:237–244, 1977.
- [652] C.H. Papadimitriou. *Computational Complexity*. Addison-Wesley, New York, 1994.
- [653] C.H. Papadimitriou and P.C. Kanellakis. Flowshop scheduling with limited temporary storage. *J. Assoc. Comput. Mach.*, 27:533–549, 1980.
- [654] C.H. Papadimitriou and K. Steiglitz. Some examples of difficult traveling salesman problems. *Oper. Res.*, 26(3):434–443, 1978.
- [655] C.H. Papadimitriou and K. Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Prentice-Hall, Englewood Cliffs, NJ, 1982.
- [656] C.H. Papadimitriou and K. Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Dover, New York, 1998.
- [657] C.H. Papadimitriou and U.V. Vazirani. On two geometric problems related to the travelling salesman problem. *J. Algorithms*, 5:231–246, 1984.
- [658] C.H. Papadimitriou and M. Yannakakis. Optimization, approximation and complexity classes. *J. Comput. System Sci.*, 43:425–440, 1991.
- [659] C.H. Papadimitriou and M. Yannakakis. The traveling salesman problem with distances one and two. *Math. Oper. Res.*, 18:1–11, 1993.
- [660] J.K. Park. A special case of the  $n$ -vertex traveling salesman problem that can be solved in  $O(n)$  time. *Inform. Process. Lett.*, 40:247–254, 1991.
- [661] R.G. Parker and R.L. Rardin. Guaranteed performance heuristics for the bottleneck traveling salesperson problem. *Oper. Res. Lett.*, 12:269–272, 1982.
- [662] W.L. Pearn and R.C. Chien. Improved solutions for the traveling purchaser problem. *Comput. Oper. Res.*, 25:879–885, 1998.
- [663] J.F. Pekny and D.L. Miller. An exact parallel algorithm for the resource constrained traveling salesman problem with application to scheduling with an aggregate deadline. Research Report EDRC-05-44-89, Institute of Complex Engineered Systems, Carnegie Mellon University, 1989.
- [664] J.F. Pekny and D.L. Miller. A parallel branch-and-bound algorithm for solving large asymmetric traveling salesman problems. *Math. Prog., Ser. A*, 55:17–33, 1992.

- [665] J.F. Pekny, D.L. Miller, and D. Stodolsky. A note on exploiting the Hamiltonian cycle problem substructure of the Asymmetric Traveling Salesman Problem. *Oper. Res. Lett.*, 10:173–176, 1991.
- [666] K. Penavic. Optimal firing sequences for CAT scans. Technical report, Dept. Appl. Math., SUNY Stony Brook, 1994.
- [667] E. Pesch and F. Glover. TSP ejection chains. *Discrete Appl. Math.*, 76:165–181, 1997.
- [668] J.M. Phillips, A.P. Punnen, and S.N. Kabadi. A linear time algorithm for the bottleneck travelling salesman problem on a Halin graph. *Inform. Process. Lett.*, 67:105–110, 1998.
- [669] J.C. Picard and M. Queyranne. The time-dependent traveling salesman problem and its application to the tardiness problem in one machine scheduling. *Oper. Res.*, 26:86–110, 1978.
- [670] R.D. Plante. The nozzle guide vane problem. *Oper. Res.*, 36:18–33, 1988.
- [671] L. K. Platzmann and J. J. Bartholdi, III. Spacefilling curves and the planar travelling salesman problem. *J. Assoc. Comput. Mach.*, 36:719–737, 1989.
- [672] L. Pósa. Hamiltonian circuits in random graphs. *Discrete Math.*, 14:359–364, 1976.
- [673] F.P. Preparata and M.I. Shamos. *Computational Geometry - An Introduction*. Springer, New York, 1985.
- [674] H.N. Psaraftis. A dynamic programming solution to the single-vehicle many-to-many immediate request dial-a-ride problem. *Transport. Sci.*, 14:130–154, 1980.
- [675] H.N. Psaraftis. A dynamic programming approach for sequencing groups of identical jobs. *Oper. Res.*, 28(6): 1347–1359, 1980.
- [676] W.R. Pulleyblank. Polyhedral Combinatorics. In *Handbook in OR and MS*, volume 1. North-Holland, Amsterdam, 1989.
- [677] A.P. Punnen. A linear time algorithm for the bottleneck strongly connected spanning subgraph problem. Research report, IIT Kanpur, India, 1989.
- [678] A.P. Punnen. Traveling salesman problem under categorization. *Oper. Res. Lett.*, 12:89–95, 1992.
- [679] A.P. Punnen. Combinatorial optimization with multiplicative objective function. Technical report, Department of Mathematical Sciences, University of New Brunswick-Saint John, 2000.
- [680] A.P. Punnen. The traveling salesman problem: new polynomial approximation algorithms and domination analysis. *J. Inform. Optim. Sci.*, 22:191–206, 2001.
- [681] A.P. Punnen and Y.P. Aneja. On  $k$ -sum optimization problems. *Oper. Res. Lett.*, 18:233–236, 1996.
- [682] A.P. Punnen and F. Glover. Implementing ejection chains with combinatorial leverage for the TSP. Research report, University of Colorado-Boulder, 1996.
- [683] A.P. Punnen and S. Kabadi. Domination analysis of some heuristics for the asymmetric traveling salesman problem. *Discrete Appl. Math.*, to appear, 2001.
- [684] A.P. Punnen, F. Margot, and S. Kabadi. TSP heuristics: domination analysis and complexity. Tech. Report 6, Dept. Math., University of Kentucky, 2001.



- [685] A.P. Punnen and K.P.K. Nair. A fast and simple algorithm for the bottleneck biconnected spanning subgraph problem. *Inform. Process. Lett.*, 50:283–286, 1994.
- [686] A.P. Punnen and K.P.K. Nair. Improved complexity bound for the maximum cardinality bottleneck bipartite matching problem. *Discrete Appl. Math.*, 55:91–93, 1994.
- [687] M. Queyranne and Y. Wang. Hamiltonian path and symmetric travelling salesman polytopes. *Math. Program. Ser. A*, 58:89–110, 1993.
- [688] M. Queyranne and Y. Wang. On the Chvátal rank of certain inequalities. Tech. report, University of British Columbia, Vancouver, BC, 1994.
- [689] M. Queyranne and Y. Wang. Symmetric inequalities and their composition for asymmetric travelling salesman polytopes. *Math. Oper. Res.*, 20:838–863, 1995.
- [690] L.V. Quintas and F. Supnik. On some properties of shortest Hamiltonian circuits. *Amer. Math. Mon.*, 72:977–980, 1965.
- [691] R. Ramakrishnan, A.P. Punnen, and P. Sharma. An efficient heuristic algorithm for the bottleneck traveling salesman problem. Technical report, IIT Kanpur, India, 2001.
- [692] R. Ramesh and R.C. Brown. An efficient four-phase heuristic for the generalized orienteering problem. *Comput. Oper. Res.*, 18:151–165, 1991.
- [693] R. Ramesh, Y.-S. Yoon, and M.H. Karwan. An optimal algorithm for the orienteering tour problem. *ORSA J. Comput.*, 4:155–165, 1992.
- [694] T. Ramesh. Traveling purchaser problem. *Opsearch*, 18:78–91, 1981.
- [695] M.R. Rao. A note on the multiple traveling salesman problem. *Oper. Res.*, 28:628–632, 1980.
- [696] S. Rao and W. Smith. Approximating geometric graphs via “spanners” and “banyans”. In *Proc. 30th Ann. ACM Symp. Theory Comput.*, pages 540–550, 1998.
- [697] R. Rardin and R.G. Parker. An efficient algorithm for producing a Hamiltonian cycle in the square of a biconnected graph. Industrial and Systems Engineering Report Series J-82, Georgia Institute of Technology, 1983.
- [698] R.L. Rardin, C.A. Tovey, and M.G. Pilcher. Analysis of random cut test instance generator for the TSP. In P.M. Pardalos, editor, *Complexity in Numerical Optimization*, pages 387–405. World Scientific, Singapore, 1993.
- [699] H.D. Ratliff and A.S. Rosenthal. Order-picking in a rectangular warehouse: a solvable case of the traveling salesman problem. *Oper. Res.*, 31(3):507–521, 1983.
- [700] S.S. Reddi and V. Ramamoorthy. On the flow-shop sequencing problem with no wait in process. *Oper. Res. Quart.*, 23(3):323–331, 1972.
- [701] C. Redmond. *Boundary Rooted Graphs and Euclidean Matching Algorithms*. PhD thesis, Dept. Math., Lehigh University, 1993.
- [702] C. Redmond and J.E. Yukich. Limit theorems and rates of convergence for subadditive Euclidean functionals. *Ann. Appl. Prob.*, 4:1057–1073, 1994.
- [703] C. Redmond and J.E. Yukich. Asymptotics for Euclidean functionals with power weighted edges. *Stock. Proc. Appl.*, 61:289–304, 1996.

- [704] C. Rego. Relaxed tours and path ejections for the traveling salesman problem. *Eur. J. Oper. Res.*, 106:522–538, 1998.
- [705] C. Rego. A subpath ejection method for the vehicle routing problem. *Manage. Sci.*, 44(10):1447–1459, 1998.
- [706] C. Rego. Node ejection chains for the vehicle routing problem: Sequential and parallel algorithms. *Parallel Comput.*, 27:201–222, 2001.
- [707] C. Rego and B. Alidaee. *Adaptive Memory and Evolution: Tabu Search and Scatter Search*. OR/CS Interfaces Series. Kluwer, Boston, to appear.
- [708] C. Rego and P. Leão. A Scatter Search Tutorial for Graph-Based Permutation Problems. Technical Report HCES-10-00, Hearin Center for Enterprise Science, School of Business Administration, University of Mississippi, MS, 2000.
- [709] G. Reinelt. TSPLIB — a traveling salesman problem library. *ORSA J. Comput.*, 3:376–384, 1991. <http://www.crpc.rice.edu/softlib/tsplib/>.
- [710] G. Reinelt. Fast heuristics for large geometric traveling salesman problems. *ORSA J. Comput.*, 4:206–217, 1992.
- [711] G. Reinelt. *The Travelling Salesman: Computational Solutions for TSP Applications*. Springer-Verlag, Berlin, 1994.
- [712] A.G. Reinhold. Some results on minimal covertex polygons. Manuscript written under the supervision of Fred Supnick, City College of New York, 1965.
- [713] J. Renaud, F.F. Boctor, and G. Laporte. A fast composite heuristic for the symmetric traveling salesman problem. *INFORMS J. Comput.*, 8:134–143, 1996.
- [714] B.W. Repetto. *Upper and Lower Bounding Procedures for the Asymmetric Traveling Salesman Problem*. PhD thesis, Graduate School of Industrial Administration, Carnegie-Mellon University, 1994.
- [715] W. Rhee. On the fluctuations of the stochastic traveling salesperson problem. *Math. Oper. Res.*, 16:482–489, 1991.
- [716] W. Rhee. On the traveling salesperson problem in many dimensions. *Random Struct. Algorithms*, 3:227–233, 1992.
- [717] W. Rhee. A matching problem and subadditive Euclidean functionals. *Ann. Appl. Prob.*, 3:794–801, 1993.
- [718] W. Rhee. On the stochastic Euclidean traveling salesperson problem for distributions with unbounded support. *Math. Oper. Res.*, 18:292–299, 1993.
- [719] W. Rhee. Boundary effects in the traveling salesperson problem. *Oper. Res. Lett.*, 16:19–25, 1994.
- [720] W. Rhee and M. Talagrand. Martingale inequalities and NP-complete problems. *Math. Oper. Res.*, 12:177–181, 1987.
- [721] W. Rhee and M. Talagrand. A sharp deviation inequality for the stochastic traveling salesman problem. *Ann. Prob.*, 17:1–8, 1989.
- [722] W. Rhee and M. Talagrand. On the long edges in the shortest tour through  $n$  random points. *Combinatorica*, 12:323–330, 1992.
- [723] F.S. Roberts. *Discrete Mathematical Models with Applications to Social, Biological, and Environmental Problems*. Prentice Hall, New Jersey, 1976.

- [724] N. Robertson and P.D. Seymour. Graph Minors X. Obstructions to tree decomposition. *J. Comb. Theory, Ser. B*, 52:153–190, 1991.
- [725] J.B. Robinson. On the Hamiltonian game: a traveling salesman problem. Technical report, RAND Research Memorandum RM-303, 1949.
- [726] R.W. Robinson and N.C. Wormald. Almost all cubic graphs are Hamiltonian. *Random Struct. Algorithms*, 3:117–126, 1992.
- [727] R.W. Robinson and N.C. Wormald. Almost all regular graphs are Hamiltonian. *Random Struct. Algorithms*, 5:363–374, 1994.
- [728] H. Rock. The three-machine no-wait flow-shop in NP-complete. *J. Assoc. Comput. Mach.*, 31:336–345, 1984.
- [729] A. Rohe, 2001. Personal communication.
- [730] D.J. Rosenkrantz, R.E. Stearns, and P.M. Lewis II. An analysis of several heuristics for the traveling salesman problem. *SIAM J. Comput.*, 6:563–581, 1977.
- [731] G. Rote. *Two Solvable Cases of The Traveling Salesman Problem*. PhD thesis, Technical University Graz, Graz, Austria, 1988.
- [732] G. Rote. The  $n$ -line traveling salesman problem. *Networks*, 22:91–108, 1992.
- [733] V.I. Rublineckii. Estimates of the accuracy of procedures in the traveling salesman problem. *Numerical Mathematics and Computer Technology*, no. 4:18–23, 1973. (in Russian).
- [734] S. Sahni and T. Gonzalez. P-complete approximation problem. *J. Assoc. Comput. Mach.*, 23:555–565, 1976.
- [735] J.J. Salazar. *Algoritmi per il problema del Commesso Viaggiatore Generalizzato*. PhD thesis, Royal Spanish College in Bologna, 1992. (in Italian).
- [736] J.J. Salazar. The polytope of the cycle problem on undirected graphs. Working paper, University of La Laguna, 1994.
- [737] D. Sanders. *On Extreme Circuits*. PhD thesis, City University of New York, 1968.
- [738] V.I. Sarvanov. On the minimization of a linear form on a set of all  $n$ -elements cycles. *Vestsi Akad. Navuk BSSR, Ser. Fiz.-Mat. Navuk*, no. 4:17–21, 1976. (in Russian).
- [739] V.I. Sarvanov. The mean value of the functional of the assignment problem. *Vestsi Akad. Navuk BSSR Ser. Fiz. -Mat. Navuk*, no. 2:111–114, 1976. (in Russian).
- [740] V.I. Sarvanov. *On Quadratic Choice Problems*. PhD thesis, Inst. Mat. AN BSSR, Minsk, USSR, 1978. (in Russian).
- [741] V.I. Sarvanov. On complexity of minimization of a linear form on a set of cyclic permutations. *Dokl. Akad. Nauk SSSR*, 253:533–535, 1980. (in Russian).
- [742] V.I. Sarvanov. On the complexity of minimizing a linear form on a set of cyclic permutations. *Soviet Math. Dokl.*, 22:118–210, 1980. (in Russian).
- [743] V.I. Sarvanov and N.N. Doroshko. The approximate solution of the traveling salesman problem by a local algorithm that searches neighborhoods of exponential cardinality in quadratic time. In *Software: Algorithms and Programs*,

- volume 31, pages 8–11. Math. Institute of the Belorussian Acad. Sci., Minsk, 1981. (in Russian).
- [744] V.I. Sarvanov and N.N. Doroshko. The approximate solution of the traveling salesman problem by a local algorithm with scanning neighborhoods of factorial cardinality in cubic time. In *Software: Algorithms and Programs*, volume 31, pages 11–13. Math. Institute of the Belorussian Acad. Sci., Minsk, 1981. (in Russian).
  - [745] F. Scholz. Coordination hole tolerance stacking. Technical Report BCSTECH-93-048, Boeing Computer Services, 1993.
  - [746] F. Scholz. Tolerance stack analysis methods. Technical Report BCSTECH-95-030, Boeing Computer Services, 1995.
  - [747] A. Schrijver. *Theory of Linear and Integer Programming*. Wiley, Chichester, 1986.
  - [748] F. Semet and J. Renaud. A tabu search algorithm for the symmetric generalized traveling salesman problem. Working paper, University of Montreal, 1999.
  - [749] M.M. Sepehri. *The symmetric generalized travelling salesman problem*. PhD thesis, University of Tennessee, Knoxville, 1991.
  - [750] T. Seppäläinen and J.E. Yukich. Large deviation principles for Euclidean functionals and other nearly additive processes. *Prob. Theory Relat. Fields*, 120:309–345, 2001.
  - [751] A.I. Serdyukov. An algorithm with an estimate for the traveling salesman problem of the maximum. *Upravlyaemye Sistemy*, no. 25:80–86, 1984. (in Russian).
  - [752] A.I. Serdyukov. Complexity of the traveling salesman problem with a description on graphs with small degrees of vertices. *Upravlyaemye Sistemy*, 85:73–82, 1985. (in Russian).
  - [753] A.I. Serdyukov. An asymptotically exact algorithm for the traveling salesman problem for a maximum in Euclidean space. *Upravlyaemye Sistemy*, no. 27:79–87, 1987. (in Russian).
  - [754] A.I. Serdyukov. Polynomial time algorithms with estimates of accuracy of solutions for one class of the maximum cost TSP. In *Kombinatorno-Algebraicheskie Metody v Diskretnoi Optimizatsii, Mezhvuzovskiy sbornik*, pages 107–114. Nauka, N. Novgorod, 1991. (in Russian).
  - [755] A.I. Serdyukov. The maximum-weight traveling salesman problem in finite-dimensional real spaces. *Diskretnyi Analiz i Issledovanie Operatsii*, 2(1):50–56, 1995. in Russian, Translated in *Operations research and discrete analysis. Translation of Diskret. Anal. Issled. Oper. 2 (1995), no. 1-4*, Mathematics and its Applications, 391, Kluwer, Dordrecht, 1997; 233–239.
  - [756] S.I. Sergeev. Algorithms for the minmax problem of the traveling salesman I: An approach based on dynamic programming. *Autom. Remote Control*, 56:1027–1032, 1995.
  - [757] S.I. Sergeev and A.B. Chernyshenko. Algorithms for the minmax problem of the traveling salesman II: Dual approach. *Autom. Remote Control*, 56:1155–1168, 1995.

- [758] S.P. Sethi, C. Sriskandarajah, G. Sorger, J. Blazewicz, and W. Kubiak. Sequencing of parts and robot moves in a robotic cell. *Internat. J. Flexible Manufacturing Systems*, 4:331–358, 1992.
- [759] E. Shamir. How many random edges make a graph Hamiltonian? *Combinatorica*, 3:123–132, 1983.
- [760] M.I. Shamos. *Computational Geometry*. PhD thesis, Yale University, 1975.
- [761] D.B. Shmoys and D.P. Williamson. Analyzing the Held-Karp TSP bound: A monotonicity property with application. *Inform. Proc. Lett.*, 35:281–285, 1990.
- [762] K.N. Singh and D.L. van Oudheusden. A branch and bound algorithm for the traveling purchaser problem. *Eur. J. Oper. Res.*, 97:571–579, 1997.
- [763] S.S. Skiena. *Implementing Discrete Mathematics: Combinatorics and Graph Theory in Mathematica*. Addison-Wesley, Redwood City, CA, 1990.
- [764] D.D. Sleator and R.E. Tarjan. Self-adjusting binary search trees. *J. Assoc. Comput. Mach.*, 32:652–686, 1985.
- [765] D.H. Smith and S. Hurley. Bounds for the frequency assignment problem. *Discrete Math.*, 167/168:571–582, 1997.
- [766] W.D. Smith. Finding the optimum  $N$ -city traveling salesman tour in the Euclidean plane in subexponential time and polynomial space. Manuscript, 1988.
- [767] S. Somhom, A. Modares, and T. Enkawa. Competition-based neural network for the multiple travelling salesman problem with minmax objective. *Comput. Oper. Res.*, 26:395–407, 1999.
- [768] J.M. Steele. Subadditive Euclidean functionals and non-linear growth in geometric probability. *Ann. Prob.*, 9:365–376, 1981.
- [769] J.M. Steele. Probabilistic algorithm for the directed traveling salesman problem. *Math. Oper. Res.*, 11:343–350, 1986.
- [770] J.M. Steele. Growth rates of Euclidean minimal spanning trees with power weighted edges. *Ann. Prob.*, 16:1767–1787, 1988.
- [771] J.M. Steele. Probabilistic and worst case analyses of classical problems of combinatorial optimization in Euclidean space. *Math. Oper. Res.*, 15:749–770, 1990.
- [772] J.M. Steele. *Probability Theory and Combinatorial Optimization*. SIAM, 1997.
- [773] J.M. Steele and T.L. Snyder. Worst case growth rates of some classical problems of combinatorial optimization. *SIAM J. Comput.*, 18:278–287, 1989.
- [774] P.C. Steven, L.G. Bruce, and E.A. Wasil. A computational study of smoothing heuristics for the Traveling Salesman Problem. *Eur. J. Oper. Res.*, 124:15–27, 1999.
- [775] P.S. Sundararaghavan. The unit capacity pickup and delivery problem on a one-way loop. *Opsearch*, 21(4):209–226, 1984.
- [776] F. Supnick. Extreme Hamiltonian lines. *Ann. Math.*, 66:179–201, 1957.
- [777] D.A. Suprunenko. The traveling-salesman problem. *Cybernetics*, 11:799–803, 1975.
- [778] D.A. Suprunenko. On the minimum of the value  $v(A, t)$  on the set of cycles. Preprint 14, Inst. Mat. Akad. Nauk. BSSR, Minsk, USSR, 1976. (in Russian).

- [779] M.M. Syslo, N. Deo, and J.S. Kowalik. *Discrete Optimization Algorithms with Pascal Programs*. Prentice-Hall, Englewood Cliffs, NJ, 1983.
- [780] M. Talagrand. Complete convergence of the directed TSP. *Math. Oper. Res.*, 16:881–887, 1991.
- [781] M. Talagrand. Concentration of measure and isoperimetric inequalities in product spaces. *Publications Mathématiques de l’I.H.E.S.*, 81:73–205, 1995.
- [782] M. Talagrand. A new look at independence. *Ann. Prob.*, 24:1–34, 1996.
- [783] M. Talagrand and J.E. Yukich. The integrability of the square exponential transportation cost. *Ann. Appl. Prob.*, 3:1100–1111, 1993.
- [784] A. Tamir and J.S.B. Mitchell. A maximum  $b$ -matching problem arising from median location models with applications to the roommate problem. *Math. Program.*, 80:171–194, 1998.
- [785] S.P. Tarasov. Properties of the trajectories of the appointments problem and the travelling-salesman problem. *U.S.S.R. Comput. Maths. Math. Phys.*, 21(1):167–174, 1981.
- [786] É. Tardos. A strongly polynomial minimum cost circulation algorithm. *Combinatorica*, 5(3):247–255, 1985.
- [787] R.E. Tarjan. Depth first search and linear graph algorithms. *SIAM J. Comput.*, 1:2:146–160, 1972.
- [788] R.E. Tarjan. Finding optimum branchings. *Networks*, 7:25–35, 1977.
- [789] A. Tetsuka. Optimal lacings. *Math. Intell.*, 19:6, 1997.
- [790] S. Thienel. *ABACUS—A Branch-And-CUt System*. PhD thesis, Universität zu Köln, 1995.
- [791] A. Thomason. A simple linear expected time algorithm for Hamilton cycles. *Discrete Math.*, 75:373–379, 1989.
- [792] T.W. Tillson. A Hamiltonian decomposition of  $K_{2m}^*$ ,  $2m \geq 8$ . *J. Combin. Theory, Ser. B*, 29(1):68–74, 1980.
- [793] E.A. Timofeev. Minmax di-associated subgraphs and the bottleneck traveling salesman problem. *Cybernetics*, 4:75–79, 1979.
- [794] L. Trevisan. When Hamming meets Euclid: the approximability of geometric TSP and MST. In *Proc. 29th ACM Symp. Theory Comput.*, pages 21–39, 1997.
- [795] L. Trevisan. When Hamming meets Euclid: the approximability of geometric TSP and Steiner tree. *SIAM J. Comput.*, 30:475–485, 2000.
- [796] T. Tsiligirides. Heuristic methods applied to orienteering. *J. Oper. Res. Soc.*, 35:797–809, 1984.
- [797] P. Underground. On graphs with Hamiltonian squares. *Discrete Math.*, 21:323, 1978.
- [798] R. van Dal. *Special Cases of the Traveling Salesman Problem*. PhD thesis, University of Groningen, 1992.
- [799] R. van Dal, J.A.A. van der Veen, and G. Sierksma. Small and large TSP: two polynomially solvable cases of the traveling salesman problem. *Eur. J. Oper. Res.*, 69:107–120, 1993.

- [800] E.S. van der Poort, M. Libura, G. Sierksma, and J.A.A. van der Veen. Solving the  $k$ -best traveling salesman problem. *Comput. Oper. Res.*, 26:409–425, 1999.
- [801] J.A.A. van der Veen. *Solvable Cases of the Traveling Salesman Problem with Various Objective Functions*. PhD thesis, University of Groningen, 1992.
- [802] J.A.A. van der Veen. An  $O(n)$  algorithm to solve the bottleneck traveling salesman problem restricted to ordered product matrice. *Discrete Appl. Math.*, 47:57–75, 1993.
- [803] J.A.A. van der Veen, A new class of pyramidally solvable symmetric traveling salesman problems. *SIAM J. Discrete Math.*, 7(4):585–592, 1994.
- [804] J.A.A. van der Veen, G. Sierksma, and R. van Dal. Pyramidal tours and the traveling salesman problem. *Eur. J. Oper. Res.*, 52:90–102, 1991.
- [805] J.A.A. van der Veen and R. van Dal. Solvable cases of the no-wait flow-shop scheduling problem. *J. Oper. Res. Soc.*, 42(11):971–980, 1991.
- [806] J.A.A. van der Veen, G.J. Woenginger, and S. Zhang. Sequencing jobs that require common resources on a single machine: a solvable case of the TSP. *Math. Program. Ser. A*, 82:235–254, 1998.
- [807] J.A.A. van der Veen and S. Zhang. Low-complexity algorithms for sequencing jobs with a fixed number of job-classes. *Comput. Oper. Res.*, 23(11):1059–1067, 1996.
- [808] D. van Wieren and Q.F. Stout. Random graph algorithms for the mesh with row and column subbuses. In *Proc. 2nd Workshop Reconfigurable Architectures*, pages 1–13. Springer, 1995.
- [809] S. Vishwanathan. An approximation algorithm for the asymmetric traveling salesman problem with distances one or two. *Inform. Process. Lett.*, 44:297–302, 1992.
- [810] V.G. Vizing. Values of the target functional in a priority problem that are majorized by the mean value. *Kibernetika, Kiev*, no. 5:76–78, 1973. (in Russian).
- [811] R.V. Vohra. Probabilistic analysis of the longest hamiltonian tour. *Networks*, 18(1):13–18, 1988.
- [812] O. Vornberger. Easy and hard cycle covers. Technical report, Univerität Paderborn, Germany, 1980.
- [813] O. Vornberger. *Komplexität von Wegeproblemen in Graphen*. PhD thesis, Gesamthochschule Paderborn, West Germany, 1980.
- [814] S. Voss. Dynamic tabu search strategies for the traveling purchaser problem. *Ann. Oper. Res.*, 63:253–275, 1996.
- [815] C. Voudouris and E. Tsang. Guided local search and its application to the travelling salesman problem. *Eur. J. Oper. Res.*, 113:469–499, 1999.
- [816] K. Wagner. Bermerkungen zu hadwigers vermutung. *Math. Ann.*, 141:433–451, 1960.
- [817] D.W. Walkup. On the expected value of a random asignment problem. *SIAM J. Comput.*, 8:440–442, 1979.
- [818] C. Walshaw. A multilevel approach to the travelling salesman problem. *Oper. Res.*, to appear. Preliminary version available as Mathematics Research Report

- 00/1M/63, Computing and Mathematical Sciences Department, University of Greenwich.
- [819] R.H. Warren. Pyramidal tours for the traveling salesman. *Optimization*, 29(1):81–87, 1994.
  - [820] R.H. Warren. Special cases of the traveling salesman problem. *Appl. Math. Comput.*, 60:171–177, 1994.
  - [821] D.J.A. Welsh. *Matroid Theory*. Academic Press, London, 1976.
  - [822] K. Wenger. Kaktus-Repräsentation der minimalen Schnitte eines Graphen und Anwendung im Branch-and-Cut Ansatz für das TSP. Diplomarbeit, Ruprecht-Karls-Universität Heidelberg, Institut für Informatik, Im Neuenheimer Feld 368, 69120 Heidelberg, Germany, [www.informatik.uni-heidelberg.de/groups/comopt](http://www.informatik.uni-heidelberg.de/groups/comopt), 1999. (in German).
  - [823] H. Weyl. Elementare Theorie der Konvexen Polyeder. *Comment. Math. Helv.*, 7:290–306, 1935.
  - [824] D.A. Wismer. Solution of the flowshop scheduling problem with no intermediate queues. *Oper. Res.*, 20:689–697, 1972.
  - [825] L.A. Wolsey. Heuristic analysis, linear programming and branch and bound. *Math. Program. Study*, 13:121–134, 1980.
  - [826] L.A. Wolsey. *Integer Programming*. Wiley, Chichester, 1998.
  - [827] R. Wong. Integer programming formulations of the traveling salesman problem. In *Proc. IEEE Internat. Confer. Circuits Comput.*, pages 149–152, 1980.
  - [828] N. Wormald. Models of random regular graphs. In *Surveys in Combinatorics Proc. 17th British Combinatorial Conference*, pages 239–298. Cambridge Press, 1999.
  - [829] J. Xu, S. Chiu, and P. Glover. Tabu Search and Evolutionary Scatter Search for 'Tree-Star' Network Problems, with Applications to Leased-Line Network Design. In *Telecommunications Optimization*. Wiley, Chichester, to appear.
  - [830] M. Yagiura, T. Ibaraki, and F. Glover. An Ejection Chain Approach for the Generalized Assignment Problem. Technical report, Graduate School of Informatics, Kyoto University, 2000.
  - [831] Q.F. Yang, R.E. Burkard, E. Cela, and G.J. Woeginger. Hamiltonian cycles in circulant digraphs with two stripes. *Discrete Math.*, 176:233–254, 1997.
  - [832] M. Yannakakis. Computational Complexity. In E.H.L. Aarts and J.K. Lenstra, editors, *Local Search in Combinatorial Optimization*, pages 19–56. Wiley, Chichester, 1997.
  - [833] J.E. Yukich. Asymptotics for the stochastic TSP with power weighted edges. *Prob. Theory Relat. Fields*, 102:203–220, 1995.
  - [834] J.E. Yukich. Ergodic theorems for some classical optimization problems. *Ann. Appl. Prob.*, 6:1006–1023, 1996.
  - [835] J.E. Yukich. *Probability Theory of Classical Euclidean Optimization Problems*, volume 1675 of *Lect. Notes Math.* Springer-Verlag, 1998.
  - [836] A.Z. Zelikovsky. An 11/6-approximation algorithm for the network Steiner Problem. *Algorithmica*, 9:463–470, 1993.



- [837] W. Zhang. Truncated branch-and-bound: A case study on the asymmetric TSP. In *Proc. AAAI 1993 Spring Symp. AI and NP-Hard Problems*, pages 160–166, Stanford, CA, 1993.
- [838] W. Zhang. Depth-first branch-and-bound versus local search: A case study. In *Proc. 17th National Conf. Artif. Intell. (AAAI-2000)*, pages 930–935, Austin, TX, 2000.

# List of Figures

|      |                                                                        |    |
|------|------------------------------------------------------------------------|----|
| 1.1  | Eyelets on a shoe                                                      | 4  |
| 1.2  | Zig-zag Lacing                                                         | 6  |
| 1.3  | Straight Lacing                                                        | 6  |
| 1.4  | quick Lacing                                                           | 6  |
| 1.5  | diagonal Lacing                                                        | 6  |
| 2.1  | The diamond graph                                                      | 38 |
| 2.2  | A graph and one of its closed walks                                    | 41 |
| 2.3  | Excluded minors for Theorem 15                                         | 43 |
| 2.4  | Examples of “shortcuts”                                                | 46 |
| 2.5  | Examples of 1-node liftings                                            | 56 |
| 2.6  | Examples of edge cloning                                               | 57 |
| 2.7  | Examples of clonable and non clonable edges                            | 58 |
| 2.8  | Example of a comb ( $t = 3$ )                                          | 60 |
| 2.9  | Example of intervals                                                   | 63 |
| 2.10 | Example of a path ( $h = 6, t = 3$ )                                   | 64 |
| 2.11 | Example of minimal trace with $T_2$ not tight ( $r = 3, s - 1 = 4$ )   | 65 |
| 2.12 | A facet inducing star (non-path) inequality; RHS=24                    | 66 |
| 2.13 | Examples of non facet and facet inducing star inequalities             | 66 |
| 2.14 | Example of a Clique Tree                                               | 68 |
| 2.15 | Example of a Bipartition                                               | 69 |
| 2.16 | Traces of tight tours with 2 and 4 edges in $\delta(T)$                | 70 |
| 2.17 | Traces of tight tours with 2, 4 and 6 edges in $\delta(T^*)$           | 71 |
| 2.18 | Example of a ladder                                                    | 72 |
| 2.19 | Trace of a tight tour containing $e \in (H_1 \cap T_1 : H_2 \cap T_2)$ | 73 |
| 2.20 | Example of a facet defining inequality                                 | 76 |
| 2.21 | Some traces of Hamiltonian cycles                                      | 77 |
| 2.22 | Example of domino inequality                                           | 79 |
| 2.23 | Tight tours for the example of Figure 2.22                             | 79 |
| 2.24 | Example of a fractional point                                          | 88 |
| 2.25 | The graph of Figure 2.24 after shrinking                               | 88 |
| 2.26 | Examples of violated combs from Figure 2.24                            | 88 |
| 2.27 | Two crossing minimum cuts                                              | 91 |
| 2.28 | Edges in maximally violated comb                                       | 92 |
| 2.29 | Examples of non violated comb inequalities                             | 94 |

|      |                                                                                                                                      |     |
|------|--------------------------------------------------------------------------------------------------------------------------------------|-----|
| 2.30 | Examples of violated path inequalities with 2 handles                                                                                | 95  |
| 2.31 | Example of a violated path inequality in ts225                                                                                       | 95  |
| 2.32 | Examples of violated clique trees                                                                                                    | 96  |
| 2.33 | Example of violated clique tree                                                                                                      | 96  |
| 2.34 | Violated clique tree and path inequalities from pr299                                                                                | 97  |
| 2.35 | Example how ladders can be violated                                                                                                  | 97  |
| 2.36 | Example of ladder violation in gil262                                                                                                | 98  |
| 2.37 | Example of violated ladders from pr439                                                                                               | 98  |
| 2.38 | Violated ladder from pr136 with correcting term                                                                                      | 99  |
| 2.39 | Violated ladders from pr439 with correcting term                                                                                     | 99  |
| 2.40 | Part of a fractional solution of fnl4461                                                                                             | 101 |
| 2.41 | The same solution after shrinking                                                                                                    | 102 |
| 2.42 | Flowchart of a Branch-and-Cut algorithm                                                                                              | 106 |
| 2.43 | Illustration for the third branching rule                                                                                            | 112 |
| 3.1  | A clique tree with $r = 3$ handles and $s = 9$ teeth                                                                                 | 122 |
| 3.2  | Curtain inequality for the ATSP and corresponding inequality for the STSP.                                                           | 123 |
| 3.3  | Fork inequality for the ATSP and corresponding inequality for the STSP.                                                              | 124 |
| 3.4  | The support graph of a     (left) and of a $D_k^-$ (right) inequality when $k = 5$ .                                                 | 125 |
| 3.5  | The support graph of a $T_k$ inequality ( $k = 3$ ).                                                                                 | 127 |
| 3.6  | The support graph of a C2 inequality.                                                                                                | 127 |
| 3.7  | The support graph of a C3 inequality.                                                                                                | 128 |
| 3.8  | Illustration of the merge lifting operation.                                                                                         | 140 |
| 3.9  | A $T_2$ inequality (a) and its canonical form (b).                                                                                   | 145 |
| 3.10 | Some odd CAT's.                                                                                                                      | 147 |
| 3.11 |                                                                                                                                      | 149 |
| 3.12 | The support graph of a primitive SD inequality.                                                                                      | 151 |
| 3.13 | A general SD inequality                                                                                                              | 153 |
| 3.14 | Crossing and noncrossing chords. The internal nodes w.r.t. $(i_a, i_b)$ are marked with +, and the chords are drawn in double lines. | 157 |
| 3.15 | A noose $Q$ with $q = 4$ .                                                                                                           | 157 |
| 3.16 | The dashed lines represent chords with coefficient 0.                                                                                | 158 |
| 3.17 | Support graphs of two lifted cycle inequalities having a single chord with coefficient 2.                                            | 159 |
| 3.18 | Counterexample for the converse of Theorem 57.                                                                                       | 161 |
| 3.19 | Support graphs of two shell inequalities.                                                                                            | 162 |
| 3.20 | Support graphs of two fork inequalities.                                                                                             | 163 |
| 3.21 | 2-liftable chord sets of curtain inequalities.                                                                                       | 165 |
| 3.22 | Support graphs of curtain inequalities for $ C  = 5, 6, 7$ .                                                                         | 166 |
| 4.1  | Branching on subtour $G_p = (V_p, A_p)$ , where $V_p = \{a, b, c\}$ and $A_p = \{(a, b), (b, c), (c, a)\}$ .                         | 173 |
| 5.1  | The dissection                                                                                                                       | 210 |
| 5.2  | The enclosing box contains the bounding box and is twice as large, but is shifted by random amounts in the     and $y$ directions.   | 211 |

|      |                                                                                                                                                                                                                                                                                               |     |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 5.3  | This portal-respecting tour enters and leaves the square 10 times, and the portion inside the square is a union of 5 disjoint paths.                                                                                                                                                          | 213 |
| 5.4  | Every crossing is moved to the nearest portal by adding a “detour.”                                                                                                                                                                                                                           | 214 |
| 5.5  | The tour crossed this line segment 6 times, but breaking it and reconnecting on each side (also called “patching”) reduced the number of crossings to 2.                                                                                                                                      | 216 |
| 6.1  | Example when $n = 6$ and $k = 3$ . The path connecting the thick nodes correspond to the tour $x_1x_3x_4x_2x_6x_5x_1$ .                                                                                                                                                                       | 236 |
| 7.1  | A rotation with $x_0$ as fixed end point.                                                                                                                                                                                                                                                     | 262 |
| 7.2  | Patching $C_1$ and $C_2$ together                                                                                                                                                                                                                                                             | 271 |
| 8.1  | 3-opt obtained by two successive 2-opt moves.                                                                                                                                                                                                                                                 | 317 |
| 8.2  | The String/Unstring Neighborhood Structure                                                                                                                                                                                                                                                    | 320 |
| 8.3  | Possible moves at the first level of the Lin-Kernighan process                                                                                                                                                                                                                                | 322 |
| 8.4  | The General Lin-Kernighan Procedure                                                                                                                                                                                                                                                           | 326 |
| 8.5  | The <i>double-bridge</i> neighborhood.                                                                                                                                                                                                                                                        | 328 |
| 8.6  | Two levels of a multi-node insertion ejection chain                                                                                                                                                                                                                                           | 332 |
| 8.7  | Two levels of a multi-node exchange ejection chain                                                                                                                                                                                                                                            | 333 |
| 8.8  | Neighborhood search iteration for node-based ejection chains.                                                                                                                                                                                                                                 | 334 |
| 8.9  | The stem-and-cycle ejection chain                                                                                                                                                                                                                                                             | 337 |
| 8.10 | An Iteration of the Stem-and-Cycle Procedure                                                                                                                                                                                                                                                  | 339 |
| 8.11 | The Filter and Fan Model                                                                                                                                                                                                                                                                      | 343 |
| 8.12 | A General Filter and Fan Procedure                                                                                                                                                                                                                                                            | 346 |
| 8.13 | A Scatter Search Template                                                                                                                                                                                                                                                                     | 358 |
| 8.14 | Path Relinking                                                                                                                                                                                                                                                                                | 365 |
| 9.1  | Running times for benchmark greedy code as a function of instance size for a variety of microprocessor-based machines. The microprocessors are listed in the order of their average times for 1000-city instances.                                                                            | 376 |
| 9.2  | Ratios of predicted running time to actual running time on a Compaq 500-Mhz Alpha processor for the Benchmark Greedy code and for the Johnson-McGeoch implementation of Lin-Kernighan.                                                                                                        | 378 |
| 9.3  | Average tradeoffs between tour quality and normalized running time for a variety of heuristics applied to 10,000-city Random Uniform Euclidean Instances. The full abbreviations for the heuristic names are given in Table 9.2 below and explained in Table 9.19 at the end of this chapter. | 382 |
| 9.4  | Tour length and normalized running time comparisons: Nearest Insertion versus Nearest Neighbor.                                                                                                                                                                                               | 384 |
| 9.5  | Tour quality comparisons for the Strip and Spacefilling Curve heuristics.                                                                                                                                                                                                                     | 389 |
| 9.6  | Average percentage excess for pure augmentation heuristics. (For an explanation of the abbreviations, see the text or Table 9.19.) Note that the ranges of $N$ are different for the two classes of instances.                                                                                | 395 |
| 9.7  | Tour quality comparisons for Farthest Insertion and Savings heuristics.                                                                                                                                                                                                                       | 402 |
| 9.8  | Tour quality comparisons for CCA and Savings heuristics.                                                                                                                                                                                                                                      | 403 |
| 9.9  | Tour quality for Christofides with greedy shortcuts.                                                                                                                                                                                                                                          | 404 |

|       |                                                                                                                                                                                                                                                           |     |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 9.10  | Tour quality comparisons between the Rego-Glover-Gamboa implementation of the Stem-and-Cycle variant of Lin-Kernighan with Boruvka starting tours and the Johnson-McGeoch implementation of basic Lin-Kernighan.                                          | 422 |
| 9.11  | Tour quality comparisons between the Helsgaun variant on Lin-Kernighan and the Johnson-McGeoch implementation of basic Lin-Kernighan.                                                                                                                     | 423 |
| 9.12  | Tour quality comparisons between taking the best of ten runs of CLK-ABCC-N and taking the result of a single run of CLK-ABCC-10N.                                                                                                                         | 434 |
| 9.13  | Tour quality comparisons between a $0.1N$ iteration run of Helsgaun's LK variant and taking the best of ten runs of CLK-ABCC-N.                                                                                                                           | 435 |
| 9.14  | Tour quality comparisons between an $N$ -iteration run of Helsgaun's LK variant and taking the best of five tour-merges, each involving ten CLK-ABCC-N tours.                                                                                             | 437 |
| 11.1  | Patching pseudograph $G_p^\pi$                                                                                                                                                                                                                            | 495 |
| 11.2  | The digraph $G_\pi$                                                                                                                                                                                                                                       | 497 |
| 11.3  | Order digraph $G_O = (T^*, E_O)$                                                                                                                                                                                                                          | 498 |
| 11.4  | A simple chain on $S = \{3, 4, 5, 6, 7\}$                                                                                                                                                                                                                 | 500 |
| 11.5  | Cost structure of a pyramidal tour                                                                                                                                                                                                                        | 502 |
| 11.6  | Digraph $G_B$ corresponding to warehouse order-picking problem                                                                                                                                                                                            | 546 |
| 11.7  | The convex-hull-and- $k$ -line TSP                                                                                                                                                                                                                        | 551 |
| 11.8  | Papadimitriou-Steiglitz trap                                                                                                                                                                                                                              | 574 |
| 12.1  | Constructing a Hamiltonian cycle when $n$ is even                                                                                                                                                                                                         | 592 |
| 12.2  | Constructing a partial tour $T_2$ when $n$ is odd                                                                                                                                                                                                         | 593 |
| 12.3  | Patching cycles                                                                                                                                                                                                                                           | 596 |
| 12.4  | Constructing $P_i$ and $Q_j$                                                                                                                                                                                                                              | 597 |
| 12.5  | Patching runs into Hamiltonian cycles                                                                                                                                                                                                                     | 604 |
| 13.1  | Infeasible GTSP solution satisfying (14)–(17). The drawn edges have the blank nodes $y_v = 0$ , and the black nodes $y_v = 1/2$ .                                                                                                                         | 619 |
| 13.2  | The layered network $LN$ .                                                                                                                                                                                                                                | 634 |
| 13.3  | Fractional point violating a cycle cover inequality for the OP instance with $t_0=6$ and $t_e=1$ for all $e \in E$ ( $0 < \alpha \leq 1/2$ ). Here $T = \{(1, i_1), (i_1, i_2), \dots, (i_6, 1)\}$ , $x(T) = 2 + 5\alpha$ , and $y(V(T)) = 3 + 4\alpha$ . | 645 |
| 13.4  | A fractional point violating a path inequality for the OP instance with $t_0=6$ and $t_e=1$ for all $e \in E$ ( $0 < \alpha \leq 1/2$ ).                                                                                                                  | 646 |
| 14.1  | A typical solution.                                                                                                                                                                                                                                       | 666 |
| 14.2  | Primitive comb inequality for $P^*$                                                                                                                                                                                                                       | 672 |
| 14.3  | Primitive odd CAT inequalities for $P^*$                                                                                                                                                                                                                  | 674 |
| 14.4  | Primitive SD inequality for $P^*$ .                                                                                                                                                                                                                       | 676 |
| 14.5  | Primitive clique tree for $P_0$ with two handles.                                                                                                                                                                                                         | 678 |
| 14.6  | Primitive lifted cycle inequalities for $P_0$ .                                                                                                                                                                                                           | 680 |
| 14.7  | Cloning of two nodes of $T_2$                                                                                                                                                                                                                             | 682 |
| 14.8  | Cloning every node of $T_2$                                                                                                                                                                                                                               | 682 |
| 14.9  | SD inequality obtained by clique-lifting                                                                                                                                                                                                                  | 684 |
| 14.10 | Lifted 3-cycle inequality with two quasi-clones                                                                                                                                                                                                           | 685 |
| 14.11 | Lifted 3-cycle inequality with one quasi-clone for every node                                                                                                                                                                                             | 685 |

|                                  |     |
|----------------------------------|-----|
| <i>List of Figures</i>           | 811 |
| 14.12 Illustration of Example 19 | 690 |

*This page intentionally left blank*

## List of Tables

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |     |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 2.1 | Some statistics on $STSP(n)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 36  |
| 2.2 | Computational results                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 115 |
| 2.3 | Very large instances with Concorde                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 116 |
| 4.1 | ATSP instances.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 196 |
| 4.2 | FT-b&c without and with Fractionality Persistency (10,000 seconds time limit).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 198 |
| 4.3 | Sensitivity of Concorde to 3- and 2-node transformations and to the chunk size parameter.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 199 |
| 4.4 | Comparison of branch-and-bound codes. Time limit of 1,000 seconds.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 201 |
| 4.5 | Comparison of branch-and-cut codes. Time limit of 10,000 seconds.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 202 |
| 4.6 | Comparison of branch-and-bound codes. Time limit of 1,000 seconds.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 203 |
| 4.7 | Comparison of branch-and-cut codes. Time limit of 10,000 seconds.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 204 |
| 9.1 | For instances in the Challenge test suite that have known optimal solutions, the percent by which the optimal tour length exceeds the Held-Karp bound and the normalized running times in seconds for computing each using Concorde with its default settings. (“–” indicates that the default settings did not suffice.) For random instances, suffixes 1k, 3k, and 10k stand for 1,000, 3,162, and 10,000 cities respectively. The number of cities in a TSPLIB instance is given by its numerical suffix.                                                                                                                                                                                                                                                               | 380 |
| 9.2 | Average tour quality and normalized running times for various heuristics on the 10,000-city instances in our Random Uniform Euclidean testbed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 382 |
| 9.3 | Average comparisons between Nearest Insertion (NI) and Nearest Neighbor (NN) on our geometric testbeds. Bentley’s implementations [103] of both heuristics were used. A positive entry in the “Excess” table indicates that the NI tours are longer by the indicated percentage on average. As in all subsequent tables of this sort, the TSPLIB averages are over the following instances: pr1002, pcb1173, r11304, and nrw1379 for $N = 1,000$ , pr2392, pcb3038, and fn14461 for $N = 3,162$ , pla7397 and bfd14051 for $N = 10K$ , pla33810 for $N = 31K$ , and pla85900 for $N = 100K$ . These may not be completely typical samples as we had to pick instances that most codes could handle, thus ruling out the many TSPLIB instances with fractional coordinates. | 383 |



|      |                                                                                                                                                                                                                                                                                                                                                                                                       |     |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 9.4  | Average normalized times for reading instances using the standard I/O routines of C, compiled for MIPS processors using gcc.                                                                                                                                                                                                                                                                          | 386 |
| 9.5  | Average performance of the <code>strip</code> heuristic.                                                                                                                                                                                                                                                                                                                                              | 387 |
| 9.6  | Average performance of the Spacefilling Curve heuristic.                                                                                                                                                                                                                                                                                                                                              | 388 |
| 9.7  | Normalized running times in seconds for Pure Augmentation heuristics and Random Uniform Euclidean instances.                                                                                                                                                                                                                                                                                          | 396 |
| 9.8  | Average percentage excesses over the Held-Karp bound and normalized running times for Bentley's implementations of Insertion, Addition, and Augmented Addition heuristics applied to 100,000-city Random Uniform Euclidean instances. For comparison purposes, the last column gives results for the Johnson-McGeoch implementation of Savings.                                                       | 401 |
| 9.9  | Results for the more powerful tour construction heuristics on Random Uniform Euclidean instances. <code>Sav</code> , <code>ACh</code> , and <code>Chr</code> stand for Savings, AppChristo, and Christo, respectively.                                                                                                                                                                                | 406 |
| 9.10 | Average percent excess over the HK bound and normalized running times for 100,000-city Uniform and Clustered instances and for TSPLIB instance <code>pla85900</code> . All codes were run on the same machine.                                                                                                                                                                                        | 411 |
| 9.11 | Average percent excesses over the HK bound and normalized running times for Random Matrix instances of sizes from 1,000 to 10,000 cities. All codes were run on the same machine.                                                                                                                                                                                                                     | 412 |
| 9.12 | Results for 3-Opt and four implementations of Lin-Kernighan. Averages for TSPLIB are taken over the same instances as in Figure 9.3.                                                                                                                                                                                                                                                                  | 419 |
| 9.13 | Average percent excesses over the HK bound and normalized running times for Random Matrix instances of sizes from 1,000 to 10,000 cities.                                                                                                                                                                                                                                                             | 421 |
| 9.14 | Results for the Helsgaun variant on Lin-Kernighan as compared to those for the Johnson-McGeoch implementation of basic Lin-Kernighan.                                                                                                                                                                                                                                                                 | 424 |
| 9.15 | Tour quality results for repeated local search heuristics, with Helsgaun and LK-JM included for comparison purposes.                                                                                                                                                                                                                                                                                  | 428 |
| 9.16 | Normalized running times for repeated local search heuristics. The "3M" column was omitted so the table would fit on a page. The missing entries are 1255 seconds for LK-JM and 94700 seconds for CLK-ACR-N.                                                                                                                                                                                          | 429 |
| 9.17 | Best results obtained for all testbed instances whose optimal solutions are known. Four heuristics sufficed to generate these results: Helsgaun-N (abbreviated in the table as <code>h</code> ), Tourmerge-b5 ( <code>m</code> ), ILK-JM-10N ( <code>i10</code> ), and best of ten runs of ILK-JM-N ( <code>ib</code> ). Where more than one heuristic found the best solution, we list all that did. | 439 |
| 9.18 | Tour quality and normalized running times for TSPLIB instance <code>d15112</code> . General conclusions should not be drawn from small differences in quality or time.                                                                                                                                                                                                                                | 442 |

|       |                                                                                                                                                                                                                                                                                                                                                                                |     |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 9.19  | Abbreviated names used in this chapter. The symbol $X$ stands for an abbreviation of the implementers' names: "ABCC" for Applegate, Bixby, Chvátal, and Cook (Concorde), "ACR" for Applegate, Cook, and Rohe, "B" for Bentley, "JM" for Johnson-McGeoch, "Neto" for Neto, and "R" for Rohe. Adding the suffix "-bn" to any name means that one is taking the best of $n$ runs. | 443 |
| 10.1  | Comparisons between normalized 196 Mhz MIPS processor time and actual running time on the 500 Mhz Alpha target machine.                                                                                                                                                                                                                                                        | 450 |
| 10.2  | For the 100-, 316-, and 1000-city instances of each class, the average percentage shortfall of the AP bound from the HK bound and the average asymmetry and triangle inequality metrics as defined in the text. "-" stands for .000.                                                                                                                                           | 456 |
| 10.3  | Metrics for <code>realworld</code> testinstances.                                                                                                                                                                                                                                                                                                                              | 456 |
| 10.4  | Tour quality and normalized times for corresponding symmetric (-S) and asymmetric (-A) codes. Hgaun is an abbreviation for Helsgaun. LK is the Johnson-McGeoch implementation of (symmetric) Lin-Kernighan covered in Chapter 9.                                                                                                                                               | 465 |
| 10.5  | Best average percentage above the HK bound obtained within various normalized running time thresholds for the 316-, 1,000-, and 3162-city instances of the nine asymmetric classes.                                                                                                                                                                                            | 467 |
| 10.6  | Tour quality and normalized running times for fast heuristics.                                                                                                                                                                                                                                                                                                                 | 469 |
| 10.7  | Tour quality and normalized running times for Cycle Cover Heuristics.                                                                                                                                                                                                                                                                                                          | 471 |
| 10.8  | Normalized running times for reading an instance and solving the corresponding Assignment Problem with the AP codes used by COE, Patch, and Zhang.                                                                                                                                                                                                                             | 472 |
| 10.9  | Tourquality and normalized running times for KP, I3opt, and Ihyper-3.                                                                                                                                                                                                                                                                                                          | 474 |
| 10.10 | Tour quality and normalized running times for IKP and Helsgaun.                                                                                                                                                                                                                                                                                                                | 475 |
| 10.11 | Results for optimization via Concorde. A "-" entry indicates that optimization was not feasible via this approach. Running times include the time for running Zhang (first four classes) or Helsgaun (lastfive) to obtain an initial upper bound, but not the (relatively small) time for the ATSP-to-STSP transformation.                                                     | 476 |
| 10.12 | Tourquality and normalized running times for classes <code>amat</code> (Random Asymmetric Matrices) and <code>tmat</code> (Random Asymmetric Matrices closed under shortest paths).                                                                                                                                                                                            | 478 |
| 10.13 | Tourquality and normalized running times for classes <code>shop</code> (50 Processor No-Wait Flowshop Scheduling) and <code>disk</code> (Random Disk Head Motion Scheduling).                                                                                                                                                                                                  | 479 |
| 10.14 | Tourquality and normalized running times for classes <code>super</code> (Approximate Shortest Common Superstring) and <code>crane</code> (Stacker-Crane Scheduling).                                                                                                                                                                                                           | 480 |
| 10.15 | Tour quality and normalized running times for classes <code>coin</code> (Coinbox Collection Routing) and <code>stilt</code> (Tilted Supnorm Routing).                                                                                                                                                                                                                          | 481 |
| 10.16 | Tourquality and normalized running times for class <code>rtilt</code> (Tilted Rectilinear Routing). Optima for 1000-city instances were computed by symmetric code applied to equivalent <code>rect</code> .                                                                                                                                                                   | 482 |
| 10.17 | Robustness metrics for the heuristics in this study.                                                                                                                                                                                                                                                                                                                           | 484 |

|       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |     |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 10.18 | Results for realworld instances. Patch, Zhang, and Helsgaun are abbreviated by P, Z, and H respectively. The running time for OPT includes both the time to obtain an initial upper bound using Zhang and the time to run Concorde using its default settings on the transformed instance. The relatively small time needed to perform the ATSP to STSP transformation is not included. Instance atex600 could not be optimized in reasonable time using Concorde's default settings. | 485 |
| 13.1  | Root node statistics.                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 639 |
| 13.2  | Branch-and-cut statistics.                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 640 |
| 13.3  | Some computational results with different clusterizations.                                                                                                                                                                                                                                                                                                                                                                                                                            | 642 |
| 13.4  | Results for problems of Class I (OP and VRP instances) with $\alpha = 0.50$ .                                                                                                                                                                                                                                                                                                                                                                                                         | 657 |
| 13.5  | Additional results for Class I ( $\alpha = 0.50$ ) problems                                                                                                                                                                                                                                                                                                                                                                                                                           | 658 |
| 13.6  | Results for Class II (TSPLIB instances) and Generation 1 ( $\alpha = 0.50$ )                                                                                                                                                                                                                                                                                                                                                                                                          | 659 |
| 13.7  | Results for Class II (TSPLIB instances) and Generation 2 ( $\alpha = 0.50$ )                                                                                                                                                                                                                                                                                                                                                                                                          | 659 |
| 13.8  | Results for Class II (TSPLIB instances) and Generation 3 ( $\alpha = 0.50$ )                                                                                                                                                                                                                                                                                                                                                                                                          | 660 |
| 13.9  | Average results over 10 random instances of Class III                                                                                                                                                                                                                                                                                                                                                                                                                                 | 661 |
| 13.10 | Average results over 5 random instances of Class IV                                                                                                                                                                                                                                                                                                                                                                                                                                   | 661 |

# Topic Index

- 1-1 extension, 492
- 1-arborescence, 181
- 1-arc, 188–189, 193
- 1-edge, 189
- 1-extension, 491
- 1-factor, 590
  - almost, 593
  - maximum weight, 590, 597, 602
- 1-factors, 607
- 1-graphic matroid, 23
- 1-sum, 563
- 1-tree, 636
- 2-cycle, 673
- 2-cycles, 591
- 2-factor, 589
  - 3-restricted, 590
  - 4-restricted, 589
  - 5-restricted, 590
  - k-restricted, 589
  - maximum weight, 593
  - minimum weight, 282
  - near random, 272
- 2-liftable chord, 162
- 2-liftable chords, 158, 160
- 2-liftable set, 156
- 2-matching, 182, 589
- 2-max bound, 704, 714
- 2-Opt, 462
- 2-parity problem, 536
- 2-sum, 563
  - stable, 52
- 2-sums, 51–52
- 3-Opt, 371
- 3-splicing, 569
- 3-sum, 563
- A-path, 525, 528, 715
  - lower, 525
  - upper, 525
- ABACUS, 113
- ABCC, 113
- Absolutely continuous, 297
- Adaptive memory programming, 340, 350, 360, 364
- Additive approach, 176–177
- Additive bound, 169
- Additive bounding, 177
- Admissible neighborhood, 350
- Admissible quadruplet, 556
- Affinely independent, 670
- Airline hubs, 598
- Algorithm
  - Miller and Pekny, 175
  - $\delta$ -approximate, 706
  - additive bounding, 178, 181
  - approximation, 207, 705
  - Arora's, 208
  - branch and bound, 703
  - branch and cut, 105, 184, 705
  - Carr's, 90
  - chained Lin-Kernighan, 327
  - Christofides, 207
  - deterministic, 269
  - ejection chain, 250
  - Eppstein, 527
  - extension-rotation, 272, 277
  - fixed dissection, 293
  - GENI, 319
  - GENIUS, 319, 355
  - Gilmore-Gomory, 493
  - Gilmore Gomory, 505, 539
  - greedy, 251, 279, 631, 648
  - greedy covering, 239
  - HAM, 267
  - Hartvigsen's, 589
  - Held-Karp, 255
  - Helsgaun's, 348
  - iterated, 327
  - Karp's, 279
  - Karp's dissection, 209
  - labelling, 174
  - Lawler's, 715
  - Lawler, 538
  - Lin-Kernighan, 250, 312
  - MINCUT, 193
  - minimum cut, 34
  - Nagamoshi-Ibaraki, 84
  - nearest neighbor, 278

- odd-cut separation, 648
- p-relaxation, 712
- Park, 533
- patching, 174–176, 272, 277, 279, 495
- randomized, 267, 594, 598
- rotation-closure, 279
- Sedyukov, 591
- separation, 182, 193, 629, 648
- Smith's, 208
- subtour patching, 607
- vertex insertion, 247
- Alternating path, 131
  - dynamic, 337
- Alternating paths, 337
- Alternating trail, 146
- Amplification factor, 348
- Angle TSP, 25
- Anti greedy, 227
- Antipodal points, 549
- Arborescence, 171, 180, 192
- Arborescence problem, 178
- Arc routing, 13
- Arcs
  - compatible, 156
  - geodesic, 549
  - inessential, 547
  - reduced cost, 175
- Articulation point, 180
- Articulation set, 677
- Aspiration criteria, 350
- Assignment bound, 704
- Assignment problem
  - quadratic, 330
- Assignment problem, 16, 22, 145, 169, 171–172, 231, 275
- Assignment problem, 449
- Assignment problem, 518, 537, 568, 570, 693
  - bottleneck, 715, 724
  - generalized, 328
- Assignment
  - partial, 145
- Asymmetric TSP, 169
- Asymptotics, 208, 292, 297, 305
- Attribute set, 348
- Attributive memory, 350
- Augmenting path, 174–175
- Automorphisms, 247
- B-join, 491
- Basic solution, 190
- Basis, 48
- BBSSP, 704
- Beam search, 340
  - filtered, 340
- Benchmark, 447
- Benchmark code, 370, 375, 449
- Benchmark codes, 372
- Benchmark machine, 375, 379
- Biconnected, 704
- Bijection, 39
- Binomial random variable, 259–260
- Binomial random variables, 295
- Bipartition, 68
- Black and white TSP, 25
- Block, 87
- Blocked approximation, 298
- Blocked distributions, 297
- Blossoms, 589
- Bottleneck TSP, 7, 697
- Boundary effects, 604–605
- Boundary functional, 258, 282, 284
- Boundary TSP, 286–287, 291, 295
- Bounding procedure, 179
- Branch-and-bound, 169, 451, 634, 658
- Branch-and-cut, 169, 619, 642–643, 652
- Branch and bound, 32
- Branch and Bound, 110
- Branch and bound, 113, 172, 174–176, 195, 200, 666
- Branch and cut, 35, 104, 108, 113, 181, 190, 193, 195, 200
- Branch decision, 173
- Branching, 110
- Branching, 637
- Branching node, 172–173
- Branching nodes, 176
- Branching scheme, 172
- Branching
  - on cuts, 638
  - on variables, 638
  - strong, 112
- Bricks, 44
- Bridge, 42
- BSS bound, 704
- BTGG-scheme, 724
- BTGG scheme, 721
- BTGG Scheme, 722
- BTGG scheme, 723
- BTSP, 698
- Bucket sort, 387
- Cactus, 91
- Candidate list
  - fan, 342
  - filter, 342
  - preferred attribute, 348
- Candidate lists, 338, 348
- Cardinality-constrained TSP, 328
- CAT, 185, 673
  - odd, 673
- Catalan numbers, 213
- Categorization TSP, 734
- CATs, 146
- Catter search, 360
- Cell inventory, 664
- Cellular manufacturing, 11

- Characteristic vector, 35, 188, 191
- Chemical industry, 195
- Chernoff's bound, 259–260
- Chord
  - type 1, 675
- Chunk size, 198
- Chunking, 350
- Chvátal-rank, 160
- Chvátal rank, 164
- Circuit, 495
- Clique-liftable, 136
- Clique-lifting, 138
- Clique lifting, 672, 683, 689
- Clique tree, 67, 677
- Clonable, 57
- Clones, 133
- Cloning, 133, 136, 672, 681, 689, 691, 694
  - 2-cycle, 133, 141
  - edge, 141
- Closed walk
  - almost Hamiltonian, 48
  - minimal, 45
  - minimum f-length, 52
- Closure, 569
- Clustered TSP, 8
- Clusterizations, 641
- Coboundaries, 69, 109
- Coboundary, 30, 61, 64, 70, 75–76, 79, 86, 92, 98
- Cocycle, 30
- Coded attributes, 351
- Coloring, 269
- Column generation, 103
- Comb, 54, 194, 673
- Combinatorial explosion, 352
- Combinatorial implosion, 352
- Combinatorial rank, 586, 588, 600
- Compact formulations, 30
- Compact representations, 17
- Compatible, 489
- Complexity, 180, 293, 527, 530, 532, 534, 542, 546–547, 557, 595, 604, 710, 712, 724
- Components
  - connected, 47
  - strongly connected, 242
- Concave, 705
- Concentration estimate, 286
- Concord, 197
- CONCORDE, 113
- Concorde, 205, 310
- Concorde, 372, 379, 410, 418
- Concorde, 448, 462, 476
- Configuration, 264–265
- Configuration model, 264
- Conjecture, 493
- Connected component, 519, 523, 543
- Connected components, 495, 542
- Connected subset, 589
- Consecutive ones, 93
- Constant-TSP, 492
- Constant TSP, 37, 489–491
- Constraint pool, 193
- Constraints
  - 2-matching, 21
  - circuit packing, 16
  - clique packing, 16
  - coupling, 18
  - generalized 2-matching, 631
  - indegree, 119
  - MTZ, 17
  - network flow, 17
  - outdegree, 119
  - subtour elimination, 16, 22, 119, 170
- Convex-hull-and- $k$ -line TSP, 551–552
- Convex-hull-and- $k$ -line TSP, 553
- Convex-hull-and-additional-point TSP, 558
- Convex-hull-and-line-TSP
  - generalized, 554, 557
- Convex-hull-and-line TSP, 551, 555
- Convex-hull TSP, 582
- Convex distance, 301
- Convex distribution, 606
- Convex dual, 303
- Convex hull, 35, 118
- Convex hull TSP, 548–549
- Convex hull
  - control distance, 301
- Cost ranges, 176
- Covering problem, 239
- Covering salesman, 27
- Covering tour problem, 616
- CPLEX, 103
- Crew scheduling, 328
- Crossover operations, 358
- Cryptography, 274
- CT-digraph, 491–492
- CT-graph, 491–492
- Cubic graphs, 264
- Curve reconstruction, 44, 575
- Cut, 30, 35
  - 2-mod, 84
  - minimum capacity, 33, 628
  - minimum crossing, 91
  - mod- $k$ , 83
  - mod-2, 59
  - value of, 30
- Cutnode, 87
- Cuts
  - Chvatal-Gomory, 186
  - conditional, 643, 647
- Cutset, 564, 732
  - 3-edge, 564
  - $k$ -edge, 564

- Cutter, 92
- Cutting plane, 35
- Cycle cover, 275, 458, 590
  - minimum weight, 275
  - near, 277
- Cycle extension, 268
- Cycle relaxation, 644
- D-heap, 175
- Data compression, 587
- Data structure, 193, 391
- De-Poissonize, 296
- Decrease subtours, 175
- Deficiency of node, 539
- Degree balanced, 539, 576
- Degree constraints, 44
- Delaunay triangulation, 106–107, 391, 405, 548
- Deletable, 269
- Delivery man problem, 25
- Demidenko conditions, 515, 517
- Density matrix, 493, 497
- Depth-first search, 580
- Depth, 329
- Derandomizations, 210
- Deterministic estimate, 292
- Dicycle, 687
- Dicycles, 686
- Differential encoding, 358
- Digraph
  - auxiliary, 570
  - circulant, 582, 729
  - k-regular, 250
  - random regular, 273
  - semicomplete, 580
- Dimension, 40
- Directed pseudograph, 539
- Discrete distributions, 607
- Dissection, 210
  - randomized, 211–212
- Divide and conquer, 209
- DLB approach, 353
- DNA sequencing, 537, 587
- Dominant, 130
- Domination analysis, 223, 225, 227
- Domination number, 226, 244, 252, 254
  - maximum, 245, 250
- Domination ratio, 226
- Domino, 78
  - half, 78, 80
- Don't-look bits, 409
- Double-bridge, 327, 416, 462–463
- DS-digraph, 543, 545
- DS-graph, 544
- Dual problem, 107
- Dual solution, 178, 180, 190–191
- Dual variables, 103
- Dynamic programming, 29, 209, 212–213, 219, 255, 271, 399, 432, 525, 527, 530–531, 547, 553, 577, 594, 612, 705
- Ear, 90
- EC methods, 312
- Edge-based encoding, 359
- Edge-exchanges, 314
- Edge coloring, 56
- Edge
  - cloning, 58
  - f-clonable, 57
  - power weighted, 304
  - rim, 562
  - spoke, 562
- Edges
  - crossing, 52
  - intersect, 549
  - strictly intersect, 549
- EGGTSP, 560, 563
- Ejection chain, 321, 329, 348, 354
  - multi-node, 334
  - node based, 330
  - stem-and-cycle, 336, 355
  - subpath, 335
- Ejection Chains, 312, 325
- Ejection moves, 329
- Ellipsoid algorithm, 34
- Ellipsoid method, 568
- Entropic characterization, 304
- Ergodic theorem, 288–289
  - super-additive, 290
- Error bound
  - worst case, 711
- Euclidean functional
  - smooth sub-additive, 284
- Euclidean norm, 588, 605
- Euclidean TSP, 207, 209, 256, 258, 287, 548, 571
  - higher-dimensional, 221
- Euler tour, 399, 458
- Eulerian graph, 41
- Eulerian path, 291
- Eulerian tour, 540, 709
- Eulerian traversal, 216
- Evolutionary method, 356, 360
- Evolutionary methods, 313
- Expected time, 271
- Extension-rotation, 257, 264
- Extreme points
  - integral, 43
- F-adjacent, 50
- F-connected, 50–51
- F&F process, 344
- Facet-lifting, 120
- Facet-lifting, 669
- Facet defining, 667
- Facet

- composition of, 51
- lifting, 133
- nontrivial, 130
- primitive, 672
- regular, 133, 135
- Facets, 39
  - bad, 128
  - trivial, 130
- Fan, 565
- Farkas lemma, 103
- Farthest neighbor, 605
- Fathoming, 192
- Fence, 552, 554
- Fibonacci number, 233
- Film-Copy Deliverer, 26
- Filter width, 340
- Flowshop, 454
- Formulation
  - Integer LP, 610
  - cycle shrink, 31
  - double index, 31–32, 43
  - multicommodity flow, 20
  - multistage insertion, 31
  - single commodity flow, 19
  - subtour elimination, 31
  - two commodity, 19
- Fractionality Persistence, 197
- Frequency assignment, 13
- Fubini's theorem, 296
- G-G problem, 541, 543–544, 546
- Gap, 455
- Generalized TSP, 8
- Generalized TSP, 610, 615, 617
  - E-GTSP, 615, 618
- Genetic algorithms, 357, 371
- Geometric norm, 207, 548
- Geometric STSP, 370
- Geometric TSP, 207, 547–548
- GG-TSP, 503–505, 533, 539–540
- GG scheme, 506–507, 517–518
- GG Scheme, 521
- GG scheme, 522, 524–525, 530, 532, 534, 536, 545
- Gilmore-Gomory scheme, 496, 503–504, 506
- Gilmore-Gomory TSP, 494
- Graph process, 263
- Graph
  - $\alpha$ -adjacency, 132
  - $k$ -quadrant neighbor, 347
  - auxiliary, 571
  - biconnected, 87, 710
    - square, 710
  - bipartite, 627
  - elementary, 569
  - Eulerian, 576
  - Halin, 565, 569, 730
  - hypohamiltonian, 81
  - outer-facial, 566
  - Peterson, 81
  - power of, 707
  - prismatic, 570
  - rank, 624
  - series parallel, 562
- Graphical TSP, 41
  - even generalized, 560
  - generalized, 560–562, 565
- Graphs
  - isomorphic, 51
- GRASP, 371
- Greedy matching, 405
- Greedy scheme, 537
- Greedy tour, 279
- Gridpoint, 210
- Growth bounds, 283
- Hölder continuity, 284
- Hamiltonian decomposition, 250
- Hamming distance, 285, 300
- Handle, 59, 78, 121, 126, 675, 677
- Handles, 62, 73
- Heap queue, 174
- Held and Karp, 321
- Held and Karp bound, 34
- Helicopter touring, 663
- Hereditary property, 510, 716
  - lower, 509, 717
  - upper, 509, 512, 717
- Heuristic
  - Karps's dissection, 208
  - $\epsilon$ -optimal, 294
  - 2-opt, 407, 410
  - 2-Opt, 632
  - 2.5-opt, 407, 410
  - 2.5-Opt, 414
  - 2.5opt-B, 412
  - 2opt-JM, 413
  - 3-opt-JM, 413
  - 3-opt, 407, 410, 461, 632
  - 3opt-JM, 412
  - 4-opt, 407
  - AppChristo, 406
  - asymptotically optimal, 606
  - Boruvka, 393–394
  - bottleneck double tree, 708
  - boundary, 293
  - canonical, 293
  - CCA, 402, 405
  - cheapest insertion, 398, 632, 636
  - CHII, 398
  - Christo-G, 405, 412
  - Christo-HK, 405, 412
  - Christofides, 208, 215, 251, 293, 385, 403, 458
  - CI, 398
  - composite, 311



- concorde, 396
- construction, 464
- contract or patch, 459, 470
- cycle cover, 470, 473
- DENN, 392, 394, 396
- DMST, 399, 403
- double tree, 227, 251
- farthest city, 606
- farthest insertion, 632, 636
- fast, 386
- FRP, 390
- generalized 2-opt, 632
- GENI, 414
- GENIUS, 414
- greedy, 227, 375, 377, 383, 392, 396
- Greedy, 464
- Heuristic
  - greedy
    - minimum cut, 84
  - Held-Karp, 293
  - Helsgaun's, 449–450, 462
  - Helsgaun, 427, 435, 476
  - HyperOpt, 415
  - hyperopt, 462
  - insertion variants, 400
  - iterated 3-opt, 462
  - Iterated 3-Opt, 473
  - Iterated Hyperopt, 473
  - iterated KP, 462
  - k-opt, 407
  - Karp's partitioning, 399
  - KP, 461, 464, 473
  - Lin-Kernighan, 110, 321, 325, 385, 415, 714
    - chained, 385
    - Johnson-McGeoch, 377
  - Litke's, 400
  - NA, 397
  - NA+, 397
  - nearest insertion, 383, 385, 632, 636
  - nearest neighbor, 2, 227, 252, 383, 391
    - double-ended, 392
  - NI, 397
  - NN-ABCC, 413
  - NN, 396
  - Or-opt, 407, 414
  - patching, 215
  - partitioning, 607
  - Q-Boruvka, 393–394, 418
  - RA, 470
  - recursive partitioning, 386, 389
  - repeated assignment, 458
  - repeated local search, 461
  - RNN, 252
  - savings, 390, 393–394, 396, 402
  - scaled, 294
- Heuristics
  - assignment based, 447, 457
  - construction, 311, 447, 457
  - cycle cover, 458
  - cycle patching, 458
  - experimental analysis, 713
  - Greedy, 457
  - improvement, 311
  - local search, 447, 457, 460
  - Nearest Neighbor, 457
  - probability theory, 293
  - variable depth, 666
- Hitting time, 263
- HK bound, 448, 455, 464
- Homogeneity, 283
- Horizontal separation, 219
- Hungarian method, 449
- Hypergraph, 71
- Hypo-Hamiltonian, 128
- Hypo-semi-Hamiltonian, 128
- Inequality
  - path, 65
- Implementation Challenge, 338, 346, 370–371, 451
- Improvement methods, 313–314
- In-edges, 276–278
- Incidence vector, 35, 50
- Incompatibility graph, 186
- Incompatible, 673
- Incremental gap, 177
- Indirect method, 144
- Inequalities
  - $D_k^-$ , 126
  - $D_k^+$ , 126
  - $D_k^+$ , 183
  - 2-matching, 644, 648
  - ATS, 120, 128
  - bipartition, 68
  - C2 and  $T_k$ , 140
  - C3, 142
  - CAT, 142, 145
  - chain, 58, 142
  - comb, 55, 60, 182, 618, 623
  - separation
    - comb inequality, 631
    - conditional cuts, 651
    - cycle cover, 650
  - spacefilling curve, 385–386
  - strip, 385
  - Strip, 386
  - subtour patching, 606
  - Tabu Search, 431
  - tour construction, 385
  - tourmerge, 436
  - undominated, 381
  - Walshaw's, 435
  - Zhang's, 448, 459, 470

- composition of, 51
- concentration, 258
- cover, 644
- curtain, 125
- equivalent, 142
- facet defining, 120
- facet inducing, 181, 183
- fork, 125
- generalized comb, 634
- isoperimetric, 282
- lifted cycle, 120, 128, 155, 678
- lifting, 667
- maximally 2-lifted cycle, 164
- multi-handle, 98
- odd CAT, 193
- odd CAT and C2, 154
- primitive, 134
- SD, 142, 153
- subtour elimination, 92, 158, 618, 679
- symmetric, 182
- valid, 182
- Inequality
  - subtour elimination, 67
  - TT facet, 55, 58
  - $h$ -canonical form, 143
  - $T_2$ , 144
  - $T_k$ , 126, 138
  - 2-cycle, 133
  - 2-matching, 117
  - arithmetic-geometric mean, 238
  - binested, 73
  - bipartition, 67, 69, 71, 77, 95
  - C2, 126, 140
  - C3, 126
  - canonical, 48
  - canonical form, 144
  - CAT, 186
  - Chebyshev's, 265, 606
  - clique-tree, 121
  - clique tree, 67, 73, 95, 139, 677, 686
  - closed-set form, 73
  - comb, 59, 62, 64, 79, 109, 121
  - crown, 82
  - cycle, 183
  - cycle cover, 645
  - degree, 45
  - domino, 78–79
  - domino parity, 78
  - external, 45
  - facet defining, 122, 671
  - facet inducing, 38
  - fan, 627
  - FDA, 686
  - generalized 2-matching, 631
  - generalized Chebyshev, 258
  - generalized comb, 623
  - generalized subtour elimination, 627
  - $h$ -liftable, 52
  - hypohamiltonian, 81
  - isoperimetric, 285, 300
  - Jensen's, 258, 288, 292
  - KP, 645
  - ladder, 71, 73, 77, 97
  - lifted 3-cycle, 686
  - lifted cycle, 159, 686
  - merge-lifted, 140
  - multiple handle, 94
  - non negativity, 62
  - odd CAT, 148, 675, 691
  - path, 63, 646, 649
  - primitive CAT, 681
  - primitive comb, 672–673, 677, 684
  - primitive odd CAT, 675, 677, 684
  - primitive SD, 675, 684, 689
  - SD, 151, 675, 683
  - SD primitive, 152
  - simple, 47
  - simple hypohamiltonian, 81
  - star, 62, 65, 76
  - subtour elimination, 31, 44, 62, 74, 101, 109, 164
  - T-lifted, 138
  - tight-triangular, 625
  - tight triangular, 45, 49, 137
  - triangular, 41
  - trivial, 45
  - TT, 47, 49, 625
  - TT facet, 49, 51, 54
  - twisted comb, 79–80
  - valid, 45, 188–189
  - ZT-clique tree, 141–142
- Instance generators, 372
- Instance
  - phylogenetic problem, 451
  - Random Matrix, 451
  - Random Uniform Euclidean, 451
- Instances
  - Additive norm, 452
  - benchmark, 447
  - code optimization, 455
  - coin collection, 454
  - common superstring, 454
  - Disk Drive, 453
  - geometric, 446
  - no-wait flowshop, 454
  - random, 446
  - random 2D Rectilinear, 451
  - random matrix, 451
  - realworld, 455
  - robotic motion, 455
  - Stacker Crane, 453
  - STSP testbed, 448
  - sup norm, 453
  - table compression, 455

- tape drive reading, 455
- Tilted Drilling Machine, 452
- TSPLIB, 446
- vehicle routing, 455
- Integer linear program, 310
- Integer program, 566, 579
- Integer programming, 109
- Integer programming, 170
- Integer programming, 668, 698
- Integrability condition, 301
- Intensification approach, 353
- Intermediate structure, 312
- Internal nodes, 731
- International TSP, 615
- International TSP, 664
- Interportal distance, 214
- Intersection graph, 121, 521
- Interval property, 63
- Isoperimetric, 258, 285
- Isoperimetric methods, 300
- Isoperimetric theory, 300
- JNRT, 114
- Join, 491
- K-cycles, 687
- K-peripatetic salesman, 27
- Kalmanson TSP, 549
- KCAP, 666
- La-path, 525
- Lacing
  - diagonal, 5
  - quick, 5
  - straight, 5
  - zig-zag, 5
- Lagrangian relaxation, 381
- Lagrangian relaxations, 321, 356
- Lagrangian subproblems, 666
- Lagrangian relaxation, 29, 635–637, 639
- Lagrangian
  - restricted, 177
- Large cycles, 278
- Large deviation principle, 303
- Large vertices, 268
- Laundry van problem, 2
- LCF-free, 552, 554
- Legitimacy conditions, 329, 337
- Legitimacy restrictions, 329
- Legitimate structure, 333
- Level, 329
- Lifting, 54, 692, 694
- Lifting coefficients, 125, 669
- Lifting procedure, 621
- Lifting sequence, 621
- Lifting set, 691
- Lifting theorem, 621
- Lifting
  - T, 133
  - 0-node, 137
  - 1-node, 55
  - 2-node, 56
  - clique, 54, 81, 133, 136, 187
  - merge, 133, 139
  - node, 55
  - sequence, 148
  - sequential, 52, 74, 125
  - T, 137–138, 142
- Liftings, 671
- Limit theorems, 258
- Lin-Kernighan, 312, 319, 321, 335, 337, 348, 353–355, 360, 367, 371, 425
- Lin-Kernighan procedure, 325
- Lin-Kernighan
  - basic, 417
  - CLK-ABC-N, 381
  - CLK-ABCC, 426
  - CLK-ACR, 426
  - Helsgaun-.1N, 381
  - Helsgaun-k, 426
  - Helsgaun, 422, 426
  - Iterated, 371
  - iterated, 425
  - ILK-JM, 426
  - LK-Neto, 426
  - LH-JM, 417
  - LK-ABCC, 417
  - LK-ACR, 418
  - LK-Neto, 417
  - MLLK-.1N, 381
- Linear equations, 36
- Linear program, 103, 105, 280, 543
  - bottleneck, 705
- Linear programming, 15, 178, 209, 448, 566, 573, 605, 705
- Linear programs, 33
- Linear relaxation, 32
- Linear representation, 3
- LK method, 325
- LK procedure, 327, 337
- LK process, 323
- Local cut, 100
- Local search, 101, 209
- Local search, 224
- Local search, 226, 249, 329, 347, 373, 385
  - chained, 425
- Local search
  - iterated
  - 3-Opt, 425
- Long chord, 552
- Longest path, 261
- Longest string, 587
- Look-ahead, 325
- Look ahead, 311
- Lookup table, 213
- Lower bound, 171–173, 178–179, 181, 191, 197, 200, 263, 279

- Assignment, 448
- Assignment, 447
- asymmetric HK, 448
- Balas and Christofides, 177
- Christofides, 177
- Held-Karp, 379, 388, 394, 399, 447
- Held and Karp, 573
- Lower minor, 717
- Lower minors, 510
- Lower
  - minor, 509
- M-salesman TSP, 8
- Machine sequencing, 9
- Makespan, 504
- Manhattan norm, 601
- Marginal value, 107
- Martingale estimates, 302
- Master tour, 582
- Matching problem, 566
- Matching
  - b-matching, 559
  - fractional 2-matching, 567
  - maximum, 231
  - maximum weight, 305, 591, 603
  - minimum cost, 531
  - minimum weight, 231
  - near perfect, 264
  - non-bipartite, 612
  - partial, 552
  - perfect, 216, 276, 281
    - minimum weight, 275, 589
  - weighted, 530
- Matrix
  - reduced cost, 174
  - AM, 517
  - BCT, 493
  - BR, 579, 581
  - Brownian, 581
  - Burkov-Rubinshtein, 579
  - constant tour, 490
  - CT, 490
  - DD-graded, 720, 727
  - Demidenko, 514–515
  - doubly graded, 720, 726
  - downward graded, 720
  - DU-graded, 720
  - generalized Kalmanson, 514–515
  - generalized Supnick, 514–515
  - Gilmore Gomory, 534
  - graded, 725
    - diagonally inward graded, 720
    - diagonally outward, 720
  - Hamming, 301
  - inward graded, 727
  - Kalmanson, 514–515, 549–550, 554, 558, 582
    - non-adjacent, 550
    - non-degenerate, 550
- Matrix
  - kalmanson
    - non-degenerate, 555
  - Klyaus, 517
  - max-distribution, 720, 723, 726
  - max-Klyaus, 721
  - max-Lawler, 714
  - Monge, 522
  - NE-staircase, 534
  - NK, 550
  - ordered product, 508, 725
  - permuted cost, 495, 721
  - product, 506, 524
  - pseudo-large, 505
  - reduced cost, 181
  - residual cost, 177
  - SDCI, 727
  - small, 535, 725
  - sparse, 176
  - Supnick, 514–515
  - Suprunenko-I, 517
  - type 2-GG, 533
  - type k-GG, 534, 536
  - UD-graded, 720
  - upper triangular, 538, 715
  - upward graded, 720
  - UU-graded, 720, 725, 727
  - Van der Veen, 516
- Matroid, 630
- Matroid intersection, 23
- Max-BK-II( $k$ ), 718
- MAX-SNP-hard, 208
- MAX TSP, 7, 586, 701
- Maximum flow, 628
- Maximum length tour, 305
- Maximum scatter TSP, 698
- MBTSP, 731
- MCNDP, 541, 543–544
- Mean execution time, 293
- Mean value, 284
- Median cycle problem, 616
- Memetic algorithm, 457
- Memory
  - frequency-based, 350, 352
  - long-term, 350
  - recency-based, 350, 352
  - residence frequency, 352
  - semi-explicit, 351
  - short-term, 350, 354
  - transition frequency, 352
- Messenger problem, 1, 7
- Meta-heuristic, 312
- Meta-heuristics, 255
- Metaheuristics, 313, 367, 371, 457
- Method
  - ejection chain, 331

- Methods
  - constructive neighborhood, 313
  - ejection chain, 327
  - filter and fan, 341
  - noising, 339
  - population-based neighborhood, 314
  - sequential fan, 340
  - transition neighborhood, 314
  - variable depth, 329
- Metric problem, 586
- Metric TSP, 207–208
- Minimal cover, 667
- Minimum-cost cover, 448
- Minimum cost flow, 568
- Minimum cut, 82, 568
- Minimum latency problem, 25
- Minimum makespan, 10
- Minimum Steiner tree, 220
- Minkowski functional, 600–601
- Minkowski norm, 207
- Minor, 43
- Mixed pseudograph, 519–520
- Monotonicity, 283
- Monotonization, 129–130, 145
  - generalized, 130
- Move
  - 2-opt, 409
  - 4-Opt, 425
  - compound, 329
  - k-opt, 407
  - transition, 329
- Moves
  - $k$ -opt, 367
  - $L$ -compound, 341
  - 2-opt, 322
  - 3-opt, 322
  - 4-opt, 321–322
  - 5-opt, 321
  - compound, 341
  - cycle-ejection, 336
  - double bridge, 327
  - legitimate, 344
  - LK, 353
  - multi-node exchange, 330, 332
  - multi-node insertion, 330
  - single node-insertion, 344
  - stem-ejection, 336
  - string, 321
  - unstring, 319
  - variable depth, 327
- Moving target TSP, 27
- MSTSP, 698, 706, 711, 727
- Multi-path, 525, 530, 724
- Multi-set, 265
- Multi-star, 532
- Multi-tour, 525, 530
- Multi-tree, 525, 530–531, 724
- Multigraph, 41, 85, 247, 264, 529–530, 576
  - series-parallel, 562
- Multiple visit TSP, 7
- Multiset, 216, 576
- MVI, 510, 512
- Natural formulations, 17
- Natural partition, 499
- Nearest Neighbor, 381, 464, 606
- Nearly additive, 284
- Necklace, 92
- Necklace Condition, 571
- Neighbor lists, 338
- Neighborhood, 228
- Neighborhood digraph, 240
- Neighborhood search
  - F&F, 344
  - variable, 462
- Neighborhood size, 237
  - upper bound, 238
- Neighborhood structure, 240, 311, 329, 354, 407
  - String/Unstring, 319
- Neighborhood structures, 314
  - compound, 311
- Neighborhood tree, 342
- Neighborhood
  - $k$ -exchange, 314
  - 2-opt, 230
  - assign, 230
  - Balas-Simonetti, 232
  - cardinality, 225
  - Carlier-Villon, 249
  - compound, 330
  - constructive, 355
  - destructive, 355
  - diameter of, 240
  - exponential, 227
  - k-opt, 407
  - Or-opt, 349
  - polynomially searchable, 228
  - PVC, 230, 241
  - pyramidal, 229, 241
  - restricted 4-Opt, 415
  - shifting property, 228
  - structure, 228
- Neighborhoods
  - exponential, 223, 340, 371
  - non-sequential, 327
  - sequential, 327
- Neural nets, 371
- Neutral 2-cycle, 147
- No-wait flow shop, 10
- Node-insertion, 314
- Node deficiency, 541
- Node lifting, 47, 51
- Non-basic variables, 280
- Non-trivial subtours, 495, 519

- Normalization, 375, 447, 449
- Normalized form, 40
- Normed space, 586
- Odd CATs, 148
- Odd cut
  - minimum capacity, 631
- Odd cycle, 39
- Optimal assignment, 277
- Optimum assignment, 279
- Or-insertion, 314
- Or-opt, 312
- Or-Opt, 314
- Or-opt, 318, 335
- Order digraph, 499–500, 502
- Ordered cluster TSP, 26
- Ordered partition, 552
- Orienteering problem, 26
- Orienteering Problem, 610, 614, 643
- Orienteering problem, 664
- Out-edges, 276–278
- Outer nodes, 731
- Overload segment, 217
- Pairing, 264
- Parallel algorithms, 272
- Parallelization, 175
- Parametric technique, 173–174
- Parametrization, 174
- Parity function, 560
- Partition matroid, 23
- Partitioning heuristic, 293
- Patch edges, 276
- Patching, 529, 721
- Patching lemma, 217
- Patching
  - adjacent, 496, 503, 538
  - cycles, 271
  - GG, 503, 508
  - non-adjacent, 538
  - optimal, 604
  - pseudograph, 495–496, 522, 530, 544
  - subtour, 605
- Patchings, 725
- Path-Relinking, 364
- Path-relinking, 366
- Path Relinking, 355
- Path
  - alternating, 715
  - bottleneck, 715
  - minimum weight, 529
  - portal-respecting, 214
  - self-intersecting, 549
- PCTSP, 665
- Peak, 508, 581
  - proper, 581
  - set-back, 581
- Peaks, 718
- Penalty triplets, 731
- Pendant tooth, 142
- Perfect matching, 37
  - minimum cost, 448
- Performance bound, 709–710, 712
- Performance estimate, 607
- Performance ratio, 590–591, 595, 597, 604
- Permutation
  - cost of relative to, 501
  - cyclic, 310, 540
  - dense, 499
  - identity, 495
  - minimum cost, 538
  - product of, 495
  - random, 276
- Permutations, 228
  - cyclic, 704
- Perturbation, 210
- Pick-and-place machine, 10
- Pigeonhole principle, 283
- Points, 264
- Poisson random variable, 295
- Poisson
  - asymptotically, 263
- Poissonized version, 295
- Polyhedral norm, 560, 587–588, 599, 601
- Polyhedron
  - full dimensional, 39
  - graphical TSP, 129
  - unbounded, 41
- Polynomial 2-opt, 249
- Polynomial restriction, 254
- Polynomially equivalent, 532
- Polytope, 181, 566–567
  - monotone ATS, 128
  - 1-arborescence, 129
  - 2-matching, 39, 567, 569
  - AA, 145, 150
  - APA, 148
  - assignment, 129
  - ATS, 118, 150, 667, 681, 689, 691
    - subtour elimination, 668
    - valid inequality, 671
  - ATSP, 189
  - cycle-and-loops, 689, 694
  - cycle, 667, 686, 689, 691, 693
  - dimension, 36
  - E-GTSP, 618, 623
  - flat STS, 137
  - full dimensional, 39, 42
  - general 0-1, 135
  - GTSP, 620
  - knapsack, 667
  - monotone, 129
  - monotone ATS, 136
  - monotone STSP, 39
  - monotonization, 59
  - PCTS, 681, 689

- PCTSP, 673
- STS, 118
- subtour elimination, 33–34, 44, 567, 573
- symmetric TSP, 35
- traveling salesman, 567
- TSP, 189, 626
- vertex packing, 146
- Population-based approach, 355
- Population-based methods, 313
- Portal, 212
  - parameter, 212
  - respecting tour, 212
- Portals, 217
- Postal routing, 615
- PQ-tree, 91
- Pre-processing, 505
- Precedence-constrained TSP, 663
- Precedence constrained TSP, 27
- Preferred attributes, 348
- Price out, 191
- Pricing, 107–108, 190–191, 193–194, 653
  - AP, 191
  - heuristic, 192
  - Lagrangian, 191
  - loop, 193
  - standard, 191–192
- Primal degeneracy, 191
- Primal dual, 173
- Primal solution, 190
- Principal submatrix, 507
- Priority queue, 391
- Prism, 570
- Prize-Collecting TSP, 610, 613–614
  - capacitated, 614
- Prize Collecting, 664
- Probability measure, 257
- Probability space, 605
- Property
  - Baki-II, 516–517
  - Baki-Kabadi-II, 517
  - Baki-Kabadi, 511
  - Baki-Kabadi III, 513
  - Baki-Kabadi IV, 513
  - G-G, 544
  - GG1, 522, 524
  - GG2, 522, 524
  - GG3, 523
  - GG4, 523, 536
  - max patching - II, 723
  - max patching I, 722
  - MPP-I, 722
  - MPP-II, 723
- Pseudo-node, 85, 101
- Pseudograph
  - patching, 722
- Pseudorandom, 541
- PTAS, 208
- Pyramidal
  - circuit, 500
  - path, 499, 508
  - permutation, 499
  - set-back tour, 581
  - solvability, 510, 513
  - tour, 502, 508, 514, 526, 533, 600
  - tours, 513
- Pyramidally solvable, 508–509, 511–514, 518, 523–525, 715, 718–720, 726
- Quad-neighbor list, 409
- Quadratic programming, 22
- Quadratic representation, 3
- Quadtree, 213
- Quasi-convex, 705
- Quasi-parallel, 551
- Quasi clone, 683, 686, 691
- Quasi clones, 681
- Quasi parallel, 553
- R-SAADP, 178, 180
- R-SAAP, 171, 178
- R-SADP, 172, 178, 180
- R-SAP, 171, 178
- Random dissection, 211
- Range, 519
- Raw data, 372
- Reduced cost, 108, 174–176, 178–179, 191–192
- Reduced costs, 193, 280
- Reference solutions, 365
- Reference structure, 322, 325
  - Hamiltonian path, 322
  - stem-and-cycle, 322, 335
- Refueling, 663
- Region, 519
- Regular graph
  - random, 272
- Regular graphs
  - random, 264
- Relaxation, 171
  - Held-Karp, 209
  - linear programming, 379
- Remote TSP, 27
- Residual cost, 178
- Resource constrained TSP, 26
- ROLL-A-ROUND, 666
- Rolling mills, 666
- Root node, 33
- Rooted tree, 278
- Rotation, 261
- Routing applications, 195
- Row generation, 33
- Row rank, 130
- Running time
  - normalized, 370, 381
- Rural Postman Problem, 541
- S-components, 562

- S-sum, 51
- Saturated set, 188
- Scatter Search, 355
- Scatter search, 356–357
- Scheduling, 535, 664
  - flow shop, 504
  - m-machine flow shop, 514
  - no wait, 504
  - robotic cell, 535
- SCSS bound, 704
- SEC, 119
- Seed node, 135
- Selective TSP, 26
- Selective TSP, 614
- Semi-regular curve, 575
- Semimetric problem, 586
- Separation, 190
- Separation problem, 82, 181, 627
- Separation procedure, 205
- Separation
  - 2-matching, 82
  - bipartition, 83
  - comb, 83
    - heuristic, 86
  - cover inequality, 650
  - domino, 83
  - heuristic, 85
  - simple comb, 82
  - subtour elimination, 82
- Sequencing, 578
- Sequential Fan tree, 340
- Sequential lifting, 668
- Serdyukov TSP, 26
- Shoelace problem, 4
- Short-cutting, 709
- Shortcut, 47
- Shortest path, 180, 233
- Shortest path condition, 137
- Shortest superstring, 587
- Shrinking, 86
- Shrinking procedure, 188–189
- Shrinking technique, 630
- Shrinkings, 85
  - legal, 85
- Sidelength, 210
- Simple chain, 500
- Simple cycle problem, 609
- Simplex method, 568
- Simulated annealing, 371
- Small cycles, 277
- Small vertex, 269
- Small vertices, 268
- Sorting network, 241
- Space smoothing, 339
- Spanner, 220
- Spanning closed walk, 40
- Spanning subgraph
  - biconnected, 710
- Spanning tree, 132, 321, 498, 503–504, 519–520, 530–531, 542, 544, 722
  - bottleneck, 709, 725
  - minimum, 630, 648
- Sparsification, 175–176
- Spherical volume, 603
- Star, 530
- Steel rolling mill, 664
- Steiner node, 541
- Steiner nodes, 541
- Steiner tree, 544
- Stem-and-cycle, 321, 338
- Stem-and-Cycle, 354, 360, 367
- Stem and cycle, 417, 421
- Stirling's formula, 233
- Stochastic convergence, 258
- Strategic oscillation, 319
- Strategic oscillation, 354
- Strong connectivity, 170, 179
- Strongly connected, 179, 579, 704
- Strongly Hamiltonian, 491
- Structuring matrices, 14
- STSP Challenge, 447
- Sub-additivity, 283–284, 295
- Sub-additivity, 295
- Sub-additivity
  - geometric, 283, 287
- Sub-cube, 283, 288, 292
- Sub-Gaussian tail, 302
- Sub-multi-tree, 531
- Sub-path, 532
- Subchains, 558
- Subgradient optimization, 636
- Submatrix
  - lower principal, 509
  - principal, 523, 536
  - upper principal, 509
- Submissive, 132
- Submoves, 341
- Subpath, 549
- Subpaths, 315, 319, 335
- Subroots, 336
- Subtour cover, 607
- Subtour elimination, 172
- Subtour merging, 173, 175
- Subtour
  - non-trivial, 498, 717
- Subtours
  - vertex disjoint, 171
  - arc disjoint, 544
- Super-additivity, 283–284, 287
  - geometric, 287, 299
  - strong, 289
- Supernodes, 335
- Superstring, 454
- Support graph, 85



- Support reduced, 130–131, 136
  - strongly, 131–132
- Surrogate constraints, 356
- Surrogate relaxations, 356
- Symmetric TSP, 169
- Tabu list, 351
- Tabu restrictions, 350
- Tabu Search, 345
- Tabu search, 350
- Tabu search, 353, 355, 371
- Tabu Search, 381, 385
- Tabu search
  - reactive, 351
- Tabu tenure, 351
- Tailing off, 110, 194
- Taxicab ripoff” problem, 601
- Taxicab ripoff problem, 585
- Teeth, 59, 61–62, 72, 78, 126, 675, 677
  - odd, 80
- Testbed, 310, 381, 449, 464
  - random, 372
  - random Euclidean, 373
    - clustered, 373
  - random matrix, 374
  - TSPLIB, 373
- Testbeds, 370, 372, 446–447, 450
- Threshold, 260
- Threshold algorithm, 701, 712, 714
  - generalized, 701
    - approximate, 714
- Threshold method, 175
- Threshold
  - Hamiltonicity, 260
- Time dependent TSP, 25
- Time window TSP, 27
- Tooth
  - degenerate, 68
- Total unimodularity, 543
- Tour merging, 436
- Tour
  - affinely independent, 36
  - compatible, 102
  - portal-respecting, 214–215
  - pyramidal, 727
- Tours
  - portal-respecting, 213
- Transformation, 195
- Translation invariance, 283
- Transportation problem, 578
- Transposition, 495
  - adjacent, 495, 501
- Traps, 44, 573
- Traveling purchaser problem, 617
  - capacitated, 617
- Traveling repairman problem, 25
- Tree-composition, 123
- Tree-like structure, 67
- Tree
  - 4-ary, 212
- Triagle inequality, 23
- Trial move, 329, 332
- Triangle inequality, 244, 508, 571, 596
  - parameterized, 23
- Troublesome vertices, 274, 276
- TSP Challenge, 310
- TSP polytope, 490, 610
  - dimension, 490
- TSPLIB, 34, 107, 411, 450
- TT-form, 49
- TT, 45
- Tunneling, 365
- Tunneling TSP, 558
- Twisted comb, 80
- Type GG, 532
- Ua-path, 525
- Upper bound, 174, 183, 197
- Upper minor, 509, 717
- Upper minors, 510
- Upperbound, 110, 112, 218
- V-compatibility graph, 624–625
- V-restriction, 624, 626
- V-tree, 572
- Valid inequality
  - primitive, 671
- Valley, 508
- Valleys, 718
- Variable-depth, 462
- Variable depth methods, 311
- Variable fixing, 107
- Variable fixing, 172, 193, 620
- Vehicle routing, 328
- Vertex
  - insertion, 228
  - k-critical, 52
  - removal, 227
- Violated cuts, 188, 193
- Vocabulary building, 350, 355
- Voronoi diagram, 572
- Wallpaper cutting, 536, 545
- Warehouse, 586
- Warehouse TSP, 598
- Weighted girth problem, 611
- Wheel, 562, 569, 731
- Whp, 258
- With high probability, 258
- Worst-case analysis, 257, 279
- Worst case analysis, 706
- Zero-lifting, 55
- Zero lifting, 104 $\Omega$